DSA NOTES

Time and Space Complexity

Analysis of Algorithms (Background)

What is meant by Algorithm Analysis?

Algorithm analysis is an important part of computational complexity theory, which provides theoretical estimation for the required resources of an algorithm to solve a specific computational problem. Analysis of algorithms is the determination of the amount of time and space resources required to execute it.

Why Analysis of Algorithms is important?

- To predict the behavior of an algorithm without implementing it on a specific computer.
- It is much more convenient to have simple measures for the efficiency of an algorithm than to implement the algorithm and test the efficiency every time a certain parameter in the underlying computer system changes.
- It is impossible to predict the exact behaviour of an algorithm. There are too many influencing factors.
- The analysis is thus only an approximation; it is not perfect.
- More importantly, by analyzing different algorithms, we can compare them to determine the best one for our purpose.

Types of Algorithm Analysis:

1. Best case
2. Worst case
3. Average case

Asymptotic Analysis

*In Asymptotic Analysis, we **evaluate the performance of an algorithm in terms of input size** (we don't measure the actual running time). We calculate, how the time (or space) taken by an algorithm increases with the input size.*

Why performance analysis?

There are many important things that should be taken care of, like user-friendliness, modularity, security, maintainability, etc. Why worry about performance? The answer to this is simple, we can have all the above things only if we have performance. So performance is like currency through which we can buy all the above things. Another reason for studying performance is – speed is fun! To summarize, performance == scale. Imagine a text editor that can load 1000 pages, but can spell check 1 page per minute OR an image editor that takes 1 hour to rotate your image 90 degrees left OR … you get it. If a software feature can not cope with the scale of tasks users need to perform – it is as good as dead.

Given two algorithms for a task, how do we find out which one is better?

One naive way of doing this is – to implement both the algorithms and run the two programs on your computer for different inputs and see which one takes less time. There are many problems with this approach for the analysis of algorithms.

- It might be possible that for some inputs, the first algorithm performs better than the second. And for some inputs second performs better.
- It might also be possible that for some inputs, the first algorithm performs better on one machine, and the second works better on another machine for some other inputs.

*Asymptotic Analysis is the big idea that handles the above issues in analyzing algorithms. In Asymptotic Analysis, we **evaluate the performance of an algorithm in terms of input size** (we don't measure the actual running time). We calculate, how the time (or space) taken by an algorithm increases with the input size.*

For example, let us consider the search problem (searching a given item) in a sorted array.

The solution to above search problem includes:

- **Linear Search** (order of growth is linear)
- **Binary Search** (order of growth is logarithmic).

To understand how Asymptotic Analysis solves the problems mentioned above in analyzing algorithms,

- let us say:
  - we run the Linear Search on a fast computer A and
  - Binary Search on a slow computer B and
  - pick the constant values for the two computers so that it tells us exactly how long it takes for the given machine to perform the search in seconds.
- Let's say the constant for A is 0.2 and the constant for B is 1000 which means that A is 5000 times more powerful than B.
- For small values of input array size n, the fast computer may take less time.
- **But, after a certain value of input array size, the Binary Search will definitely start taking less time compared to the Linear Search even though the Binary Search is being run on a slow machine**.

Input Size	Running time on A	Running time on B
10	2 sec	~ 1 h
100	20 sec	~ 1.8 h
10^6	~ 55.5 h	~ 5.5 h
10^9	~ 6.3 years	~ 8.3 h

- The reason is the order of growth of Binary Search with respect to input size is logarithmic while the order of growth of Linear Search is linear.
- **So the machine-dependent constants can always be ignored after a certain value of input size.**

Running times for this example:

- Linear Search running time in seconds on A: 0.2 * n
- Binary Search running time in seconds on B: 1000*log(n)

Does Asymptotic Analysis always work?

Asymptotic Analysis is not perfect, but that's the best way available for analyzing algorithms. For example, say there are two sorting algorithms that take 1000nLogn and 2nLogn time respectively on a machine. Both of these algorithms are asymptotically the same (order of growth is nLogn). So, With Asymptotic Analysis, we can't judge which one is better as we ignore constants in Asymptotic Analysis.

Also, in Asymptotic analysis, we always talk about input sizes larger than a constant value. It might be possible that those large inputs are never given to your software and an asymptotically slower algorithm always performs better for your particular situation. So, you may end up choosing an algorithm that is Asymptotically slower but faster for your software.

Worst, Average and Best Case Time Complexities

It is important to analyze an algorithm after writing it to find it's efficiency in terms of time and space in order to improve it if possible.

When it comes to analyzing algorithms, the asymptotic analysis seems to be the best way possible to do so. This is because asymptotic analysis analyzes algorithms in terms of the input size. It checks how are the time and space growing in terms of the input size.

In this article, we will take an example of Linear Search and analyze it using Asymptotic analysis.

We can have three cases to analyze an algorithm:

1. Worst Case
2. Average Case
3. Best Case

Below is the algorithm for performing linear search:

C

```c
// Linearly search x in arr[].
// If x is present then return the index,
// otherwise return -1
int search(int arr[], int n, int x)
{
    int i;
    for (i=0;i<n;i++){
        if(arr[i]==x){
```

```
                return i;
            }
        }
        return -1;
}

//Driver program to test above functions
int main(){
        int arr[]={2,8,12,9};
        int x=12;
        int n=sizeof(arr)/sizeof(arr[0]);
        printf("%d is present in %d index",x,search(arr,n,x));
        getchar();
        return 0;
}
```

Worst Case Analysis (Usually Done): In the worst case analysis, we calculate upper bound on running time of an algorithm. We must know the case that causes the maximum number of operations to be executed. For Linear Search, the worst case happens when the element to be searched (x in the above code) is not present in the array. When x is not present, the search() functions compares it with all the elements of arr[] one by one. Therefore, the worst case time complexity of linear search would be O(N), where N is the number of elements in the array.

Average Case Analysis (Sometimes done): In average case analysis, we take all possible inputs and calculate computing time for all of the inputs. Sum all the calculated values and divide the sum by total number of inputs. We must know (or predict) distribution of cases. For the linear search problem, let us assume that all cases are uniformly distributed (including the case of x not being present in array). So we sum all the cases and divide the sum by (N+1). Following is the value of average case time complexity.

$$\text{Average Case Time} = \frac{\sum_{i=1}^{n+1} \theta(i)}{(n+1)}$$

$$= \frac{\theta((n+1)*(n+2)/2)}{(n+1)}$$

$$= \theta(n)$$

Best Case Analysis (Bogus): In the best case analysis, we calculate lower bound on running time of an algorithm. We must know the case that causes minimum number of

operations to be executed. In the linear search problem, the best case occurs when x is present at the first location. The number of operations in the best case is constant (not dependent on N). So time complexity in the best case would be O(1).

Example-

C++

```cpp
 #include <bits/stdc++.h>
using namespace std;

// Linearly search x in arr[].
// If x is present then return the index,
// otherwise return -1
int search(int arr[], int n, int x)
{
    int i;
    for (i = 0; i < n; i++) {
        if (arr[i] == x)
            return i;
    }
    return -1;
}

// Driver's Code
int main()
{
    int arr[] = { 1, 10, 30, 15 };
    int x = 30;
    int n = sizeof(arr) / sizeof(arr[0]);

    // Function call
    cout << x << " is present at index "
         << search(arr, n, x);

    return 0;
}
```

Time Complexity Analysis: (In Big-O notation)

- **Best Case:** O(1), This will take place if the element to be searched is on the first index of the given list. So, the number of comparisons, in this case, is 1.
- **Average Case:** O(n), This will take place if the element to be searched is on the middle index of the given list.
- **Worst Case:** O(n), This will take place if:
    - The element to be searched is on the last index

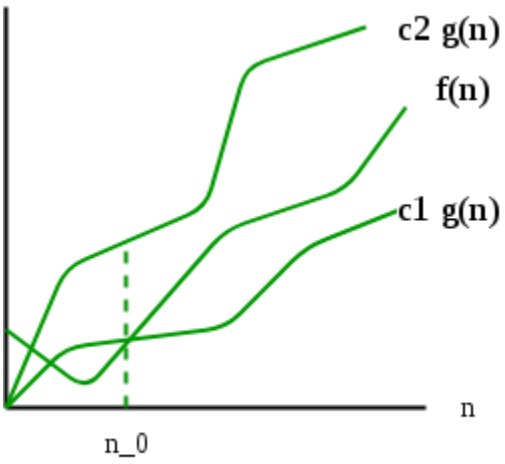
- The element to be searched is not present on the list

Important Points:

- Most of the times, we do the worst case analysis to analyze algorithms. In the worst analysis, we guarantee an upper bound on the running time of an algorithm which is a good piece of information.
- The average case analysis is not easy to do in most of the practical cases and it is rarely done. In the average case analysis, we must know (or predict) the mathematical distribution of all possible inputs.
- The Best Case analysis is bogus. Guaranteeing a lower bound on an algorithm doesn't provide any information as in the worst case, an algorithm may take years to run.

Asymptotic Notations

Asymptotic notations are mathematical tools to represent the time complexity of algorithms for asymptotic analysis. The following 3 asymptotic notations are mostly used to represent the time complexity of algorithms:



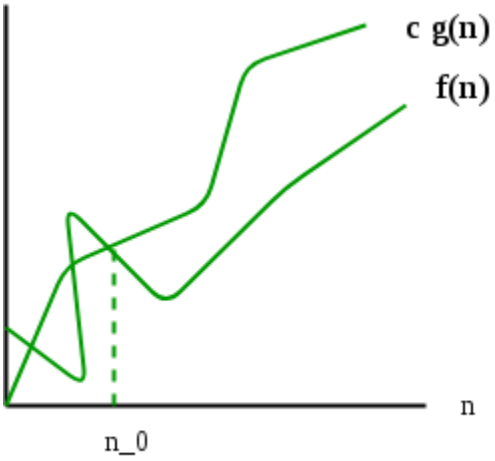
f(n) = theta(g(n))

1) Θ Notation: The theta notation bounds a function from above and below, so it defines exact asymptotic behavior.
A simple way to get the Theta notation of an expression is to drop low-order terms and ignore leading constants. For example, consider the following expression.
$3n^3 + 6n^2 + 6000 = \Theta(n^3)$
Dropping lower order terms is always fine because there will always be a number(n) after

which $\Theta(n^3)$ has higher values than $\Theta(n^2)$ irrespective of the constants involved.
For a given function g(n), we denote $\Theta(g(n))$ is following set of functions.

```
Θ(g(n)) = {f(n): there exist positive constants c1, c2 and n0
such
                  that 0 <= c1*g(n) <= f(n) <= c2*g(n) for all n
>= n0}
```

The above definition means, if f(n) is theta of g(n), then the value f(n) is always between c1*g(n) and c2*g(n) for large values of n (n >= n0). The definition of theta also requires that f(n) must be non-negative for values of n greater than n0.

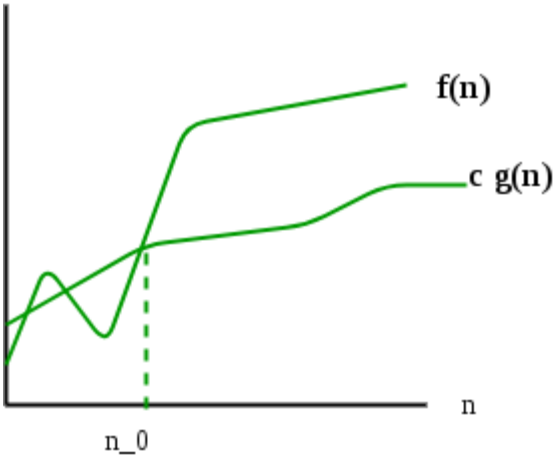


f(n) = O(g(n))

2) Big O Notation: The Big O notation defines an upper bound of an algorithm, it bounds a function only from above. For example, consider the case of Insertion Sort. It takes linear time in best case and quadratic time in worst case. We can safely say that the time complexity of Insertion sort is O(n^2). Note that O(n^2) also covers linear time.
If we use O notation to represent time complexity of Insertion sort, we have to use two statements for best and worst cases:
1. The worst case time complexity of Insertion Sort is O(n^2).
2. The best case time complexity of Insertion Sort is O(n).

The Big O notation is useful when we only have upper bound on time complexity of an algorithm. Many times we easily find an upper bound by simply looking at the algorithm.

```
O(g(n)) = { f(n): there exist positive constants c and
                  n0 such that 0 <= f(n) <= c*g(n) for
                  all n >= n0}
```



f(n) = Omega(g(n))

3) Ω Notation: Just as Big O notation provides an asymptotic upper bound on a function, Ω notation provides an asymptotic lower bound.

Ω Notation can be useful when we have lower bound on time complexity of an algorithm. The Omega notation is the least used notation among all three.

For a given function g(n), we denote by Ω(g(n)) the set of functions.

```
Ω (g(n)) = {f(n): there exist positive constants c and
                  n0 such that 0 <= c*g(n) <= f(n) for
                  all n >= n0}.
```

Big O Notation

In this article, we will discuss the analysis of the algorithm using Big - O asymptotic notation in complete detail.

Big-O Analysis of Algorithms

We can express algorithmic complexity using the big-O notation. For a problem of size N:

- A constant-time function/method is "order 1" : O(1)
- A linear-time function/method is "order N" : O(N)
- A quadratic-time function/method is "order N squared" : $O(N^2)$

Definition: Let g and f be functions from the set of natural numbers to itself. The function f is said to be O(g) (read big-oh of g), if there is a constant $\mathbf{c > 0}$ and a natural number $\mathbf{n_0}$ such that $f(n) \leq cg(n)$ for all $n >= n_0$.

Note: O(g) is a set!

Abuse of notation: f = O(g) does not mean $f \in O(g)$.
The Big-O Asymptotic Notation gives us the Upper Bound Idea, mathematically described below:

$f(n) = O(g(n))$ if there exists a positive integer n_0 and a positive constant c, such that $f(n) \leq c.g(n)\ \forall\ n \geq n_0$

The general step wise procedure for Big-O runtime analysis is as follows:

1. Figure out what the input is and what n represents.
2. Express the maximum number of operations, the algorithm performs in terms of n.
3. Eliminate all excluding the highest order terms.
4. Remove all the constant factors.

Some of the useful properties of Big-O notation analysis are as follow:

- Constant Multiplication:

If $f(n) = c.g(n)$, then $O(f(n)) = O(g(n))$; where c is a nonzero constant.

- Polynomial Function:

If $f(n) = a_0 + a_1.n + a_2.n^2 + \text{----} + a_m.n^m$, then $O(f(n)) = O(n^m)$.

- Summation Function:

If $f(n) = f_1(n) + f_2(n) + \text{----} + f_m(n)$ and $f_i(n) \leq f_{i+1}(n)\ \forall\ i=1, 2, \text{----}, m$,
then $O(f(n)) = O(\max(f_1(n), f_2(n), \text{----}, f_m(n)))$.

- Logarithmic Function:

If $f(n) = \log_a n$ and $g(n)=\log_b n$, then $O(f(n))=O(g(n))$
; all log functions grow in the same manner in terms of Big-O.

Basically, this asymptotic notation is used to measure and compare the worst-case scenarios of algorithms theoretically. For any algorithm, the Big-O analysis should be straightforward as long as we correctly identify the operations that are dependent on n, the input size.

Runtime Analysis of Algorithms

In general cases, we mainly used to measure and compare the worst-case theoretical running time complexities of algorithms for the performance analysis.
The fastest possible running time for any algorithm is O(1), commonly referred to as *Constant Running Time*. In this case, the algorithm always takes the same amount of time to execute, regardless of the input size. This is the ideal runtime for an algorithm, but it's rarely achievable.
In actual cases, the performance (Runtime) of an algorithm depends on n, that is the size of the input or the number of operations is required for each input item.
The algorithms can be classified as follows from the best-to-worst performance (Running Time Complexity):

- A logarithmic algorithm - O(logn)

Runtime grows logarithmically in proportion to n.

- A linear algorithm - O(n)

Runtime grows directly in proportion to n.

- A superlinear algorithm - O(nlogn)

Runtime grows in proportion to n.

- A polynomial algorithm - $O(n^c)$

Runtime grows quicker than previous all based on n.

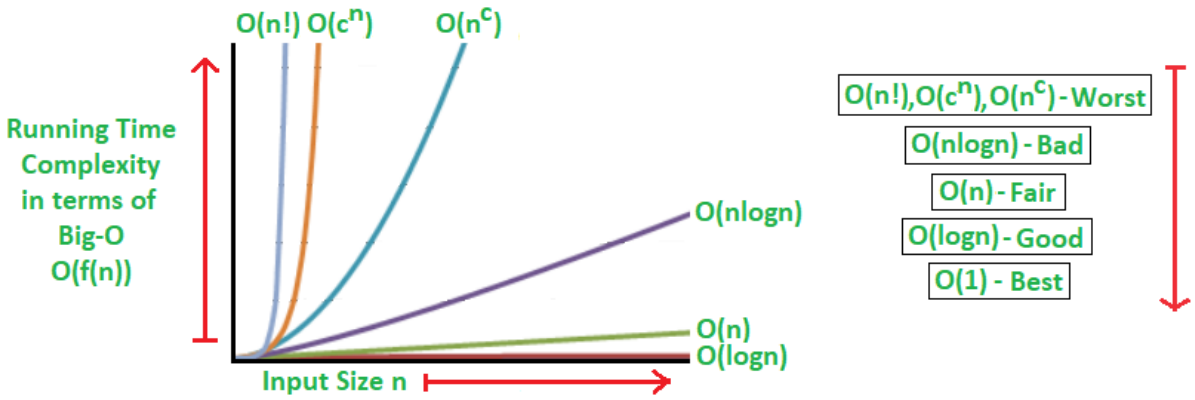
- A exponential algorithm - $O(c^n)$

Runtime grows even faster than polynomial algorithm based on n.

- A factorial algorithm - O(n!)

Runtime grows the fastest and becomes quickly unusable for even small values of n.

Where, n is the input size and c is a positive constant.



Algorithmic Examples of Runtime Analysis:
Some of the examples of all those types of algorithms (in worst-case scenarios) are mentioned below:

- Logarithmic algorithm - O(logn) - Binary Search.
- Linear algorithm - O(n) - Linear Search.
- Superlinear algorithm - O(nlogn) - Heap Sort, Merge Sort.
- Polynomial algorithm - O(n^c) - Strassen's Matrix Multiplication, Bubble Sort, Selection Sort, Insertion Sort, Bucket Sort.
- Exponential algorithm - O(c^n) - Tower of Hanoi.
- Factorial algorithm - O(n!) - Determinant Expansion by Minors, Brute force Search algorithm for Traveling Salesman Problem.

Mathematical Examples of Runtime Analysis:
The performances (Runtimes) of different orders of algorithms separate rapidly as n (the input size) gets larger. Let's consider the mathematical example:

```
If n = 10,
    log(10) = 1;
    10 = 10;
    10log(10)=10;
    10²=100;
```

```
If n=20,
    log(20) = 2.996;
    20 = 20;
    20log(20)=59.9;
    20²=400;
```

$2^{10}=1024;$ $2^{20}=1048576;$
$10!=3628800;$ $20!=2.432902e+18^{18};$

Memory Footprint Analysis of Algorithms

For performance analysis of an algorithm, runtime measurement is not only relevant metric but also we need to consider the memory usage amount of the program. This is referred to as the Memory Footprint of the algorithm, shortly known as Space Complexity. Here also, we need to measure and compare the worst case theoretical space complexities of algorithms for the performance analysis.
It basically depends on two major aspects described below:

- Firstly, the implementation of the program is responsible for memory usage. For example, we can assume that recursive implementation always reserves more memory than the corresponding iterative implementation of a particular problem.
- And the other one is n, the input size or the amount of storage required for each item. For example, a simple algorithm with a high amount of input size can consume more memory than a complex algorithm with less amount of input size.

Algorithmic Examples of Memory Footprint Analysis: The algorithms with examples are classified from the best-to-worst performance (Space Complexity) based on the worst-case scenarios are mentioned below:

- Ideal algorithm - O(1) - Linear Search, Binary Search, Bubble Sort, Selection Sort, Insertion Sort, Heap Sort, Shell Sort.
- Logarithmic algorithm - O(log n) - Merge Sort.
- Linear algorithm - O(n) - Quick Sort.
- Sub-linear algorithm - O(n+k) - Radix Sort.

Space-Time Tradeoff and Efficiency

There is usually a trade-off between optimal memory use and runtime performance.
In general for an algorithm, space efficiency and time efficiency reach at two opposite ends and each point in between them has a certain time and space efficiency. So, the more time efficiency you have, the less space efficiency you have and vice versa.
For example, Mergesort algorithm is exceedingly fast but requires a lot of space to do the operations. On the other side, Bubble Sort is exceedingly slow but requires the minimum space.
At the end of this topic, we can conclude that finding an algorithm that works in less

running time and also having less requirement of memory space, can make a huge difference in how well an algorithm performs.

Example of Big-oh noatation:

C++

```
// C++ program to findtime complexity for single for loop
#include <bits/stdc++.h>
using namespace std;
// main Code
int main()
{
    //declare variable
    int a = 0, b = 0;
    //declare size
    int N = 5, M = 5;
    // This loop runs for N time
    for (int i = 0; i < N; i++) {
        a = a + 5;
    }
    // This loop runs for M time
    for (int i = 0; i < M; i++) {
        b = b + 10;
    }
    //print value of a and b
    cout << a << ' ' << b;
    return 0;
}
```

Output

```
25 50
```

Explanation :

First Loop runs *N* Time whereas Second Loop runs *M* Time. The calculation takes *O(1)times*. So by adding them the time complexity will be O (N + M + 1) = O(N + M).

Time Complexity : O(N + M)

Space Complexity

Space Complexity:
The term Space Complexity is misused for Auxiliary Space at many places. Following are the correct definitions of Auxiliary Space and Space Complexity.

Auxiliary Space is the extra space or temporary space used by an algorithm.

Space Complexity of an algorithm is the total space taken by the algorithm with respect to the input size. Space complexity includes both Auxiliary space and space used by input.

For example, if we want to compare standard sorting algorithms on the basis of space, then Auxiliary Space would be a better criterion than Space Complexity. Merge Sort uses O(n) auxiliary space, Insertion sort, and Heap Sort use O(1) auxiliary space. The space complexity of all these sorting algorithms is O(n) though.

Space complexity is a parallel concept to time complexity. If we need to create an array of size n, this will require O(n) space. If we create a two-dimensional array of size n*n, this will require $O(n^2)$ space.

In recursive calls stack space also counts.

Example :

```
int add (int n){
    if (n <= 0){
        return 0;
    }
    return n + add (n-1);
}
```

```
Here each call add a level to the stack :

1.  add(4)
2.    -> add(3)
3.      -> add(2)
4.        -> add(1)
5.          -> add(0)

Each of these calls is added to call stack and takes up actual
memory.
So it takes O(n) space.
```

However, just because you have n calls total doesn't mean it takes O(n) space.

Look at the below function :

```
int addSequence (int n){
    int sum = 0;
    for (int i = 0; i < n; i++){
        sum += pairSum(i, i+1);
    }
    return sum;
}

int pairSum(int x, int y){
    return x + y;
}

There will be roughly O(n) calls to pairSum. However, those
calls do not exist simultaneously on the call stack,
so you only need O(1) space.
```

Note: It's necessary to mention that space complexity depends on a variety of things such as the programming language, the compiler, or even the machine running the algorithm.

Analysis of Common Techniques

Analysis of Common loops

In this article, an analysis of iterative programs with simple examples is discussed.

1) O(1): Time complexity of a function (or set of statements) is considered as O(1) if it doesn't contain loop, recursion, and call to any other non-constant time function.

```
// set of non-recursive and non-loop statements
```

For example, swap() function has O(1) time complexity.
A loop or recursion that runs a constant number of times is also considered as O(1). For example, the following loop is O(1).

```
// Here c is a constant
for (int i = 1; i <= c; i++) {
```

```
        // some O(1) expressions
    }
```

2) O(n): Time Complexity of a loop is considered as O(n) if the loop variables are incremented/decremented by a constant amount. For recursive call in recursive function, the time complexity is considered as O(n). For example following functions have O(n) time complexity.

```
    // Here c is a positive integer constant
    for (int i = 1; i <= n; i += c) {
        // some O(1) expressions
    }

    for (int i = n; i > 0; i -= c) {
        // some O(1) expressions
    }


//Recursive function
void recurse(n)
{
    if(n==0)
        return;
    else{
        // some O(1) expressions
    }
    recurse(n-1);
}
```

3) O(n^c): Time complexity of nested loops is equal to the number of times the innermost statement is executed. For example, the following sample loops have O(n^2) time complexity

```
    for (int i = 1; i <=n; i += c) {
        for (int j = 1; j <=n; j += c) {
           // some O(1) expressions
        }
    }

    for (int i = n; i > 0; i -= c) {
        for (int j = i+1; j <=n; j += c) {
           // some O(1) expressions
    }
```

For example, Selection sort and Insertion Sort have $O(n^2)$ time complexity.

4) O(Logn) Time Complexity of a loop is considered as O(Logn) if the loop variables are divided/multiplied by a constant amount.

```
for (int i = 1; i <=n; i *= c) {
    // some O(1) expressions
}
for (int i = n; i > 0; i /= c) {
    // some O(1) expressions
}
```

For example, Binary Search(refer iterative implementation) has O(Logn) time complexity. Let us see mathematically how it is O(Log n). The series that we get in the first loop is 1, c, c^2, c^3, ... c^k. If we put k equals to $Log_c n$, we get $c^{Log_c n}$ which is n.

5) O(LogLogn) Time Complexity of a loop is considered as O(LogLogn) if the loop variables are reduced/increased exponentially by a constant amount.

```
// Here c is a constant greater than 1
for (int i = 2; i <=n; i = pow(i, c)) {
    // some O(1) expressions
}
//Here fun is sqrt or cuberoot or any other constant root
for (int i = n; i > 1; i = fun(i)) {
    // some O(1) expressions
}
```

See this for mathematical details.

How to combine the time complexities of consecutive loops?
When there are consecutive loops, we calculate time complexity as a sum of time complexities of individual loops.

```
for (int i = 1; i <=m; i += c) {
     // some O(1) expressions
}
for (int i = 1; i <=n; i += c) {
     // some O(1) expressions
}
Time complexity of above code is O(m) + O(n) which is O(m+n)
If m == n, the time complexity becomes O(2n) which is O(n).
```

How to calculate time complexity when there are many if, else statements inside loops?

As discussed here, worst-case time complexity is the most useful among best, average and worst. Therefore we need to consider the worst case. We evaluate the situation when values in if-else conditions cause a maximum number of statements to be executed.
For example, consider the linear search function where we consider the case when an element is present at the end or not present at all.
When the code is too complex to consider all if-else cases, we can get an upper bound by ignoring if-else and other complex control statements.

How to calculate the time complexity of recursive functions?

The time complexity of a recursive function can be written as a mathematical recurrence relation. To calculate time complexity, we must know how to solve recurrences.

The following is a cheat sheet of the time complexities of various algorithms.

Algorithms Cheat Sheet

Algorithm	**Best Case**	**Average Case**	**Worst Case**
Selection Sort	O(n^2)	O(n^2)	O(n^2)
Bubble Sort	O(n)	O(n^2)	O(n^2)

Algorithm	Best Case	Average Case	Worst Case
Insertion Sort	O(n)	O(n^2)	O(n^2)
Tree Sort	O(nlogn)	O(nlogn)	O(n^2)
Radix Sort	O(dn)	O(dn)	O(dn)
Merge Sort	O(nlogn)	O(nlogn)	O(nlogn)
Heap Sort	O(nlogn)	O(nlogn)	O(nlogn)
Quick Sort	O(nlogn)	O(nlogn)	O(n^2)
Bucket Sort	O(n+k)	O(n+k)	O(n^2)
Counting Sort	O(n+k)	O(n+k)	O(n+k)

Libraries in 1 Go

The C++ Standard Template Library (STL)

The Standard Template Library (STL) is a set of C++ template classes to provide common programming data structures and functions such as lists, stacks, arrays, etc. It is a library of container classes, algorithms, and iterators. It is a generalized library and so, its components are parameterized.

The C++ Standard Template Library (STL) is a collection of algorithms, data structures, and other components that can be used to simplify the development of C++ programs. The STL provides a range of containers, such as vectors, lists, and maps, as well as algorithms for searching, sorting and manipulating data.

One of the key benefits of the STL is that it provides a way to write generic, reusable code that can be applied to different data types. This means that you can write an algorithm once, and then use it with different types of data without having to write separate code for each type.

The STL also provides a way to write efficient code. Many of the algorithms and data structures in the STL are implemented using optimized algorithms, which can result in faster execution times compared to custom code.

Some of the key components of the STL include:

1. Containers: The STL provides a range of containers, such as vector, list, map, set, and stack, which can be used to store and manipulate data.
2. Algorithms: The STL provides a range of algorithms, such as sort, find, and binary_search, which can be used to manipulate data stored in containers.
3. Iterators: Iterators are objects that provide a way to traverse the elements of a container. The STL provides a range of iterators, such as forward_iterator, bidirectional_iterator, and random_access_iterator, that can be used with different types of containers.
4. Function Objects: Function objects, also known as functors, are objects that can be used as function arguments to algorithms. They provide a way to pass a function to an algorithm, allowing you to customize its behavior.
5. Adapters: Adapters are components that modify the behavior of other components in the STL. For example, the reverse_iterator adapter can be used to reverse the order of elements in a container.

By using the STL, you can simplify your code, reduce the likelihood of errors, and improve the performance of your programs.

STL has 4 components:

- **Algorithms**
- **Containers**
- **Functors**
- **Iterators**

1. Algorithms

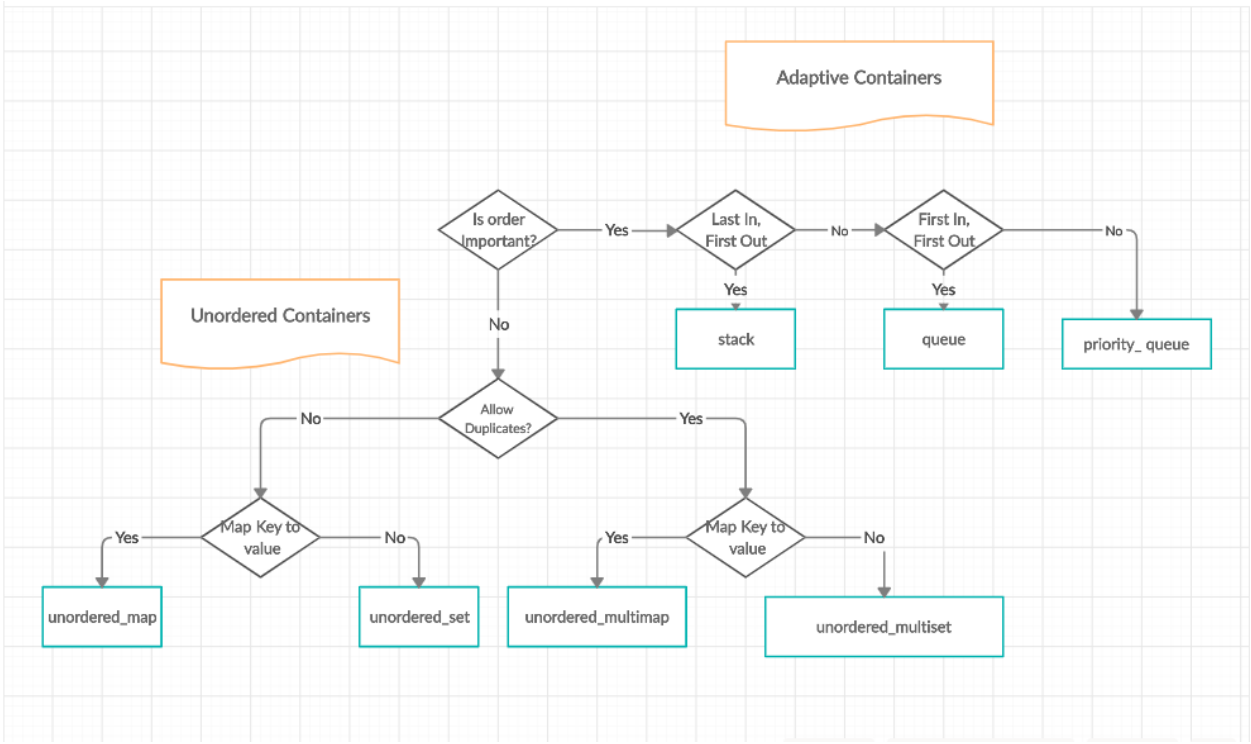
The header algorithm defines a collection of functions specially designed to be used on a range of elements. They act on containers and provide means for various operations for the contents of the containers.

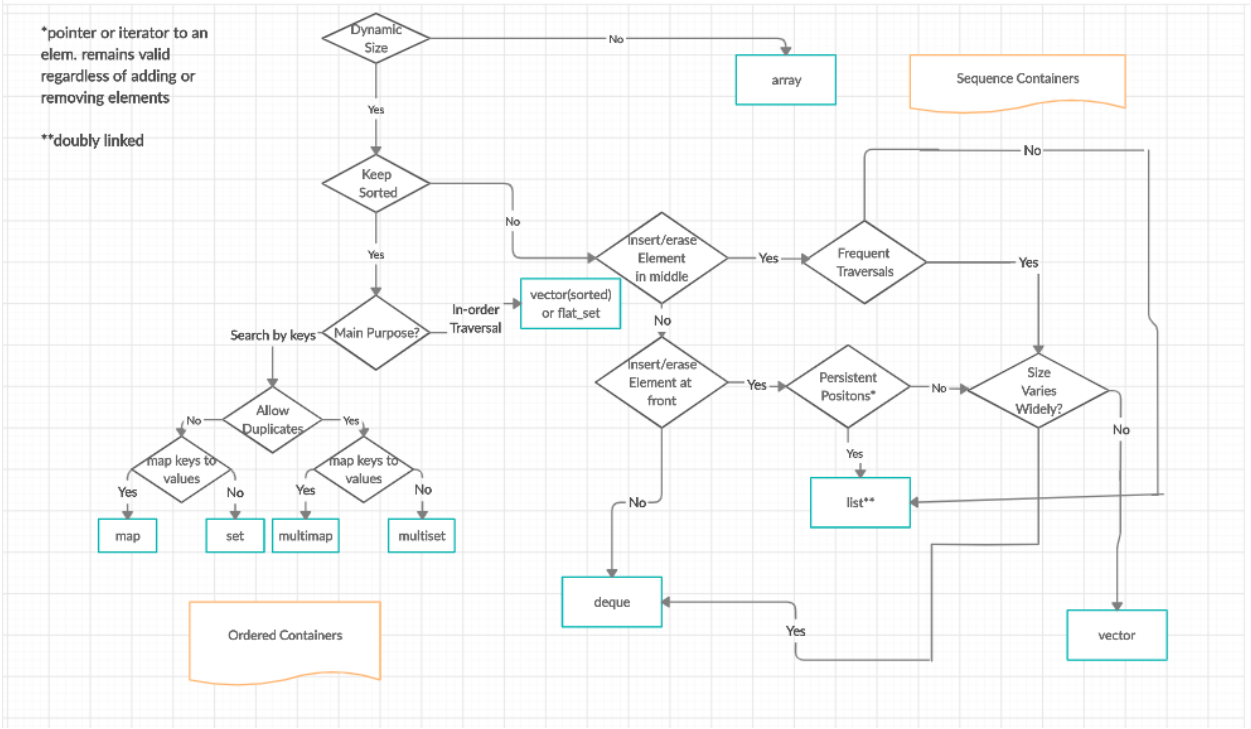
- Algorithm
  - Sorting
  - Searching
  - Important STL Algorithms
  - Useful Array algorithms
  - Partition Operations
- Numeric
  - valarray class

2. Containers

Containers or container classes store objects and data. There are in total seven standards "first-class" container classes and three container adaptor classes and only seven header files that provide access to these containers or container adaptors.

- Sequence Containers: implement data structures that can be accessed in a sequential manner.
  - vector
  - list
  - deque
  - arrays
  - forward_list(Introduced in C++11)
- Container Adaptors: provide a different interface for sequential containers.
  - queue
  - priority_queue
  - stack
- Associative Containers: implement sorted data structures that can be quickly searched (O(log n) complexity).
  - set
  - multiset
  - map
  - multimap
- Unordered Associative Containers: implement unordered data structures that can be quickly searched
  - unordered_set (Introduced in C++11)
  - unordered_multiset (Introduced in C++11)
  - unordered_map (Introduced in C++11)
  - unordered_multimap (Introduced in C++11)


Adaptive Containers
Is order Important?
Yes
Last In, First Out
No
First In, First Out
No
Yes
Yes
stack
queue
priority_ queue
Unordered Containers
No
Allow Duplicates?
No
Yes
Map Key to value
Map Key to value
Yes
No
Yes
No
unordered_map
unordered_set
unordered_multimap
unordered_multiset


*pointer or iterator to an elem. remains valid regardless of adding or removing elements
**doubly linked
Dynamic Size
No
array
Sequence Containers
Yes
Keep Sorted
No
Yes
Insert/erase Element in middle
Yes
Frequent Traversals
Yes
No
Search by keys
Main Purpose?
In-order Traversal
vector(sorted) or flat_set
No
Insert/erase Element at front
Yes
Persistent Positons*
No
Size Varies Widely?
No
Yes
Allow Duplicates
No
Yes
map keys to values
map keys to values
Yes
No
Yes
No
map
set
multimap
multiset
list**
No
deque
Yes
vector
Ordered Containers

3. Functors

The STL includes classes that overload the function call operator. Instances of such classes are called function objects or functors. Functors allow the working of the associated function to be customized with the help of parameters to be passed. **Must Read** - Functors

4. Iterators

As the name suggests, iterators are used for working on a sequence of values. They are the major feature that allows generality in STL. **Must Read** - Iterators

Utility Library

Defined in header <utility>. **Must Read** - Pair in C++ STL

For more details, refer to the Recent Articles on STL!

Advantages of the C++ Standard Template Library (STL):

1. Reusability: One of the key advantages of the STL is that it provides a way to write generic, reusable code that can be applied to different data types. This can lead to more efficient and maintainable code.
2. Efficient algorithms: Many of the algorithms and data structures in the STL are implemented using optimized algorithms, which can result in faster execution times compared to custom code.
3. Improved code readability: The STL provides a consistent and well-documented way of working with data, which can make your code easier to understand and maintain.
4. Large community of users: The STL is widely used, which means that there is a large community of developers who can provide support and resources, such as tutorials and forums.

Disadvantages of the C++ Standard Template Library (STL):

1. Learning curve: The STL can be difficult to learn, especially for beginners, due to its complex syntax and use of advanced features like iterators and function objects.
2. Lack of control: When using the STL, you have to rely on the implementation provided by the library, which can limit your control over certain aspects of your code.
3. Performance: In some cases, using the STL can result in slower execution times compared to custom code, especially when dealing with small amounts of data.

Arrays

Introduction to Arrays

What is an Array?

*An **array** is a collection of items of the same variable type stored that are stored at contiguous memory locations. It's one of the most popular and simple data structures and is often used to implement other data structures. Each item in an array is indexed starting with 0.*

The dream of every programmer is to become not just a good, but also a great programmer. We all want to achieve our goals and to achieve our goals, we must have a great plan with us. In this context, we have decided to provide a complete guide for Arrays interview preparation, which will help you to tackle the problems that are mostly asked in the interview, such as What is an Array, What is Array in C language, How do you initialize an Array in C, How to sort an Array, etc. We have also covered the topics such as Top **Theoretical interview questions** and **Top interview coding questions** in this complete guide for Array interview preparation.

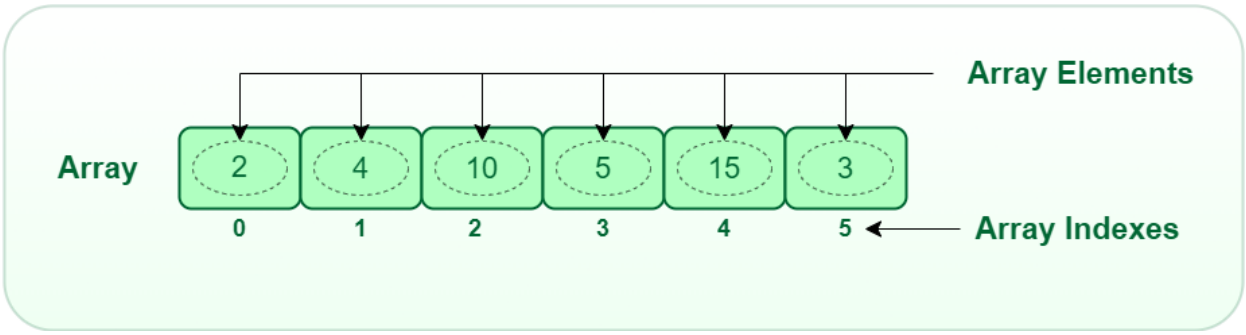
We can directly access an array element by using its index value.

Basic terminologies of array

- **Array Index:** In an array, elements are identified by their indexes. Array index starts from 0.
- **Array element:** Elements are items stored in an array and can be accessed by their index.
- **Array Length:** The length of an array is determined by the number of elements it can contain.

Representation of Array

The representation of an array can be defined by its declaration. A declaration means allocating memory for an array of a given size.



Array

Arrays can be declared in various ways in different languages. For better illustration, below are some language-specific array declarations.

Javascript

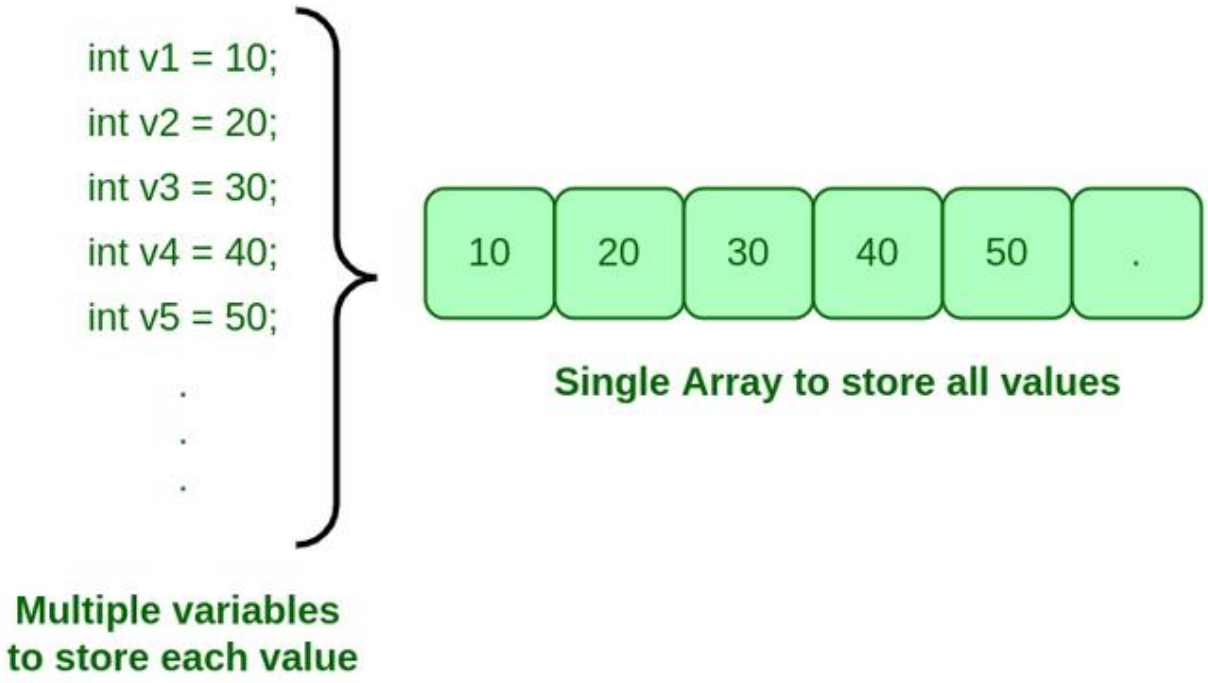
```
var arr = new Array(5); // This array will store integer type element
var arr = new Array(10); // This array will store char type element
var arr = new Array(20); // This array will store float type element
```

However, the above declaration is **static** or **compile-time** memory allocation, which means that the array element's memory is allocated when a program is compiled. Here only a fixed size (i,e. the size that is mentioned in square brackets **[]**) of memory will be allocated for storage, but don't you think it will not be the same situation as we know the size of the array every time, there might be a case where we don't know the size of the array. If we declare a larger size and store a lesser number of elements will result in a wastage of memory or either be a case where we declare a lesser size then we won't get enough memory to store the rest of the elements. In such cases, static memory allocation is not preferred.

Why Array Data Structures is needed?

Assume there is a class of five students and if we have to keep records of their marks in examination then, we can do this by declaring five variables individual and keeping track of records but what if the number of students becomes very large, it would be challenging to manipulate and maintain the data.

What it means is that, we can use normal variables (v1, v2, v3, ..) when we have a small number of objects. But if we want to store a large number of instances, it becomes difficult to manage them with normal variables. **The idea of an array is to represent many instances in one variable**..



Need for Array

Operations on Arrays

Types of Array operations:

- Traversal: Traverse through the elements of an array.
- Insertion: Inserting a new element in an array.

CC++Java

```c
#include <stdio.h>

int insert(int arr[], int n, int x, int cap, int pos)
{
        if(n == cap)
                return n;

        int idx = pos - 1;

        for(int i = n - 1; i >= idx; i--)
        {
                arr[i + 1] = arr[i];
```

```
        }

        arr[idx] = x;

        return n + 1;
}



int main() {

        int arr[5], cap = 5, n = 3;

        arr[0] = 5; arr[1] = 10; arr[2] = 20;

        printf("Before Insertion \n");

        for(int i=0; i < n; i++)
        {
            printf("%d ",arr[i]);
        }

        printf("\n");

        int x = 7, pos = 2;

        n = insert(arr, n, x, cap, pos);

        printf("After Insertion \n");

        for(int i=0; i < n; i++)
        {
                printf("%d ",arr[i]);
        }

}
```

- Deletion: Deleting element from the array.
- Searching: Search for an element in the array.

CC++Java

```
#include <stdio.h>

int search(int arr[], int n, int x)
{
        for(int i = 0; i < n; i++)
        {
                if(arr[i] == x)
                        return i;
        }
```

```
        return -1;
}



int main() {

        int arr[] = {20, 5, 7, 25}, x = 5;

    printf("%d ",search(arr, 4, x));
}
```

- Sorting: Maintaining the order of elements in the array.

Advantages of using Arrays:

- Arrays allow random access to elements. This makes accessing elements by position faster.
- Arrays have better cache locality which makes a pretty big difference in performance.
- Arrays represent multiple data items of the same type using a single name.
- Arrays store multiple data of similar types with the same name.
- Array data structures are used to implement the other data structures like linked lists, stacks, queues, trees, graphs, etc.

Disadvantages of Array:

- As arrays have a fixed size, once the memory is allocated to them, it cannot be increased or decreased, making it impossible to store extra data if required. An array of fixed size is referred to as a static array.
- Allocating less memory than required to an array leads to loss of data.
  An array is homogeneous in nature so, a single array cannot store values of different data types.
- Arrays store data in contiguous memory locations, which makes deletion and insertion very difficult to implement. This problem is overcome by implementing linked lists, which allow elements to be accessed randomly.

Application of Array:

- They are used in the implementation of other data structures such as array lists, heaps, hash tables, vectors, and matrices.
- Database records are usually implemented as arrays.
- It is used in lookup tables by computer.

- It is used for different sorting algorithms such as bubble sort insertion sort, merge sort, and quick sort.

Marked as Read
Report An Issue

Insertion and Deletion in Arrays

Insertion in Arrays

Given an array of a given size. The task is to insert a new element in this array. There are two possible ways of inserting elements in an array:

1. Insert elements at the end of the array.
2. Insert element at any given index in the array.

Special Case: A special case is needed to be considered is that whether the array is already full or not. If the array is full, then the new element can not be inserted.

Consider the given array is **arr[]** and the initial size of the array is N, that is the array can contain a maximum of N elements and the length of the array is **len**. That is, there are *len* number of elements already present in this array.

- **Insert an element K at end in arr[]**: The first step is to check if there is any space left in the array for new element. To do this check,

```
if(len < N)
     // space left
else
     // array is full
```

- If there is space left for the new element, insert it directly at the end at position **len + 1** and index **len**:

```
arr[len] = k;
```

- ***Time Complexity*** of this insert operation is constant, i.e. O(1) as we are directly inserting the element in a single operation.
- **Insert an element K at position, pos in arr[]**: The first step is to check if there is any space left in the array for new element. To do this check,

```
if(len < N)
     // space left
else
     // array is full
```

- Now, if there is space left, the element can be inserted. The index of the new element will be **idx = pos - 1**. Now, before inserting the element at the index idx, shift all elements from the index idx till end of the array to the right by 1 place. This can be done as:

```
for(i = len-1; i >= idx; i--)
{
    arr[i+1] = arr[i];
}
```

- After shifting the elements, place the element K at index idx.

```
arr[idx] = K;
```

- ***Time Complexity*** in worst case of this insertion operation can be linear i.e. O(N) as we might have to shift all of the elements by one place to the right.

Deletion in Arrays

To delete a given element from an array, we will have to first search the element in the array. If the element is present in the array then delete operation is performed for the element otherwise the user is notified that the array does not contains the given element.

Consider the given array is **arr[]** and the initial size of the array is N, that is the array can contain a maximum of N elements and the length of the array is **len**. That is, there are *len* number of elements already present in this array.

Deleting an element K from the array arr[]: Search the element K in the array arr[] to find the index at which it is present.

```
for(i = 0; i < N; i++)
{
    if(arr[i] == K)
        idx = i; return;
    else
        Element not Found;
}
```

Now, to delete the element present at index **idx**, left shift all of the elements present

after *idx* by one place and finally reduce the length of the array by 1.

```
for(i = idx+1; i < len; i++)
{
    arr[i-1] = arr[i];
}

len = len-1;
```

Time Complexity in worst case of this deletion operation can be linear i.e. O(N) as we might have to shift all of the elements by one place to the left.

Stock Buy and Sell Problem

The cost of a stock on each day is given in an array. Find the maximum profit that you can make by buying and selling on those days. If the given array of prices is sorted in decreasing order, then profit cannot be earned at all.

Examples:

Input: *arr[] = {100, 180, 260, 310, 40, 535, 695}*
Output: *865*
Explanation: *Buy the stock on day 0 and sell it on day 3 => 310 – 100 = 210*
Buy the stock on day 4 and sell it on day 6 => 695 – 40 = 655
Maximum Profit = 210 + 655 = 865

Input: *arr[] = {4, 2, 2, 2, 4}*
Output: *2*
Explanation: *Buy the stock on day 1 and sell it on day 4 => 4 – 2 = 2*
Maximum Profit = 2

A simple approach is to try buying the stocks and selling them every single day when profitable and keep updating the maximum profit so far.

Follow the steps below to solve the problem:

- Try to buy every stock from **start** to **end – 1**
- After that again call the maxProfit function to calculate answer

- curr_profit = price[j] – price[i] + maxProfit(start, i – 1) + maxProfit(j + 1, end)
- profit = max(profit, curr_profit)

Below is the implementation of the above approach:

C++Java

```cpp
// C++ implementation of the approach
#include <bits/stdc++.h>
using namespace std;

// Function to return the maximum profit
// that can be made after buying and
// selling the given stocks
int maxProfit(int price[], int start, int end)
{

        // If the stocks can't be bought
        if (end <= start)
                return 0;

        // Initialise the profit
        int profit = 0;

        // The day at which the stock
        // must be bought
        for (int i = start; i < end; i++) {

                // The day at which the
                // stock must be sold
                for (int j = i + 1; j <= end; j++) {

                        // If buying the stock at ith day and
                        // selling it at jth day is profitable
                        if (price[j] > price[i]) {

                                // Update the current profit
                                int curr_profit
                                        = price[j] - price[i]
                                        + maxProfit(price, start, i - 1)
                                        + maxProfit(price, j + 1, end);

                                // Update the maximum profit so far
                                profit = max(profit, curr_profit);
                        }
                }
        }
        return profit;
}
```

```
// Driver code
int main()
{
        int price[] = { 100, 180, 260, 310, 40, 535, 695 };
        int n = sizeof(price) / sizeof(price[0]);

        cout << maxProfit(price, 0, n - 1);

        return 0;
}
```

Output

```
865
```

Time Complexity: O(N^2), Trying to buy every stock and exploring all possibilities.
Auxiliary Space: O(1)

Stock Buy Sell to Maximize Profit using Valley Peak Approach:

In this approach, we just need to find the next greater element and subtract it from the current element so that the difference keeps increasing until we reach a minimum. If the sequence is a decreasing sequence, so the maximum profit possible is 0.

Follow the steps below to solve the problem:

- maxProfit = 0
- if price[i] > price[i – 1]
    - maxProfit = maxProfit + price[i] – price[i – 1]

Below is the implementation of the above approach:

C++Java

```
#include <iostream>
#include <cmath>
using namespace std;


int maxProfit(int price[], int n)
{
        int profit = 0;

        for(int i = 1; i < n; i++)
        {
```

```
                if(price[i] > price[i - 1])
                        profit += price[i] - price[i -1];
        }

        return profit;

}


int main() {

      int arr[] = {1, 5, 3, 8, 12}, n = 5;

      cout<<maxProfit(arr, n);

}
```

Output

```
13
```

Time Complexity: O(N), Traversing over the array of size N.
Auxiliary Space: O(1)

Move Zeros to End

Given an array of random numbers, Push all the zero's of a given array to the end of the array. For example, if the given arrays is {1, 9, 8, 4, 0, 0, 2, 7, 0, 6, 0}, it should be changed to {1, 9, 8, 4, 2, 7, 6, 0, 0, 0, 0}. The order of all other elements should be same. Expected time complexity is O(n) and extra space is O(1).

Example:

```
 Input :  arr[] = {1, 2, 0, 4, 3, 0, 5, 0};
Output : arr[] = {1, 2, 4, 3, 5, 0, 0, 0};

Input : arr[]  = {1, 2, 0, 0, 0, 3, 6};
Output : arr[] = {1, 2, 3, 6, 0, 0, 0};
```

There can be many ways to solve this problem. Following is a simple and interesting way to solve this problem.

Traverse the given array 'arr' from left to right. While traversing, maintain count of non-zero elements in array. Let the count be 'count'. For every non-zero element arr[i], put the element at 'arr[count]' and increment 'count'. After complete traversal, all non-zero elements have already been shifted to front end and 'count' is set as index of first 0. Now all we need to do is that run a loop which makes all elements zero from 'count' till end of the array.
Below is the implementation of the above approach.

CC++JavaPython3C#PHPJavascript

```c
// A C program to move all zeroes at the end of array
#include <stdio.h>

// Function which pushes all zeros to end of an array.
void pushZerosToEnd(int arr[], int n)
{
    int count = {0};  // Count of non-zero elements

    // Traverse the array. If element encountered is non-
    // zero, then replace the element at index 'count'
    // with this element
    for (int i = 0; i < n; i++)
        if (arr[i] != 0)
            arr[count++] = arr[i]; // here count is
                                   // incremented

    // Now all non-zero elements have been shifted to
    // front and  'count' is set as index of first 0.
    // Make all elements 0 from count to end.
    while (count < n)
        arr[count++] = 0;
}

// Driver program to test above function
int main()
{
    int arr[] = {1, 9, 8, 4, 0, 0, 2, 7, 0, 6, 0, 9};
    int n = sizeof(arr) / sizeof(arr[0]);
    pushZerosToEnd(arr, n);
    printf("%s\n", "Array after pushing all zeros to end of array:");
    for (int i = 0; i < n; i++)
      printf("%d ", arr[i]);
    return 0;
}
```

Output

```
Array after pushing all zeros to end of array :
1 9 8 4 2 7 6 9 0 0 0 0
```

Time Complexity: O(n) where n is number of elements in input array.
Auxiliary Space: O(1)

Sliding Window Technique

This technique shows how a nested for loop in few problems can be converted to single for loop and hence reducing the time complexity.

Let's start with a problem for illustration where we can apply this technique:

```
Given an array of integers of size 'n'.
Our aim is to calculate the maximum sum of 'k'
consecutive elements in the array.

Input  : arr[] = {100, 200, 300, 400}
         k = 2
Output : 700

Input  : arr[] = {1, 4, 2, 10, 23, 3, 1, 0, 20}
         k = 4
Output : 39
We get maximum sum by adding subarray {4, 2, 10, 23}
of size 4.

Input  : arr[] = {2, 3}
         k = 3
Output : Invalid
There is no subarray of size 3 as size of whole
array is 2.
```

The **Naive Approach** to solve this problem is to calculate sum for each of the blocks of K consecutive elements and compare which block has the maximum sum possible. The time complexity of this approach will be O(n * k).

Window Sliding Technique

The above problem can be solved in Linear Time Complexity by using Window Sliding Technique by avoiding the overhead of calculating sum repeatedly for each block of k elements.

The technique can be best understood with the window pane in bus, consider a window of length **n** and the pane which is fixed in it of length **k**. Consider, initially the pane is at extreme left i.e., at 0 units from the left. Now, co-relate the window with array arr[] of size n and plane with current_sum of size k elements. Now, if we apply force on the window such that it moves a unit distance ahead. The pane will cover next **k** consecutive elements.

Consider an array **arr[]** = {5 , 2 , -1 , 0 , 3} and value of **k** = 3 and **n** = 5

Applying sliding window technique :

1. We compute the sum of first k elements out of n terms using a linear loop and store the sum in variable window_sum.
2. Then we will graze linearly over the array till it reaches the end and simultaneously keep track of maximum sum.
3. To get the current sum of block of k elements just subtract the first element from the previous block and add the last element of the current block .

C++ code for sliding window Algorithm: -

```
#include <bits/stdc++.h>
using namespace std;

// Returns maximum sum in a subarray of size k.
int maxSum(int arr[], int n, int k) {
    // n must be greater
    if (n < k) {
        cout << "Invalid";
        return -1;
    }

    //sum of first window of size k
    int window_sum = 0;
    for (int i = 0; i < k; i++)
        window_sum += arr[i];

    // Compute sums of remaining windows by
```

```
    // removing first element of previous
    // window and adding last element of
    // current window.
    int max_sum = window_sum;
    for (int i = k; i < n; i++) {
        window_sum += (arr[i] - arr[i - k]);
        max_sum = max(max_sum, window_sum);
    }
    return max_sum;
}

int main(){
    int n = 6 , k = 3;;
    int arr[] = { 16,12,9,19,11,8};
    cout << maxSum(arr, n, k);
    return 0;
}
```

Prefix Sum Technique

Given an array arr[] of size n, its prefix sum array is another array prefixSum[] of the same size, such that the value of prefixSum[i] is arr[0] + arr[1] + arr[2] … arr[i].

Examples :
Input *: arr[] = {10, 20, 10, 5, 15}*
Output : *prefixSum[] = {10, 30, 40, 45, 60}*
Explanation : *While traversing the array, update the element by adding it with its previous element.*
prefixSum[0] = 10,
prefixSum[1] = prefixSum[0] + arr[1] = 30,
prefixSum[2] = prefixSum[1] + arr[2] = 40 and so on.
To fill the prefix sum array, we run through index 1 to last and keep on adding the present element with the previous value in the prefix sum array.
Below is the implementation :

Implementation:
Python

```
def getSum(arr, l, r):
    res=0
    for i in range(l, r+1):
```

```
        res+=arr[i]
    return res


arr=[2, 8, 3, 9, 6, 5, 4]
print(getSum(arr, 0, 2))
print(getSum(arr, 1, 3))
print(getSum(arr, 2, 6))
```

Output:

```
13
20
27
```

Time Complexity: O(n)

Auxiliary Space: O(n)

Efficient Approach

Given an array arr[] of size n. Given Q queries and in each query given L and R, print sum of array elements from index L to R.

Approach: To solve the problem follow the given steps:

- Declare a new array **prefixSum[]** of the same size as the input array
- Run a for loop to traverse the input array
- For each index add the value of the current element and the previous value of the prefix sum array
- Now calculate required queries .

Example

Python

```
def getSum(l, r):
    if(l==0):
        return pSum[r]
    else:
        return pSum[r]-pSum[l-1]

arr=[2, 8, 3, 9, 6, 5, 4]
n=len(arr)
pSum=[None]*n
pSum[0]=arr[0]
for i in range(1, n):
    pSum[i]=pSum[i-1]+arr[i]

print(getSum(2, 6))
```

Output:

```
27
```

Time Complexity: O(n)
Auxiliary Space: O(n)

Explaination

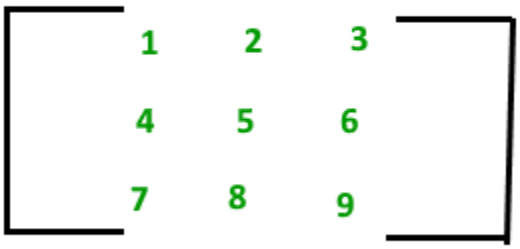
- **Function Definition:**
  - `getSum(l, r)` calculates the sum of elements in the array `arr` from index `l` to `r` using a precomputed prefix sum array `pSum`.
- **Prefix Sum Array Initialization:**
  - `arr = [2, 8, 3, 9, 6, 5, 4]`: Defines the input array.
  - `n = len(arr)`: Gets the length of `arr`.
  - `pSum = [None] * n`: Initializes a list `pSum` of the same length as `arr`.
  - `pSum[0] = arr[0]`: Sets the first element of `pSum` to the first element of `arr`.
- **Compute Prefix Sum Array:**
  - `for i in range(1, n): pSum[i] = pSum[i-1] + arr[i]`: Computes the cumulative sum for each element in `arr` and stores it in `pSum`.
- **Get Sum Function:**
  - `if l == 0: return pSum[r]`: If `l` is `0`, return the prefix sum at index `r`.
  - `else: return pSum[r] - pSum[l-1]`: Otherwise, return the difference between the prefix sums at indices `r` and `l-1`.
  - Now calculate required queries

Matrix

Introduction to Matrix

A **matrix** represents a collection of numbers arranged in order of rows and columns. It is necessary to enclose the elements of a matrix in parentheses or brackets.

A matrix with 9 elements is shown below:



The above Matrix M has 3 rows and 3 columns. Each element of matrix [M] can be referred

to by its row and column number. For example, $M_{23} = 6$

Order of a Matrix : The order of a matrix is defined in terms of its number of rows and columns.

```
Order of a matrix = No. of rows × No. of columns

Therefore, Matrix [M] is a matrix of order 3 × 3.
```

Transpose of a Matrix

The transpose $[M]^T$ of an **m x n** matrix [M] is the n x m matrix obtained by interchanging the rows and columns of [M].

Transpose of a matrix A is defined as:

if $A = [a_{ij}]$ mxn:
then $A^T = [b_{ij}]$ nxm where $b_{ij} = a_{ji}$

For Example, transpose of matrix M, M^T will be:

```
M =1 2 3
   4 5 6
   7 8 9
```

$\mathbf{M^T}$ = 1 4 7
2 5 8
3 6 9

Properties of transpose of a matrix:

- $(A^T)^T = A$
- $(A+B)^T = A^T + B^T$
- $(AB)^T = B^T A^T$

Properties of Matrix addition and multiplication:

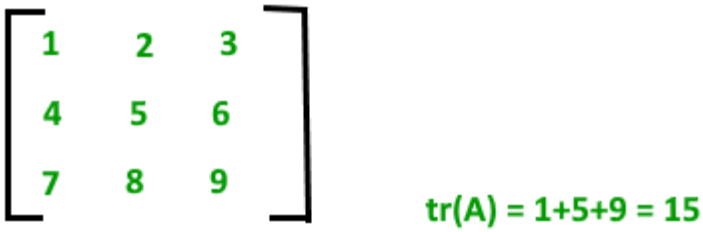
1. A+B = B+A (Commutative)
2. (A+B)+C = A+ (B+C) (Associative)
3. AB ≠ BA (Not Commutative)
4. (AB) C = A (BC) (Associative)
5. A (B+C) = AB+AC (Distributive)

Terminologies

- **Square Matrix:** A square Matrix has as many rows as it has columns. i.e. no of rows = no of columns.
- **Symmetric matrix:** A square matrix is said to be symmetric if the transpose of original matrix is equal to its original matrix. i.e. $(A^T) = A$.
- **Skew-symmetric:** A skew-symmetric (or antisymmetric or antimetric[1]) matrix is a square matrix whose transpose equals its negative.i.e. $(A^T) = -A$.
- **Diagonal Matrix:**A diagonal matrix is a matrix in which the entries outside the main diagonal are all zero. The term usually refers to square matrices.
- **Identity Matrix:**A square matrix in which all the elements of the principal diagonal are ones and all other elements are zeros.Identity matrix is denoted as I.
- **Orthogonal Matrix:** A matrix is said to be orthogonal if $AA^T = A^T A = I$.
- **Idemponent Matrix:** A matrix is said to be idemponent if $A^2 = A$.
- **Involutary Matrix:** A matrix is said to be Involutary if $A^2 = I$.
- **Singular Matrix**: A square matrix is said to be singular matrix if its determinant is zero i.e. $|A|=0$
- **Nonsingular Matrix**: A square matrix is said to be non-singular matrix if its determinant is non-zero.

Note: Every Square Matrix can uniquely be expressed as the sum of a symmetrix matrix and skew symmetric matrix. $A = 1/2\ (A^T + A) + 1/2\ (A - A^T)$.

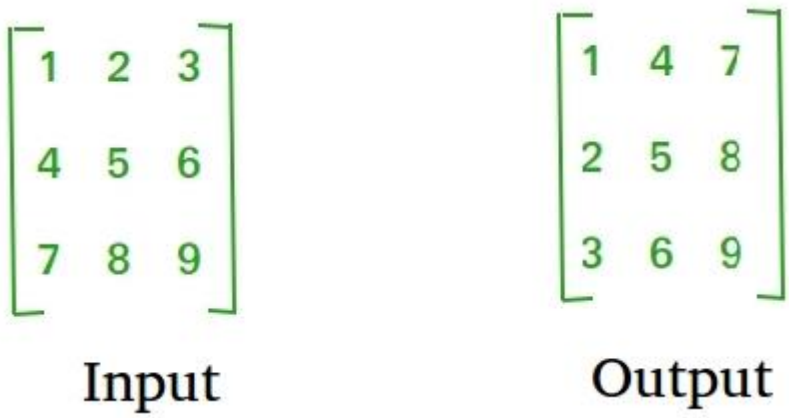
Trace of a matrix: trace of a matrix is denoted as tr(A) which is used only for square matrix and equals the sum of the diagonal elements of the matrix. For example:



Transpose of a Matrix

Transpose of a matrix is obtained by changing rows to columns and columns to rows. In other words, transpose of A[N][M] is obtained by changing A[i][j] to A[j][i].

Example:



Approach: Follow the given steps to solve the problem:

- Run a nested loop using two integer pointers i and j for 0 <= i < N and 0 <= j < M
- Set B[i][j] equal to A[j][i]

Below is the implementation of the above approach:

Python3

```
# Python3 Program to find
# transpose of a matrix

N = 4

# This function stores
# transpose of A[][] in B[][]


def transpose(A, B):

        for i in range(N):
                for j in range(N):
                        B[i][j] = A[j][i]


# Driver code
if __name__ == '__main__':
        A = [[1, 1, 1, 1],
                [2, 2, 2, 2],
                [3, 3, 3, 3],
                [4, 4, 4, 4]]


# To store result
B = [[0 for x in range(N)] for y in range(N)]

# Function call
transpose(A, B)

print("Result matrix is")
for i in range(N):
        for j in range(N):
                print(B[i][j], " ", end='')
        print()
```

Output

```
Result matrix is
 1  2  3  4
 1  2  3  4
 1  2  3  4
 1  2  3  4
```

Time complexity: O(M x N).

Auxiliary Space: O(M x N), since M x N extra space has been used.

Program to find the transpose of a matrix using constant space:

Follow the given steps to solve the problem:

- Run a nested loop using two integer pointers i and j for 0 <= i < N and 0 <= j < M
- Swap A[i][j] with A[j][i]

Below is the implementation of the above approach:

Python3

```
# Python3 Program to find
# transpose of a matrix

N = 4

# Finds transpose of A[][] in-place


def transpose(A):

        for i in range(N):
                for j in range(i+1, N):
                        A[i][j], A[j][i] = A[j][i], A[i][j]


# driver code
A = [[1, 1, 1, 1],
        [2, 2, 2, 2],
        [3, 3, 3, 3],
        [4, 4, 4, 4]]

transpose(A)

print("Modified matrix is")
for i in range(N):
        for j in range(N):
                print(A[i][j], " ", end='')
        print()

# This code is contributed
# by Anant Agarwal.
```

Output

```
Modified matrix is
1 2 3 4
1 2 3 4
```

```
1 2 3 4
1 2 3 4
```

Time complexity: O(N^2).

Auxiliary Space: O(1)

Mark as Read

Report An Issue

Recursion

Recursion Basics

What is Recursion?

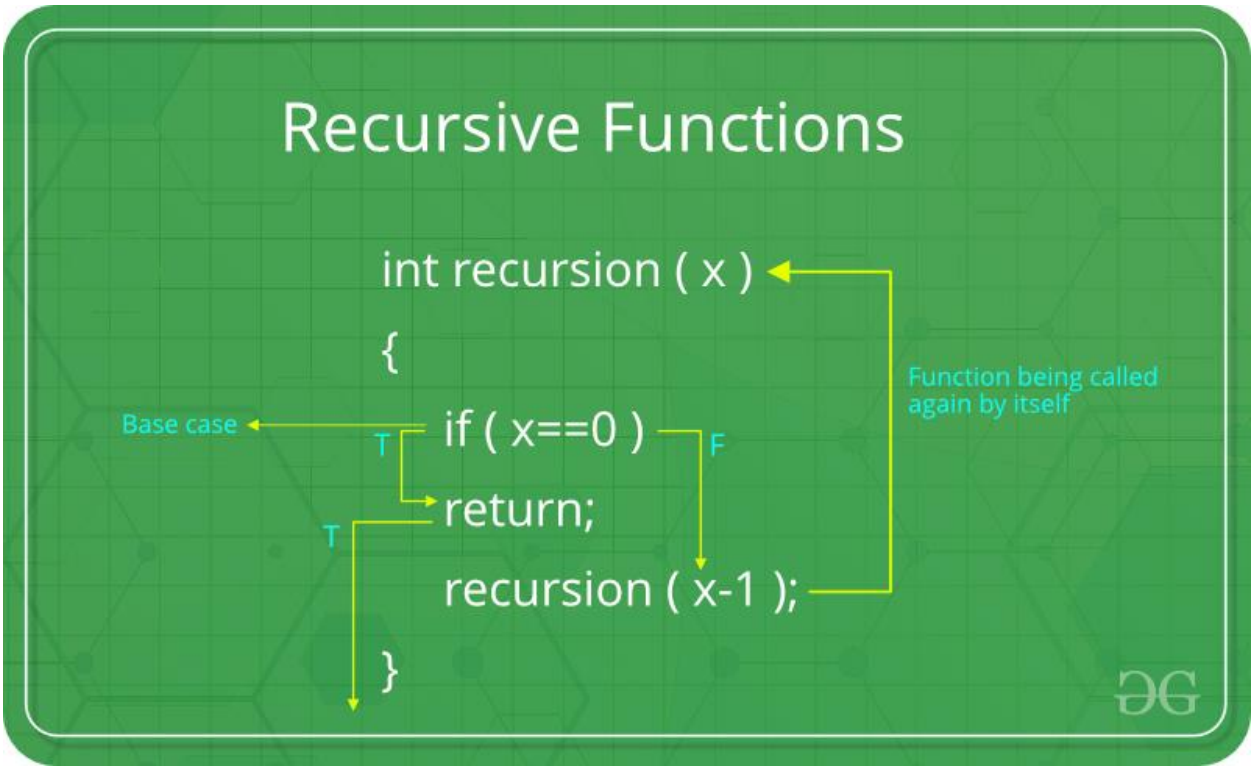
The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called as recursive function. Using recursive algorithm, certain problems can be solved quite easily. Examples of such problems are Towers of Hanoi (TOH), Inorder/Preorder/Postorder Tree Traversals, DFS of Graph, etc.

What is the base condition in recursion?

In a recursive program, the solution to the base case is provided and the solution of bigger problem is expressed in terms of smaller problems.

```
function fact(n) {
    if (n <= 1) { // base case
        return 1;
    } else {
        return n * fact(n - 1);
    }
}
```

In the above example, the base case for n < = 1 is defined and a larger value of a number can be solved by converting to a smaller one till the base case is reached.



How a particular problem is solved using recursion?

The idea is to represent a problem in terms of one or more smaller problems, and add one or more base conditions that stop recursion. For example, we compute factorial n if we know factorial of (n-1). The base case for factorial would be n = 0. We return 1 when n = 0.

Why Stack Overflow error occurs in recursion?
If the base case is not reached or not defined, then the stack overflow problem may arise. Let us take an example to understand this.

```
function fact(n)
{
    // wrong base case (it may cause
    // stack overflow).
    if (n == 100)
        return 1;
```

```
    else
        return n*fact(n-1);
}
```

If fact(10) is called, it will call fact(9), fact(8), fact(7), and so on but the number will never reach 100. So, the base case is not reached. If the memory is exhausted by these functions on the stack, it will cause a stack overflow error.

How memory is allocated to different function calls in recursion?

When any function is called from main(), the memory is allocated to it on stack. A recursive function calls itself, the memory for the called function is allocated on top of memory allocated to the calling function and a different copy of local variables is created for each function call. When the base case is reached, the function returns its value to the function by whom it is called and memory is de-allocated and the process continues. Let us take the example of how recursion works by taking a simple function:

```
function printFun(test)

{

    if (test < 1)

        return;

    else

    {

        print test;

        printFun(test-1);    // statement 2

        print test;

        return;

    }

}


// Calling function printFun()
```

```
var test = 3;

printFun(test);
```

Output :

```
3 2 1 1 2 3
```

When **printFun(3)** is called from main(), memory is allocated to **printFun(3)** and a local variable test is initialized to 3 and statement 1 to 4 are pushed on the stack as shown in below diagram. It first prints '3'. In statement 2, **printFun(2)** is called and memory is allocated to **printFun(2)** and a local variable test is initialized to 2 and statements 1 to 4 are pushed in the stack.
Similarly, **printFun(2)** calls **printFun(1)** and **printFun(1)** calls **printFun(0)**. **printFun(0)** goes to if statement and it returns to **printFun(1)**. Remaining statements of **printFun(1)** are executed and it returns to **printFun(2)** and so on. In the output, values from 3 to 1 are printed and then 1 to 3 are printed.

Disadvantage of Recursion: Note that both recursive and iterative programs have the same problem-solving powers, i.e., every recursive program can be written iteratively and vice versa. Recursive program has greater space requirements than iterative program as all functions will remain in the stack until the base case is reached. A recursive program also has greater time requirements because of function calls and return overhead. **Advantages of Recursion**: Recursion provides a clean and simple way to write code. Some problems are inherently recursive like tree traversals, Tower of Hanoi, etc. For such problems, it is preferred to write recursive code. We can write such codes also iteratively with the help of the stack data structure.

Tail Recursion

As we read before, that recursion is defined when a function invokes/calls itself.

Tail Recursion: A recursive function is said to be following Tail Recursion if it invokes itself at the end of the function. That is, if all of the statements are executed before the function invokes itself then it is said to be following Tail Recursion.

For Example:

CPPJava

```
// This is a Tail Recursion

void printN(int N)
{
    if(N==0)
        return;
    else
        cout<<N<<" ";

    printN(N-1);
}
```

The above function call for **N = 5** will print:

```
5 4 3 2 1
```

Which one is Better-Tail Recursive or Non Tail-Recursive?

The tail-recursive functions are considered better than non-tail recursive functions as tail-recursion can be optimized by the compiler. The idea used by compilers to optimize tail-recursive functions is simple, since the recursive call is the last statement, there is nothing left to do in the current function, so saving the current function's stack frame is of no use.

Can a non-tail recursive function be written as tail-recursive to optimize it?

Consider the following function to calculate factorial of N. Although it looks like Tail Recursive at first look, it is a non-tail-recursive function. If we take a closer look, we can see that the value returned by **fact(N-1)** is used in **fact(N)**, so the call to fact(N-1) is not the last thing done by fact(N).

```
int fact(int N)
{
    if (N == 0)
```

```
        return 1;

    return N*fact(N-1);
}
```

The above function can be written as a Tail Recursive function. The idea is to use one more argument and accumulate the factorial value in the second argument. When N reaches 0, return the accumulated value.

```
int factTR(int N, int a)
{
    if (N == 0)
        return a;

    return factTR(N-1, N*a);
}
```

Mark as Read
Report An Issue

Rope Cutting Problem

Given a rope of length **N** meters, and the rope can be cut in only 3 sizes **A**, **B** and **C**. The task is to maximizes the number of cuts in rope. If it is impossible to make cut then print the number else print **-1**. **Examples:**

Input:

N = 17, A = 10, B = 11, C = 3

Output: 3

Explanation: The maximum cut can be obtain after making 2 cut of length 3 and one cut of length 11.

Input: N = 10, A = 9, B = 7, C = 11

Output: -1

Explanation: It is impossible to make any cut so output will be -1.

Naive Approach: Using Recursion

C++Java

```
#include <iostream>
using namespace std;


int maxCuts(int n, int a, int b, int c)
{
        if(n == 0)
                return 0;
        if(n <= -1)
                return -1;

        int res = max(maxCuts(n-a, a, b, c),
                  max(maxCuts(n-b, a, b, c),
                  maxCuts(n-c, a, b, c)));

        if(res == -1)
                return -1;

        return res + 1;
}
int main() {

        int n = 5, a = 2, b = 1, c = 5;

        cout<<maxCuts(n, a, b, c);

        return 0;
}
```

Output:

5

Time Complexity : O(3^n)

Space Complexity : O(n), due to recursive call stack.

Hashing

Introduction to Hashing

Hashing is a method of storing and retrieving data from a database efficiently, Suppose that we want to design a system for storing employee records keyed using phone numbers. And we want the following queries to be performed efficiently:

1. Insert a phone number and the corresponding information.
2. Search a phone number and fetch the information.
3. Delete a phone number and the related information.

We can think of using the following data structures to maintain information about different phone numbers.

1. An array of phone numbers and records.
2. A linked list of phone numbers and records.
3. A balanced binary search tree with phone numbers as keys.
4. A direct access table.

For **arrays and linked lists**, we need to search in a linear fashion, which can be costly in practice. If we use arrays and keep the data sorted, then a phone number can be searched in O(Logn) time using Binary Search, but insert and delete operations become costly as we have to maintain sorted order. With a **balanced binary search tree**, we get a moderate search, insert and delete time. All of these operations can be guaranteed to be in O(Logn) time. Another solution that one can think of is to use a **direct access table** where we make a big array and use phone numbers as indexes in the array. An entry in the array is NIL if the phone number is not present, else the array entry stores pointer to records corresponding to the phone number. Time complexity wise this solution is the best of all, we can do all operations in O(1) time. For example, to insert a phone number, we create a record with details of the given phone number, use the phone number as an index and store the pointer to the record created in the table. This solution has many practical limitations. The first problem with this solution is that the extra space required is huge. For example, if the phone number is of n digits, we need $O(m * 10^n)$ space for the table where m is the size of a pointer to the record. Another problem is an integer in a programming language may not store n digits. Due to the above limitations, the Direct Access Table cannot always be used. **Hashing** is the solution that can be used in almost all such situations and performs extremely well as compared to above data structures like Array, Linked List, Balanced BST in practice. With hashing, we get O(1) search time on average (under reasonable assumptions) and O(n) in the worst case.

Hashing is an improvement over Direct Access Table. The idea is to use a hash function that converts a given phone number or any other key to a smaller number and uses the small number as an index in a table called a hash table.

Hash Function: A function that converts a given big phone number to a small practical integer value. The mapped integer value is used as an index in the hash table. In simple terms, a hash function maps a big number or string to a small integer that can be used as an index in the hash table. A good hash function should have following properties:

1. It should be efficiently computable.
2. It should uniformly distribute the keys (Each table position be equally likely for each key).

For example, for phone numbers, a bad hash function is to take the first three digits. A better function will consider the last three digits. Please note that this may not be the best hash function. There may be better ways.

Hash Table: An array that stores pointers to records corresponding to a given phone number. An entry in hash table is NIL if no existing phone number has hash function value equal to the index for the entry.

Collision Handling: Since a hash function gets us a small number for a big key, there is a possibility that two keys result in the same value. The situation where a newly inserted key maps to an already occupied slot in the hash table is called collision and must be handled using some collision handling technique. Following are the ways to handle collisions:

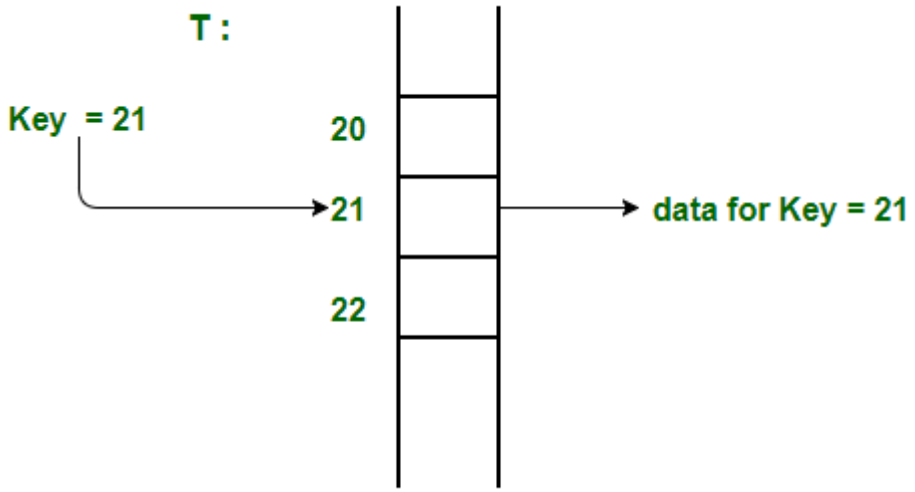
- **Chaining:**The idea is to make each cell of the hash table point to a linked list of records that have the same hash function value. Chaining is simple, but it requires additional memory outside the table.
- **Open Addressing:** In open addressing, all elements are stored in the hash table itself. Each table entry contains either a record or NIL. When searching for an element, we one by one examine the table slots until the desired element is found or it is clear that the element is not present in the table.

Direct Address Table

Direct Address Table

Direct Address Table is a data structure that has the capability of mapping records to their corresponding keys using arrays. In direct address tables, records are placed using their key values directly as indexes. They facilitate fast searching, insertion and deletion operations.

We can understand the concept using the following example. We create an array of size equal to maximum value plus one (assuming 0 based index) and then use values as indexes. For example, in the following diagram key 21 is used directly as index.



Advantages:

1. **Searching in O(1) Time:** Direct address tables use arrays which are random access data structure, so, the key values (which are also the index of the array) can be easily used to search the records in O(1) time.
2. **Insertion in O(1) Time:** We can easily insert an element in an array in O(1) time. The same thing follows in a direct address table also.
3. **Deletion in O(1) Time:** Deletion of an element takes O(1) time in an array. Similarly, to delete an element in a direct address table we need O(1) time.

Limitations:

1. Prior knowledge of maximum key value
2. Practically useful only if the maximum value is very less.
3. It causes wastage of memory space if there is a significant difference between total records and maximum value.

Hashing can overcome these limitations of direct address tables.

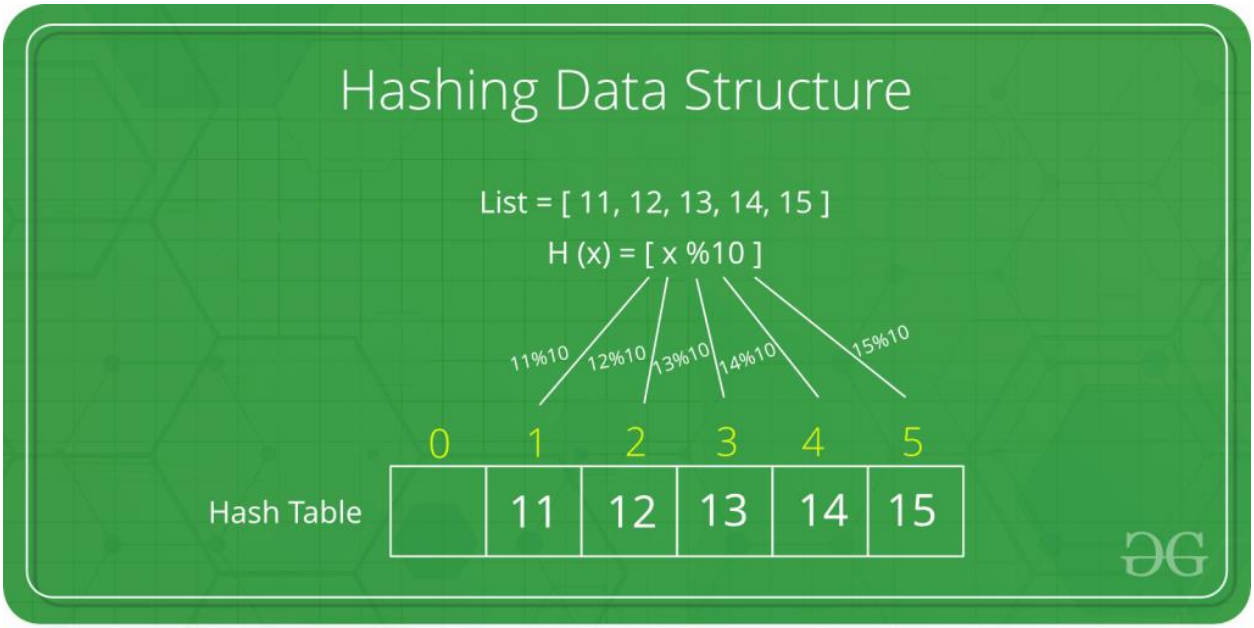
How to handle collisions?

Collisions can be handled like Hashing. We can either use Chaining or open addressing to handle collisions. The only difference from hashing here is, we do not use a hash function to find the index. We rather directly use values as indexes.

Hashing Functions

Hashing is a technique or process of mapping keys, and values into the hash table by using a hash function. It is done for faster access to elements. The efficiency of mapping depends on the efficiency of the hash function used.

Let a hash function H(x) maps the value x at the index **x%10** in an Array. For example if the list of values is [11,12,13,14,15] it will be stored at positions {1,2,3,4,5} in the array or Hash table respectively.



There are many hash functions that use numeric or alphanumeric keys. This article focuses on discussing different hash functions:

Division Method.

Mid Square Method.

Folding Method.

Multiplication Method.

Let's begin discussing these methods in detail.

1. Division Method:

This is the most simple and easiest method to generate a hash value. The hash function divides the value k by M and then uses the remainder obtained.

Formula:

h(K) = k mod M

Here,

k is the key value, and

M is the size of the hash table.

It is best suited that M is a prime number as that can make sure the keys are more uniformly distributed. The hash function is dependent upon the remainder of a division.

Example:

k = 12345

M = 95

h(12345) = 12345 mod 95

= 90

k = 1276

M = 11

h(1276) = 1276 mod 11

= 0

Pros:

This method is quite good for any value of M.

The division method is very fast since it requires only a single division operation.

Cons:

This method leads to poor performance since consecutive keys map to consecutive hash values in the hash table.

Sometimes extra care should be taken to choose the value of M.

2. Mid Square Method:

The mid-square method is a very good hashing method. It involves two steps to compute the hash value-

Square the value of the key k i.e. k2

Extract the middle r digits as the hash value.

Formula:

h(K) = h(k x k)

Here,

k is the key value.

The value of r can be decided based on the size of the table.

Example:

Suppose the hash table has 100 memory locations. So r = 2 because two digits are required to map the key to the memory location.

k = 60

k x k = 60 x 60

= 3600

h(60) = 60

The hash value obtained is 60

Pros:

The performance of this method is good as most or all digits of the key value contribute to the result. This is because all digits in the key contribute to generating the middle digits of the squared result.

The result is not dominated by the distribution of the top digit or bottom digit of the original key value.

Cons:

The size of the key is one of the limitations of this method, as the key is of big size then its square will double the number of digits.

Another disadvantage is that there will be collisions but we can try to reduce collisions.

3. Digit Folding Method:

This method involves two steps:

Divide the key-value k into a number of parts i.e. k1, k2, k3,….,kn, where each part has the same number of digits except for the last part that can have lesser digits than the other parts.

Add the individual parts. The hash value is obtained by ignoring the last carry if any.

Formula:

k = k1, k2, k3, k4, ….., kn

s = k1+ k2 + k3 + k4 +….+ kn

h(K)= s

Here,s is obtained by adding the parts of the key k

Example:

k = 12345

k1 = 12, k2 = 34, k3 = 5

s = k1 + k2 + k3

= 12 + 34 + 5

= 51

h(K) = 51

Note:

The number of digits in each part varies depending upon the size of the hash table. Suppose for example the size of the hash table is 100, then each part must have two digits except for the last part which can have a lesser number of digits.

4. Multiplication Method

This method involves the following steps:

Choose a constant value A such that 0 < A < 1.

Multiply the key value with A.

Extract the fractional part of kA.

Multiply the result of the above step by the size of the hash table i.e. M.

The resulting hash value is obtained by taking the floor of the result obtained in step 4.

Formula:

h(K) = floor (M (kA mod 1))

Here,

M is the size of the hash table.

k is the key value.

A is a constant value.

Example:

k = 12345

A = 0.357840

M = 100

h(12345) = floor[100 (12345*0.357840 mod 1)]

= floor[100 (4417.5348 mod 1)]

= floor[100 (0.5348)]

= floor[53.48]

= 53

Pros:

The advantage of the multiplication method is that it can work with any value between 0 and 1, although there are some values that tend to give better results than the rest.

Cons:

The multiplication method is generally suitable when the table size is the power of two, then the whole process of computing the index by the key using multiplication hashing is very fast.

Collision Handling

Collision Handling: Since a hash function gets us a small number for a big key, there is possibility that two keys result in same value. The situation where a newly inserted key maps to an already occupied slot in hash table is called collision and must be handled using some collision handling technique. Following are the ways to handle collisions:

- **Chaining:**The idea is to make each cell of hash table point to a linked list of records that have same hash function value. Chaining is simple, but requires additional memory outside the table.
- **Open Addressing:** In open addressing, all elements are stored in the hash table itself. Each table entry contains either a record or NIL. When searching for an element, we examine the table slots one by one until the desired element is found or it is clear that the element is not in the table.

There are three types of chaining which we will discuss ahead in detail:

1) Separate Chaining

Each bucket in the hash table holds a linked list or other data structure to handle collisions.

Colliding elements are stored in these data structures at the same index.

Offers flexibility and simplicity but may consume more memory due to additional data structures.

2) Open Addressing

All elements are stored directly within the hash table, without using external data structures.

When a collision occurs, the algorithm probes for an alternative empty slot in the table.

Avoids memory overhead but can suffer from clustering and degraded performance.

3) Double Hashing

A probing technique used in open addressing to resolve collisions.

Utilizes a secondary hash function to calculate the interval between probes.

Helps distribute elements more evenly, reducing clustering and improving performance compared to simpler probing techniques.

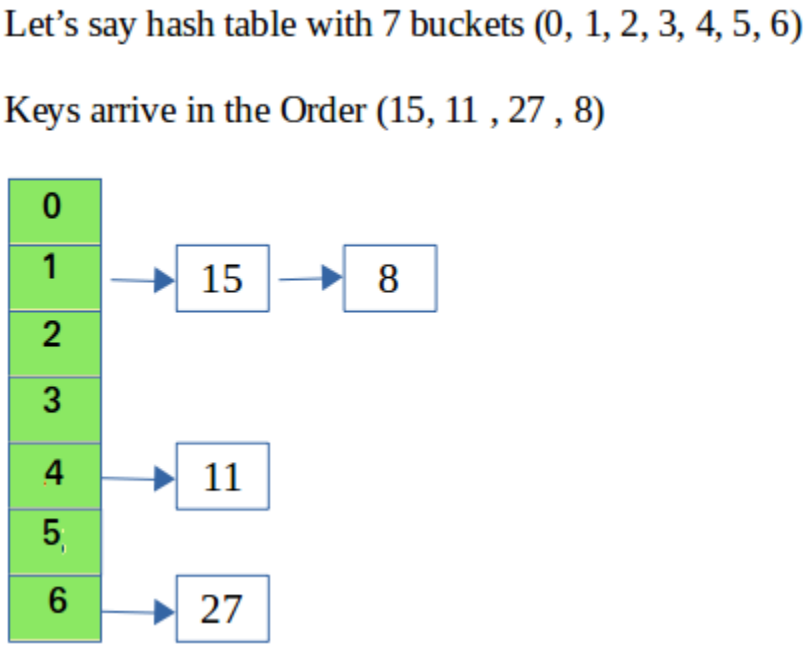
Implementation of Chaining

In hashing there is a hash function that maps keys to some values. But these hashing functions may lead to a collision that is two or more keys are mapped to same value. **Chain hashing** avoids collision. The idea is to make each cell of hash table point to a linked list of records that have same hash function value.

Let's create a hash function, such that our hash table has 'N' number of buckets.

To insert a node into the hash table, we need to find the hash index for the given key. And it could be calculated using the hash function.

Example: hashIndex = key % noOfBuckets
Insert: Move to the bucket corresponding to the above-calculated hash index and insert the new node at the end of the list.
Delete: To delete a node from hash table, calculate the hash index for the key, move to the bucket corresponding to the calculated hash index, and search the list in the current bucket to find and remove the node with the given key (if found).



Please refer **Hashing | Set 2 (Separate Chaining)** for details.
We use a list in C++ which is internally implemented as linked list (Faster insertion and deletion).

Method - 1 :
This method has not concept of rehashing. It only has a fixed size array i.e. fixed numbers of buckets.

CPP

```
// CPP program to implement hashing with chaining
#include<bits/stdc++.h>
using namespace std;

class Hash
{
    int BUCKET;    // No. of buckets
```

```
    // Pointer to an array containing buckets
    list<int> *table;
public:
    Hash(int V);  // Constructor

    // inserts a key into hash table
    void insertItem(int x);

    // deletes a key from hash table
    void deleteItem(int key);

    // hash function to map values to key
    int hashFunction(int x) {
        return (x % BUCKET);
    }

    void displayHash();
};

Hash::Hash(int b)
{
    this->BUCKET = b;
    table = new list<int>[BUCKET];
}

void Hash::insertItem(int key)
{
    int index = hashFunction(key);
    table[index].push_back(key);
}

void Hash::deleteItem(int key)
{
  // get the hash index of key
  int index = hashFunction(key);

  // find the key in (index)th list
  list <int> :: iterator i;
  for (i = table[index].begin();
           i != table[index].end(); i++) {
    if (*i == key)
      break;
  }

  // if key is found in hash table, remove it
  if (i != table[index].end())
    table[index].erase(i);
}
```

```
// function to display hash table
void Hash::displayHash() {
  for (int i = 0; i < BUCKET; i++) {
    cout << i;
    for (auto x : table[i])
      cout << " --> " << x;
    cout << endl;
  }
}

// Driver program
int main()
{
  // array that contains keys to be mapped
  int a[] = {15, 11, 27, 8, 12};
  int n = sizeof(a)/sizeof(a[0]);

  // insert the keys into the hash table
  Hash h(7);   // 7 is count of buckets in
               // hash table
  for (int i = 0; i < n; i++)
    h.insertItem(a[i]);

  // delete 12 from hash table
  h.deleteItem(12);

  // display the Hash table
  h.displayHash();

  return 0;
}
```

Output

```
0
1 --> 15 --> 8
2
3
4 --> 11
5
6 --> 27
```

Time Complexity:

- **Search :** O(1+(n/m))
- **Delete :** O(1+(n/m))
  where n = Total elements in hash table
  m = Size of hash table
- Here n/m is the **Load Factor**.
- Load Factor ($\propto$) must be as small as possible.

- If load factor increases,then possibility of collision increases.
- Load factor is trade of space and time .
- Assume , uniform distribution of keys ,
- Expected chain length : O(∝)
- Expected time to search : O(1 + ∝)
- Expected time to insert/ delete : O(1 + ∝)

Auxiliary Space: O(1), since no extra space has been taken.

Method - 2 :

Let's discuss another method where we have no boundation on number of buckets. Number of buckets will increase when value of load factor is greater than 0.5.

We will do rehashing when the value of load factor is greater than 0.5. In rehashing, we double the size of array and add all the values again to new array (doubled size array is new array) based on hash function. Hash function should also be change as it is depends on number of buckets. Therefore, hash function behaves differently from the previous one.

- Our Hash function is : **(ascii value of character * some prime number ^ x) % total number of buckets.** In this case prime number is 31.
- Load Factor = number of elements in Hash Map / total number of buckets
- Our key should be string in this case.
- We can make our own Hash Function but it should be depended on the size of array because if we do rehashing then it must reflect changes and number of collisions should reduce.

C++

```
#include <iostream>
#define ll long long int

using namespace std;

// Linked List
template <typename T>
class node
{
public:
    string key;
    T value;
    node *next;

    node(string key, T value) // constructor
    {
        this->key = key;
        this->value = value;
        this->next = NULL;
    }
    node(node &obj) // copy constructor
    {
```

```
        this->key = obj.key;
        this->value = obj.value;
        this->next = NULL;
    }

    ~node() // destructor
    {
        node *head = this;
        while (head != NULL)
        {
            node *currNode = head;
            head = head->next;
            delete currNode;
        }
    }
};

// hash table
template <typename T>
class unordered_map
{
public:
    int numOfElements, capacity;
    node<T> **arr; // want a array which stores pointers to node<T> i.e.
head of a Linked List

    unordered_map() // constructor
    {
        this->capacity = 1;
        this->numOfElements = 0;
        this->arr = new node<T> *[this->capacity];
        this->arr[0] = NULL;
    }

    int hashFunction(string key) // hash function for hashing a string
    {
        int bucketIndex;
        ll sum = 0, factor = 31;
        for (int i = 0; i < key.size(); i++)
        {
              // sum = sum + (ascii value of character * (prime number ^
x)) % total number of buckets
              // factor = factor * prime number i.e. prime number ^ x
            sum = ((sum % this->capacity) + ((int(key[i])) * factor) %
this->capacity) % this->capacity;
            factor = ((factor % INT16_MAX) * (31 % INT16_MAX)) %
INT16_MAX;
        }
```

```
        bucketIndex = sum;
        return bucketIndex;
    }

    float getLoadFactor()
    {
          // number of elements in hash table / total numbers of buckets
        return (float)(this->numOfElements + 1) / (float)(this->capacity);
    }

    void rehashing()
    {
        int oldCapacity = this->capacity;
        node<T> **temp = this->arr; // temp is hodling current array

        this->capacity = oldCapacity * 2; // doubling the size of current
capacity
        this->arr = new node<T> *[this->capacity]; // points to new array
of doubled size

        for (int i = 0; i < this->capacity; i++)
        {
            arr[i] = NULL;
        }
        for (int i = 0; i < oldCapacity; i++) // copying all the previous
values in new array
        {
            node<T> *currBucketHead = temp[i];
            while (currBucketHead != NULL) // copying whole linked list
            {
                this->insert(currBucketHead->key, currBucketHead->value);
// insert function have now updated hash function as capacity is doubled
                currBucketHead = currBucketHead->next;
            }
        }
        delete[] temp; // deleting old array from heap memory
        return;
    }

    void insert(string key, T value)
    {
        while (this->getLoadFactor() > 0.5f) // when load factor > 0.5
        {
            this->rehashing();
        }

        int bucketIndex = this->hashFunction(key);
        if (this->arr[bucketIndex] == NULL) // when there is no linked
list at bucket
```

```
        {
            node<T> *newNode = new node<T>(key, value);
            arr[bucketIndex] = newNode;
        }
        else // adding at the head of current linked list
        {
            node<T> *newNode = new node<T>(key, value);
            newNode->next = this->arr[bucketIndex];
            this->arr[bucketIndex] = newNode;
        }
        return;
    }

    int search(string key)
    {
        int bucketIndex = this->hashFunction(key); // getting bucket index
        node<T> *bucketHead = this->arr[bucketIndex];
        while (bucketHead != NULL) // searching in the linked list which
is present at bucket for given key
        {
            if (bucketHead->key == key)
            {
                return bucketHead->value;
            }
            bucketHead = bucketHead->next; // moving to next node in
linked list
        }
        cout << "Oops!! Data not found." << endl; // when key is not
matched...
        return -1;
    }
};

int main()
{
    unordered_map<int> mp; // int is value....in our case key must be of
string type
      mp.insert("Manish", 16);
      mp.insert("Vartika", 14);
      mp.insert("ITT", 5);
      mp.insert("elite_Programmer", 4);
      mp.insert("pluto14", 14);
      mp.insert("GeeksForGeeks", 11);

      cout << "Value of GeeksForGeeks : " << mp.search("GeeksForGeeks") <<
endl;
      cout << "Value of ITT : " << mp.search("ITT") << endl;
      cout << "Value of Manish : " << mp.search("Manish") << endl;
      cout << "Value of Vartika : " << mp.search("Vartika") << endl;
```

```
      cout << "Value of elite_Programmer : " <<
mp.search("elite_Programmer") << endl;
      cout << "Value of pluto14 : " << mp.search("pluto14") << endl;

        // prints Oops!! Data not found and return -1
      mp.search("GFG"); // case when there is no key present in Hash Map..

      return 0;
}
```

Output

```
Value of GeeksForGeeks : 11
Value of ITT : 5
Value of Manish : 16
Value of Vartika : 14
Value of elite_Programmer : 4
Value of pluto14 : 14
Oops!! Data not found.
```

Complexity analysis of Insert:

- **Time Complexity:** O(N), It takes O(N) time complexity because we are checking the load factor each time and when it is greater than 0.5 we call rehashing function which takes O(N) time.
- **Space Complexity**: O(N), It takes O(N) space complexity because we are creating a new array of doubled size and copying all the elements to the new array.

Complexity analysis of Search:

- **Time Complexity:** O(N), It takes O(N) time complexity because we are searching in a linked list of size N.
- **Space Complexity:** O(1), It takes O(1) space complexity because we are not using any extra space for searching.

Open Addressing

Open Addressing: Open addressing is a method for handling collisions. In Open Addressing, all elements are stored in the hash table itself. So at any point, the size of the table must be greater than or equal to the total number of keys (Note that we can increase table size by copying old data if needed).

Important Operations:

- Insert(k): Keep probing until an empty slot is found. Once an empty slot is found, insert k.
- Search(k): Keep probing until the slot's key doesn't become equal to k or an empty slot is reached.
- Delete(k): ***Delete operation is interesting***. If we simply delete a key, then the search may fail. So slots of the deleted keys are marked specially as "deleted".

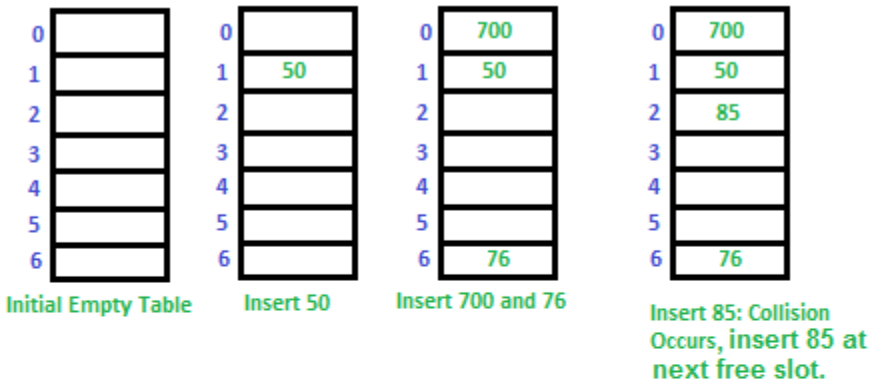
Insert can insert an item in a deleted slot, but the search doesn't stop at a deleted slot.

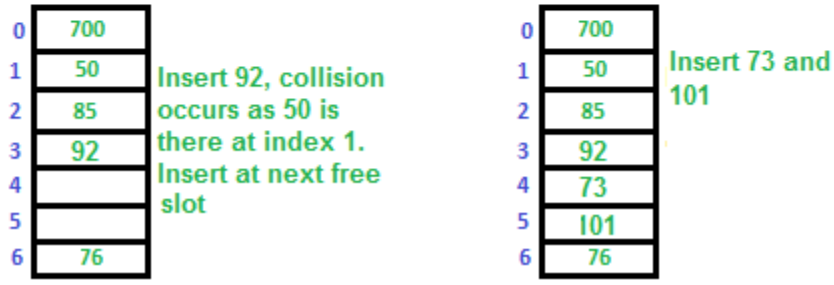
Open Addressing is done in the following ways:

1. ***Linear Probing:*** In linear probing, we linearly probe for the next slot. For example, the typical gap between the two probes is 1 as taken in the below example also.
   let **hash(x)** be the slot index computed using a hash function and **S** be the table size.

```
If slot hash(x) % S is full, then we try (hash(x) + 1) % S
If (hash(x) + 1) % S is also full, then we try (hash(x) + 2) % S
If (hash(x) + 2) % S is also full, then we try (hash(x) + 3) % S
.................................................
.................................................
```

1. Let us consider a simple hash function as “key mod 7” and a sequence of keys as 50, 700, 76, 85, 92, 73, 101.





Clustering: The

main problem with linear probing is clustering, many consecutive elements form groups and it starts taking time to find a free slot or to search an element.

2. ***Quadratic Probing*** We look for i^2'th slot in i'th iteration.

let hash(x) be the slot index computed using hash function. If slot hash(x) % S is full, then we try (hash(x) + 1*1) % S If (hash(x) + 1*1) % S is also full, then we try (hash(x) + 2*2) % S If (hash(x) + 2*2) % S is also full, then we try (hash(x) + 3*3) % S ....

1. **Double Hashing** We use another hash function hash2(x) and look for i*hash2(x) slot in i'th rotation.

let hash(x) be the slot index computed using hash function. If slot hash(x) % S is full, then we try (hash(x) + 1*hash2(x)) % S If (hash(x) + 1*hash2(x)) % S is also full, then we try (hash(x) + 2*hash2(x)) % S If (hash(x) + 2*hash2(x)) % S is also full, then we try (hash(x) + 3*hash2(x)) % S .....
.....

1. See this for step by step diagrams.

Comparison of above three:

- Linear probing has the best cache performance but it suffers from clustering. One more advantage of Linear probing that it is easy to compute.
- Quadratic probing lies between the two in terms of cache performance and clustering.
- Double hashing has poor cache performance but no clustering. Double hashing requires more computation time as two hash functions need to be computed.

S.No.	Seperate Chaining	Open Addressing
1.	Chaining is Simpler to implement.	Open Addressing requires more computation.
2.	In chaining, Hash table never fills up, we can always add more elements to chain.	In open addressing, table may become full.

S.No.	Seperate Chaining	Open Addressing
3.	Chaining is Less sensitive to the hash function or load factors.	Open addressing requires extra care for to avoid clustering and load factor.
4.	Chaining is mostly used when it is unknown how many and how frequently keys may be inserted or deleted.	Open addressing is used when the frequency and number of keys is known.
5.	Cache performance of chaining is not good as keys are stored using linked list.	Open addressing provides better cache performance as everything is stored in the same table.
6.	Wastage of Space (Some Parts of hash table in chaining are never used).	In Open addressing, a slot can be used even if an input doesn't map to it.
7.	Chaining uses extra space for links.	No links in Open addressing

Performance of Open Addressing:

Like Chaining, the performance of hashing can be evaluated under the assumption that each key is equally likely to be hashed to any slot of the table (simple uniform hashing).

```
m = Number of slots in the hash table
n = Number of keys to be inserted in the hash table

Load factor α = n/m  ( < 1 )
```

```
Expected time to search/insert/delete < 1/(1 - α)

So Search, Insert and Delete take (1/(1 - α)) time
```

Double Hashing

Double Hashing

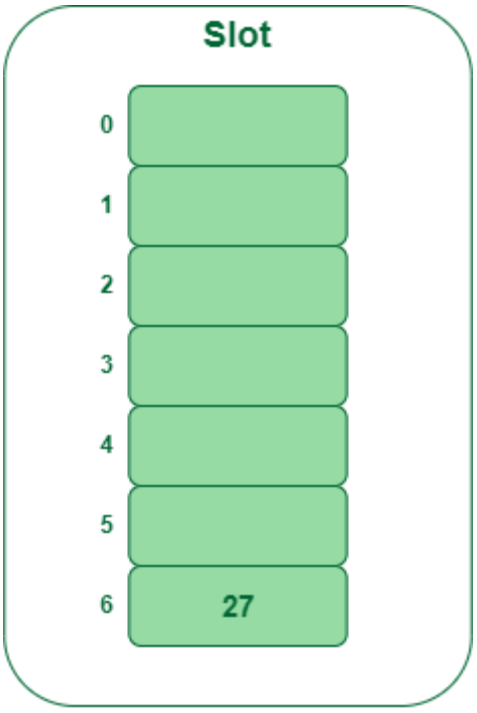
The intervals that lie between probes are computed by another hash function. Double hashing is a technique that reduces clustering in an optimized way. In this technique, the increments for the probing sequence are computed by using another hash function. We use another hash function hash2(x) and look for the i*hash2(x) slot in the **i**th rotation.

let hash(x) be the slot index computed using hash function.

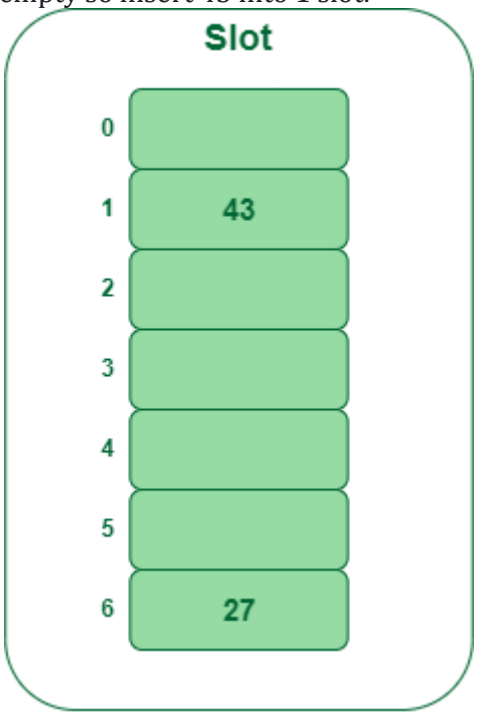
*If slot hash(x) % S is full, then we try (hash(x) + 1*hash2(x)) % S*
*If (hash(x) + 1*hash2(x)) % S is also full, then we try (hash(x) + 2*hash2(x)) % S*
*If (hash(x) + 2*hash2(x)) % S is also full, then we try (hash(x) + 3*hash2(x)) % S*
……
……

Example: Insert the keys 27, 43, 98, 72 into the Hash Table of size 7. where first hash-function is **h1(k) = k mod 7** and second hash-function is **h2(k) = 1 + (k mod 5)**

- **Step 1:** Insert 27
  - 27 % 7 = 6, location 6 is empty so insert 27 into 6 slot.



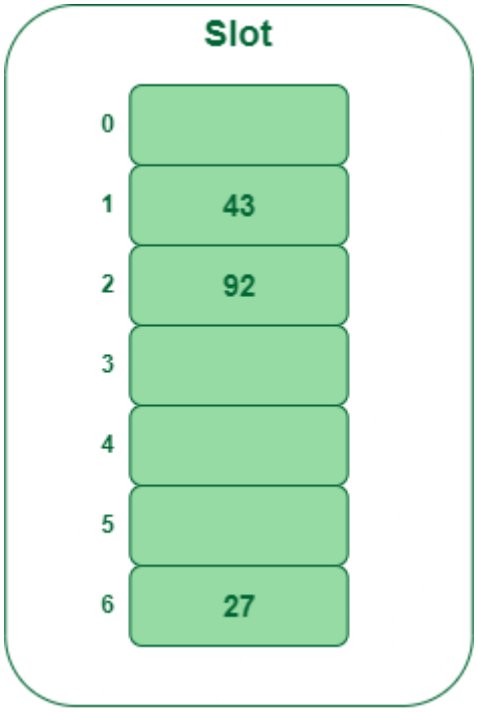
- **Step 2:** Insert 43
  - 43 % 7 = 1, location 1 is empty so insert 43 into 1 slot.



- **Step 3:** Insert 98
    - 98 % 7 = 6, but location 6 is already being occupied and this is a collision
    - So we need to resolve this collision using double hashing.

h_{new} *= [h1(98) + i * (h2(98] % 7*
*= [6 + 1 * (1 + 98 % 5)] % 7*
= 9 % 7
= 2

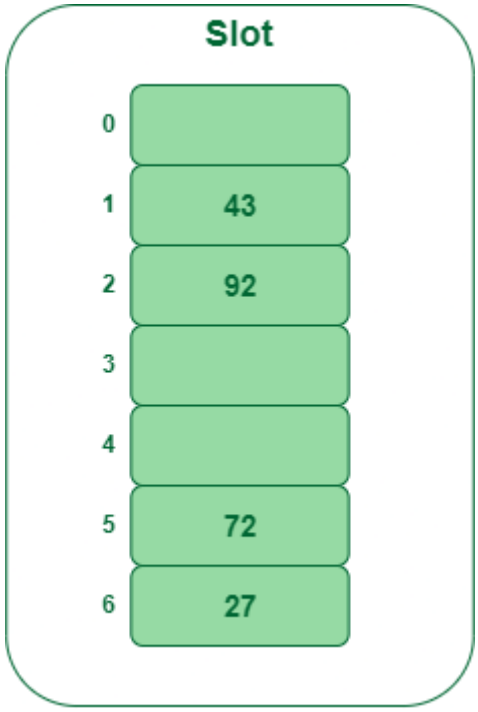
Now, as 2 is an empty slot,
so we can insert 98 into 2nd slot.



- **Step 4:** Insert 72
    - 72 % 7 = 2, but location 2 is already being occupied and this is a collision.
    - So we need to resolve this collision using double hashing.

*h_{new} = [h1(72) + i * (h2(72)] % 7*
*= [2 + 1 * (1 + 72 % 5)] % 7*
= 5 % 7
= 5,

Now, as 5 is an empty slot,
so we can insert 72 into 5th slot.



Performance of Open Addressing:

Cache performance of chaining is not good because when we traverse a Linked List, we are basically jumping from one node to another, all across the computer's memory. For this reason, the CPU cannot cache the nodes which aren't visited yet, this doesn't help us. But with Open Addressing, data isn't spread, so if the CPU detects that a segment of memory is constantly being accessed, it gets cached for quick access.

Like Chaining, the performance of hashing can be evaluated under the assumption that each key is equally likely to be hashed to any slot of the table (simple uniform hashing)

m = Number of slots in the hash table

n = Number of keys to be inserted in the hash table

Load factor α = n/m (< 1)

Expected time to search/insert/delete < 1/(1 – α)

So Search, Insert and Delete take (1/(1 – α)) time

Implementation of Open Addressing

The task is to design a general Hash Table data structure with Collision case handled and that supports the **Insert()**, **Find(),** and **Delete()** functions.

Examples:

Suppose the operations are performed on an array of pairs, {{1, 5}, {2, 15}, {3, 20}, {4, 7}}. And an array of capacity 20 is used as a Hash Table:

1. **Insert(1, 5**): Assign the pair {1, 5} at the index (1%20 =1) in the Hash Table.
2. **Insert(2, 15**): Assign the pair {2, 15} at the index (2%20 =2) in the Hash Table.
3. **Insert(3, 20**): Assign the pair {3, 20} at the index (3%20 =3) in the Hash Table.
4. **Insert(4, 7**): Assign the pair {4, 7} at the index (4%20 =4) in the Hash Table.
5. **Find(4):** The key 4 is stored at the index (4%20 = 4). Therefore, print the 7 as it is the value of the key, 4, at index 4 of the Hash Table.
6. **Delete(4):** The key 4 is stored at the index (4%20 = 4). After deleting Key 4, the Hash Table has keys {1, 2, 3}.
7. **Find(4):** Print -1, as the key 4 does not exist in the Hash Table.

Approach: The given problem can be solved by using the modulus Hash Function and using an array of structures as Hash Table, where each array element will store the **{key, value}** pair to be hashed. The collision case can be handled by Linear probing, open addressing. Follow the steps below to solve the problem:

- Define a node, structure say **HashNode,** to a key-value pair to be hashed.

- Initialize an array of the pointer of type **HashNode**, say ***arr[]** to store all key-value pairs.
- **Insert(Key, Value):** Insert the pair **{Key, Value}** in the Hash Table.
  - Initialize a **HashNode** variable, say **temp**, with value **{Key, Value}.**
  - Find the index where the key can be stored using the, Hash Function and then store the index in a variable say **HashIndex**.
  - If **arr[HashIndex]** is not empty or there exists another **Key**, then do linear probing by continuously updating the **HashIndex** as **HashIndex =(HashIndex+1)%capacity.**
  - If **arr[HashIndex]** is not null, then insert the given Node by assigning the address of **temp** to **arr[HashIndex].**
- **Find(Key):** Finds the value of the **Key** in the Hash Table.
  - Find the index where the **key** may exist using a Hash Function and then store the index in a variable, say **HashIndex**.
  - If the **arr[HashIndex]** contains the key, **Key** then returns the value of it.
  - Otherwise, do linear probing by continuously updating the **HashIndex** as **HashIndex =(HashIndex+1)%capacity.** Then**,** if **Key** is found, then return the value of the **Key** at that **HashIndex** and then return **true**.
  - If the **Key** is not found, then return **-1** representing not found. Otherwise, return the value of the **Key.**
- **Delete(Key):** Deletes the **Key** from the Hash Table.
  - Find the index where the **key** may exist using a Hash Function and then store the index in a variable, say **HashIndex**.
  - If the **arr[HashIndex**] contains the key, **Key** then delete by assigning **{-1, -1}** to the **arr[HashIndex]** and then return **true**.
  - Otherwise, do linear probing by continuously updating the **HashIndex** as **HashIndex =(HashIndex+1)%capacity.** Then**,** if **Key** is found then delete the value of the **Key** at that **HashIndex** and then return **true**.
  - If the **Key** is not found, then the return is false.

Below is the implementation of the above approach:

C++C

```
// C++ program for the above approach
#include <bits/stdc++.h>
using namespace std;

struct HashNode {
    int key;
    int value;
};

const int capacity = 20;
int size = 0;
```

```
struct HashNode** arr;
struct HashNode* dummy;

// Function to add key value pair
void insert(int key, int V)
{

    struct HashNode* temp
        = (struct HashNode*)malloc(sizeof(struct HashNode));
    temp->key = key;
    temp->value = V;

    // Apply hash function to find
    // index for given key
    int hashIndex = key % capacity;

    // Find next free space
    while (arr[hashIndex] != NULL
           && arr[hashIndex]->key != key
           && arr[hashIndex]->key != -1) {
        hashIndex++;
        hashIndex %= capacity;
    }

    // If new node to be inserted
    // increase the current size
    if (arr[hashIndex] == NULL || arr[hashIndex]->key == -1)
        size++;

    arr[hashIndex] = temp;
}

// Function to delete a key value pair
int deleteKey(int key)
{
    // Apply hash function to find
    // index for given key
    int hashIndex = key % capacity;

    // Finding the node with given
    // key
    while (arr[hashIndex] != NULL) {
        // if node found
        if (arr[hashIndex]->key == key) {
            // Insert dummy node here
            // for further use
            arr[hashIndex] = dummy;

            // Reduce size
```

```
            size--;

            // Return the value of the key
            return 1;
        }
        hashIndex++;
        hashIndex %= capacity;
    }

    // If not found return null
    return 0;
}

// Function to search the value
// for a given key
int find(int key)
{
    // Apply hash function to find
    // index for given key
    int hashIndex = (key % capacity);

    int counter = 0;

    // Find the node with given key
    while (arr[hashIndex] != NULL) {

        int counter = 0;
        // If counter is greater than
        // capacity
        if (counter++ > capacity)
            break;

        // If node found return its
        // value
        if (arr[hashIndex]->key == key)
            return arr[hashIndex]->value;

        hashIndex++;
        hashIndex %= capacity;
    }

    // If not found return
    // -1
    return -1;
}

// Driver Code
int main()
{
```

```
    // Space allocation
    arr = (struct HashNode**)malloc(sizeof(struct HashNode*)
                                    * capacity);
    // Assign NULL initially
    for (int i = 0; i < capacity; i++)
        arr[i] = NULL;

    dummy
        = (struct HashNode*)malloc(sizeof(struct HashNode));

    dummy->key = -1;
    dummy->value = -1;

    insert(1, 5);
    insert(2, 15);
    insert(3, 20);
    insert(4, 7);
    if (find(4) != -1)
        cout << "Value of Key 4 = " << find(4) << endl;
    else
        cout << ("Key 4 does not exists\n");

    if (deleteKey(4))
        cout << ("Node value of key 4 is deleted "
                 "successfully\n");
    else {
        cout << ("Key does not exists\n");
    }

    if (find(4) != -1)
        cout << ("Value of Key 4 = %d\n", find(4));
    else
        cout << ("Key 4 does not exists\n");
}

// This code is contributed by Lovely Jain
```

Output

```
Value of Key 4 = 7
Node value of key 4 is deleted successfully
Key 4 does not exists
```

Time Complexity: *O(capacity), for each operation*

Auxiliary Space: *O(capacity)*

Chaining vs Open Addressing

Chaining vs Open Addressing

S.No.	Separate Chaining	Open Addressing
1.	Chaining is Simpler to implement.	Open Addressing requires more computation.
2.	In chaining, Hash table never fills up, we can always add more elements to chain.	In open addressing, table may become full.
3.	Chaining is Less sensitive to the hash function or load factors.	Open addressing requires extra care to avoid clustering and load factor.
4.	Chaining is mostly used when it is unknown how many and how frequently keys may be inserted or deleted.	Open addressing is used when the frequency and number of keys is known.
5.	Cache performance of chaining is not good as keys are stored using linked list.	Open addressing provides better cache performance as everything is stored in the same table.
6.	Wastage of Space (Some Parts of hash table in chaining are never used).	In Open addressing, a slot can be used even if an input doesn’t map to it.
7.	Chaining uses extra space for links.	No links in Open addressing

Count Distinct Elements

Given an unsorted array, count all distinct elements in it.

Examples:

```
Input :   arr[] = {10, 20, 20, 10, 30, 10}
Output : 3
There are three distinct elements 10, 20 and 30.

Input :   arr[] = {10, 20, 20, 10, 20}
Output : 2
```

A **simple solution** is to run two loops. For every element, check if it has appeared before. If yes, increment count of distinct elements.

C++Java

```
// C++ program to count distinct elements
// in a given array
#include <iostream>
using namespace std;

int countDistinct(int arr[], int n)
{
    int res = 1;

    // Pick all elements one by one
    for (int i = 1; i < n; i++) {
        int j = 0;
        for (j = 0; j < i; j++)
            if (arr[i] == arr[j])
                break;

        // If not printed earlier, then print it
        if (i == j)
            res++;
    }
    return res;
}

// Driver program to test above function
```

```
int main()
{
    int arr[] = { 12, 10, 9, 45, 2, 10, 10, 45 };
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << countDistinct(arr, n);
    return 0;
}
```

Output

5

Time Complexity of above solution is O(n^2). We can **Use Sorting** to solve the problem in O(nLogn) time. The idea is simple, first sort the array so that all occurrences of every element become consecutive. Once the occurrences become consecutive, we can traverse the sorted array and count distinct elements in O(n) time. Following is the implementation of the idea.

C++Java

```
// C++ program to count all distinct elements
// in a given array
#include <algorithm>
#include <iostream>
using namespace std;

int countDistinct(int arr[], int n)
{
    // First sort the array so that all
    // occurrences become consecutive
    sort(arr, arr + n);

    // Traverse the sorted array
    int res = 0;
    for (int i = 0; i < n; i++) {

        // Move the index ahead while
        // there are duplicates
        while (i < n - 1 && arr[i] == arr[i + 1])
            i++;

        res++;
    }
```

```
    return res;
}

// Driver program to test above function
int main()
{
    int arr[] = { 6, 10, 5, 4, 9, 120, 4, 6, 10 };
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << countDistinct(arr, n);
    return 0;
}
```

Output
6

We can **Use Hashing** to solve this in O(n) time on average. The idea is to traverse the given array from left to right and keep track of visited elements in a hash set , as a set consists of only unique elements.
Following is the implementation of the idea.

C++Java

```
/* CPP program to print all distinct elements
   of a given array */
#include <bits/stdc++.h>
using namespace std;

// This function prints all distinct elements
int countDistinct(int arr[], int n)
{
    // Creates an empty hashset
    unordered_set<int> s;

    // Traverse the input array
    int res = 0;
    for (int i = 0; i < n; i++) {

        // If not present, then put it in
```

```
        // hashtable and increment result
        if (s.find(arr[i]) == s.end()) {
            s.insert(arr[i]);
            res++;
        }
    }

    return res;
}

// Driver program to test above function
int main()
{
    int arr[] = { 6, 10, 5, 4, 9, 120, 4, 6, 10 };
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << countDistinct(arr, n);
    return 0;
}
```

Output

6

Set STL approach:

Sets are a type of associative containers in which each element has to be unique, because the value of the element identifies it. The value of the element cannot be modified once it is added to the set, though it is possible to remove and add the modified value of that element. We exploit the very basic property of set STL that it stores only unique numbers.

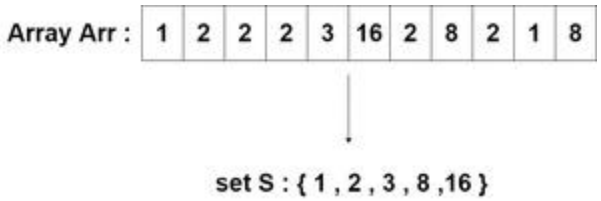
Examples: Arr = **[1,2,2,2,3,16,2,8,2,1,8]**

Distinct elements present = **[1,2,3,16,8]**

Total number of distinct elements in the array are **5**

Algorithm :

1. Insert all the elements into the set **S** one by one.



2. Store the total size **s** of the set using **set::size()**.

3.The total size **s** is the number of distinct elements present in the array.

C++Java

```cpp
#include <bits/stdc++.h>
using namespace std;
// function that accepts the array and it's size and returns
// the number of distince elements
int distinct(int* arr, int len)
{
    set<int> S; // declaring a set container using STL
    for (int i = 0; i < len; i++) {
        S.insert(arr[i]); // inserting all elements of the
                          // array into set
    }
    int ans = S.size(); // calculating the size of the set
    return ans;
}
int main()
{
    int arr[] = { 12, 10, 9, 45, 2, 10, 10, 45};
    int l = sizeof(arr)
            / sizeof(
                int); // calculating the size of the array
    int dis_elements
        = distinct(arr, l); // calling the function on array
    cout << dis_elements << endl;
    return 0;
}
```

Output

5

String

Introduction to Strings

Strings are defined as a stream of characters. Strings are used to represent text and are generally represented by enclosing text within quotes as: *"This is a sample string!"*.

Different programming languages have different ways of declaring and using Strings. We will learn to implement strings in **C/C++ and Java**.

Strings in C/C++

In C/C++, Strings are defined as an array of characters. The difference between a character array and a string is that the string is terminated with a special character '\0'.

Declaring Strings: Declaring a string is as simple as declaring a one-dimensional array. Below is the basic syntax for declaring a string.

```
char str_name[size];
```

In the above syntax, *str_name* is any name given to the string variable and size is used to define the length of the string, i.e the number of characters strings will store. Please keep in mind that there is an extra terminating character which is the Null character ('\0') used to indicate the termination of string which differs strings from normal character arrays.

Initializing a String: A string can be initialized in different ways. We will explain this with the help of an example. Below is an example to declare a string with the name as *str* and initialize it with **"GeeksforGeeks"**.

1. char str[] = "GeeksforGeeks";

2. char str[50] = "GeeksforGeeks";

3. char str[] = {'G','e','e','k','s','f','o','r','G','e','e','k','s','\0'};

4. char str[14] = {'G','e','e','k','s','f','o','r','G','e','e','k','s','\0'};

Printing a string array: Unlike arrays we do not need to print a string, character by character. The C/C++ language does not provide an inbuilt data type for strings but it has an access specifier "**%s**" which can be used to directly print and read strings.

CPP

```
// C/C++ program to illustrate strings

#include<bits/stdc++.h>

int main()
{
    // declare and initialize string
    char str[] = "Geeks";

    // print string
    printf("%s",str);

    return 0;
}
```

Output:

```
Geeks
```

Passing strings to function: As strings are character arrays, so we can pass strings to function in the same way we pass an array to a function. Below is a sample program to do this:

CPP

```
// C/C++ program to illustrate how to
// pass strings to function

#include<bits/stdc++.h>

void printStr(char str[])
{
    printf("String is : %s",str);
}

int main()
```

```
{
    // declare and initialize string
    char str[] = "GeeksforGeeks";

    // print string by passing string
    // to a different function
    printStr(str);

    return 0;
}
```

Output:

```
String is : GeeksforGeeks
```

std::string Class in C++

C++ has in its definition a way to represent the sequence of characters as an object of a class. This class is called **std::string**. The String class stores the characters as a sequence of bytes with functionality of allowing access to single byte character.

string Class vs Character array:

- A character array is simply an array of characters can terminated by a null character. A string is a class which defines objects that be represented as stream of characters.
- Size of the character array has to allocated statically, more memory cannot be allocated at run time if required. Unused allocated memory is wasted in case of character array. In case of strings, memory is allocated dynamically. More memory can be allocated at run time on demand. As no memory is preallocated, no memory is wasted.
- Implementation of character array is faster than std:: string. Strings are slower when compared to implementation than character array.
- Character array does not offer much inbuilt functions to manipulate strings. String class defines a number of functionalities which allow manifold operations on strings.

Declaration Syntax: Declaring string using string class is simple and can be done using the *string* keyword as shown below.

```
string string_name = "Sample String";
```

Sample Program:

CPP

```cpp
// C++ program to illustrate strings

#include<bits/stdc++.h>
using namespace std;

int main()
{
    // declare and initialize string
    string str = "Geeks";

    // print string
    cout<<str;

    return 0;
}
```

Output:

```
Geeks
```

To learn about std::string in details, refer: [std::string class in C++](#).

Strings in Java

String is a sequence of characters. In java, objects of String are immutable which means a constant and cannot be changed once created.

Creating a String

There are two ways to create string in Java:

- ***String literal***
- `String s = "GeeksforGeeks";`
- **Using *new* keyword**
- `String s = new String ("GeeksforGeeks");`

String Methods

1. **int length():** Returns the number of characters in the String.

```
"GeeksforGeeks".length();  // returns 13
```

2. **Char charAt(int i):** Returns the character at i^{th} index.

```
"GeeksforGeeks".charAt(3); // returns  'k'
```

3. **String substring (int i):** Return the substring from the ith index character to end.

```
"GeeksforGeeks".substring(3); // returns "ksforGeeks"
```

4. **String substring (int i, int j):** Returns the substring from i to j-1 index.

```
 "GeeksforGeeks".substring(2, 5); // returns "eks"
```

5. **String concat(String str):** Concatenates specified string to the end of this string.

```
String s1 = "Geeks";
String s2 = "forGeeks";
String output = s1.concat(s2); // returns "GeeksforGeeks"
```

6. **int indexOf (String s):** Returns the index within the string of the first occurrence of the specified string.

```
String s = "Learn Share Learn";
int output = s.indexOf("Share"); // returns 6
```

7. **int indexOf (String s, int i):** Returns the index within the string of the first occurrence of the specified string, starting at the specified index.

```
String s = "Learn Share Learn";
int output = s.indexOf('a',3);// returns 8
```

8. **Int lastIndexOf(String s):** Returns the index within the string of the last occurrence of the specified string.

```
String s = "Learn Share Learn";
int output = s.lastIndexOf('a'); // returns 14
```

9. **boolean equals(Object otherObj):** Compares this string to the specified object.

```
Boolean out = "Geeks".equals("Geeks"); // returns true
Boolean out = "Geeks".equals("geeks"); // returns false
```

10. **boolean equalsIgnoreCase (String anotherString):** Compares string to another string, ignoring case considerations.

```
Boolean out= "Geeks".equalsIgnoreCase("Geeks"); //
returns true
Boolean out = "Geeks".equalsIgnoreCase("geeks"); //
returns true
```

11. **int compareTo(String anotherString):** Compares two string lexicographically.

```
int out = s1.compareTo(s2);  // where s1 ans s2 are
                             // strings to be compared

This returns difference s1-s2. If :
out < 0  // s1 comes before s2
out = 0  // s1 and s2 are equal.
out > 0   // s1 comes after s2.
```

42. **int compareToIgnoreCase(String anotherString):** Compares two string lexicographically, ignoring case considerations.

```
43.  int out = s1.compareToIgnoreCase(s2);
44. // where s1 ans s2 are
45. // strings to be compared
46.
47.  This returns difference s1-s2. If :
48.  out < 0  // s1 comes before s2
49.  out = 0   // s1 and s2 are equal.
50.  out > 0   // s1 comes after s2.
51.
```

Note- In this case, it will not consider case of a letter (it will ignore whether it is uppercase or lowercase).

52. **String toLowerCase():** Converts all the characters in the String to lower case.

```
53. String word1 = "HeLLo";
54. String word3 = word1.toLowerCase(); // returns "hello"
55.
```

56. **String toUpperCase():** Converts all the characters in the String to upper case.

```
57. String word1 = "HeLLo";
58. String word2 = word1.toUpperCase(); // returns "HELLO"
59.
```

60. **String trim():** Returns the copy of the String, by removing whitespaces at both ends. It does not affect whitespaces in the middle.

```
61. String word1 = " Learn Share Learn ";
62. String word2 = word1.trim(); // returns "Learn Share
    Learn"
63.
```

64. **String replace (char oldChar, char newChar):** Returns new string by replacing all occurrences of *oldChar* with *newChar.*

```
65. String s1 = "feeksforfeeks";
66. String s2 = "feeksforfeeks".replace('f' ,'g'); // returns
    "geeksgorgeeks"
67.
```

Note:- s1 is still feeksforfeeks and s2 is geeksgorgeeks

Program to illustrate all string methods:

Java

```java
// Java code to illustrate different constructors and methods
// String class.

import java.io.*;
import java.util.*;
class Test
{
    public static void main (String[] args)
    {
```

```
        String s= "GeeksforGeeks";
        // or String s= new String ("GeeksforGeeks");

        // Returns the number of characters in the String.
        System.out.println("String length = " + s.length());

        // Returns the character at ith index.
        System.out.println("Character at 3rd position = "
                           + s.charAt(3));

        // Return the substring from the ith index character
        // to end of string
        System.out.println("Substring " + s.substring(3));

        // Returns the substring from i to j-1 index.
        System.out.println("Substring = " + s.substring(2,5));

        // Concatenates string2 to the end of string1.
        String s1 = "Geeks";
        String s2 = "forGeeks";
        System.out.println("Concatenated string = " +
                            s1.concat(s2));

        // Returns the index within the string
        // of the first occurrence of the specified string.
        String s4 = "Learn Share Learn";
        System.out.println("Index of Share " +
                           s4.indexOf("Share"));

        // Returns the index within the string of the
        // first occurrence of the specified string,
        // starting at the specified index.
        System.out.println("Index of a = " +
                           s4.indexOf('a',3));

        // Checking equality of Strings
        Boolean out = "Geeks".equals("geeks");
        System.out.println("Checking Equality " + out);
        out = "Geeks".equals("Geeks");
        System.out.println("Checking Equality " + out);

        out = "Geeks".equalsIgnoreCase("gEeks ");
        System.out.println("Checking Equality " + out);

        int out1 = s1.compareTo(s2);
        System.out.println("If s1 = s2 " + out);

        // Converting cases
        String word1 = "GeeKyMe";
```

```
        System.out.println("Changing to lower Case " +
                           word1.toLowerCase());

        // Converting cases
        String word2 = "GeekyME";
        System.out.println("Changing to UPPER Case " +
                           word1.toUpperCase());

        // Trimming the word
        String word4 = " Learn Share Learn ";
        System.out.println("Trim the word " + word4.trim());

        // Replacing characters
        String str1 = "feeksforfeeks";
        System.out.println("Original String " + str1);
        String str2 = "feeksforfeeks".replace('f' ,'g') ;
        System.out.println("Replaced f with g -> " + str2);
    }
}
```

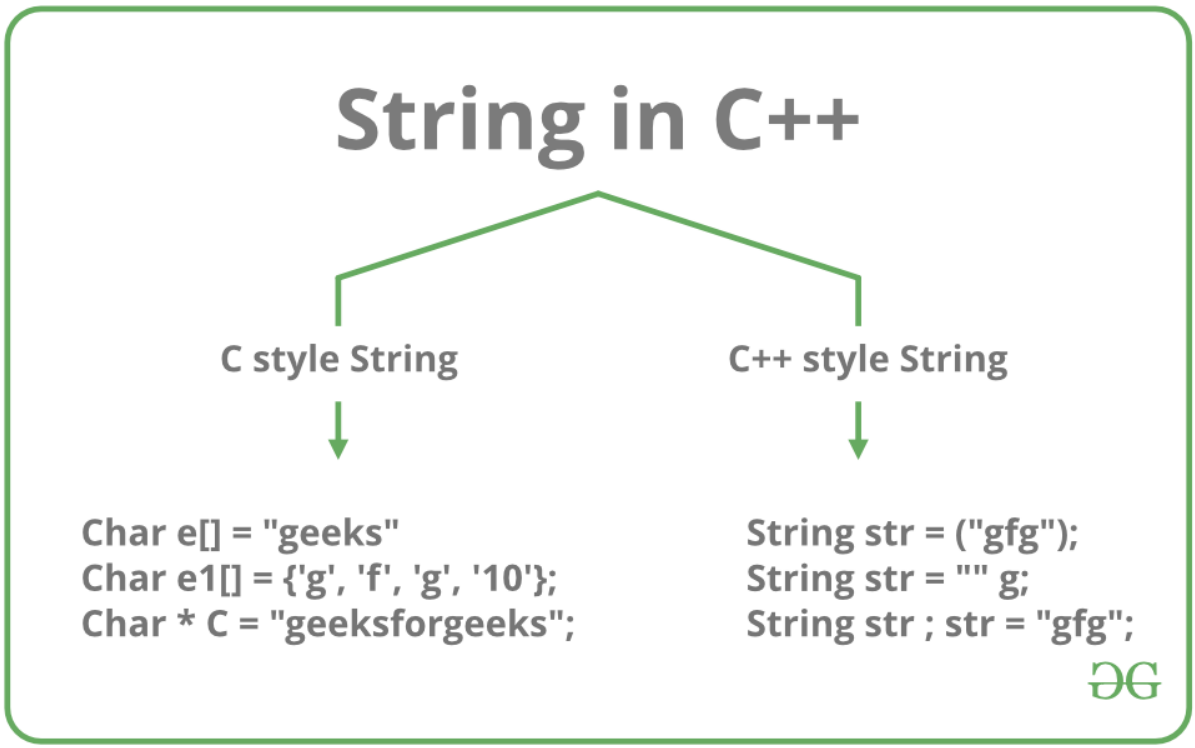
Output :

```
String length = 13
Character at 3rd position = k
Substring ksforGeeks
Substring = eks
Concatenated string = GeeksforGeeks
Index of Share 6
Index of a = 8
Checking Equality false
Checking Equality true
Checking Equality false
If s1 = s2 false
Changing to lower Case geekyme
Changing to UPPER Case GEEKYME
Trim the word Learn Share Learn
Original String feeksforfeeks
Replaced f with g -> geeksgorgeeks
```

Strings in C++

Strings in C++ are used to store text or sequence of characters. In C++ strings can be stored in one of the two following ways:

- C style string (using characters)
- String class



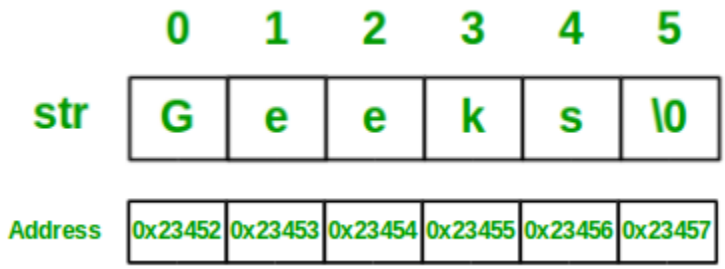
Each of the above methods is discussed below:

1. **C style string:** In C, strings are defined as an array of characters. The difference between a character array and a string is the string is terminated with a special character '\0'. In C, the string is actually represented as an array of characters terminated by a null string. Therefore the size of the character array is always one more than that of the number of characters in the actual string. This thing continues to be supported in C++ too. The C++ compiler automatically sets "\0" at the end of the string, during initialization of the array.

Initializing a String in C++:

```
1. char str[] = "Geeks";
2. char str[6] = "Geeks";
3. char str[] = {'G', 'e', 'e', 'k', 's', '\0'};
4. char str[6] = {'G', 'e', 'e', 'k', 's', '\0'};
```

Below is the memory representation of a string "Geeks" in C++.



Let's look at some examples to better understand the string representation in C++, using C style:

C++

```
// C++ program to demonstrate
// Strings using C style

#include <iostream>
using namespace std;

int main()
{

        // Declare and initialize string
        char str[] = "Geeks";

        // Print string
        cout << str;

        return 0;
}
```

Output

```
Geeks
```

Standard String representation and String Class: In C++, one can directly store the collection of characters or text in a string variable, surrounded by double-quotes. C++ provides string class, which supports various operations like copying strings, concatenating strings etc.

Initializing string in C++:

```
1. string str1 = "Geeks";
2. string str2 = "Welcome to GeeksforGeeks!";
```

Example:

C++

```cpp
// C++ program to demonstrate String
// using Standard String representation

#include <iostream>
#include <string>
using namespace std;

int main()
{

        // Declare and initialize the string
        string str1 = "Welcome to GeeksforGeeks!";

        // Initialization by raw string
        string str2("A Computer Science Portal");

        // Print string
        cout << str1 << endl << str2;

        return 0;
}
```

Output

```
Welcome to GeeksforGeeks!
A Computer Science Portal
```

Check for Palindrome in Cpp

A palindrome is a word, phrase, number, or other sequences of characters which reads the same forward and backward (ignoring spaces, punctuation, and capitalization). In this article, we'll discuss how to check if a given string is a palindrome or not in C++.

Method 1: Using Iterators One approach to check if a string is a palindrome is to use iterators to iterate over the characters in the string and compare them. Here's an example of how to use iterators to check if a string is a palindrome:

C++

```cpp
#include <iostream>
#include <string>
using namespace std;

bool isPalindrome(string input) {
```

```
    string::iterator begin = input.begin();
    string::iterator end = input.end() - 1;
    while (begin < end) {
        if (*begin != *end) {
            return false;
        }
        begin++;
        end--;
    }
    return true;
}

int main() {
    string input;
    cout << "Enter a string: ";
    cin >> input;
    if (isPalindrome(input)) {
        cout << input << " is a palindrome" << endl;
    } else {
        cout << input << " is not a palindrome" << endl;
    }
    return 0;
}
```

In this example, the **isPalindrome()** function takes a string as input and uses iterators to iterate over the characters in the string from both the beginning and the end. It compares the characters at each position, and if any pair of characters is not equal, it returns false, indicating that the string is not a palindrome. If the function completes the iteration and all pairs of characters are equal, it returns true, indicating that the string is a palindrome.

Method 2: Using the **string.rbegin()** and **string.rend()** function Another approach to check if a string is a palindrome is to use the **string.rbegin()** and **string.rend()** function. This will give you the reverse iterator to the beginning and end of the string respectively.

C++

```
#include <iostream>
#include <string>
using namespace std;

bool isPalindrome(string input) {
    for (int i = 0; i < input.length(); i++) {
        if(input[i] != input[input.length() - i - 1])
            return false;
    }
    return true;
```

```
}

int main() {
    string input;
    cout << "Enter a string: ";
    cin >> input;
    if (isPalindrome(input)) {
        cout << input << " is a palindrome" << endl;
    } else {
        cout << input << " is not a palindrome" << endl;
    }
    return 0;
}
```

Searching

Binary Search (Recursive)

Problem: Given a sorted array **arr[]** of **n** elements, write a function to search a given element **x** in **arr[]** and return the index of x in the array.

Consider array is 0 base index.

Examples:

Input: *arr[] = {10, 20, 30, 50, 60, 80, 110, 130, 140, 170}, x = 110*
Output: *6*
Explanation: *Element x is present at index 6.*

Input: *arr[] = {10, 20, 30, 40, 60, 110, 120, 130, 170}, x = 175*
Output: *-1*
Explanation: *Element x is not present in arr[].*

Binary Search Approach:

Binary Search *is a searching algorithm used in a sorted array by* ***repeatedly dividing the search interval in half****. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(Log n).*

Binary Search Algorithm: The basic steps to perform Binary Search are:

- Begin with the mid element of the whole array as a search key.
- If the value of the search key is equal to the item then return an index of the search key.
- Or if the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half.
- Otherwise, narrow it to the upper half.
- Repeatedly check from the second point until the value is found or the interval is empty.

```
                high = mid - 1
```

Recursive Method (The recursive method follows the divide and conquer approach)

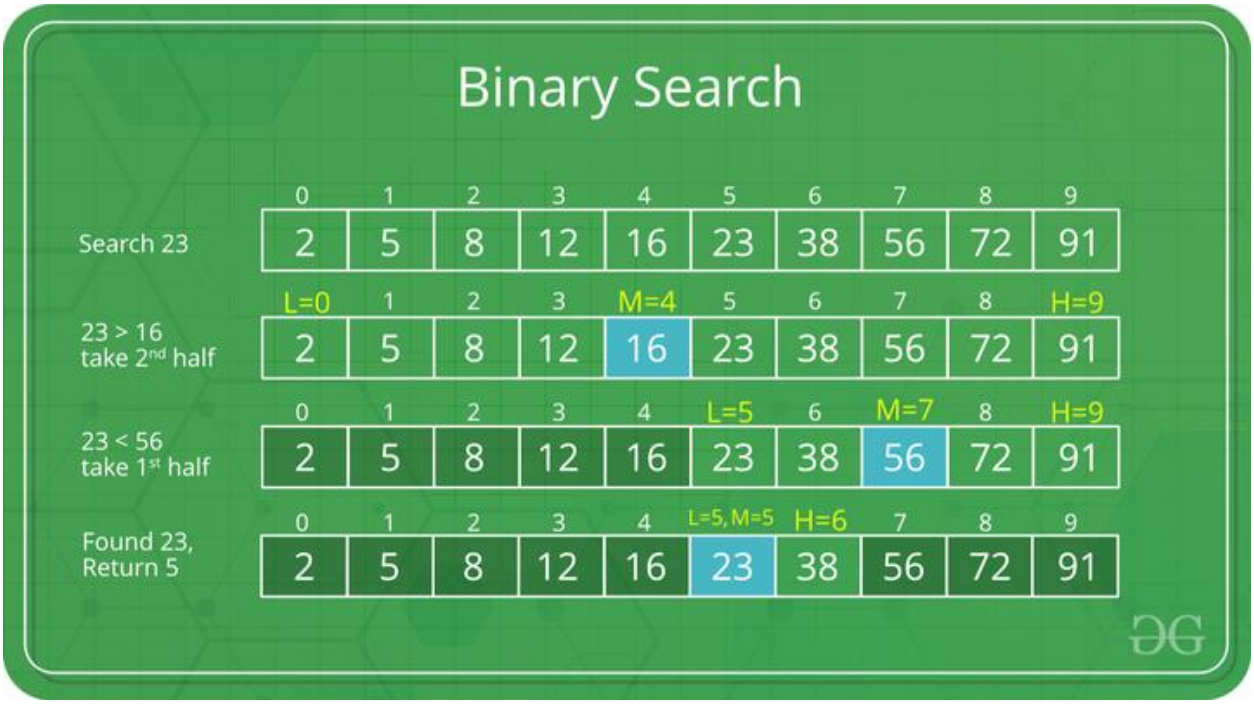
```
    binarySearch(arr, x, low, high)
           if low > high
               return False

           else
               mid = (low + high) / 2
                   if x == arr[mid]
                   return mid

               else if x > arr[mid]         // x is on the right
side
                   return binarySearch(arr, x, mid + 1, high)

               else                         // x is on the left
side
                   return binarySearch(arr, x, low, mid - 1)
```

Illustration of Binary Search Algorithm:



Example of Binary Search Algorithm

Step-by-step Binary Search Algorithm: We basically ignore half of the elements just after one comparison.

1. Compare x with the middle element.
2. If x matches with the middle element, we return the mid index.
3. Else If x is greater than the mid element, then x can only lie in the right half subarray after the mid element. So we recur for the right half.
4. Else (x is smaller) recur for the left half.

Recursive implementation of Binary Search:

C++Java

```
// C++ program to implement recursive Binary Search
#include <bits/stdc++.h>
using namespace std;

// A recursive binary search function. It returns
// location of x in given array arr[l..r] is present,
// otherwise -1
int binarySearch(int arr[], int l, int r, int x)
{
        if (r >= l) {
                int mid = l + (r - l) / 2;

                // If the element is present at the middle
                // itself
                if (arr[mid] == x)
```

```
                        return mid;

                // If element is smaller than mid, then
                // it can only be present in left subarray
                if (arr[mid] > x)
                        return binarySearch(arr, l, mid - 1, x);

                // Else the element can only be present
                // in right subarray
                return binarySearch(arr, mid + 1, r, x);
        }

        // We reach here when element is not
        // present in array
        return -1;
}

int main(void)
{
        int arr[] = { 2, 3, 4, 10, 40 };
        int x = 10;
        int n = sizeof(arr) / sizeof(arr[0]);
        int result = binarySearch(arr, 0, n - 1, x);
        (result == -1)
                ? cout << "Element is not present in array"
                : cout << "Element is present at index " << result;
        return 0;
}
```

Output

```
Element is present at index 3
```

Time Complexity: O(log n)
Auxiliary Space: O(log n)

Binary Search Functions in C++ STL

So far, we have discussed the Binary Search algorithm and its implementation by writing a function. The C++ standard template library have some built-in functions that can be used to perform Binary Search operation directly on a sequential list or container.

Some of these functions are:

- **binary_search()**
- **upper_bound()**
- **lower_bound()**

binary_search()

This function only checks if a particular element is present in a sorted container or not. It accepts the starting iterator, ending iterator and the element to be checked as parameters and returns True if the element is present otherwise False.

Syntax:

```
binary_search(start_ptr, end_ptr, ele)
```

Below program illustrate the working of binary_search() function with both Arrays and Vectors:

CPP

```cpp
// C++ code to demonstrate the working
// of binary_search()

#include<bits/stdc++.h>
using namespace std;

int main()
{
    /*** USING binary_search() ON VECTOR ***/

    // initializing vector of integers
    vector<int> vec = {10, 15, 20, 25, 30, 35};

    // using binary_search to check if 15 exists
    if (binary_search(vec.begin(), vec.end(), 15))
        cout << "15 exists in vector";
    else
        cout << "15 does not exist";

    cout << endl;

    /*** USING binary_search() ON ARRAYS ***/
    int arr[] = {10, 15, 20, 25, 30, 35};
    int n = sizeof(arr)/sizeof(arr[0]);

    // using binary_search to check if 20 exists
    if (binary_search(arr, arr + n, 20))
        cout << "20 exists in Array";
    else
```

```
        cout << "20 does not exist";

    return 0;
}
```

Output:

```
15 exists in vector
20 exists in Array
```

Note: This function only checks if the element is present or not, it does not give any information about the location of the element if it exists.

upper_bound()

The upper_bound() function is used to find the upper bound of an element present in a container. That is it finds the location of an element just greater than a given element in a container. This function accepts the start iterator, end iterator and the element to be checked as parameters and returns the iterator pointing to the element just greater than the element passed as the parameter. If there does not exist any such element than the function returns an iterator pointing to the last element.

Syntax:

```
upper_bound(first_itr, last_itr, ele)
```

Return Value: Returns an iterator pointing to the element just greater than *ele*.

Examples:

```
Input : 10 20 30 30 40 50
Output : upper_bound for element 30 will return
         an iterator pointing to the element 40.

Input : 10 20 30 40 50
Output : upper_bound for element 45 will return
         an iterator pointing to the element 50.

Input : 10 20 30 40 50
Output : upper_bound for element 60 will
         return end iterator.
```

Note: We can calculate the exact index position of the elements by subtracting the beginning iterator from the returned iterator.

Below program illustrate the working of upper_bound() function with both Vectors and Arrays:

CPP

```cpp
// CPP program to illustrate using upper_bound()
// with vectors and arrays

#include <bits/stdc++.h>
using namespace std;

// Driver code
int main()
{
    /*** USING upper_bound() WITH VECTOR ***/

    vector<int> v{ 10, 20, 30, 40, 50 };

    // Print vector
    cout << "Vector contains :";
    for (int i = 0; i < v.size(); i++)
        cout << " " << v[i];
    cout << "\n";

    vector<int>::iterator upper1, upper2;

    // std :: upper_bound
    upper1 = upper_bound(v.begin(), v.end(), 35);
    upper2 = upper_bound(v.begin(), v.end(), 45);

    cout << "\nUpper_bound for element 35 is at position : "
            << (upper1 - v.begin());
    cout << "\nUpper_bound for element 45 is at position : "
            << (upper2 - v.begin())<<"\n\n";


    /*** USING upper_bound() WITH ARRAY ***/

    int arr[] = { 10, 20, 30, 40, 50 };

    // Print elements of array
    cout << "Array contains :";
    for (int i = 0; i < 5; i++)
        cout << " " << arr[i];
    cout << "\n";

    // using upper_bound
    int up1 = upper_bound(arr, arr+5, 35) - arr;
    int up2 = upper_bound(arr, arr+5, 45) - arr;

    cout << "\nupper_bound for element 35 is at position : "
              << (up1);
```

```
    cout << "\nupper_bound for element 45 is at position : "
              << (up2);

    return 0;
}
```

Output:

```
Vector contains : 10 20 30 40 50

Upper_bound for element 35 is at position : 3
Upper_bound for element 45 is at position : 4

Array contains : 10 20 30 40 50

upper_bound for element 35 is at position : 3
upper_bound for element 45 is at position : 4
```

lower_bound()

The lower_bound() function is used to find the lower bound of an element present in a container. That is it finds the location of an element just smaller than a given element in a container. This function accepts the start iterator, end iterator and the element to be checked as parameters and returns the iterator pointing to the lower bound of the element passed as the parameter. If all elements of the container are smaller are less than the element passed, then it returns the last iterator.

Syntax:

```
lower_bound(first_itr, last_itr, ele)
```

Return Value: Returns an iterator pointing to the lower bound of the element *ele*. That is if *ele* exists in the container, it returns an iterator pointing to *ele* otherwise it returns an iterator pointing to the element just greater than ele.

Below program illustrate the working of lower_bound() function with both Vectors and Arrays:

CPP

```
// CPP program to illustrate lower_bound()
// for both vectors and array

#include <bits/stdc++.h>
using namespace std;

// Driver code
int main()
{
```

```
    /*** USING lower_bound() ON VECTORS ***/

    vector<int> v{ 10, 20, 30, 40, 50 };

    // Print vector
    cout << "Vector contains :";
    for (int i = 0; i < v.size(); i++)
        cout << " " << v[i];
    cout << "\n";

    vector<int>::iterator low1, low2;

    // using lower_bound()
    low1 = lower_bound(v.begin(), v.end(), 20);
    low2 = lower_bound(v.begin(), v.end(), 55);

    cout << "\nlower_bound for element 20 at position : "
         << (low1 - v.begin());
    cout << "\nlower_bound for element 55 at position : "
         << (low2 - v.begin());


    /*** USING lower_bound() ON ARRAYS ***/

    int arr[] = { 10, 20, 30, 40, 50 };

    // Print elements of array
    cout << "\n\nArray contains :";
    for (int i = 0; i < 5; i++)
        cout << " " << arr[i];
    cout << "\n";

    // using lower_bound()
    int lb1 = lower_bound(arr, arr + 5, 20) - arr;
    int lb2 = lower_bound(arr, arr + 5, 55) - arr;

    cout << "\nlower_bound for element 20 is at position : "
         << (lb1);
    cout << "\nlower_bound for element 55 is at position : "
         << (lb2);

    return 0;
}
```

Output:

```
Vector contains : 10 20 30 40 50

lower_bound for element 20 at position : 1
```

```
lower_bound for element 55 at position : 5

Array contains : 10 20 30 40 50

lower_bound for element 20 is at position : 1
lower_bound for element 55 is at position : 5
```

Square root

Given an integer **X**, find its square root. If **X** is not a perfect square, then return **floor(√x)**.

Examples :

Input: *x = 4*
Output: *2*
Explanation: *The square root of 4 is 2.*

Input: *x = 11*
Output: *3*
Explanation: *The square root of 11 lies in between 3 and 4 so floor of the square root is 3.*

Naive Approach: To find the floor of the square root, try with all-natural numbers starting from 1. Continue incrementing the number until the square of that number is greater than the given number.

Follow the steps below to implement the above idea

1. Create a variable (counter) ***i*** and take care of some base cases, (i.e when the given number is 0 or 1).
2. Run a loop until ***i*i <= n***, where n is the given number. Increment i by 1.
3. The floor of the square root of the number is *i – 1*

Below is the implementation of the above approach:

C++Java

```cpp
// A C++ program to find floor(sqrt(x)
#include <bits/stdc++.h>
using namespace std;

// Returns floor of square root of x
int floorSqrt(int x)
{
```

```
        // Base cases
        if (x == 0 || x == 1)
                return x;

        // Starting from 1, try all numbers until
        // i*i is greater than or equal to x.
        int i = 1, result = 1;
        while (result <= x) {
                i++;
                result = i * i;
        }
        return i - 1;
}

// Driver program
int main()
{
        int x = 11;
        cout << floorSqrt(x) << endl;
        return 0;
}
```

Output

```
3
```

Complexity Analysis:

- **Time Complexity:** O(√X). Only one traversal of the solution is needed, so the time complexity is O(√X).
- **Auxiliary Space:** O(1).

Square root an integer using Binary search:

*The idea is to find the largest integer **i** whose square is less than or equal to the given number. The values of **i * i** is monotonically increasing, so the problem can be solved using binary search.*

Below is the implementation of the above idea:

1. Base cases for the given problem are when the given number is **0** or **1**, then return **X**;
2. Create some variables, for storing the lower bound say ***l = 0,*** *and for upper bound* ***r = X / 2*** *(i.e, The floor of the square root of x cannot be more than x/2 when x > 1).*
3. Run a loop until ***l <= r***, the search space vanishes

4. Check if the square of mid **(*mid = (l + r)/2*)** is less than or equal to **X**, If yes then search for a larger value in the second half of the search space, i.e **l = mid + 1**, update **ans = mid**
5. Else if the square of mid is more than **X** then search for a smaller value in the first half of the search space, i.e **r = mid – 1**
6. Finally, Return the **ans**

Below is the implementation of the above approach:

C++Java

```
#include <iostream>

using namespace std;
int floorSqrt(int x)
{
        // Base cases
        if (x == 0 || x == 1)
                return x;

        // Do Binary Search for floor(sqrt(x))
        int start = 1, end = x / 2, ans;
        while (start <= end) {
                int mid = (start + end) / 2;

                // If x is a perfect square
                int sqr = mid * mid;
                if (sqr == x)
                        return mid;

                // Since we need floor, we update answer when
                // mid*mid is smaller than x, and move closer to
                // sqrt(x)

                /*
                if(mid*mid<=x)
                                {
                                                start = mid+1;
                                                ans = mid;
                                }
                        Here basically if we multiply mid with itself so
                there will be integer overflow which will throw
                tle for larger input so to overcome this
                situation we can use long or we can just divide
                        the number by mid which is same as checking
                mid*mid < x

                */
                if (sqr <= x) {
                        start = mid + 1;
```

```
                        ans = mid;
                }
                else // If mid*mid is greater than x
                        end = mid - 1;
        }
        return ans;
}

// Driver program
int main()
{
        int x = 11;
        cout << floorSqrt(x) << endl;
        return 0;
}
```

Output

```
3
```

Complexity Analysis:

- **Time Complexity:** O(log(X)).
- **Auxiliary Space:** O(1).

Find a Peak Element

Given an array **arr[]** of integers. Find a peak element i.e. an element that is **not smaller** than its neighbors.

Note: For corner elements, we need to consider only one neighbor.

Example:

Input: *array[]= {5, 10, 20, 15}*
Output: *20*
Explanation: *The element 20 has neighbors 10 and 15, both of them are less than 20.*

Input: *array[] = {10, 20, 15, 2, 23, 90, 67}*
Output: *20 or 90*

Explanation: *The element 20 has neighbors 10 and 15, both of them are less than 20, similarly 90 has neighbors 23 and 67.*

The following corner cases give a better idea about the problem.

1. If the input array is sorted in a strictly increasing order, the last element is always a peak element. For example, 50 is peak element in {10, 20, 30, 40, 50}.
2. If the input array is sorted in a strictly decreasing order, the first element is always a peak element. 100 is the peak element in {100, 80, 60, 50, 20}.
3. If all elements of the input array are the same, every element is a peak element.

It is clear from the above examples that there is always a peak element in the input array.

Naive Approach: Below is the idea to solve the problem

The array can be traversed and the element whose neighbors are less than that element can be returned.

Follow the below steps to Implement the idea:

- If the first element is greater than the second or the last element is greater than the second last, print the respective element and terminate the program.
- Else traverse the array from the second index to the second last index i.e. **1** to **N – 1**
    - If for an element array[i] is greater than both its neighbors, i.e., and , then print that element and terminate.

Below is the implementation of above idea.

C++Java

```
// A C++ program to find a peak element
#include <bits/stdc++.h>
using namespace std;

// Find the peak element in the array
int findPeak(int arr[], int n)
{
        // first or last element is peak element
        if (n == 1)
                return 0;
        if (arr[0] >= arr[1])
                return 0;
        if (arr[n - 1] >= arr[n - 2])
                return n - 1;

        // check for every other element
        for (int i = 1; i < n - 1; i++) {

                // check if the neighbors are smaller
```

```
                if (arr[i] >= arr[i - 1] && arr[i] >= arr[i + 1])
                        return i;
        }
}

// Driver Code
int main()
{
        int arr[] = { 1, 3, 20, 4, 1, 0 };
        int n = sizeof(arr) / sizeof(arr[0]);
        cout << "Index of a peak point is " << findPeak(arr, n);
        return 0;
}
```

Find a peak element using iterative Binary search

Below is the idea to solve the problem.

*Using Binary Search, check if the **middle** element is the peak element or not. If the middle element the peak element terminate the while loop and print **middle element**, then check if the element on the right side is greater than the middle element then there is always a peak element on the **right side**. If the element on the left side is greater than the middle element then there is always a peak element on the **left side**.*

Follow the steps below to implement the idea:

- Create two variables, l and r, initialize l = 0 and r = n-1
- Run a while loop till **l <= r**, lowerbound is less than the upperbound
    - Check if the mid value or index is the peak element or not, if yes then print the element and terminate.
    - Else if the element on the left side of the middle element is greater then check for peak element on the left side, i.e. update *r = mid – 1*
    - Else if the element on the right side of the middle element is greater then check for peak element on the right side, i.e. update

The below-given code is the iterative version of the above explained and demonstrated recursive based divide and conquer technique.

C++Java

```
// A C++ program to find a peak element
// using divide and conquer
#include <bits/stdc++.h>
```

```
using namespace std;

// A binary search based function
// that returns index of a peak element
int findPeak(int arr[], int n)
{
	int l = 0;
	int r = n-1;
	int mid;

	while (l <= r) {

		// finding mid by binary right shifting.
		mid = (l + r) >> 1;

		// first case if mid is the answer
		if ((mid == 0 || arr[mid - 1] <= arr[mid])
			and (mid == n - 1 || arr[mid + 1] <= arr[mid]))
			break;

		// move the right pointer
		if (mid > 0 and arr[mid - 1] > arr[mid])
			r = mid - 1;

		// move the left pointer
		else
			l = mid + 1;
	}

	return mid;
}

// Driver Code
int main()
{
	int arr[] = { 1, 3, 20, 4, 1, 0 };
	int N = sizeof(arr) / sizeof(arr[0]);
	cout << "Index of a peak point is " << findPeak(arr, N);
	return 0;
}
```

Output

```
Index of a peak point is 2
```

Time Complexity: O(log N), Where n is the number of elements in the input array. In each step our search becomes half. So it can be compared to Binary search, So the time

complexity is O(log N)
Auxiliary Space: O(1), No extra space is required, so the space complexity is constant.

Sorting

Introduction to Sorting

Sorting any sequence means to arrange the elements of that sequence according to some specific criterion.

For Example, the array arr[] = {5, 4, 2, 1, 3} after *sorting in increasing order* will be: arr[] = {1, 2, 3, 4, 5}. The same array after *sorting in descending order* will be: arr[] = {5, 4, 3, 2, 1}.

In-Place Sorting: An in-place sorting algorithm uses constant extra space for producing the output (modifies the given array only). It sorts the list only by modifying the order of the elements within the list.

In this tutorial, we will see three of such in-place sorting algorithms, namely:

- Insertion Sort
- Selection Sort
- Bubble Sort

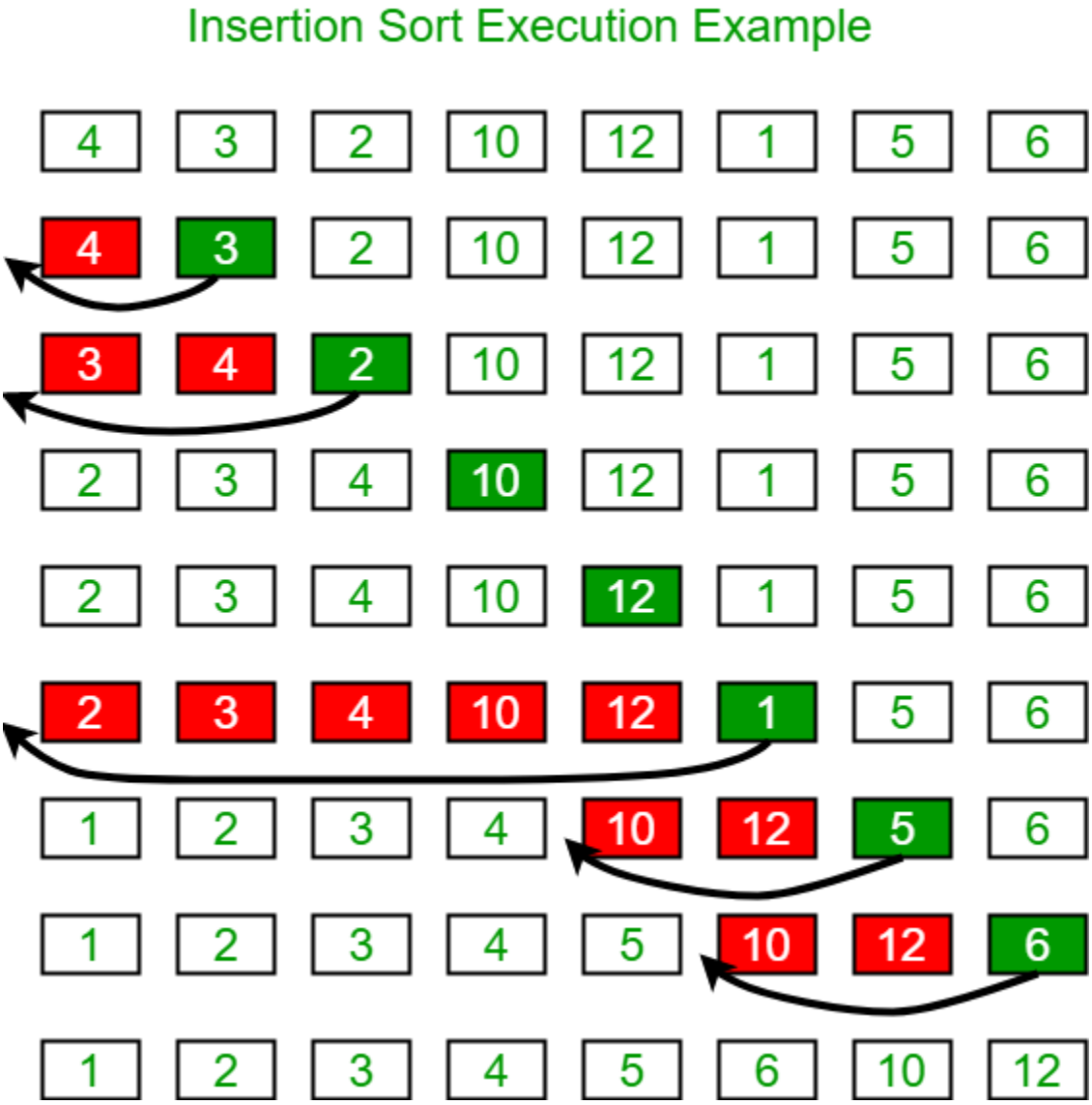
Insertion Sort

Insertion Sort is an In-Place sorting algorithm. This algorithm works in a similar way of sorting a deck of playing cards.

The idea is to start iterating from the second element of array till last element and for every element insert at its correct position in the subarray before it.

In the below image you can see, how the array [4, 3, 2, 10, 12, 1, 5, 6] is being sorted in

increasing order following the insertion sort algorithm.



Algorithm:

```
Step 1: If the current element is 1st element of array,
        it is already sorted.
Step 2: Pick next element
Step 3: Compare the current element with all elements
        in the sorted sub-array before it.
Step 4: Shift all of the elements in the sub-array before
        the current element which are greater than the current
        element by one place and insert the current element
        at the new empty space.
Step 5: Repeat step 2-3 until the entire array is sorted.
```

Another Example:
arr[] = {**12**, 11, 13, 5, 6}

Let us loop for i = 1 (second element of the array) to 4 (Size of input array - 1).

- *i = 1*, Since 11 is smaller than 12, move 12 and insert 11 before 12. **11, 12,** 13, 5, 6
- *i = 2*, 13 will remain at its position as all elements in A[0..I-1] are smaller than 13 **11, 12, 13,** 5, 6
- *i = 3*, 5 will move to the beginning and all other elements from 11 to 13 will move one position ahead of their current position. **5, 11, 12, 13,** 6
- *i = 4*, 6 will move to position after 5, and elements from 11 to 13 will move one position ahead of their current position. **5, 6, 11, 12, 13**

Function Implementation:

C++CJava

```
void insertionSort(int arr[], int n)
{

   for (int i = 1; i < n; i++)
   {
        int key = arr[i];
        int j = i-1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j+1] = arr[j];
            j = j-1;
        }
        arr[j+1] = key;
   }
}
```

Time Complexity: O(N^2), where N is the size of the array.

Bubble Sort

Bubble Sort is also an in-place sorting algorithm. This is the simplest sorting algorithm and it works on the principle that:

In one iteration if we swap all adjacent elements of an array such that after swap the first element is less than the second element then at the end of the iteration, the first element of the array will be the minimum element.

Bubble-Sort algorithm simply repeats the above steps N-1 times, where N is the size of the array.

Example: Consider the array, arr[] = *{6, 0, 3, 5}*

- **First Pass:**
  The largest element is placed in its correct position, i.e., the end of the array.

Bubble Sort Algorithm : Placing the largest element at correct position

- **Second Pass:**
  Place the second largest element at correct position

Bubble Sort Algorithm : Placing the second largest element at correct position

- **Third Pass:**
  Place the remaining two elements at their correct positions.

Bubble Sort Algorithm : Placing the remaining elements at their correct positions

Function Implementation:

C++CJava

```
void bubbleSort(int arr[], int n)
{
   for (int i = 0; i < n-1; i++)

       // Last i elements are already in place
       for (int j = 0; j < n-i-1; j++)
           if (arr[j] > arr[j+1])
              swap(&arr[j], &arr[j+1]);
}
```

Note: The above solution can be further optimized by keeping a flag to check if the array is already sorted in the first pass itself and to stop any further iteration.

Time Complexity: $O(N^2)$

Selection Sort

The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning. The algorithm maintains two subarrays in a given array.

1. The subarray which is already sorted.
2. Remaining subarray which is unsorted.

In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray.

Following example explains the above steps:

```
arr[] = 64 25 12 22 11.

// Find the minimum element in arr[0...4]
// and place it at beginning
11 25 12 22 64

// Find the minimum element in arr[1...4]
// and place it at beginning of arr[1...4]
11 12 25 22 64

// Find the minimum element in arr[2...4]
// and place it at beginning of arr[2...4]
11 12 22 25 64

// Find the minimum element in arr[3...4]
// and place it at beginning of arr[3...4]
11 12 22 25 64
```

Function Implementation:

C++CJava

```
void selectionSort(int arr[], int n)
{
    // One by one move boundary of unsorted subarray
    for (int i = 0; i < n-1; i++)
    {
        // Find the minimum element in unsorted array
        int min_idx = i;
        for (int j = i+1; j < n; j++)
          if (arr[j] < arr[min_idx])
            min_idx = j;

        // Swap the found minimum element with the first element
        swap(&arr[min_idx], &arr[i]);
    }
}
```

Time Complexity: $O(N^2)$

Stability in Sorting Algorithm

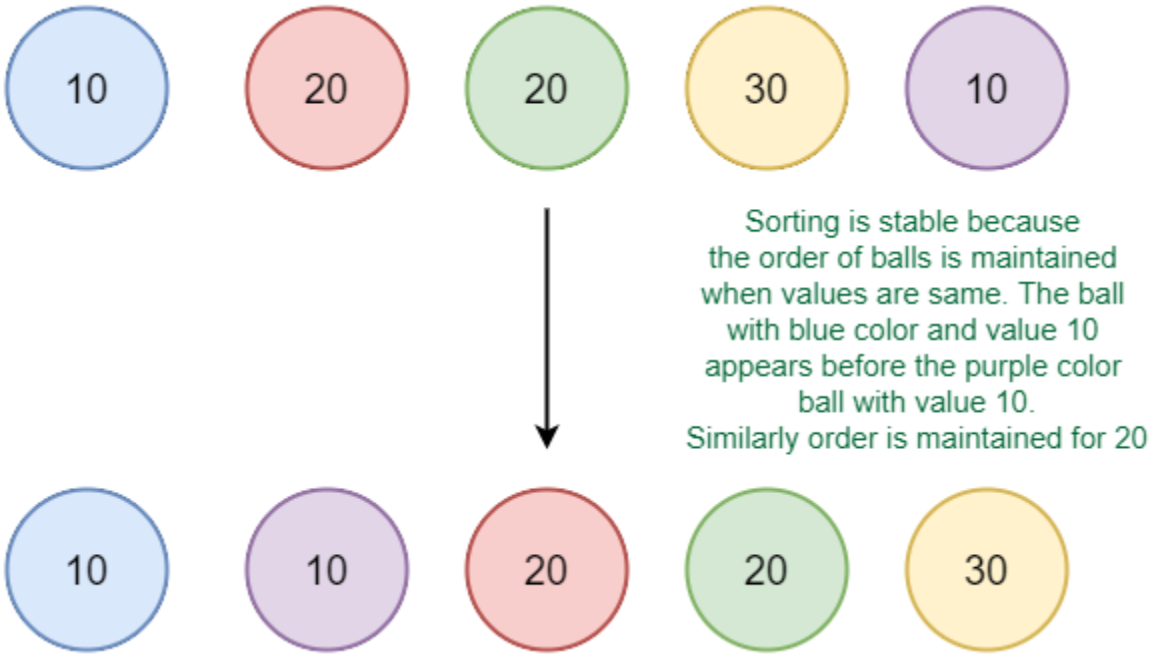
Stability is mainly essential when we have key-value pairs with duplicate keys possible (like people's names as keys and their details as values). And we wish to sort these objects by keys.

What is a stable sorting algorithm?

A sorting algorithm is said to be stable if two objects with equal keys appear in the same order in sorted output as they appear in the input data set

Formally stability may be defined as, how the algorithm treats equal elements. Let **A[]** be an array, and let **'<'** be a strict weak ordering on the elements of A[]. A sorting algorithm is stable if: i<jandA[i]≡A[j]impliesπ(i)<π(j)where πis the sorting permutation (sorting moves A[i]to position π(i)) .

Informally, stability means that equivalent elements retain their relative positions, after sorting.



Example of stable sort

Do we care for simple arrays like the array of integers?

When equal elements are indistinguishable, such as with integers, or more generally, any data where the entire element is the key, stability is not an issue. Stability is also not an issue if all keys are different.

Where stable sorting algorithms are useful?

Consider the following dataset of Student Names and their respective class sections.

(Dave,A)(Alice,B)(Ken,A)(Eric,B)(Carol,A)

If we sort this data according to name only, then it is highly unlikely that the resulting dataset will be grouped according to sections as well.

(Alice,B)(Carol,A)(Dave,A)(Eric,B)(Ken,A)

So we might have to sort again to obtain the list of students section-wise too. But in doing so, if the sorting algorithm is not stable, we might get a result like this:

(Carol,A)(Dave,A)(Ken,A)(Eric,B)(Alice,B)

The dataset is now sorted according to sections, but not according to names. In the name-sorted dataset, the tuple (Alice, B) was before (Eric, B) , but since the sorting algorithm is not stable, the relative order is lost. If on the other hand, we used a stable sorting algorithm, the result would be:

(Carol,A)(Dave,A)(Ken,A)(ALice,B)(Eric,B)

Here the relative order between different tuples is maintained. It may be the case that the relative order is maintained in an Unstable Sort but that is highly unlikely.

Which sorting algorithms are stable?

Some Sorting Algorithms are stable by nature, such as Bubble Sort, Insertion Sort, Merge Sort, Count Sort, etc. Comparison-based stable sorts such as Merge Sort and Insertion Sort maintain stability by ensuring that- Element A[j]comes before A[i]if and only if A[j]<A[i],

here i, j are indices, and i<j. The relative order is preserved i.e. comes before .

Other non-comparison-based sorts such as Counting Sort maintain stability by ensuring that the Sorted Array is filled in reverse order so that elements with equivalent keys have the same relative position. Some sorts such as Radix Sort depend on another sort, with the only requirement that the other sort should be stable.

Which sorting algorithms are unstable?

Quick Sort, Heap Sort etc., can be made stable by also taking the position of the elements into consideration. This change may be done in a way that does not compromise a lot on the performance and takes some extra space, possibly $\theta(n)$.

Can we make any sorting algorithm stable?

Any given sorting algorithm which is not stable can be modified to be stable. There can be algorithm-specific ways to make it stable, but in general, any comparison-based sorting algorithm which is not stable by nature can be modified to be stable by changing the key comparison operation so that the comparison of two keys considers position as a factor for objects with equal keys.

Bubble Sort

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst case time complexity is quite high.

How Bubble Sort Works?

Consider an array arr[] = {5, 1, 4, 2, 8}

First Pass:

- Bubble sort starts with very first two elements, comparing them to check which one is greater.
    - (**5 1** 4 2 8) --> (**1 5** 4 2 8), Here, algorithm compares the first two elements, and swaps since 5 > 1.
    - (1 **5 4** 2 8) --> (1 **4 5** 2 8), Swap since 5 > 4
    - (1 4 **5 2** 8) --> (1 4 **2 5** 8), Swap since 5 > 2
    - (1 4 2 **5 8**) --> (1 4 2 **5 8**), Now, since these elements are already in order (8 > 5), algorithm does not swap them.

Second Pass:

- Now, during second iteration it should look like this:
    - (**1 4** 2 5 8) --> (**1 4** 2 5 8)
    - (1 **4 2** 5 8) --> (1 **2 4** 5 8), Swap since 4 > 2
    - (1 2 **4 5** 8) --> (1 2 **4 5** 8)
    - (1 2 4 **5 8**) --> (1 2 4 **5 8**)

Third Pass:

- Now, the array is already sorted, but our algorithm does not know if it is completed.
- The algorithm needs one **whole** pass without **any** swap to know it is sorted.
    - (**1 2** 4 5 8) --> (**1 2** 4 5 8)
    - (1 **2 4** 5 8) --> (1 **2 4** 5 8)
    - (1 2 **4 5** 8) --> (1 2 **4 5** 8)
    - (1 2 4 **5 8**) --> (1 2 4 **5 8**)

Illustration:

i = 0	j	0	1	2	3	4	5	6	7
	0	5	3	1	9	8	2	4	7
	1	3	5	1	9	8	2	4	7
	2	3	1	5	9	8	2	4	7
	3	3	1	5	9	8	2	4	7
	4	3	1	5	8	9	2	4	7
	5	3	1	5	8	2	9	4	7
	6	3	1	5	8	2	4	9	7
i=1	0	3	1	5	8	2	4	7	9
	1	1	3	5	8	2	4	7	
	2	1	3	5	8	2	4	7	
	3	1	3	5	8	2	4	7	
	4	1	3	5	2	8	4	7	
	5	1	3	5	2	4	8	7	
i=2	0	1	3	5	2	4	7	8	
	1	1	3	5	2	4	7		
	2	1	3	5	2	4	7		
	3	1	3	2	5	4	7		
	4	1	3	2	4	5	7		
i=3	0	1	3	2	4	5	7		
	1	1	3	2	4	5			
	2	1	2	3	4	5			
	3	1	2	3	4	5			
i=4	0	1	2	3	4	5			
	1	1	2	3	4				
	2	1	2	3	4				
i=5	0	1	2	3	4				
	1	1	2	3					
i=6	0	1	2	3					
		1	2						

C++Java

```
// C++ program for implementation
// of Bubble sort
#include <bits/stdc++.h>
using namespace std;

// A function to implement bubble sort
void bubbleSort(int arr[], int n)
{
    int i, j;
```

```
    for (i = 0; i < n - 1; i++)

        // Last i elements are already
        // in place
        for (j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
}

// Function to print an array
void printArray(int arr[], int size)
{
    int i;
    for (i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

// Driver code
int main()
{
    int arr[] = { 5, 1, 4, 2, 8};
    int N = sizeof(arr) / sizeof(arr[0]);
    bubbleSort(arr, N);
    cout << "Sorted array: \n";
    printArray(arr, N);
    return 0;
}
// This code is contributed by rathbhupendra
```

Output

```
Sorted array:
1 2 4 5 8
```

Time Complexity: O(n^2), where n is the number of elements in the input array.
Space Complexity: O(1)

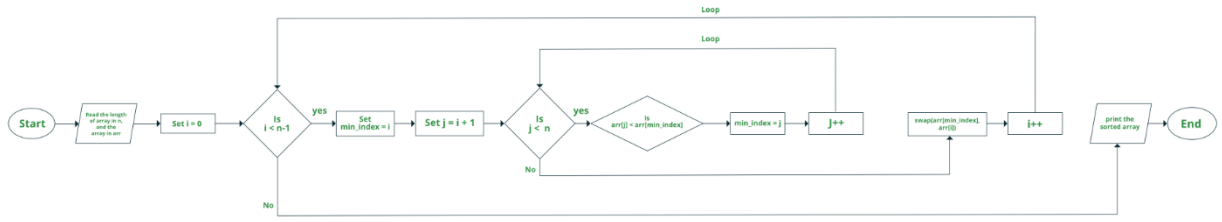
Selection Sort

The **selection sort algorithm** sorts an array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning. The algorithm maintains two subarrays in a given array.

- The subarray which is already sorted.
- Remaining subarray which is unsorted.

In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray.

Flowchart of the Selection Sort:



Flowchart for Selection Sort

How selection sort works?

Lets consider the following array as an example: **arr[] = {64, 25, 12, 22, 11}**

First pass:

- For the first position in the sorted array, the whole array is traversed from index 0 to 4 sequentially. The first position where **64** is stored presently, after traversing whole array it is clear that **11** is the lowest value.

64	25	12	22	11

- Thus, replace 64 with 11. After one iteration **11**, which happens to be the least value in the array, tends to appear in the first position of the sorted list.

11	25	12	22	64

Second Pass:

- For the second position, where 25 is present, again traverse the rest of the array in a sequential manner.

11	**25**	12	22	64

- After traversing, we found that **12** is the second lowest value in the array and it should appear at the second place in the array, thus swap these values.

11	**12**	25	22	64

Third Pass:

- Now, for third place, where **25** is present again traverse the rest of the array and find the third least value present in the array.

11	12	**25**	22	64

- While traversing, **22** came out to be the third least value and it should appear at the third place in the array, thus swap **22** with element present at third position.

11	12	**22**	25	64

Fourth pass:

- Similarly, for fourth position traverse the rest of the array and find the fourth least element in the array
- As **25** is the 4th lowest value hence, it will place at the fourth position.

11	12	22	**25**	64

Fifth Pass:

- At last the largest value present in the array automatically get placed at the last position in the array

- The resulted array is the sorted array.

11	12	22	25	64

Approach:

- Initialize minimum value(min_idx) to location 0
- Traverse the array to find the minimum element in the array
- While traversing if any element smaller than **min_idx** is found then swap both the values.
- Then, increment min_idx to point to next element
- Repeat until array is sorted

Below is the implementation of the above approach:

C++Java

```
// C++ program for implementation of
// selection sort
#include <bits/stdc++.h>
using namespace std;

//Swap function
void swap(int *xp, int *yp)
{
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}

void selectionSort(int arr[], int n)
{
    int i, j, min_idx;

    // One by one move boundary of
    // unsorted subarray
    for (i = 0; i < n-1; i++)
    {

        // Find the minimum element in
        // unsorted array
        min_idx = i;
        for (j = i+1; j < n; j++)
        if (arr[j] < arr[min_idx])
            min_idx = j;

        // Swap the found minimum element
        // with the first element
        swap(&arr[min_idx], &arr[i]);
```

```
    }
}

//Function to print an array
void printArray(int arr[], int size)
{
    int i;
    for (i=0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

// Driver program to test above functions
int main()
{
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr)/sizeof(arr[0]);
    selectionSort(arr, n);
    cout << "Sorted array: \n";
    printArray(arr, n);
    return 0;
}
// This is code is contributed by rathbhupendra
```

Output

```
Sorted array:
11 12 22 25 64
```

Time Complexity: O(n*n), where n is the number of elements in the input array.
Space Complexity: O(1)

Insertion Sort

Insertion sort is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.

Characteristics of Insertion Sort:

- This algorithm is one of the simplest algorithm with simple implementation
- Basically, Insertion sort is efficient for small data values
- Insertion sort is adaptive in nature, i.e. it is appropriate for data sets which are already partially sorted.

Working of Insertion Sort algorithm:

Consider an example: arr[]: {12, 11, 13, 5, 6}

12 11 13 5 6

First Pass:

- *Initially, the first two elements of the array are compared in insertion sort.*

12	***11***	*13*	*5*	*6*

- *Here, 12 is greater than 11 hence they are not in the ascending order and 12 is not at its correct position. Thus, swap 11 and 12.*
- *So, for now 11 is stored in a sorted sub-array.*

11	***12***	*13*	*5*	*6*

Second Pass:

- *Now, move to the next two elements and compare them*

11	***12***	***13***	*5*	*6*

- *Here, 13 is greater than 12, thus both elements seems to be in ascending order, hence, no swapping will occur. 12 also stored in a sorted sub-array along with 11*

Third Pass:

- *Now, two elements are present in the sorted sub-array which are **11** and **12***
- *Moving forward to the next two elements which are 13 and 5*

11	*12*	***13***	***5***	*6*

- *Both 5 and 13 are not present at their correct place so swap them*

11	*12*	***5***	***13***	*6*

- *After swapping, elements 12 and 5 are not sorted, thus swap again*

11	***5***	***12***	*13*	*6*

- *Here, again 11 and 5 are not sorted, hence swap again*

5	***11***	*12*	*13*	*6*

- *here, it is at its correct position*

Fourth Pass:

- *Now, the elements which are present in the sorted sub-array are **5, 11** and **12***
- *Moving to the next two elements 13 and 6*

5	*11*	*12*	***13***	***6***

- *Clearly, they are not sorted, thus perform swap between both*

5	*11*	*12*	***6***	***13***

- *Now, 6 is smaller than 12, hence, swap again*

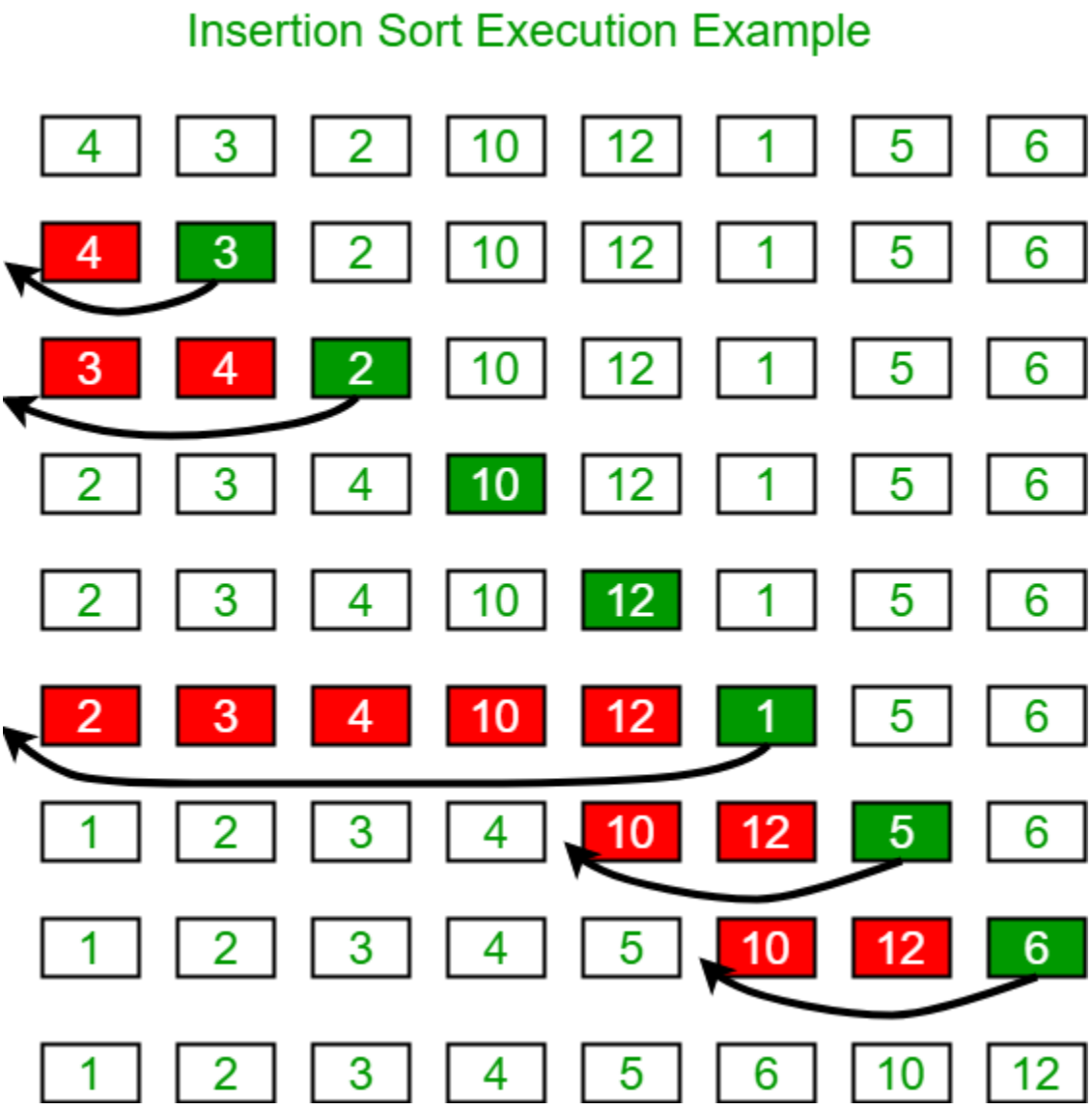
5	*11*	***6***	***12***	*13*

- *Here, also swapping makes 11 and 6 unsorted hence, swap again*

5	***6***	***11***	*12*	*13*

- *Finally, the array is completely sorted.*

Illustrations:



Insertion Sort Algorithm

To sort an array of size N in ascending order:

- Iterate from arr[1] to arr[N] over the array.
- Compare the current element (key) to its predecessor.
- If the key element is smaller than its predecessor, compare it to the elements before. Move the greater elements one position up to make space for the swapped element.

Implementation

C++Java

```
// C++ program for insertion sort
```

```cpp
#include <bits/stdc++.h>
using namespace std;

// Function to sort an array using
// insertion sort
void insertionSort(int arr[], int n)
{
    int i, key, j;
    for (i = 1; i < n; i++)
    {
        key = arr[i];
        j = i - 1;

        // Move elements of arr[0..i-1],
        // that are greater than key, to one
        // position ahead of their
        // current position
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

// A utility function to print an array
// of size n
void printArray(int arr[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

// Driver code
int main()
{
    int arr[] = { 12, 11, 13, 5, 6 };
    int N = sizeof(arr) / sizeof(arr[0]);

    insertionSort(arr, N);
    printArray(arr, N);

    return 0;
}
// This is code is contributed by rathbhupendra
```

Output

```
5 6 11 12 13
```

Time Complexity: O(N^2)
Auxiliary Space: O(1)

Merge function of Merge sort

Given two sorted arrays, the task is to merge them in a sorted manner.
Examples:

Input: arr1[] = { 1, 3, 4, 5}, arr2[] = {2, 4, 6, 8}
Output: arr3[] = {1, 2, 3, 4, 4, 5, 6, 8}
Input: arr1[] = { 5, 8, 9}, arr2[] = {4, 7, 8}
Output: arr3[] = {4, 5, 7, 8, 8, 9}

Method 1 (O(n1 * n2) Time and O(n1+n2) Extra Space)

1. Create an array arr3[] of size n1 + n2.
2. Copy all n1 elements of arr1[] to arr3[]
3. Traverse arr2[] and one by one insert elements (like insertion sort) of arr3[] to arr1[]. This step take O(n1 * n2) time.

We have discussed implementation of above method in Merge two sorted arrays with O(1) extra space

Method 2 (O(n1 + n2) Time and O(n1 + n2) Extra Space)
The idea is to use Merge function of Merge sort.

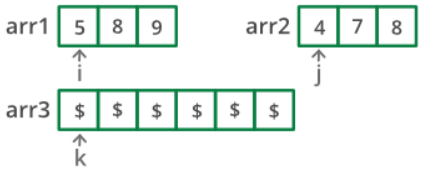
1. Create an array arr3[] of size n1 + n2.
2. Simultaneously traverse arr1[] and arr2[].

- Pick smaller of current elements in arr1[] and arr2[], copy this smaller element to next position in arr3[] and move ahead in arr3[] and the array whose element is picked.

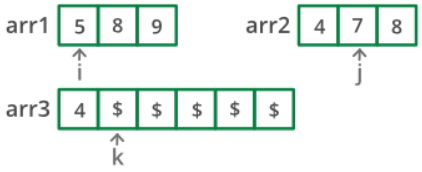
3. If there are remaining elements in arr1[] or arr2[], copy them also in arr3[].

Below image is a dry run of the above approach:

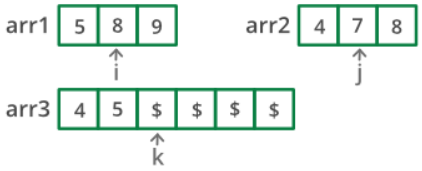
Initially:



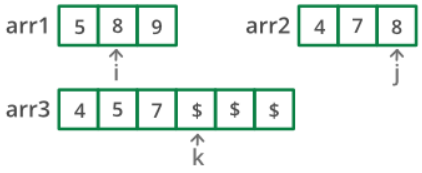
Step 1:



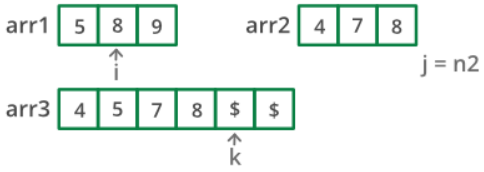
Step 2:



Step 3:



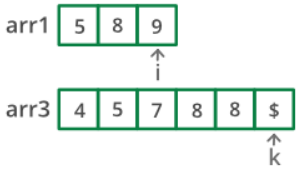
Step 4:



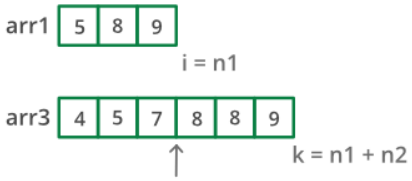
The first while loop breaks

Second while loop copies all elements from arr1 to arr3

Step 5:



Step 6:



Merged Sorted Array

Below is the implementation of the above approach:

C++Java

```cpp
// C++ program to merge two sorted arrays/
#include<iostream>
using namespace std;

// Merge arr1[0..n1-1] and arr2[0..n2-1] into
// arr3[0..n1+n2-1]
void mergeArrays(int arr1[], int arr2[], int n1,
                 int n2, int arr3[])
{
    int i = 0, j = 0, k = 0;

    // Traverse both array
    while (i<n1 && j <n2)
    {
        // Check if current element of first
        // array is smaller than current element
        // of second array. If yes, store first
        // array element and increment first array
        // index. Otherwise do same with second array
        if (arr1[i] < arr2[j])
            arr3[k++] = arr1[i++];
        else
            arr3[k++] = arr2[j++];
    }

    // Store remaining elements of first array
    while (i < n1)
        arr3[k++] = arr1[i++];

    // Store remaining elements of second array
    while (j < n2)
        arr3[k++] = arr2[j++];
}

// Driver code
int main()
{
    int arr1[] = {1, 3, 5, 7};
    int n1 = sizeof(arr1) / sizeof(arr1[0]);

    int arr2[] = {2, 4, 6, 8};
    int n2 = sizeof(arr2) / sizeof(arr2[0]);
```

```
    int arr3[n1+n2];
    mergeArrays(arr1, arr2, n1, n2, arr3);

    cout << "Array after merging" <<endl;
    for (int i=0; i < n1+n2; i++)
        cout << arr3[i] << " ";

    return 0;
}
```

Output:

```
Array after merging
1 2 3 4 5 6 7 8
```

Time Complexity : O(n1 + n2)

Auxiliary Space : O(n1 + n2)

Method 3: Using Maps (O(nlog(n) + mlog(m)) Time and O(N) Extra Space)

1. Insert elements of both arrays in a map as keys.
2. Print the keys of the map.

Below is the implementation of above approach.

CPPJava

```
// C++ program to merge two sorted arrays
//using maps
#include<bits/stdc++.h>
using namespace std;

// Function to merge arrays
void mergeArrays(int a[], int b[], int n, int m)
{
    // Declaring a map.
    // using map as a inbuilt tool
    // to store elements in sorted order.
    map<int, bool> mp;

    // Inserting values to a map.
    for(int i = 0; i < n; i++)
    mp[a[i]] = true;
```

```
    for(int i = 0;i < m;i++)
    mp[b[i]] = true;

    // Printing keys of the map.
    for(auto i: mp)
    cout<< i.first <<" ";
}

// Driver Code
int main()
{
    int a[] = {1, 3, 5, 7}, b[] = {2, 4, 6, 8};

    int size = sizeof(a)/sizeof(int);
    int size1 = sizeof(b)/sizeof(int);

    // Function call
    mergeArrays(a, b, size, size1);

    return 0;
}

//This code is contributed by yashbeersingh42
```

Output:

```
1 2 3 4 5 6 7 8
```

Time Complexity: O(nlog(n) + mlog(m))
Auxiliary Space: O(N)

Merge Sort

Merge Sort

The Merge Sort algorithm is a sorting algorithm that is considered an example of the divide and conquer strategy. So, in this algorithm, the array is initially divided into two equal halves and then they are combined in a sorted manner. We can think of it as a recursive algorithm that continuously splits the array in half until it cannot be further divided. This means that if the array becomes empty or has only one element left, the dividing will stop, i.e. it is the base case to stop the recursion. If the array has multiple elements, we split the

array into halves and recursively invoke the merge sort on each of the halves. Finally, when both the halves are sorted, the merge operation is applied. Merge operation is the process of taking two smaller sorted arrays and combining them to eventually make a larger one.

Pseudocode :

- Declare left variable to 0 and right variable to n-1
- Find mid by medium formula. mid = (left+right)/2
- Call merge sort on (left,mid)
- Call merge sort on (mid+1,right)
- Continue till left is less than right
- Then call merge function to perform merge sort.

Algorithm:

step 1: start

step 2: declare array and left, right, mid variable

step 3: perform merge function.

mergesort(array,left,right)

if left > right

return

mid= (left+right)/2

mergesort(array, left, mid)

mergesort(array, mid+1, right)

merge(array, left, mid, right)

step 4: Stop

See the following C implementation for details.

MergeSort(arr[], l, r)

If r > l

Find the middle point to divide the array into two halves:

middle m = l + (r – l)/2

Call mergeSort for first half:

Call mergeSort(arr, l, m)

Call mergeSort for second half:

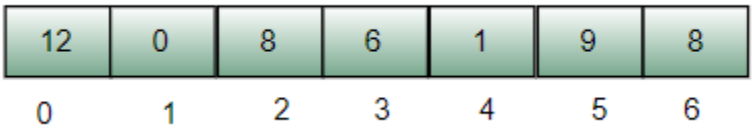
Call mergeSort(arr, m + 1, r)

Merge the two halves sorted in step 2 and 3:

Call merge(arr, l, m, r)

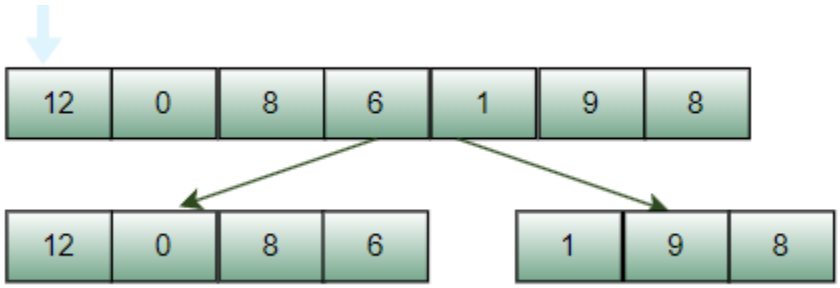
How Merge sort Works?

At first, check if the left index of array is less than the right index, if yes then calculate its mid point



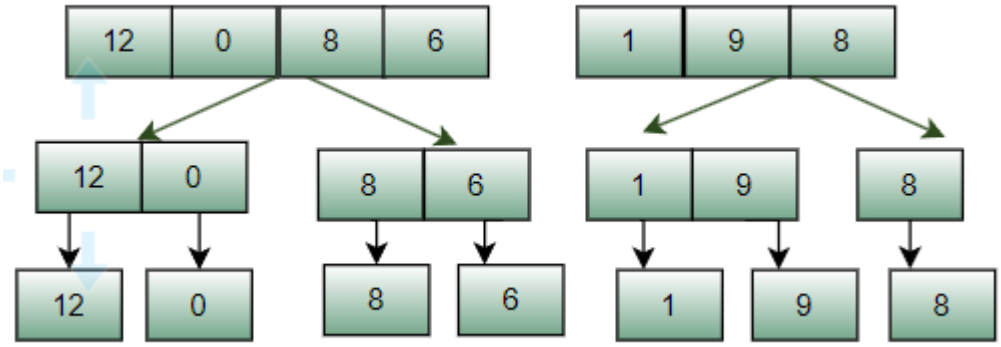
Now, as we already know that merge sort first divides the whole array iteratively into equal halves, unless the atomic values are achieved.

Here, we see that an array of 7 items is divided into two arrays of size 4 and 3 respectively.



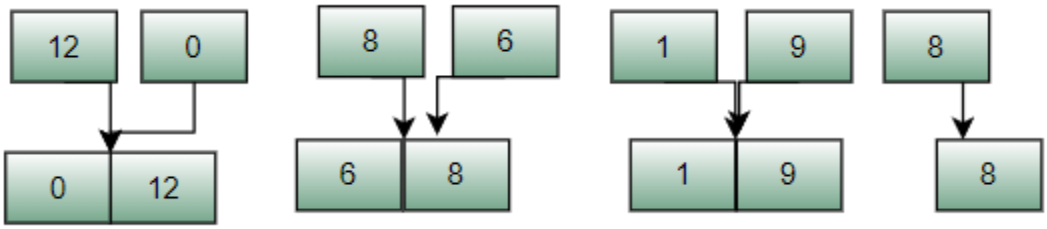
Now, again find that is left index is less than the right index for both arrays, if found yes, then again calculate mid points for both the arrays.

Now, further divide these two arrays into further halves, until the atomic units of the array is reached and further division is not possible.



After dividing the array into smallest units, start merging the elements again based on comparison of size of elements

Firstly, compare the element for each list and then combine them into another list in a sorted manner.



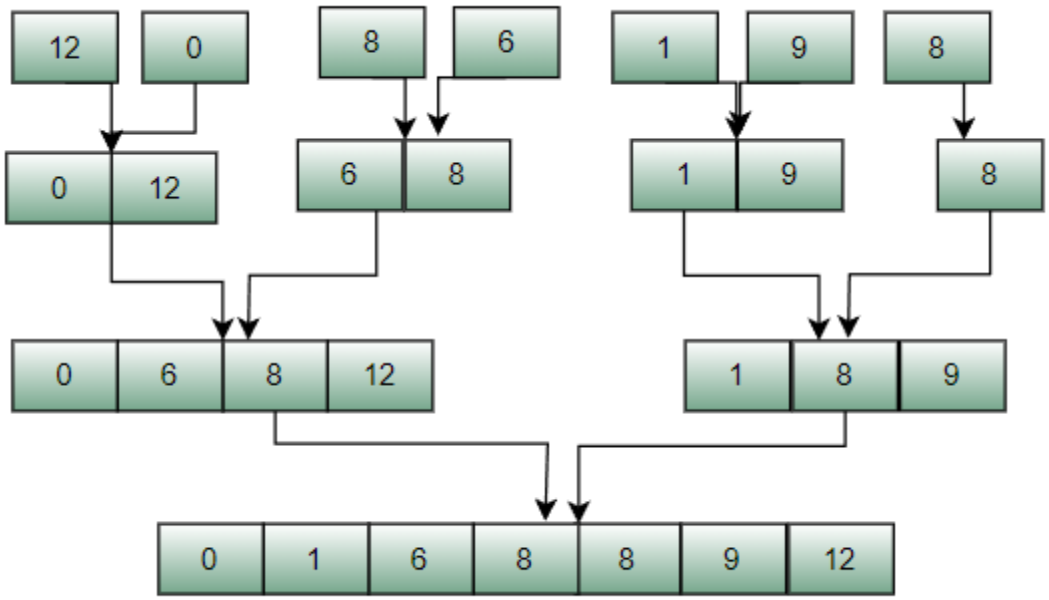
After the final merging, the list looks like this:



The following diagram shows the complete merge sort process for an example array {12, 0, 8, 6, 1, 9, 8}.

If we take a closer look at the diagram, we can see that the array is recursively divided into two halves till the size becomes 1.

Once the size becomes 1, the merge processes come into action and start merging arrays back till the complete array is merged.



Merge Sort Analysis

Like QuickSort, **Merge Sort** is a Divide and Conquer algorithm. It divides the input array into two halves, calls itself for the two halves, and then it merges the two sorted halves. **The merge() function** is used for merging two halves. The merge(arr, l, m, r) is a key process that assumes that arr[l..m] and arr[m+1..r] are sorted and merges the two sorted sub-arrays into one.

See the following C implementation for details.

MergeSort(arr[], l, r)

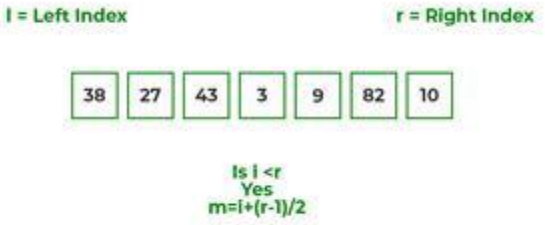
If r > l

- Find the middle point to divide the array into two halves:
  - middle m = l + (r - l)/2
- Call mergeSort for first half:
  - Call mergeSort(arr, l, m)
- Call mergeSort for second half:
  - Call mergeSort(arr, m + 1, r)
- Merge the two halves sorted in step 2 and 3:
  - Call merge(arr, l, m, r)

How Merge sort Works?

To know the functioning of merge sort, lets consider an array arr[] = {38, 27, 43, 3, 9, 82, 10}

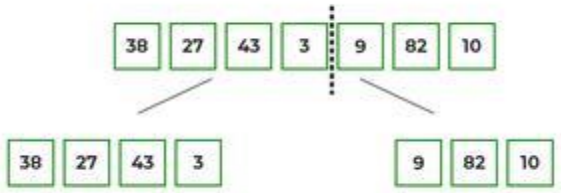
- At first, check if the left index of array is less than the right index, if yes then calculate its mid point



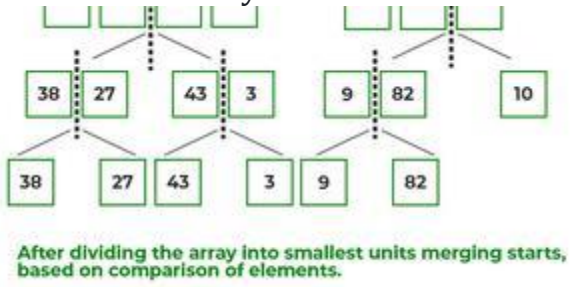
- Now, as we already know that merge sort first divides the whole array iteratively into equal halves, unless the atomic values are achieved.
- Here, we see that an array of 7 items is divided into two arrays of size 4 and 3 respectively.



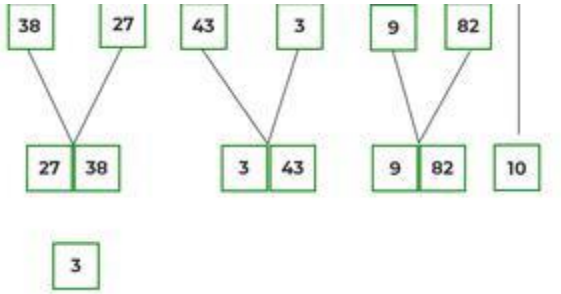
- Now, again find that is left index is less than the right index for both arrays, if found yes, then again calculate mid points for both the arrays.



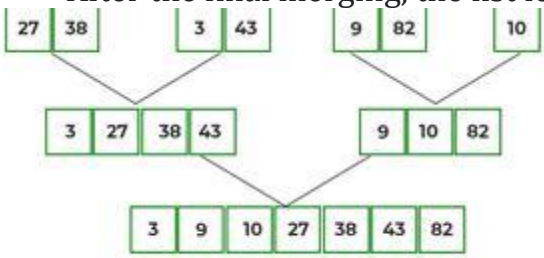
- Now, further divide these two arrays into further halves, until the atomic units of the array is reached and further division is not possible.



- After dividing the array into smallest units, start merging the elements again based on comparison of size of elements
- Firstly, compare the element for each list and then combine them into another list in a sorted manner.



- After the final merging, the list looks like this:

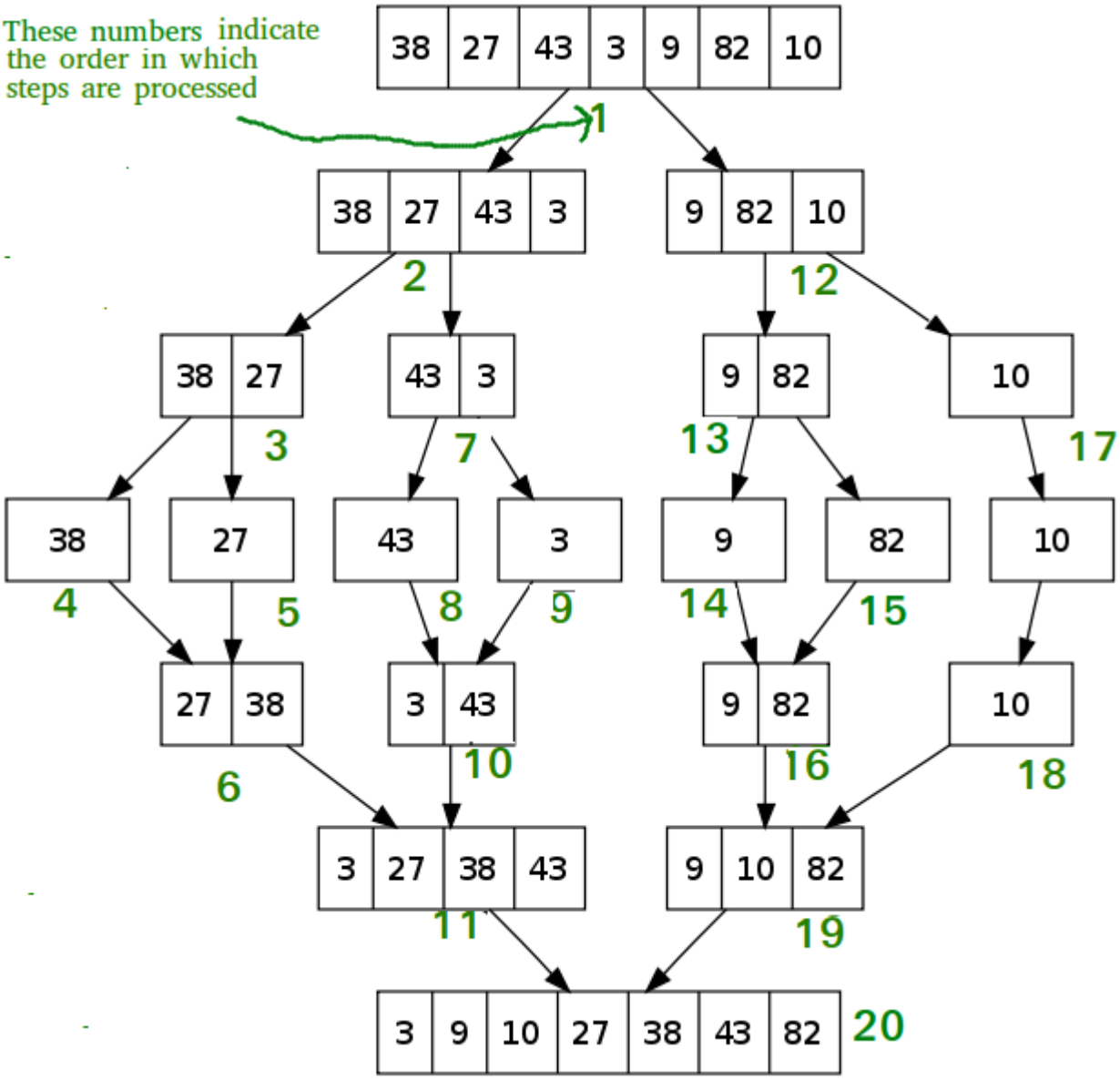


Refer to below illustrations for further clarity:

The following diagram shows the complete merge sort process for an example array {38, 27, 43, 3, 9, 82, 10}.

If we take a closer look at the diagram, we can see that the array is recursively divided into two halves till the size becomes

- Once the size becomes 1, the merge processes come into action and start merging arrays back till the complete array is merged.



C++CJavaPython3C#Javascript

```
// C++ program for Merge Sort
```

```cpp
#include <iostream>
using namespace std;

// Merges two subarrays of array[].
// First subarray is arr[begin..mid]
// Second subarray is arr[mid+1..end]
void merge(int array[], int const left, int const mid, int const right)
{
    auto const subArrayOne = mid - left + 1;
    auto const subArrayTwo = right - mid;

    // Create temp arrays
    auto *leftArray = new int[subArrayOne],
         *rightArray = new int[subArrayTwo];

    // Copy data to temp arrays leftArray[] and rightArray[]
    for (auto i = 0; i < subArrayOne; i++)
        leftArray[i] = array[left + i];
    for (auto j = 0; j < subArrayTwo; j++)
        rightArray[j] = array[mid + 1 + j];

    auto indexOfSubArrayOne = 0, // Initial index of first sub-array
        indexOfSubArrayTwo = 0; // Initial index of second sub-array
    int indexOfMergedArray = left; // Initial index of merged array

    // Merge the temp arrays back into array[left..right]
    while (indexOfSubArrayOne < subArrayOne && indexOfSubArrayTwo <
subArrayTwo) {
        if (leftArray[indexOfSubArrayOne] <=
rightArray[indexOfSubArrayTwo]) {
            array[indexOfMergedArray] = leftArray[indexOfSubArrayOne];
            indexOfSubArrayOne++;
        }
        else {
            array[indexOfMergedArray] = rightArray[indexOfSubArrayTwo];
            indexOfSubArrayTwo++;
        }
        indexOfMergedArray++;
    }
    // Copy the remaining elements of
    // left[], if there are any
    while (indexOfSubArrayOne < subArrayOne) {
        array[indexOfMergedArray] = leftArray[indexOfSubArrayOne];
        indexOfSubArrayOne++;
        indexOfMergedArray++;
    }
    // Copy the remaining elements of
    // right[], if there are any
    while (indexOfSubArrayTwo < subArrayTwo) {
```

```
        array[indexOfMergedArray] = rightArray[indexOfSubArrayTwo];
        indexOfSubArrayTwo++;
        indexOfMergedArray++;
    }
}

// begin is for left index and end is
// right index of the sub-array
// of arr to be sorted */
void mergeSort(int array[], int const begin, int const end)
{
    if (begin >= end)
        return; // Returns recursively

    auto mid = begin + (end - begin) / 2;
    mergeSort(array, begin, mid);
    mergeSort(array, mid + 1, end);
    merge(array, begin, mid, end);
}

// UTILITY FUNCTIONS
// Function to print an array
void printArray(int A[], int size)
{
    for (auto i = 0; i < size; i++)
        cout << A[i] << " ";
}

// Driver code
int main()
{
    int arr[] = { 12, 11, 13, 5, 6, 7 };
    auto arr_size = sizeof(arr) / sizeof(arr[0]);

    cout << "Given array is \n";
    printArray(arr, arr_size);

    mergeSort(arr, 0, arr_size - 1);

    cout << "\nSorted array is \n";
    printArray(arr, arr_size);
    return 0;
}

// This code is contributed by Mayank Tyagi
// This code was revised by Joshua Estes
```

Output

```
Given array is
12 11 13 5 6 7
Sorted array is
5 6 7 11 12 13
```

Time Complexity: O(n logn), Sorting arrays on different machines. Merge Sort is a recursive algorithm and time complexity can be expressed as following recurrence relation.
$T(n) = 2T(n/2) + \theta(n)$

The above recurrence can be solved either using the Recurrence Tree method or the Master method. It falls in case II of Master Method and the solution of the recurrence is $\theta(nLogn)$. Time complexity of Merge Sort is $\theta(nLogn)$ in all 3 cases (worst, average and best) as merge sort always divides the array into two halves and takes linear time to merge two halves.
Auxiliary Space: O(n)

Algorithmic Paradigm: Divide and Conquer

Is Merge sort In Place?

No in a typical implementation

Is Merge sort Stable:

Yes, merge sort is stabe.

Applications of Merge Sort:

- Merge Sort is useful for sorting linked lists in O(nLogn) time. In the case of linked lists, the case is different mainly due to the difference in memory allocation of arrays and linked lists. Unlike arrays, linked list nodes may not be adjacent in memory. Unlike an array, in the linked list, we can insert items in the middle in O(1) extra space and O(1) time. Therefore, the merge operation of merge sort can be implemented without extra space for linked lists.
  In arrays, we can do random access as elements are contiguous in memory. Let us say we have an integer (4-byte) array A and let the address of A[0] be x then to access A[i], we can directly access the memory at (x + i*4). Unlike arrays, we can not do random access in the linked list. Quick Sort requires a lot of this kind of access. In a linked list to access i'th index, we have to travel each and every node from the head to i'th node as we don't have a continuous block of memory. Therefore, the overhead increases for quicksort. Merge sort accesses data sequentially and the need of random access is low.
- Inversion Count Problem
- Used in External Sorting

Drawbacks of Merge Sort:

- Slower comparative to the other sort algorithms for smaller tasks.
- Merge sort algorithm requires an additional memory space of 0(n) for the temporary array.
- It goes through the whole process even if the array is sorted.

- Recent Articles on Merge Sort
- Coding practice for sorting.
- Quiz on Merge Sort

Other Sorting Algorithms on GeeksforGeeks:

3-way Merge Sort, Selection Sort, Bubble Sort, Insertion Sort, Merge Sort, Heap

Sort, QuickSort, Radix Sort, Counting Sort, Bucket Sort, ShellSort, Comb Sort
Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above.

QuickSort analysis

Is QuickSort stable?

The default implementation is not stable. However any sorting algorithm can be made stable by considering indexes as comparison parameter.

Is QuickSort In-place?

As per the broad definition of in-place algorithm it qualifies as an in-place sorting algorithm as it uses extra space only for storing recursive function calls but not for manipulating the input.

What is 3-Way QuickSort?

In simple QuickSort algorithm, we select an element as pivot, partition the array around pivot and recur for subarrays on left and right of pivot.
Consider an array which has many redundant elements. For example, {1, 4, 2, 4, 2, 4, 1, 2, 4, 1, 2, 2, 2, 2, 4, 1, 4, 4, 4}. If 4 is picked as pivot in Simple QuickSort, we fix only one 4 and recursively process remaining occurrences. In 3 Way QuickSort, an array arr[l..r] is divided in 3 parts:

- arr[l..i] elements less than pivot.
- arr[i+1..j-1] elements equal to pivot.
- arr[j..r] elements greater than pivot.

See this for implementation.

How to implement QuickSort for Linked Lists?

QuickSort on Singly Linked List
QuickSort on Doubly Linked List

Can we implement QuickSort Iteratively?

Yes, please refer Iterative Quick Sort.

Why Quick Sort is preferred over MergeSort for sorting Arrays ?

Quick Sort in its general form is an in-place sort (i.e. it doesn't require any extra storage) whereas merge sort requires O(N) extra storage, N denoting the array size which may be

quite expensive. Allocating and de-allocating the extra space used for merge sort increases the running time of the algorithm. Comparing average complexity we find that both type of sorts have O(NlogN) average complexity but the constants differ. For arrays, merge sort loses due to the use of extra O(N) storage space.
Most practical implementations of Quick Sort use randomized version. The randomized version has expected time complexity of O(nLogn). The worst case is possible in randomized version also, but worst case doesn't occur for a particular pattern (like sorted array) and randomized Quick Sort works well in practice.
Quick Sort is also a cache friendly sorting algorithm as it has good locality of reference when used for arrays.
Quick Sort is also tail recursive, therefore tail call optimizations is done.

Why MergeSort is preferred over QuickSort for Linked Lists ?

In case of linked lists the case is different mainly due to difference in memory allocation of arrays and linked lists. Unlike arrays, linked list nodes may not be adjacent in memory. Unlike array, in linked list, we can insert items in the middle in O(1) extra space and O(1) time. Therefore merge operation of merge sort can be implemented without extra space for linked lists.
In arrays, we can do random access as elements are continuous in memory. Let us say we have an integer (4-byte) array A and let the address of A[0] be x then to access A[i], we can directly access the memory at (x + i*4). Unlike arrays, we can not do random access in linked list. Quick Sort requires a lot of this kind of access. In linked list to access i'th index, we have to travel each and every node from the head to i'th node as we don't have continuous block of memory. Therefore, the overhead increases for quick sort. Merge sort accesses data sequentially and the need of random access is low.

How to optimize QuickSort so that it takes O(Log n) extra space in worst case?

Please see QuickSort Tail Call Optimization (Reducing worst case space to Log n

- Quiz on QuickSort
- Recent Articles on QuickSort
- Coding practice for sorting.

Advantages of Quick Sort:

- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It is a stable sort, meaning that if two elements have the same key, their relative order will be preserved in the sorted output.
- It has a low overhead, as it only requires a small amount of memory to function.

Disadvantages of Quick Sort:

- It has a worst-case time complexity of O(n^2), which occurs when the pivot is chosen poorly.

- It is not a good choice for small data sets.
- It can be sensitive to the choice of pivot.
- It is not cache-efficient.

Time Complexity

Time taken by QuickSort in general can be written as following.

```
T(n) = T(k) + T(n-k-1) + Θ(n)
```

The first two terms are for two recursive calls, the last term is for the partition process. k is the number of elements which are smaller than pivot.
The time taken by QuickSort depends upon the input array and partition strategy. Following are three cases.

Worst Case: The worst case occurs when the partition process always picks greatest or smallest element as pivot. If we consider above partition strategy where last element is always picked as pivot, the worst case would occur when the array is already sorted in increasing or decreasing order. Following is recurrence for worst case.

```
 T(n) = T(0) + T(n-1) + Θ(n)
which is equivalent to
 T(n) = T(n-1) + Θ(n)
```

The solution of above recurrence is $\Theta(n^2)$.

Best Case: The best case occurs when the partition process always picks the middle element as pivot. Following is recurrence for best case.

```
 T(n) = 2T(n/2) + Θ(n)
```

The solution of above recurrence is **Θ(nLogn)**. It can be solved using case 2 of Master Theorem.

Average Case:
To do average case analysis, we need to consider all possible permutation of array and calculate time taken by every permutation which doesn't look easy.
We can get an idea of average case by considering the case when partition puts O(n/9) elements in one set and O(9n/10) elements in other set. Following is recurrence for this case.

```
T(n) = T(n/9) + T(9n/10) + Θ(n)
```

Solution of above recurrence is also O(nLogn)

Although the worst case time complexity of QuickSort is $O(n^2)$ which is more than many other sorting algorithms like **Merge Sort** and **Heap Sort**, QuickSort is faster in practice, because its inner loop can be efficiently implemented on most architectures, and in most real-world data. QuickSort can be implemented in different ways by changing the choice of pivot, so that the worst case rarely occurs for a given type of data. However, merge sort is generally considered better when data is huge and stored in external storage.

Summary:

- Quick sort is a fast and efficient sorting algorithm with an average time complexity of O(n log n).
- It is a divide-and-conquer algorithm that breaks down the original problem into smaller subproblems that are easier to solve.
- It can be easily implemented in both iterative and recursive forms and it is efficient on large data sets, and can be used to sort data in-place.
- However, it also has some drawbacks such as worst case time complexity of O(n^2) which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets, it is not cache-efficient, and is sensitive to the choice of pivot.

Tail call elimination in Quick Sort

Tail Recursion a recursive function is tail recursive if the recursive call is the last thing executed by the function.

C++

```
#include <bits/stdc++.h>
using namespace std;
void print(int n)
{
    if (n < 0)
       return;
    cout<<n;

    // The last executed statement is recursive call
    print(n-1);
}
```

Tail-recursive is better than a non-tail recursive as tail-recursion can be optimized by modern compilers. Modern compiler basically does **tail call elimination** to optimize the tail-recursive code.

If we take a closer look at the above function, we can remove the last call with goto. Below are examples of tail call elimination.

C++

```
#include <bits/stdc++.h>
using namespace std;

void print(int n)
{
start:
    if (n < 0)
       return;
    cout<<n;

    // Update parameters of recursive call
    // and replace recursive call with goto
    n = n-1;
    goto start;
}
```

QuickSort : One more example

QuickSort is also tail recursive (Note that MergeSort is not tail recursive, this is also one of the reasons why QuickSort performs better)

C++

```
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        /* pi is partitioning index, arr[p] is now
           at right place */
        int pi = partition(arr, low, high);

        // Separately sort elements before
        // partition and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
```

```
}
```

The above function can be replaced by following after tail call elimination.

C++

```
void quickSort(int arr[], int low, int high)
{
start:
    if (low < high)
    {
        /* pi is partitioning index, arr[p] is now
           at right place */
        int pi = partition(arr, low, high);

        // Separately sort elements before
        // partition and after partition
        quickSort(arr, low, pi - 1);

        // Update parameters of recursive call
        // and replace recursive call with goto
        low = pi+1;
        high = high;
        goto start;
    }
}
```

Therefore job for compilers is to identify tail recursion, add a label at the beginning and update parameter(s) at the end followed by adding the last goto statement.

Function stack frame management in Tail Call Elimination :
Recursion uses a stack to keep track of function calls. With every function call, a new frame is pushed onto the stack which contains local variables and data of that call. Let's say one stack frame requires O(1) i.e, constant memory space, then for **N** recursive call memory required would be O(N).

Tail call elimination reduces the space complexity of recursion from O(N) to O(1). As function call is eliminated, no new stack frames are created and the function is executed in constant memory space.

It is possible for the function to execute in constant memory space because, in tail recursive function, there are no statements after call statement so preserving state and frame of

parent function is not required. Child function is called and finishes immediately, it doesn't have to return control back to the parent function.

As no computation is performed on the returned value and no statements are left for execution, the current frame can be modified as per the requirements of the current function call. So there is no need to preserve stack frames of previous function calls and function executes in constant memory space. This makes tail recursion faster and memory-friendly.

Stack

Stack Data Structure

Stack

It is a linear data structure that follows a particular order in which the operations are performed.

LIFO(Last In First Out):

This strategy states that the element that is inserted last will come out first. You can take a pile of plates kept on top of each other as a real-life example. The plate which we put last is on the top and since we remove the plate that is at the top, we can say that the plate that was put last comes out first.

Basic Operations on Stack

In order to make manipulations in a stack, there are certain operations provided to us :-

- **push()** to insert an element into the stack
- **pop()** to remove an element from the stack
- **top()** Returns the top element of the stack.
- **isEmpty()** returns true if stack is empty else false.
- **size()** returns the size of stack.

Push:

Adds an item to the stack. If the stack is full, then it is said to be an Overflow condition.

Algorithm for push:

```
begin
 if stack is full
    return
 endif
else
 increment top
 stack[top] assign value
end else
end procedure
```

Pop:

Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an Underflow condition.

Algorithm for pop:

```
begin
 if stack is empty
    return
 endif
else
 store value of stack[top]
 decrement top
 return value
end else
end procedure
```

Top:

Returns the top element of the stack.

Algorithm for Top:

```
begin
  return stack[top]
end procedure
```

isEmpty:

Returns true if the stack is empty, else false.

Algorithm for isEmpty:

```
begin
 if top < 1
    return true
 else
    return false
end procedure
```

Understanding stack practically:

There are many real-life examples of a stack. Consider the simple example of plates stacked over one another in a canteen. The plate which is at the top is the first one to be removed, i.e. the plate which has been placed at the bottommost position remains in the stack for the longest period of time. So, it can be simply seen to follow the LIFO/FILO order.

Complexity Analysis:

- **Time Complexity**

Operations Complexity

push()	O(1)
pop()	O(1)
isEmpty()	O(1)
size()	O(1)

Types of Stacks:

- **Register Stack:** This type of stack is also a memory element present in the memory unit and can handle a small amount of data only. The height of the register stack is always limited as the size of the register stack is very small compared to the memory.
- **Memory Stack:** This type of stack can handle a large amount of memory data. The height of the memory stack is flexible as it occupies a large amount of memory data.

Applications of Stack

Application of Stack Data Structure:

- Stack is used for evaluating expression with operands and operations.
- Matching tags in HTML and XML
- Undo function in any text editor.
- Infix to Postfix conversion.
- Stacks are used for backtracking and parenthesis matching.
- Stacks are used for conversion of one arithmetic notation to another arithmetic notation.
- Stacks are useful for function calls, storing the activation records and deleting them after returning from the function. It is very useful in processing the function calls.
- Stacks help in reversing any set of data or strings.
- To manage recursion, stack data structure is used to account for the previous state of the recursion call.

Application of Stack in real life:

- CD/DVD stand.
- Stack of books in a book shop.
- Undo and Redo mechanism in text editors.
- The history of a web browser is stored in the form of a stack.
- Call logs, E-mails, and Google photos in any gallery are also stored in form of a stack.
- YouTube downloads and Notifications are also shown in LIFO format(the latest appears first).

Advantages of Stack:

- Stack helps in managing data that follows the LIFO technique.
- Stacks are being used for systematic Memory Management.
- It is used in many virtual machines like JVM.
- When a function is called, the local variables and other function parameters are stored in the stack and automatically destroyed once returned from the function. Hence, efficient function management.
- Stacks are more secure and reliable as they do not get corrupted easily.
- Stack allows control over memory allocation and deallocation.
- Stack cleans up the objects automatically.
- Stacks are used to convert

Balanced Parenthesis

Given an expression string **exp**, write a program to examine whether the pairs and the orders of "{", "}", "(", ")", "[", "]" are correct in the given expression.

Example:

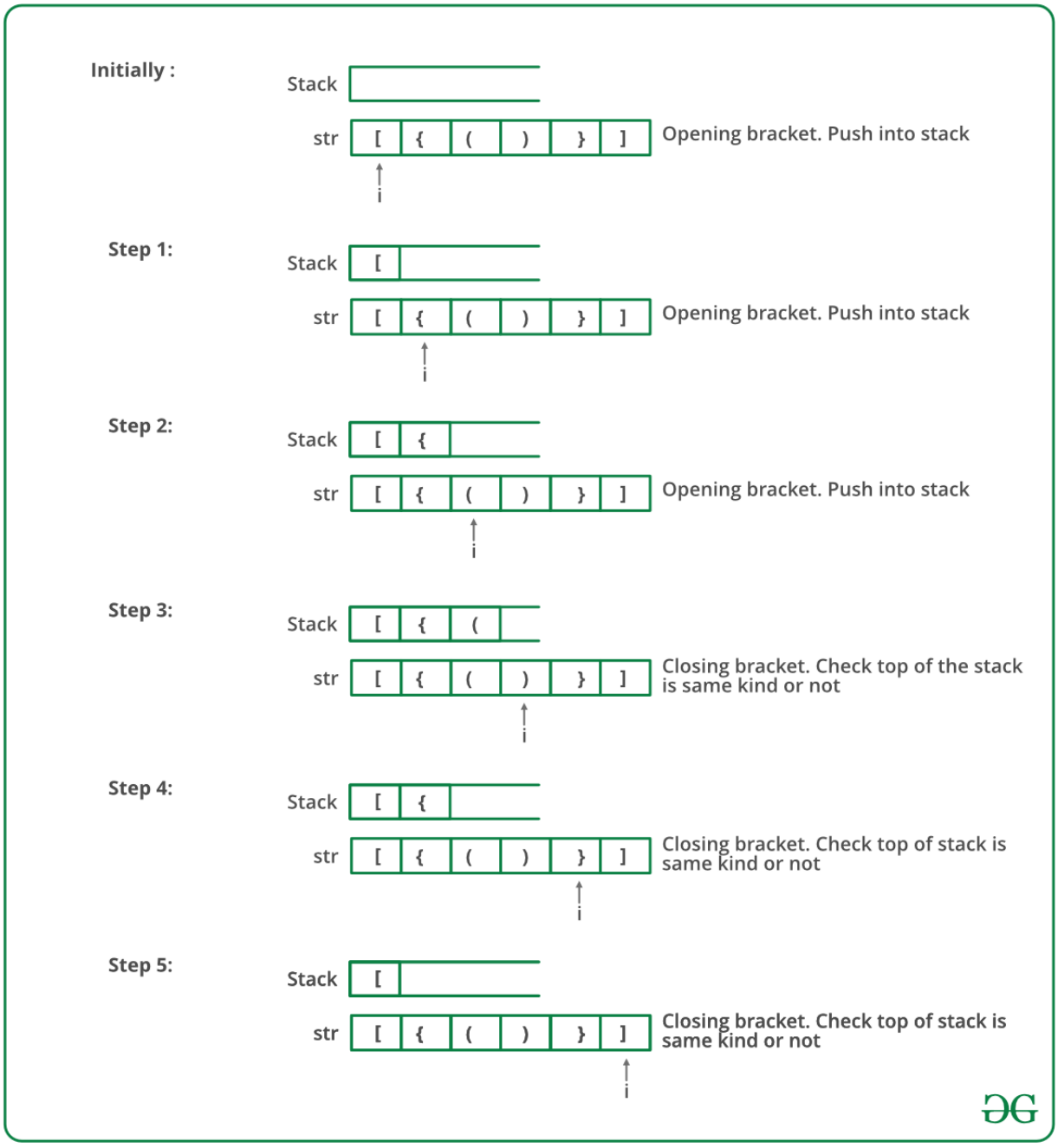
***Input**: exp = "[()]{}{[()()]()}"*
***Output**: Balanced*
***Explanation:** all the brackets are well-formed*

***Input**: exp = "[()]"*
***Output**: Not Balanced*
***Explanation: 1** and **4** brackets are not balanced because there is a closing ')' before the closing '('*

Check for Balanced Bracket expression using Stack:

The idea is to put all the opening brackets in the stack. Whenever you hit a closing bracket, search if the top of the stack is the opening bracket of the same nature. If this holds then pop the stack and continue the iteration, in the end if the stack is empty, it means all brackets are well-formed . Otherwise, they are not balanced.

Illustration:
Below is the illustration of the above approach.



Follow the steps mentioned below to implement the idea:

- Declare a character stack (say **temp**).
- Now traverse the string exp.
  - If the current character is a starting bracket (**'(' or '{' or '['**) then push it to stack.
  - If the current character is a closing bracket (**')' or '}' or ']'**) then pop from stack and if the popped character is the matching starting bracket then fine.
  - Else brackets are **Not Balanced**.

- After complete traversal, if there is some starting bracket left in stack then **Not balanced**, else **Balanced**.

Below is the implementation of the above approach:

C++Java

```
// C++ program to check for balanced brackets.

#include <bits/stdc++.h>
using namespace std;

// Function to check if brackets are balanced
bool areBracketsBalanced(string expr)
{
	// Declare a stack to hold the previous brackets.
	stack<char> temp;
	for (int i = 0; i < expr.length(); i++) {
		if (temp.empty()) {

			// If the stack is empty
			// just push the current bracket
			temp.push(expr[i]);
		}
		else if ((temp.top() == '(' && expr[i] == ')')
				|| (temp.top() == '{' && expr[i] == '}')
				|| (temp.top() == '[' && expr[i] == ']')) {

			// If we found any complete pair of bracket
			// then pop
			temp.pop();
		}
		else {
			temp.push(expr[i]);
		}
	}
	if (temp.empty()) {

		// If stack is empty return true
		return true;
	}
	return false;
}

// Driver code
int main()
{
	string expr = "{()}[]";

	// Function call
```

```
        if (areBracketsBalanced(expr))
                cout << "Balanced";
        else
                cout << "Not Balanced";
        return 0;
}
```

Output
Balanced

Time Complexity: O(N), Iteration over the string of size N one time.
Auxiliary Space: O(N) for stack.

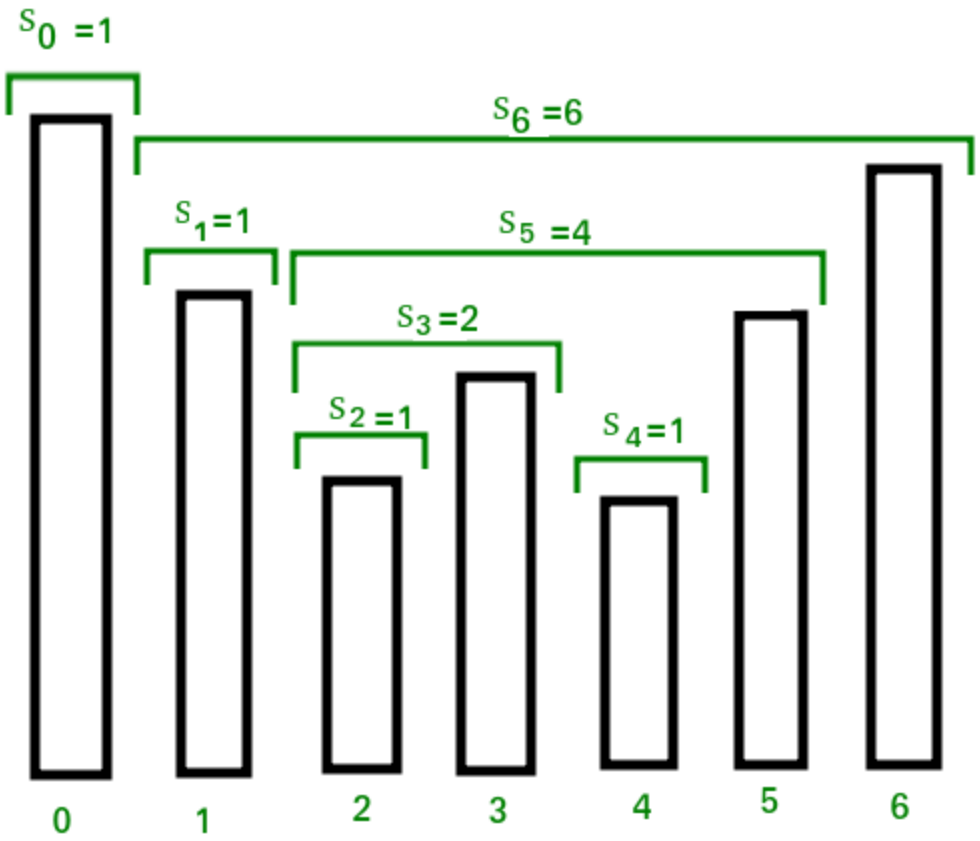
Stock span problem

The stock span problem is a financial problem where we have a series of **N** daily price quotes for a stock and we need to calculate the span of the stock's price for all N days. The span Si of the stock's price on a given day i is defined as the maximum number of consecutive days just before the given day, for which the price of the stock on the current day is less than its price on the given day.

Examples:

Input: N = 7, price[] = [100 80 60 70 60 75 85]
Output: 1 1 1 2 1 4 6
Explanation: Traversing the given input span for 100 will be 1, 80 is smaller than 100 so the span is 1, 60 is smaller than 80 so the span is 1, 70 is greater than 60 so the span is 2 and so on. Hence the output will be 1 1 1 2 1 4 6.

Input: N = 6, price[] = [10 4 5 90 120 80]
Output:1 1 2 4 5 1
Explanation: Traversing the given input span for 10 will be 1, 4 is smaller than 10 so the span will be 1, 5 is greater than 4 so the span will be 2 and so on. Hence, the output will be 1 1 2 4 5 1.



Naive Approach: To solve the problem follow the below idea:

Traverse the input price array. For every element being visited, traverse elements on the left of it and increment the span value of it while elements on the left side are smaller

Below is the implementation of the above approach:

C++Java

```
// C++ program for brute force method
// to calculate stock span values
#include <bits/stdc++.h>
using namespace std;

// Fills array S[] with span values
void calculateSpan(int price[], int n, int S[])
{
```

```
    // Span value of first day is always 1
    S[0] = 1;

    // Calculate span value of remaining days
    // by linearly checking previous days
    for (int i = 1; i < n; i++) {
        S[i] = 1; // Initialize span value

        // Traverse left while the next element
        // on left is smaller than price[i]
        for (int j = i - 1;
             (j >= 0) && (price[i] >= price[j]); j--)
            S[i]++;
    }
}

// A utility function to print elements of array
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

// Driver code
int main()
{
    int price[] = { 10, 4, 5, 90, 120, 80 };
    int n = sizeof(price) / sizeof(price[0]);
    int S[n];

    // Fill the span values in array S[]
    calculateSpan(price, n, S);

    // print the calculated span values
    printArray(S, n);

    return 0;
}
```

Output

```
1 1 2 4 5 1
```

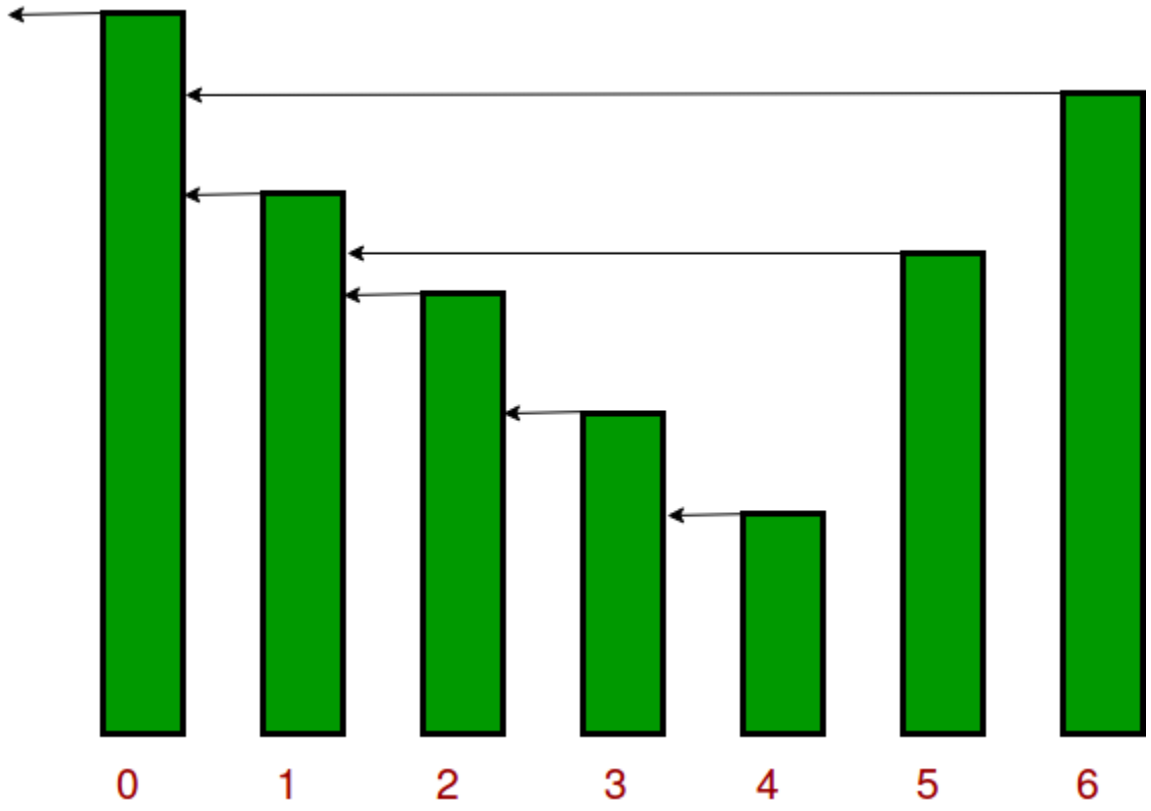
Time Complexity: $O(N^2)$

Auxiliary Space: $O(N)$

The stock span problem using stack:

To solve the problem follow the below idea:

We see that S[i] on the day **i** can be easily computed if we know the closest day preceding **i**, such that the price is greater than on that day than the price on the day i. If such a day exists, let's call it **h(i)**, otherwise, we define **h(i) = -1**
The span is now computed as **S[i] = i - h(i)**. See the following diagram



To implement this logic, we use a stack as an abstract data type to store the days i, h(i), h(h(i)), and so on. When we go from day i-1 to i, we pop the days when the price of the stock was less than or equal to price[i] and then push the value of day i back into the stack.

Follow the below steps to solve the problem:

- Create a stack of type int and push 0 in it
- Set the answer of day 1 as 1 and run a for loop to traverse the days
- While the stack is not empty and the price of st.top is less than or equal to the price of the current day, pop out the top value
- Set the answer of the current day as i+1 if the stack is empty else equal to i - st.top

- Push the current day into the stack
- Print the answer using the answer array

Below is the implementation of the above approach:

Note: We have to check also for a case when all the stock prices should be the same so therefore we have to just check whether the current stock price is bigger than the previous one or not. We will not pop from the stack when the current and previous stock prices are the same.

C++Java

```cpp
// C++ linear time solution for stock span problem
#include <bits/stdc++.h>
using namespace std;

// A stack based efficient method to calculate
// stock span values
void calculateSpan(int price[], int n, int S[])
{
    // Create a stack and push index of first
    // element to it
    stack<int> st;
    st.push(0);

    // Span value of first element is always 1
    S[0] = 1;

    // Calculate span values for rest of the elements
    for (int i = 1; i < n; i++) {
        // Pop elements from stack while stack is not
        // empty and top of stack is smaller than
        // price[i]
        while (!st.empty() && price[st.top()] <= price[i])
            st.pop();

        // If stack becomes empty, then price[i] is
        // greater than all elements on left of it,
        // i.e., price[0], price[1], ..price[i-1].  Else
        // price[i] is greater than elements after
        // top of stack
        S[i] = (st.empty()) ? (i + 1) : (i - st.top());

        // Push this element to stack
        st.push(i);
    }
}

// A utility function to print elements of array
```

```
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

// Driver code
int main()
{
    int price[] = { 10, 4, 5, 90, 120, 80 };
    int n = sizeof(price) / sizeof(price[0]);
    int S[n];

    // Fill the span values in array S[]
    calculateSpan(price, n, S);

    // print the calculated span values
    printArray(S, n);

    return 0;
}
```

Output

```
1 1 2 4 5 1
```

Time Complexity: O(N). It seems more than O(N) at first look. If we take a closer look, we can observe that every element of the array is added and removed from the stack at most once.

Auxiliary Space: O(N) in the worst case when all elements are sorted in decreasing order.

The stock span problem using dynamic programming:

To solve the problem follow the below idea:

Since the same subproblems are called again, this problem has the Overlapping Subproblems property. S0, the recomputations of the same subproblems can be avoided by constructing a temporary array to store already calculated answers

- Store the answer for every index in an array and calculate the value of the next index using previous values
- i.e check the answer for previous element and if the value of the current element is greater than the previous element
- Add the answer to the previous index into current answer and then check the value of (previous index - answer of previous index)

- Check this condition repeatedly

Below is the implementation of the above approach:

C++Java

```cpp
// C++ program for a linear time solution for stock
// span problem without using stack
#include <bits/stdc++.h>
using namespace std;

// An efficient method to calculate stock span values
// implementing the same idea without using stack
void calculateSpan(int A[], int n, int ans[])
{
    // Span value of first element is always 1
    ans[0] = 1;

    // Calculate span values for rest of the elements
    for (int i = 1; i < n; i++) {
        int counter = 1;
        while ((i - counter) >= 0
               && A[i] >= A[i - counter]) {
            counter += ans[i - counter];
        }
        ans[i] = counter;
    }
}

// A utility function to print elements of array
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

// Driver code
int main()
{
    int price[] = { 10, 4, 5, 90, 120, 80 };
    int n = sizeof(price) / sizeof(price[0]);
    int S[n];

    // Fill the span values in array S[]
    calculateSpan(price, n, S);

    // print the calculated span values
    printArray(S, n);

    return 0;
```

```
}
```

Output
```
1 1 2 4 5 1
```
Time Complexity: O(N)
Auxiliary Space: O(N)

The stock span problem using two stacks:

Follow the below steps to solve the problem:

- In this approach, I have used the data structure stack to implement this task.
- Here, two stacks are used. One stack stores the actual stock prices whereas, the other stack is a temporary stack.
- The stock span problem is solved using only the Push and Pop functions of the Stack.
- Just to take input values, I have taken array 'price' and to store output, used array 'span'.

Below is the implementation of the above approach:

C++Java

```cpp
// C++ program for brute force method
// to calculate stock span values
#include <bits/stdc++.h>
using namespace std;

vector<int> calculateSpan(int arr[], int n)
{
    // Your code here
    stack<int> s;
    vector<int> ans;
    for (int i = 0; i < n; i++) {
        while (!s.empty() and arr[s.top()] <= arr[i])
            s.pop();

        if (s.empty())
            ans.push_back(i + 1);
        else {
            int top = s.top();
            ans.push_back(i - top);
        }
        s.push(i);
    }

    return ans;
}
```

```
// A utility function to print elements of array
void printArray(vector<int> arr)
{
    for (int i = 0; i < arr.size(); i++)
        cout << arr[i] << " ";
}

// Driver code
int main()
{
    int price[] = { 10, 4, 5, 90, 120, 80 };
    int n = sizeof(price) / sizeof(price[0]);
    int S[n];

      // Function call
    vector<int> arr = calculateSpan(price, n);
    printArray(arr);

    return 0;
}
```

Output
```
1 1 2 4 5 1
```
Time Complexity: O(N), where N is the size of the array.
Auxiliary Space: O(N), where N is the size of the array.

Queue & Deque

Introduction to Queues

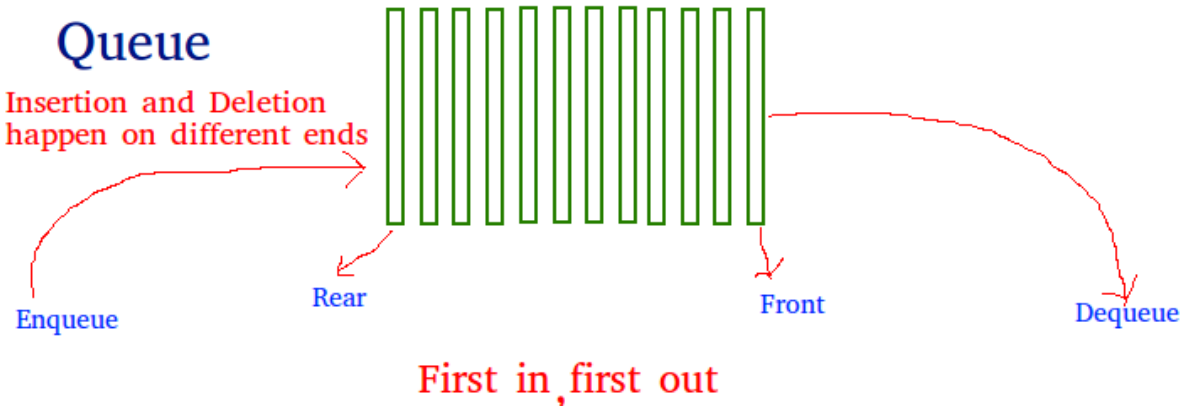
Like *Stack* data structure, ***Queue*** is also a linear data structure that follows a particular order in which the operations are performed. The order is **F**irst **I**n **F**irst **O**ut (FIFO), which means that the element that is inserted first in the queue will be the first one to be removed from the queue. A good example of queue is any queue of consumers for a resource where the consumer who came first is served first.

The difference between stacks and queues is in removing. In a stack, we remove the most recently added item; whereas, in a queue, we remove the least recently added item.

Operations on Queue:

Mainly the following four basic operations are performed on queue:

- **Enqueue:** Adds an item to the queue. If the queue is full, then it is said to be an Overflow condition.
- **Dequeue:** Removes an item from the queue. The items are popped in the same order in which they are pushed. If the queue is empty, then it is said to be an Underflow condition.
- **Front:** Get the front item from queue.
- **Rear:** Get the last item from queue.



Array implementation Of Queue

: For implementing a

queue

, we need to keep track of two indices - front and rear. We enqueue an item at the rear and dequeue an item from the front. If we simply increment front and rear indices, then there may be problems, the front may reach the end of the array. The solution to this problem is to increase front and rear in a circular manner.

Consider that an Array of size **N**

is taken to implement a queue. Initially, the size of the queue will be zero(0). The total capacity of the queue will be the size of the array i.e. N. Now initially, the index *front* will be equal to 0, and *rear* will be equal to N-1. Every time an item is inserted, so the index *rear* will increment by one, hence increment it as: **rear = (rear + 1)%N** and everytime an item is removed, so the front index will shift to right by 1 place, hence increment it as: **front = (front + 1)%N**

Example:

```
Array = queue[N].
front = 0, rear = N-1.
N = 5.

Operation 1:
enque(5);
front = 0,
rear = (N-1 + 1)%N = 0.
Queue contains: [5].

Operation 2:
enque(10);
front = 0,
rear = (rear + 1)%N = (0 + 1)%N = 1.
Queue contains: [5, 10].

Operation 3:
enque(15);
front = 0,
rear = (rear + 1)%N = (1 + 1)%N = 2.
Queue contains: [5, 10, 15].

Operation 4:
deque();
print queue[front];
front = (front + 1)%N = (0 + 1)%N = 1.
Queue contains: [10, 15].
```

Below is the Array implementation of queue in C++ and Java:

C++Java

```cpp
// CPP program for array implementation of queue
#include <bits/stdc++.h>
using namespace std;

// A structure to represent a queue
class Queue
{
    public:
    int front, rear, size;
    unsigned capacity;
    int* array;
};

// function to create a queue of a given capacity.
```

```
// It initializes the size of the queue as 0
Queue* createQueue(unsigned capacity)
{
    Queue* queue = new Queue();
    queue->capacity = capacity;
    queue->front = queue->size = 0;
    queue->rear = capacity - 1; // This is important, see the enqueue
    queue->array = new int[(queue->capacity * sizeof(int))];
    return queue;
}

// Queue is full when size
// becomes equal to the capacity
int isFull(Queue* queue)
{ return (queue->size == queue->capacity); }

// Queue is empty when size is 0
int isEmpty(Queue* queue)
{ return (queue->size == 0); }

// Function to add an item to the queue.
// It changes rear and size
void enqueue(Queue* queue, int item)
{
    if (isFull(queue))
        return;
    queue->rear = (queue->rear + 1) % queue->capacity;
    queue->array[queue->rear] = item;
    queue->size = queue->size + 1;
    cout << item << " enqueued to queue\n";
}

// Function to remove an item from the queue.
// It changes front and size
int dequeue(Queue* queue)
{
    if (isEmpty(queue))
        return INT_MIN;
    int item = queue->array[queue->front];
    queue->front = (queue->front + 1) % queue->capacity;
    queue->size = queue->size - 1;
    return item;
}

// Function to get front of queue
int front(Queue* queue)
{
    if (isEmpty(queue))
        return INT_MIN;
```

```
    return queue->array[queue->front];
}

// Function to get rear of queue
int rear(Queue* queue)
{
    if (isEmpty(queue))
        return INT_MIN;
    return queue->array[queue->rear];
}

// Driver code
int main()
{
    Queue* queue = createQueue(1000);

    enqueue(queue, 10);
    enqueue(queue, 20);
    enqueue(queue, 30);
    enqueue(queue, 40);

    cout<<dequeue(queue)<<" dequeued from queue\n";

    cout << "Front item is " << front(queue) << endl;
    cout << "Rear item is " << rear(queue) << endl;

    return 0;
}
```

Output:

```
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
10 dequeued from queue
Front item is 20
Rear item is 40
```

Time Complexity:

Time complexity of all operations such as enqueue(), dequeue(), isFull(), isEmpty(), front(), and rear() is O(1). There is no loop in any of the operations.

Applications of Queue:

Queue is used when things don't have to be processed immediatly, but have to be processed in **F**irst **I**n **F**irst **O**ut order like

Breadth First Search
. This property of Queue makes it also useful in following kind of scenarios:

1. When a resource is shared among multiple consumers. Examples include CPU scheduling, Disk Scheduling.
2. When data is transferred asynchronously (data not necessarily received at same rate as sent) between two processes. Examples include IO Buffers, pipes, file IO, etc.

Implementing Queue in C++ and Java using Built-in Classes

Queue in C++ STL

The Standard template Library in C++ offers a built-in implementation of the Queue data structure for simpler and easy use. The STL implementation of queue data structure implements all basic operations on queue such as enque(), deque(), clear() etc.

Syntax

:

```
queue< data_type > queue_name;

where,
data_type is the type of element to be stored
in the queue.
queue_name is the name of the queue data structure.
```

The functions supported by std::queue are

:

- **empty()** – Returns whether the queue is empty.
- **size()** – Returns the size of the queue.
- **swap()**: Exchange the contents of two queues but the queues must be of same type, although sizes may differ.
- **emplace()**: Insert a new element into the queue container, the new element is added to the end of the queue.
- **front():** front() function returns a reference to the first element of the queue.
- **back()**: back() function returns a reference to the last element of the queue.
- **push(g):** The push() function adds the element 'g' at the end of the queue.
- **pop()**: The pop() function deletes the first element of the queue.

CPP

```cpp
// CPP code to illustrate
// Queue in Standard Template Library (STL)
#include <iostream>
#include <queue>

using namespace std;

void showq(queue <int> gq)
{
    queue <int> g = gq;
    while (!g.empty())
    {
        cout << '\t' << g.front();
        g.pop();
    }
    cout << '\n';
}

int main()
{
    queue <int> gquiz;
    gquiz.push(10);
    gquiz.push(20);
    gquiz.push(30);
```

```
    cout << "The queue gquiz is : ";
    showq(gquiz);

    cout << "\ngquiz.size() : " << gquiz.size();
    cout << "\ngquiz.front() : " << gquiz.front();
    cout << "\ngquiz.back() : " << gquiz.back();

    cout << "\ngquiz.pop() : ";
    gquiz.pop();
    showq(gquiz);

    return 0;
}
```

Output

:

```
The queue gquiz is : 10 20 30

gquiz.size() : 3
gquiz.front() : 10
gquiz.back() : 30
gquiz.pop() : 20 30
```

Queue interface in Java

The Queue interface is available in java.util package and extends the Collection interface. The queue collection is used to hold the elements about to be processed and provides various operations like the insertion, removal etc. It is an ordered list of objects with its use limited to insert elements at the end of the list and deleting elements from the start of list i.e. it follows the FIFO or the First-In-First-Out principle. Being an interface the queue needs a concrete class for the declaration and the most commonly used classes are the PriorityQueue and LinkedList in Java. It is to be noted that both the implementations are not thread safe. PriorityBlockingQueue is one alternative implementation if thread safe implementation is needed.

Methods in Queue

:

- **add()**- This method is used to add elements at the tail of the queue. More specifically, at the last of linkedlist if it is used, or according to the priority in case of priority queue implementation.
- **peek()**- This method is used to view the head of a queue without removing it. It returns Null if the queue is empty.
- **element()**- This method is similar to peek(). It throws NoSuchElementException when the queue is empty.
- **remove()**- This method removes and returns the head of the queue. It throws NoSuchElementException when the queue is empty.
- **poll()**- This method removes and returns the head of the queue. It returns null if the queue is empty.
- **size()**- This method returns the no. of elements in the queue.

Below is a simple Java program to demonstrate these methods:

Java

```
// Java orogram to demonstrate working of Queue
// interface in Java
import java.util.LinkedList;
import java.util.Queue;

public class QueueExample
{
    public static void main(String[] args)
    {
        Queue<Integer> q = new LinkedList<>();

        // Adds elements {0, 1, 2, 3, 4} to queue
        for (int i=0; i<5; i++)
            q.add(i);
```

```
        // Display contents of the queue.
        System.out.println("Elements of queue-"+q);

        // To remove the head of queue.
        int removedele = q.remove();
        System.out.println("removed element-" + removedele);

        System.out.println(q);

        // To view the head of queue
        int head = q.peek();
        System.out.println("head of queue-" + head);

        // Rest all methods of collection interface,
        // Like size and contains can be used with this
        // implementation.
        int size = q.size();
        System.out.println("Size of queue-" + size);
    }
}
```

Output

:

```
Elements of queue-[0, 1, 2, 3, 4]
removed element-0
[1, 2, 3, 4]
head of queue-1
Size of queue-4
```

Deque Data Structure

Deque or Double Ended Queue is a generalized version of Queue data structure that allows insert and delete at both ends.

Operations on Deque: Mainly the following four basic operations are performed on queue:
insertFront(): Adds an item at the front of Deque.
insertLast(): Adds an item at the rear of Deque.
deleteFront(): Deletes an item from the front of Deque.
deleteLast(): Deletes an item from the rear of Deque. In addition to the above operations,

the following operations are also supported.
getFront(): Gets the front item from the queue.
getRear(): Gets the last item from queue.
isEmpty(): Checks whether Deque is empty or not.
isFull(): Checks whether Deque is full or not.

Applications of Deque: Since Deque supports both stack and queue operations, it can be used as both. The Deque data structure supports clockwise and anticlockwise rotations in O(1) time which can be useful in certain applications. Also, the problems where elements need to be removed and or added to both ends can be efficiently solved using Deque. For example see the Maximum of all subarrays of size k problem., 0-1 BFS, and Find the first circular tour that visits all petrol pumps. See the wiki page for another example of the A-Steal job scheduling algorithm where Deque is used as deletions operation is required at both ends.

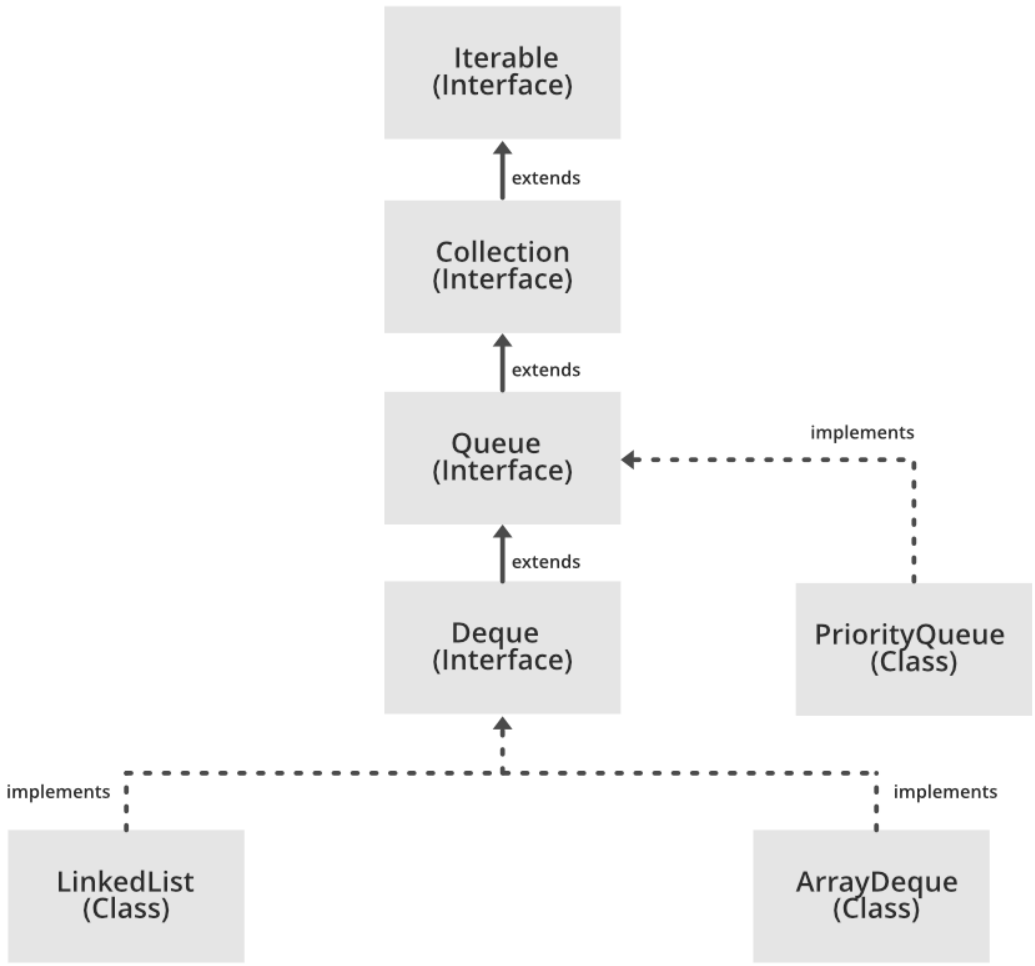
Some Practical Applications of Deque:

- Applied as both stack and queue, as it supports both operations.
- Storing a web browser's history.
- Storing a software application's list of undo operations.
- Job scheduling algorithm

Language Support: C++ STL provides an implementation of Deque as std::deque and Java provides the Deque interface. See this for more details. Deque in Java Deque in Python
Implementation: A Deque can be implemented either using a doubly-linked list or a circular array. In both implementations, we can implement all operations in O(1) time. We couldn't implement Deque using singly linked list because the delete rear operation can't be implemented in O(1) time.

Deque in Java

Deque interface present in java.util package is a subtype of the queue interface. The Deque is related to the double-ended queue that supports the addition or removal of elements from either end of the data structure. It can either be used as a queue(first-in-first-out/FIFO) or as a stack(last-in-first-out/LIFO). Deque is the acronym for double-ended queue.



Syntax: The deque interface is declared as:

```
public interface Deque extends Queue
```

Creating Deque Objects Since Deque is an interface, objects cannot be created of the type deque. We always need a class that extends this list in order to create an object. And also, after the introduction of Generics in Java 1.5, it is possible to restrict the type of object that can be stored in the Deque. This type-safe queue can be defined as:

Example: Deque

C++

```
// Java program to demonstrate the working
// of a Deque in Java
```

```
import java.util.*;

public class DequeExample {
	public static void main(String[] args)
	{
		Deque<String> deque
			= new LinkedList<String>();

		// We can add elements to the queue
		// in various ways

		// Add at the last
		deque.add("Element 1 (Tail)");

		// Add at the first
		deque.addFirst("Element 2 (Head)");

		// Add at the last
		deque.addLast("Element 3 (Tail)");

		// Add at the first
		deque.push("Element 4 (Head)");

		// Add at the last
		deque.offer("Element 5 (Tail)");

		// Add at the first
		deque.offerFirst("Element 6 (Head)");

		System.out.println(deque + "\n");

		// We can remove the first element
		// or the last element.
		deque.removeFirst();
		deque.removeLast();
		System.out.println("Deque after removing "
						+ "first and last: "
						+ deque);
	}
}
```

Output:[Element 6 (Head), Element 4 (Head), Element 2 (Head), Element 1 (Tail), Element 3 (Tail), Element 5 (Tail)] Deque after removing first and last: [Element 4 (Head), Element 2 (Head), Element 1 (Tail), Element 3 (Tail)]

Operations using the Deque Interface and the ArrayDeque class
Let's see how to perform a few frequently used operations on the deque using the ArrayDeque class.

1. Adding Elements: In order to add an element in a deque, we can use the add() method. The difference between a queue and a deque is that in deque, the addition is possible from any direction. Therefore, there are other two methods available named addFirst() and addLast() which are used to add the elements at either end.

Java

```
// Java program to demonstrate the
// addition of elements in deque

import java.util.*;
public class ArrayDequeDemo {
        public static void main(String[] args)
        {
                // Initializing an deque
                Deque<String> dq
                        = new ArrayDeque<String>();

                // add() method to insert
                dq.add("For");
                dq.addFirst("Geeks");
                dq.addLast("Geeks");

                System.out.println(dq);
        }
}
```

Output

```
[Geeks, For, Geeks]
```

2. Removing Elements: In order to remove an element from a deque, there are various methods available. Since we can also remove from both the ends, the deque interface provides us with *removeFirst()*, *removeLast()* methods. Apart from that, this interface also provides us with the poll(), pop(), pollFirst(), pollLast() methods where pop() is used to remove and return the head of the deque. However, poll() is used because this offers the same functionality as pop() and doesnt return an exception when the deque is empty.

Java

```
// Java program to demonstrate the
// removal of elements in deque

import java.util.*;
public class ArrayDequeDemo {
	public static void main(String[] args)
	{
		// Initializing an deque
		Deque<String> dq
			= new ArrayDeque<String>();

		// add() method to insert
		dq.add("For");
		dq.addFirst("Geeks");
		dq.addLast("Geeks");

		System.out.println(dq);

		System.out.println(dq.pop());

		System.out.println(dq.poll());

		System.out.println(dq.pollFirst());

		System.out.println(dq.pollLast());
	}
}
```

Output

```
[Geeks, For, Geeks]
Geeks
For
Geeks
null
```

3. Iterating through the Deque: Since a deque can be iterated from both the directions, the iterator method of the deque interface provides us two ways to iterate. One from the first and the other from the back.

Java

```
// Java program to demonstrate the
// iteration of elements in deque

import java.util.*;
```

```
public class ArrayDequeDemo {
	public static void main(String[] args)
	{
		// Initializing an deque
		Deque<String> dq
			= new ArrayDeque<String>();

		// add() method to insert
		dq.add("For");
		dq.addFirst("Geeks");
		dq.addLast("Geeks");
		dq.add("is so good");

		for (Iterator itr = dq.iterator();
			itr.hasNext();) {
			System.out.print(itr.next() + " ");
		}

		System.out.println();

		for (Iterator itr = dq.descendingIterator();
			itr.hasNext();) {
			System.out.print(itr.next() + " ");
		}
	}
}
```

Output

```
Geeks For Geeks is so good
is so good Geeks For Geeks
```

The class which implements the Deque interface is ArrayDeque.

ArrayDeque: ArrayDeque class which is implemented in the collection framework provides us with a way to apply resizable-array. This is a special kind of array that grows and allows users to add or remove an element from both sides of the queue. Array deques have no capacity restrictions and they grow as necessary to support usage. They are not thread-safe which means that in the absence of external synchronization, ArrayDeque does not support concurrent access by multiple threads. ArrayDeque class is likely to be faster than Stack when used as a stack. ArrayDeque class is likely to be faster than LinkedList when used as a queue. Let's see how to create a queue object using this class.

Java

```
// Java program to demonstrate the
```

```java
// creation of deque object using the
// ArrayDeque class in Java

import java.util.*;
public class ArrayDequeDemo {
	public static void main(String[] args)
	{
		// Initializing an deque
		Deque<Integer> de_que
			= new ArrayDeque<Integer>(10);

		// add() method to insert
		de_que.add(10);
		de_que.add(20);
		de_que.add(30);
		de_que.add(40);
		de_que.add(50);

		System.out.println(de_que);

		// clear() method
		de_que.clear();

		// addFirst() method to insert the
		// elements at the head
		de_que.addFirst(564);
		de_que.addFirst(291);

		// addLast() method to insert the
		// elements at the tail
		de_que.addLast(24);
		de_que.addLast(14);

		System.out.println(de_que);
	}
}
```

Output

```
[10, 20, 30, 40, 50]
[291, 564, 24, 14]
```

Methods of Deque Interface

The following are the methods present in the deque interface:

Method	Description

add(element)	This method is used to add an element at the tail of the queue. If the Deque is capacity restricted and no space is left for insertion, it returns an IllegalStateException. The function returns true on successful insertion.
addFirst(element)	This method is used to add an element at the head of the queue. If the Deque is capacity restricted and no space is left for insertion, it returns an IllegalStateException. The function returns true on successful insertion.
addLast(element)	This method is used to add an element at the tail of the queue. If the Deque is capacity restricted and no space is left for insertion, it returns an IllegalStateException. The function returns true on successful insertion.
contains()	This method is used to check whether the queue contains the given object or not.
descendingIterator()	This method returns an iterator for the deque. The elements will be returned in order from last(tail) to first(head).
element()	This method is used to retrieve, but not remove, the head of the queue represented by this deque.
getFirst()	This method is used to retrieve, but not remove, the first element of this deque.
getLast()	This method is used to retrieve, but not remove, the last element of this deque.

iterator()	This method returns an iterator for the deque. The elements will be returned in order from first (head) to last (tail).
offer(element)	This method is used to add an element at the tail of the queue. This method is preferable to add() method since this method does not throws an exception when the capacity of the container is full since it returns false.
offerFirst(element)	This method is used to add an element at the head of the queue. This method is preferable to addFirst() method since this method does not throws an exception when the capacity of the container is full since it returns false.
offerLast(element)	This method is used to add an element at the tail of the queue. This method is preferable to add() method since this method does not throws an exception when the capacity of the container is full since it returns false.
peek()	This method is used to retrieve the element at the head of the deque but doesn't remove the element from the deque. This method returns null if the deque is empty.
peekFirst()	This method is used to retrieve the element at the head of the deque but doesn't remove the element from the deque. This method returns null if the deque is empty.

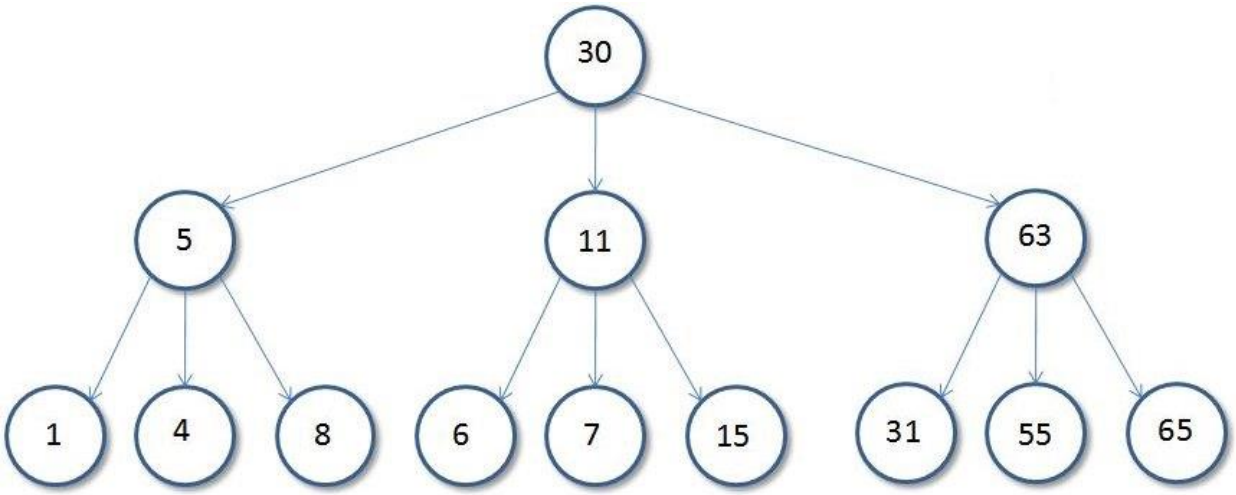
peekLast()	This method is used to retrieve the element at the tail of the deque but doesn't remove the element from the deque. This method returns null if the deque is empty.
poll()	This method is used to retrieve and remove the element at the head of the deque. This method returns null if the deque is empty.
pollFirst()	This method is used to retrieve and remove the element at the head of the deque. This method returns null if the deque is empty.
pollLast()	This method is used to retrieve and remove the element at the tail of the deque. This method returns null if the deque is empty.
pop()	This method is used to remove an element from the head and return it.
push(element)	This method is used to add an element at the head of the queue.
removeFirst()	This method is used to remove an element from the head of the queue.
removeLast()	This method is used to remove an element from the tail of the queue.
size()	This method is used to find and return the size of the deque.

Tree

Introduction to Trees

A Tree is a non-linear data structure where each node is connected to a number of nodes with the help of pointers or references.

A Sample tree is as shown below:



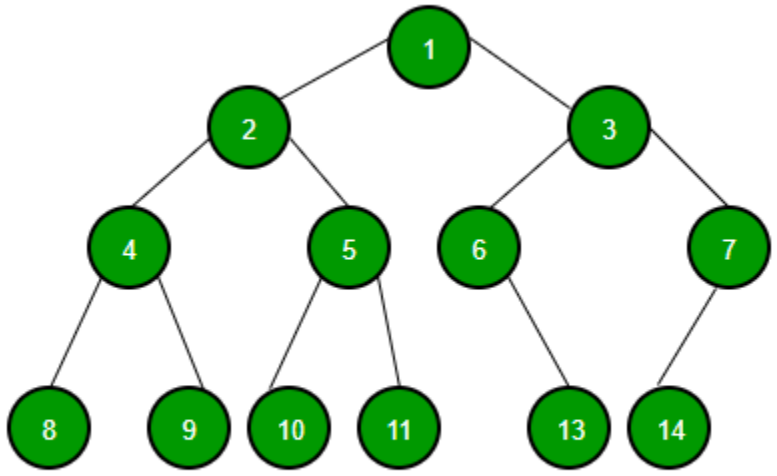
Basic Tree Terminologies:

- **Root**: The root of a tree is the first node of the tree. In the above image, the root node is the node 30.
- **Edge**: An edge is a link connecting any two nodes in the tree. For example, in the above image there is an edge between node **11** and **6**.
- **Siblings**: The children nodes of same parent are called siblings. That is, the nodes with same parent are called siblings. In the above tree, nodes *5, 11, and 63* are siblings.
- **Leaf Node**: A node is said to be the leaf node if it has no children. In the above tree, node **15** is one of the leaf nodes.
- **Height of a Tree**: Height of a tree is defined as the total number of levels in the tree or the length of the path from the root node to the node present at the last level. The above tree is of height **2**.

Binary Tree

A Tree is said to be a Binary Tree if all of its nodes have atmost 2 children. That is, all of its node can have either no child, 1 child, or 2 child nodes.

Below is a sample **Binary Tree**:



Properties of a Binary Tree:

1. ***The maximum number of nodes at level 'l' of a binary tree is*** (2^{l-1}). Level of root is 1. This can be proved by induction. For root, l = 1, number of nodes = $2^{1-1} = 1$ Assume that the maximum number of nodes on level l is 2^{l-1}. Since in Binary tree every node has at most 2 children, next level would have twice nodes, i.e. $2 * 2^{l-1}$.
2. ***Maximum number of nodes in a binary tree of height 'h' is*** $(2^h - 1)$. Here height of a tree is the maximum number of nodes on the root to leaf path. The height of a tree with a single node is considered as 1. This result can be derived from point 2 above. A tree has maximum nodes if all levels have maximum nodes. So maximum number of nodes in a binary tree of height h is $1 + 2 + 4 + .. + 2^h-1$. This is a simple geometric series with h terms and sum of this series is $2^h - 1$. In some books, the height of the root is considered as 0. In that convention, the above formula becomes $2^{h+1} - 1$.
3. ***In a Binary Tree with N nodes, the minimum possible height or the minimum number of levels is*** $Log_2(N+1)$. This can be directly derived from point 2 above. If we consider the convention where the height of a leaf node is considered 0, then above formula for minimum possible height becomes $Log_2(N+1) - 1$.

4. ***A Binary Tree with L leaves has at least ($Log_2L + 1$) levels.*** A Binary tree has maximum number of leaves (and minimum number of levels) when all levels are fully filled. Let all leaves be at level l, then below is true for number of leaves L.

$L <= 2^{l-1}$ [From Point 1]

$l = Log_2L + 1$

where l is the minimum number of levels.

9. ***In a Binary tree in which every node has 0 or 2 children, the number of leaf nodes is always one more than the nodes with two children.***

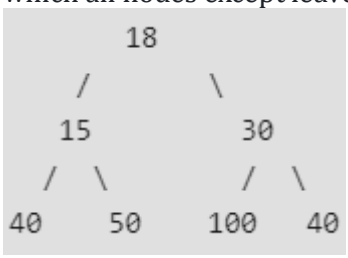
$L = T + 1$

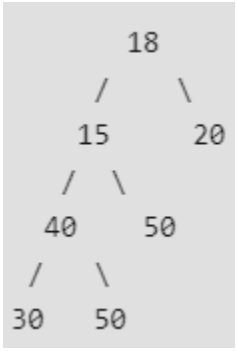
Where L = Number of leaf nodes

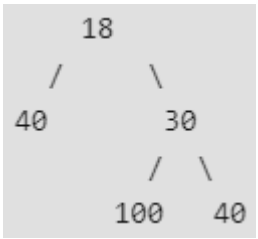
T = Number of internal nodes with two children

Types of Binary Trees: Based on the structure and number of parents and children nodes, a Binary Tree is classified into the following common types:

- **Full Binary Tree**: A Binary Tree is full if every node has either 0 or 2 children. The following are examples of a full binary tree. We can also say that a full binary tree is a binary tree in which all nodes except leave nodes have two children.







- ***In a Full Binary, the number of leaf nodes is number of internal nodes plus 1.***
-
- **Complete Binary Tree**: A Binary Tree is a complete Binary Tree if all levels are completely filled except possibly the last level and the last level has all keys as left as possible Following are the examples of Complete Binary Trees:

```
        18
     /      \
   15         30
  /  \        /  \
40    50    100   40
```

-
-

```
          18
       /      \
     15         30
    /  \        /  \
  40    50    100   40
 /  \   /
8   7  9
```

-
- **Perfect Binary Tree**: A Binary tree is a Perfect Binary Tree when all internal nodes have two children and all the leave nodes are at the same level. Following are the examples of Perfect Binary Trees:

```
          18
       /      \
     15         30
    /  \        /  \
  40    50    100   40
```

-
-

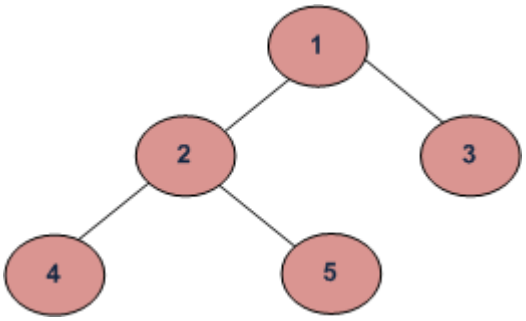
```
     18
  /      \
15         30
```

-
-

A Perfect Binary Tree of height h (where height is the number of nodes on the path from the root to leaf) has $2^h - 1$ node.

Binary Tree Traversals

Unlike linear data structures (Array, Linked List, Queues, Stacks, etc.), which have only one logical way to traverse them, trees can be traversed in different ways. Following are the generally used ways for traversing trees:



- Inorder (Left, Root, Right) : 4 2 5 1 3
- Preorder (Root, Left, Right) : 1 2 4 5 3.
- Postorder (Left, Right, Root) : 4 5 2 3 1

Let's look at each of these tree traversal algorithms in details:

- **Inorder Traversal:** In Inorder traversal, a node is processed after processing all the nodes in its left subtree. The right subtree of the node is processed after processing the node itself.
- `Algorithm Inorder(tree)`
- `   1. Traverse the left subtree, i.e.,`
- `      call Inorder(left->subtree)`
- `   2. Visit the root.`
- `   3. Traverse the right subtree, i.e.,`
- `      call Inorder(right->subtree)`
- **Example**: Inorder traversal for the above-given tree is 4 2 5 1 3.
- **Preorder Traversal:** In preorder traversal, a node is processed before processing any of the nodes in its subtree.
- `Algorithm Preorder(tree)`
- `   1. Visit the root.`
- `   2. Traverse the left subtree, i.e.,`
- `      call Preorder(left-subtree)`
- `   3. Traverse the right subtree, i.e.,`
- `      call Preorder(right-subtree)`
- **Example**: Preorder traversal for the above-given tree is 1 2 4 5 3.
- **Postorder Traversal:** In post order traversal, a node is processed after processing all the nodes in its subtrees.
- `Algorithm Postorder(tree)`
- `   1. Traverse the left subtree, i.e.,`

- call Postorder(left-subtree)
- 2. Traverse the right subtree, i.e.,
- call Postorder(right-subtree)
- 3. Visit the root.
-

Example: Postorder traversal for the above-given Tree is 4 5 2 3 1.

C++Java

```cpp
// C++ program for different tree traversals

#include <iostream>
using namespace std;

/* A binary tree node has data, pointer to left child
   and a pointer to right child */
struct Node
{
    int data;
    struct Node* left, *right;
    Node(int data)
    {
        this->data = data;
        left = right = NULL;
    }
};

// Function to print the postorder traversal
// of a Binary Tree
void printPostorder(struct Node* node)
{
    if (node == NULL)
        return;

    // first recur on left subtree
    printPostorder(node->left);

    // then recur on right subtree
    printPostorder(node->right);

    // now deal with the node
    cout << node->data << " ";
}

// Function to print the Inorder traversal
// of a Binary Tree
void printInorder(struct Node* node)
```

```
{
    if (node == NULL)
        return;

    /* first recur on left child */
    printInorder(node->left);

    /* then print the data of node */
    cout << node->data << " ";

    /* now recur on right child */
    printInorder(node->right);
}

// Function to print the PreOrder traversal
// of a Binary Tree
void printPreorder(struct Node* node)
{
    if (node == NULL)
        return;

    /* first print data of node */
    cout << node->data << " ";

    /* then recur on left sutree */
    printPreorder(node->left);

    /* now recur on right subtree */
    printPreorder(node->right);
}

// Driver Code
int main()
{
    // Contrust the Tree
    //      1
    //     /  \
    //    2    3
    //   / \
    //  4   5

    struct Node *root = new Node(1);
    root->left    = new Node(2);
    root->right    = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);

    cout << "Preorder traversal of binary tree is \n";
    printPreorder(root);
```

```
    cout << "\nInorder traversal of binary tree is \n";
    printInorder(root);

    cout << "\nPostorder traversal of binary tree is \n";
    printPostorder(root);

    return 0;
}
```

Output:

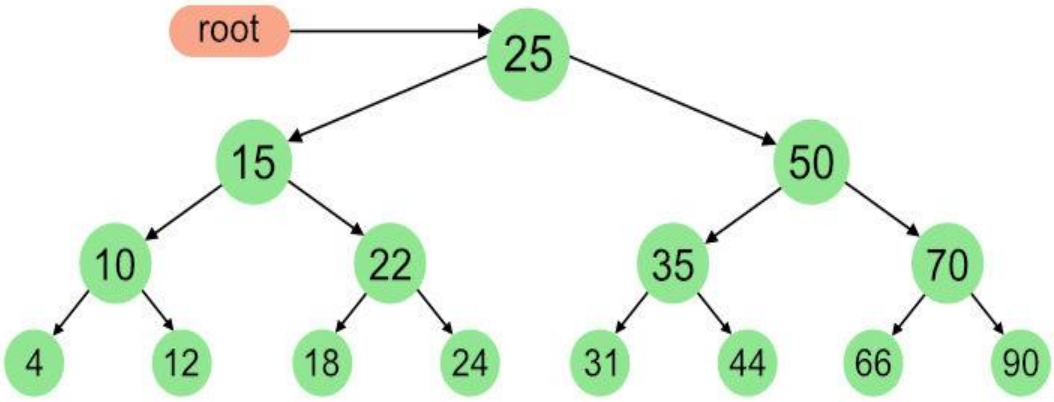
```
Preorder traversal of binary tree is
1 2 4 5 3
Inorder traversal of binary tree is
4 2 5 1 3
Postorder traversal of binary tree is
4 5 2 3 1
```

One more example:

InOrder(root) visits nodes in the following order:
4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

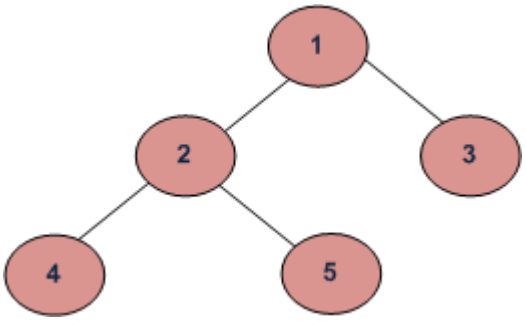
A Pre-order traversal visits nodes in the following order:
25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:
4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25



Height of Binary Tree

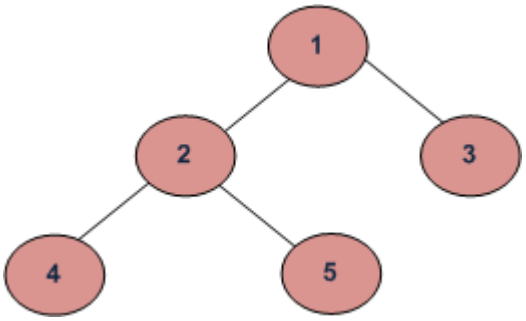
Given a binary tree, the task is to find the height of the tree. Height of the tree is the number of edges in the tree from the root to the deepest node, Height of the empty tree is **0**.



Recursively calculate height of **left** and **right** subtrees of a node and assign height to the node as **max of the heights of two children plus 1**. See below pseudo code and program for details.

Illustration:

Consider the following graph:



maxDepth('1') = max(maxDepth('2'), maxDepth('3')) + 1 = 2 + 1

because recursively
maxDepth('2') = max (maxDepth('4'), maxDepth('5')) + 1 = 1 + 1 and (as height of both '4' and '5' are 1)
maxDepth('3') = 1

Follow the below steps to Implement the idea:

- Recursively do a Depth-first search.
- If the tree is empty then return -1
- Otherwise, do the following
    - Get the max depth of the left subtree recursively i.e. call maxDepth(tree->left-subtree)
    - Get the max depth of the right subtree recursively i.e. call maxDepth(tree->right-subtree)
    - Get the max of max depths of **left** and **right** subtrees and **add 1** to it for the current node.
        -
- Return max_depth.

Below is the Implementation of the above approach:

C++Java

```cpp
// C++ program to find height of tree
#include <bits/stdc++.h>
using namespace std;

/* A binary tree node has data, pointer to left child
and a pointer to right child */
class node {
public:
	int data;
	node* left;
	node* right;
};

/* Compute the "maxDepth" of a tree -- the number of
	nodes along the longest path from the root node
	down to the farthest leaf node.*/
int maxDepth(node* node)
{
	if (node == NULL)
		return 0;
	else {
		/* compute the depth of each subtree */
		int lDepth = maxDepth(node->left);
		int rDepth = maxDepth(node->right);

		/* use the larger one */
		if (lDepth > rDepth)
			return (lDepth + 1);
		else
			return (rDepth + 1);
	}
}

/* Helper function that allocates a new node with the
given data and NULL left and right pointers. */
node* newNode(int data)
{
	node* Node = new node();
	Node->data = data;
	Node->left = NULL;
	Node->right = NULL;

	return (Node);
}

// Driver code
int main()
{
```

```
        node* root = newNode(1);

        root->left = newNode(2);
        root->right = newNode(3);
        root->left->left = newNode(4);
        root->left->right = newNode(5);

        cout << "Height of tree is " << maxDepth(root);
        return 0;
}
```

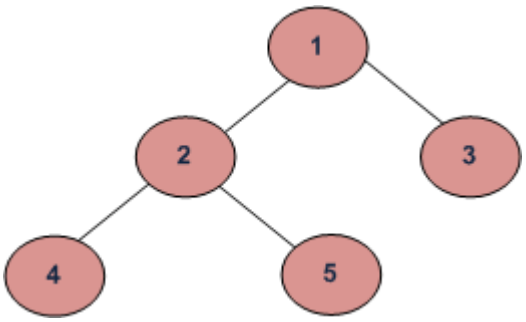
Output

```
Height of tree is 3
```

Time Complexity: O(N) (Please see our post Tree Traversal for details)
Auxiliary Space: O(N) due to recursive stack.

Size of Binary Tree

Size of a tree is the number of elements present in the tree. Size of the below tree is 5.



Size() function recursively calculates the size of a tree. It works as follows:
Size of a tree = Size of left subtree + 1 + Size of right subtree.

Algorithm:

```
size(tree)
```

```
1. If tree is empty then return 0
2. Else
     (a) Get the size of left subtree recursively  i.e., call
          size( tree->left-subtree)
     (a) Get the size of right subtree recursively  i.e., call
          size( tree->right-subtree)
     (c) Calculate size of the tree as following:
            tree_size  =  size(left-subtree) + size(right-
                                 subtree) + 1
     (d) Return tree_size
```

C++Java

```cpp
// A recursive C++ program to
// calculate the size of the tree
#include <bits/stdc++.h>
using namespace std;

/* A binary tree node has data, pointer to left child
and a pointer to right child */
class node
{
    public:
    int data;
    node* left;
    node* right;
};

/* Helper function that allocates a new node with the
given data and NULL left and right pointers. */
node* newNode(int data)
{
    node* Node = new node();
    Node->data = data;
    Node->left = NULL;
    Node->right = NULL;

    return(Node);
}

/* Computes the number of nodes in a tree. */
int size(node* node)
{
    if (node == NULL)
        return 0;
    else
        return(size(node->left) + 1 + size(node->right));
}
```

```
/* Driver code*/
int main()
{
    node *root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);

    cout << "Size of the tree is " << size(root);
    return 0;
}
```

Output:

```
Size of the tree is 5
```

Time Complexity: O(N)

As every node is visited once.

Auxiliary Space: O(N)

The extra space is due to the recursion call stack and the worst case occurs when the tree is either left skewed or right skewed.

Since this program is similar to traversal of tree, time and space complexities will be same as Tree traversal

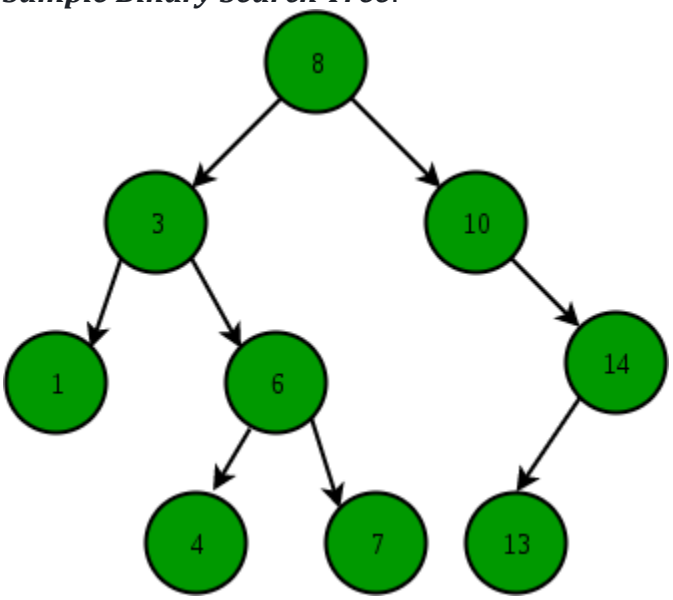
Binary Search Tree

Introduction to Binary Search Trees

Binary Search Tree is a node-based binary tree data structure which has the following properties:

- The left subtree of a node contains only nodes with keys lesser than or equal to the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree. There must be no duplicate nodes.

Sample Binary Search Tree:



The above properties of Binary Search Tree provide an ordering among keys so that the operations like search, minimum and maximum can be done fast in comparison to normal Binary Trees. If there is no ordering, then we may have to compare every key to search a given key.

Searching a Key

Using the property of Binary Search Tree, we can search for an element in O(h) time complexity where **h** is the height of the given BST.

To search a given key in Binary Search Tree, first compare it with root, if the key is present at root, return root. If the key is greater than the root's key, we recur for the right subtree of the root node. Otherwise, we recur for the left subtree.

Implementation:

C++Java

```
// C++ function to search a given key in a given BST

/* Node Structure:

   struct Node
   {
       int key;
```

```
        Node* left, right;
    };
*/

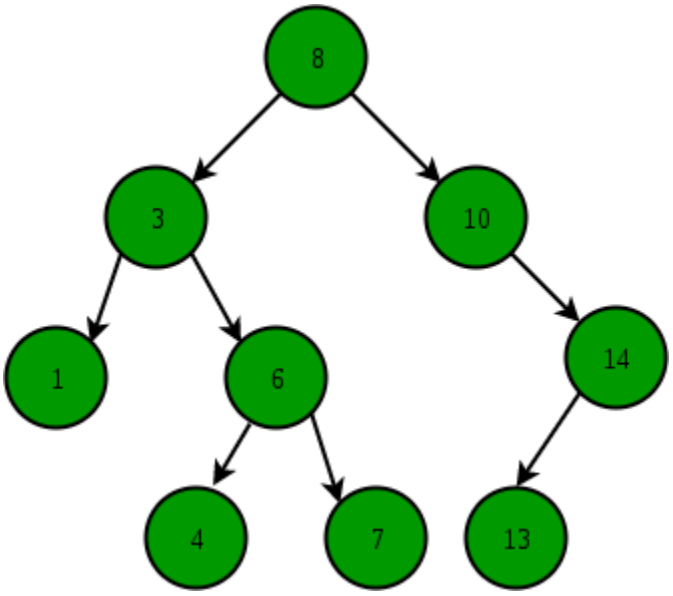
struct node* search(struct node* root, int key)
{
    // Base Cases: root is null or key is present at root
    if (root == NULL || root->key == key)
        return root;

    // Key is greater than root's key
    if (root->key < key)
        return search(root->right, key);

    // Key is smaller than root's key
    return search(root->left, key);
}
```

Illustration to search a key 6 in the below BST:

1. Start from root.
2. Compare the key element with root, if less than root, then recur for left subtree, else recur for right subtree.
3. If element to search is found anywhere, return true, else return false.



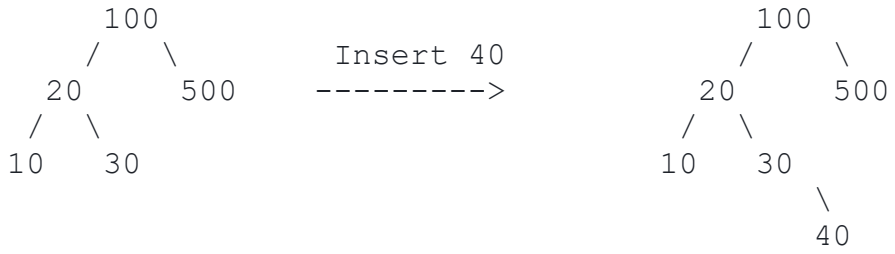
Step 1: Compare 6 with 8.
Since 6 is less than 8,

```
        move to left subtree.
Step 2: Compare 6 with 3.
        Since 6 is greater than 3,
        move to its right subtree.
Step 3: Comapre 6 with 6.
        Node Found.
```

Insertion of a Key

Inserting a new node in the Binary Search Tree is always done at the leaf nodes to maintain the order of nodes in the Tree. The idea is to start searching the given node to be inserted from the root node till we hit a leaf node. Once a leaf node is found, the new node is added as a child of the leaf node.

For Example:



Implementation:

C++Java

```cpp
// C++ program to demonstrate insert
// operation in binary search tree

#include<bits/stdc++.h>
using namespace std;

// Binary Search Tree node
struct node
{
    int key;
    struct node *left, *right;
};

// A utility function to create a new BST node
```

```
struct node *newNode(int item)
{
    node *temp = new node;
    temp->key = item;
    temp->left = temp->right = NULL;
    return temp;
}

// A utility function to do inorder traversal of BST
void inorder(struct node *root)
{
    if (root != NULL)
    {
        inorder(root->left);
        cout<<root->key<<" ";
        inorder(root->right);
    }
}

// A utility function to insert a new node
// with given key in BST
struct node* insert(struct node* node, int key)
{
    /* If the tree is empty, return a new node */
    if (node == NULL) return newNode(key);

    /* Otherwise, recur down the tree */
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);

    /* return the (unchanged) node pointer */
    return node;
}

// Driver Program to test above functions
int main()
{
    /* Let us create following BST
          50
       /     \
      30      70
     / \    / \
    20 40  60 80 */
    struct node *root = NULL;
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 20);
```

```
    insert(root, 40);
    insert(root, 70);
    insert(root, 60);
    insert(root, 80);

    // print inoder traversal of the BST
    inorder(root);

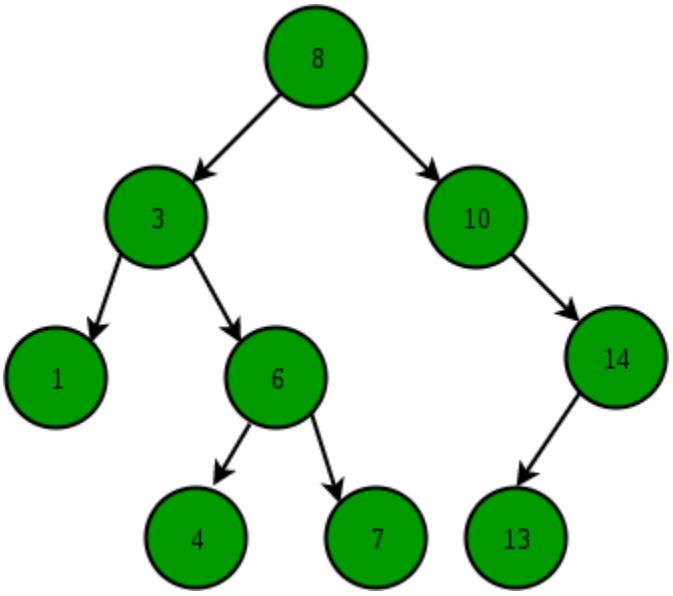
    return 0;
}
```

Output:

```
20 30 40 50 60 70 80
```

Illustration to insert 2 in the below tree:

1. Start from root.
2. Compare the element to be inserted with root, if it is less than root, then recur for left-subtree, else recur for right-subtree.
3. After reaching end,just insert that node at left(if less than current leaf node) else right.



Element to be inserted: 2

Step 1: Compare 2 with 8.
Since 2 is less than 8,

```
        move to left subtree.
Step 2: Compare 2 with 3.
        Since 2 is less than 3,
        move to its left subtree.
Step 3: Comapre 2 with 1.
        Since 2 is greater than 1 and also 1 is a leaf node.
        Insert 2 as its right child of node 1.
```

Time Complexity: The worst case time complexity of search and insert operations is O(h) where **h** is height of Binary Search Tree. In the worst case, we may have to travel from root to the deepest leaf node. The height of a skewed tree may become n and the time complexity of search and insert operation may become O(n).

Deletion in a Binary Search Tree

We have discussed the Search and Insert operations in BST in the previous post. In this post, delete operation is discussed.

That is, given a Binary Search Tree and a node to be deleted. The task is to search that node in the given BST and delete it from the BST if it is present.

When we delete a node, three cases may arise:

1. ***Node to be deleted is leaf:*** Simply remove from the tree.



2.
3. ***Node to be deleted has only one child:*** Copy the child to the node and delete the child.



4.
5.

6. ***Node to be deleted has two children:*** Find inorder successor of the node. Copy contents of the inorder successor to the node and delete the inorder successor. Note that inorder predecessor can also be used.



7.
8.

Note: *The inorder successor can be obtained by finding the minimum value in the right child of the node.*

Below is the implementation of the above three cases:

C++Java

```
// C++ program to demonstrate delete
// operation in binary search tree
#include<bits/stdc++.h>
using namespace std;

// BST Node
struct node
{
    int key;
    struct node *left, *right;
};

// A utility function to create a new BST node
struct node *newNode(int item)
{
    node *temp = new node;
    temp->key = item;
    temp->left = temp->right = NULL;
    return temp;
}

// A utility function to do inorder traversal of BST
void inorder(struct node *root)
{
    if (root != NULL)
    {
        inorder(root->left);
        cout<<root->key<<" ";
        inorder(root->right);
```

```
    }
}

// A utility function to insert a new node
// with given key in BST
struct node* insert(struct node* node, int key)
{
    // If the tree is empty, return a new node
    if (node == NULL)
        return newNode(key);

    // Otherwise, recur down the tree
    if (key < node->key)
        node->left = insert(node->left, key);
    else
        node->right = insert(node->right, key);

    // return the (unchanged) node pointer
    return node;
}

// Function to find the minimum
// valued node in a BST
struct node * minValueNode(struct node* node)
{
    struct node* current = node;

    /* loop down to find the leftmost leaf */
    while (current->left != NULL)
        current = current->left;

    return current;
}

// Function to delete a given node from the BST
struct node* deleteNode(struct node* root, int key)
{
    // base case
    if (root == NULL) return root;

    // If the key to be deleted is smaller than the root's key,
    // then it lies in left subtree
    if (key < root->key)
        root->left = deleteNode(root->left, key);

    // If the key to be deleted is greater than the root's key,
    // then it lies in right subtree
    else if (key > root->key)
        root->right = deleteNode(root->right, key);
```

```
    // if key is same as root's key, then This is the node
    // to be deleted
    else
    {
        // node with only one child or no child
        if (root->left == NULL)
        {
            struct node *temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL)
        {
            struct node *temp = root->left;
            free(root);
            return temp;
        }

        // node with two children: Get the inorder successor (smallest
        // in the right subtree)
        struct node* temp = minValueNode(root->right);

        // Copy the inorder successor's content to this node
        root->key = temp->key;

        // Delete the inorder successor
        root->right = deleteNode(root->right, temp->key);
    }

    return root;
}

// Driver Program to test above functions
int main()
{
    /* Let us create following BST
          50
        /     \
       30      70
      /  \    / \
     20  40 60  80 */
    struct node *root = NULL;
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 70);
    root = insert(root, 60);
```

```
    root = insert(root, 80);

    cout<<"Inorder traversal of the given tree \n";
    inorder(root);

    cout<<"\nDelete 20\n";
    root = deleteNode(root, 20);
    cout<<"Inorder traversal of the modified tree \n";
    inorder(root);

    cout<<"\nDelete 30\n";
    root = deleteNode(root, 30);
    cout<<"Inorder traversal of the modified tree \n";
    inorder(root);

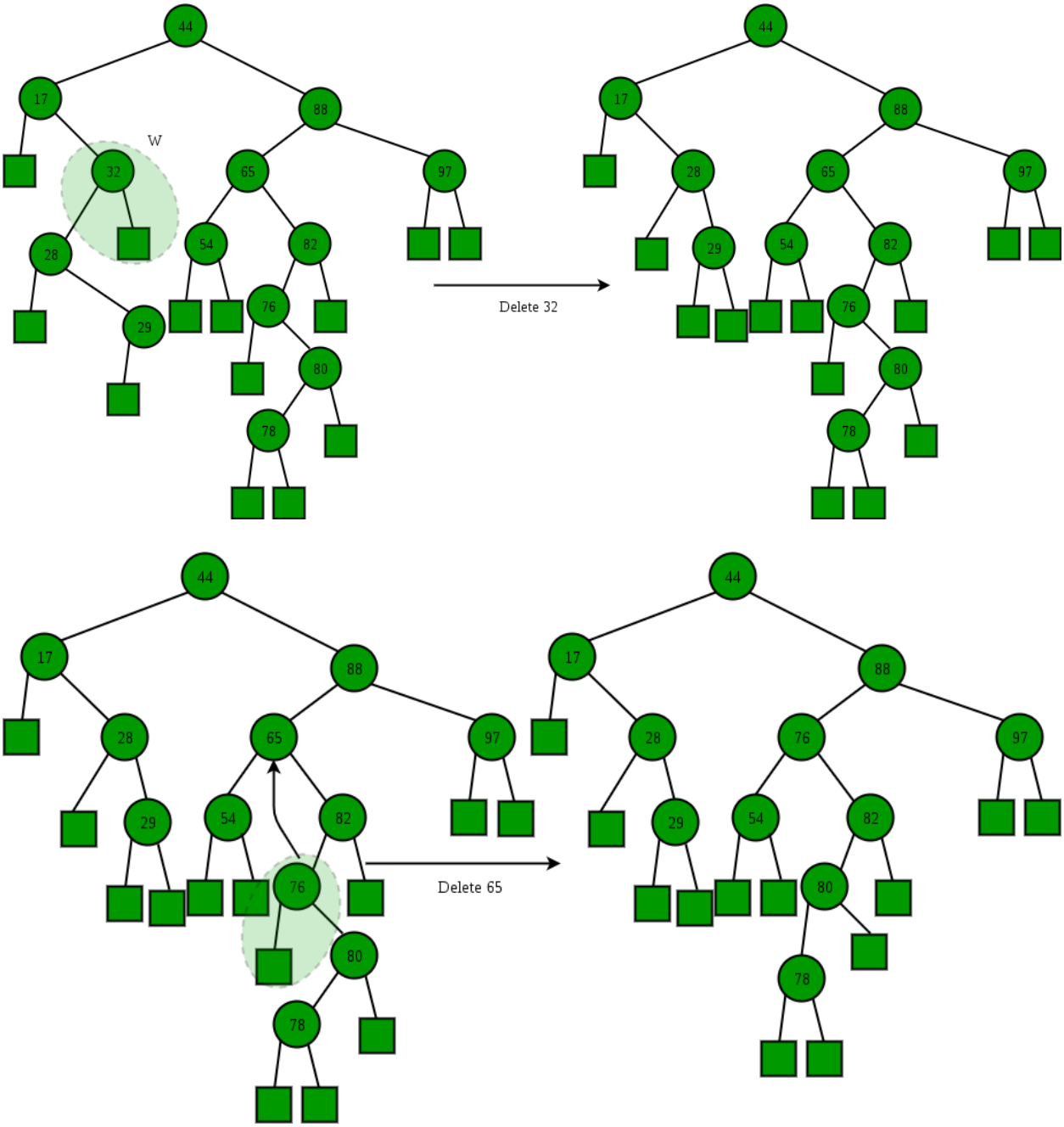
    cout<<"\nDelete 50\n";
    root = deleteNode(root, 50);
    cout<<"Inorder traversal of the modified tree \n";
    inorder(root);

    return 0;
}
```

Output:

```
Inorder traversal of the given tree
20 30 40 50 60 70 80
Delete 20
Inorder traversal of the modified tree
30 40 50 60 70 80
Delete 30
Inorder traversal of the modified tree
40 50 60 70 80
Delete 50
Inorder traversal of the modified tree
40 60 70 80
```

Illustration:



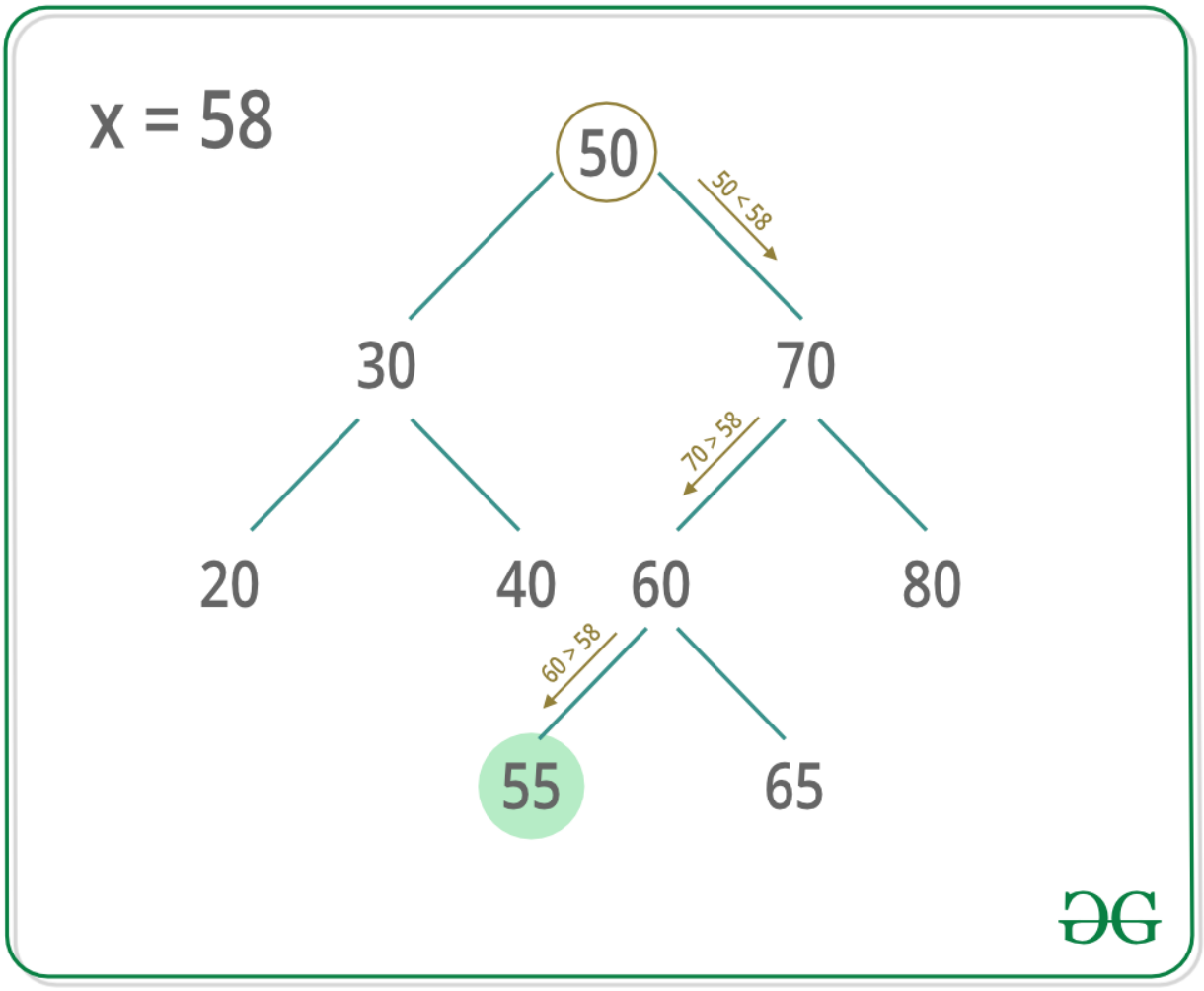
Floor in BST

Given a Binary Search Tree and a number x, find floor of x in the given BST.

```
Input : x = 14 and root of below tree
            10
          /   \
```

```
           5     15
               /   \
             12     30
Output : 12

Input : x = 15 and root of below tree
              10
            /   \
           5     15
               /   \
             12     30
Output : 15
```



A **simple solution** is to traverse the tree using (Inorder or Preorder or Postorder) and

keep track of closest smaller or same element. Time complexity of this solution is **O(n)** where n is total number of Nodes in BST.
We can **efficiently** find closest smaller or same element in **O(h)** time where h is height of BST. Algorithm to find the floor of a key in a binary search tree (BST):

```
1 Start at the root Node.
2 If root->data == key,
     floor of the key is equal
     to the root.
3 Else if root->data > key, then
     floor of the key must lie in the
     left subtree.
4 Else floor may lie in the right subtree
  but only if there is a value lesser than
  or equal to the key.If not, then root is
  the key.
```

For finding ceil of BST you can refer to this article.

C++Java

```cpp
// C++ code to find floor of a key in BST
#include <bits/stdc++.h>
using namespace std;

/*Structure of each Node in the tree*/
struct Node {
    int data;
    Node *left, *right;
};

/*This function is used to create and
initializes new Nodes*/
Node* newNode(int key)
{
    Node* temp = new Node;
    temp->left = temp->right = NULL;
    temp->data = key;
    return temp;
}

/* This function is used to insert
 new values in BST*/
Node* insert(Node* root, int key)
{
```

```
    if (!root)
        return newNode(key);
    if (key < root->data)
        root->left = insert(root->left, key);
    else
        root->right = insert(root->right, key);
    return root;
}

/*This function is used to find floor of a key*/
int floor(Node* root, int key)
{
    if (!root)
        return INT_MAX;

    /* If root->data is equal to key */
    if (root->data == key)
        return root->data;

    /* If root->data is greater than the key */
    if (root->data > key)
        return floor(root->left, key);

    /* Else, the floor may lie in right subtree
      or may be equal to the root*/
    int floorValue = floor(root->right, key);
    return (floorValue <= key) ? floorValue : root->data;
}

int main()
{
    /* Let us create following BST
              7
            /    \
           5      10
         /  \    /  \
        3    6   8   12 */
    Node* root = NULL;
    root = insert(root, 7);
    insert(root, 10);
    insert(root, 5);
    insert(root, 3);
    insert(root, 6);
    insert(root, 8);
    insert(root, 12);
    cout << floor(root, 9) << endl;
    return 0;
}
```

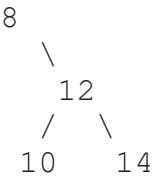
Output:

```
8
```

Ceil in BST

Given a binary tree and a key(node) value, find the ceil value for that particular key value.
Ceil Value Node: Node with the smallest data larger than or equal to the key value.

For example, Let's consider the Binary Tree below -



Key: 11 **Ceil:** 12
Key: 1 **Ceil:** 2
Key: 6 **Ceil:** 6
Key: 15 **Ceil:** -1

There are numerous applications where we need to find the floor/ceil value of a key in a binary search tree or sorted array. For example, consider designing a memory management system in which free nodes are arranged in BST. Find the best fit for the input request.

Algorithm:

```
Imagine we are moving down the tree, and assume we are root
node.
The comparison yields three possibilities,

A) Root data is equal to key. We are done, root data is ceil
value.

B) Root data < key value, certainly the ceil value can't be in
left subtree.
   Proceed to search on right subtree as reduced problem
instance.
```

C) Root data > key value, the ceil value *may be* in left subtree.
We may find a node with is larger data than key value in left subtree,
if not the root itself will be ceil node.

Here is the code for ceil value:

C++Java

```cpp
// Program to find ceil of a given value in BST
#include <bits/stdc++.h>
using namespace std;

/* A binary tree node has key, left child and right child */
class node {
public:
    int key;
    node* left;
    node* right;
};

/* Helper function that allocates a new node with the given key and
NULL left and right pointers.*/
node* newNode(int key)
{
    node* Node = new node();
    Node->key = key;
    Node->left = NULL;
    Node->right = NULL;
    return (Node);
}

// Function to find ceil of a given input in BST. If input is more
// than the max key in BST, return -1
int Ceil(node* root, int input)
{
    // Base case
    if (root == NULL)
        return -1;

    // We found equal key
    if (root->key == input)
        return root->key;

    // If root's key is smaller, ceil must be in right subtree
    if (root->key < input)
        return Ceil(root->right, input);
```

```
    // Else, either left subtree or root has the ceil value
    int ceil = Ceil(root->left, input);
    return (ceil >= input) ? ceil : root->key;
}

// Driver program to test above function
int main()
{
    node* root = newNode(8);

    root->left = newNode(4);
    root->right = newNode(12);

    root->left->left = newNode(2);
    root->left->right = newNode(6);

    root->right->left = newNode(10);
    root->right->right = newNode(14);

    for (int i = 0; i < 16; i++)
        cout << i << " " << Ceil(root, i) << endl;

    return 0;
}

// This code is contributed by rathbhupendra
```

Output

```
0 2
1 2
2 2
3 4
4 4
5 6
6 6
7 8
8 8
9 10
10 10
11 12
12 12
13 14
14 14
15 -1
```

Time complexity: O(logn)

Auxiliary Space: O(logn)

Self Balancing BST

Self-Balancing Binary Search Trees are ***height-balanced*** binary search trees that automatically keeps height as small as possible when insertion and deletion operations are performed on tree. The height is typically maintained in order of Log n so that all operations take O(Log n) time on average.

How do Self-Balancing-Tree maintain height?
A typical operation done by trees is rotation. Following are two basic operations that can be performed to re-balance a BST without violating the BST property (keys(left) < key(root) < keys(right)).

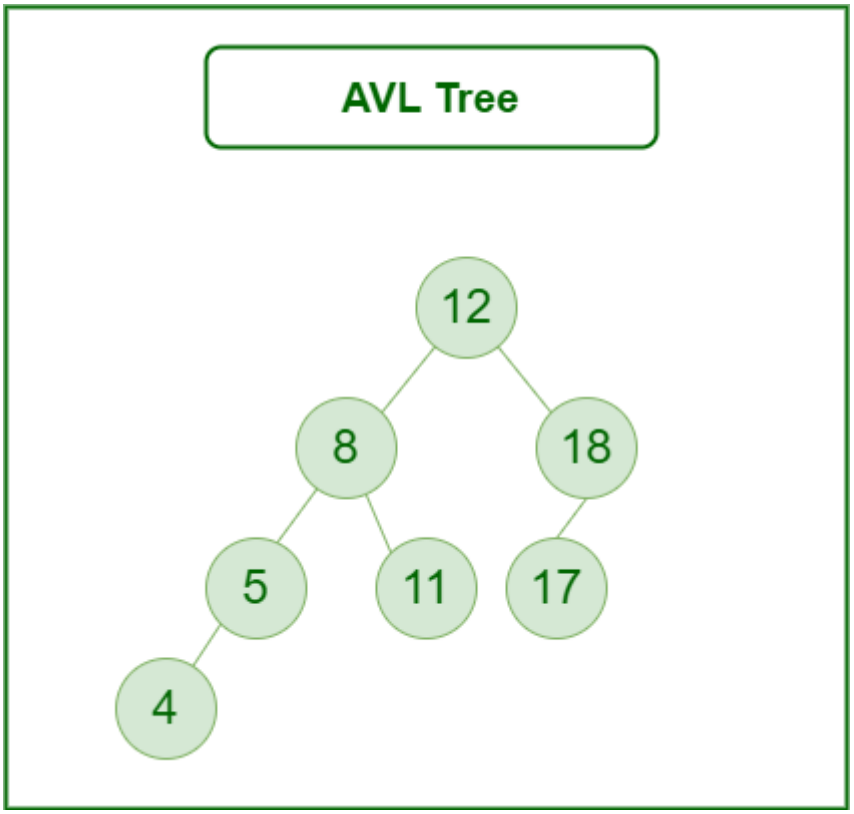
1. Left Rotation
2. Right Rotation

```
T1, T2 and T3 are subtrees of the tree
rooted with y (on the left side) or x (on
the right side)
     y                               x
    / \     Right Rotation          /  \
   x   T3   - - - - - - - >        T1   y
  / \       < - - - - - - -            / \
 T1  T2     Left Rotation            T2  T3
Keys in both of the above trees follow the
following order
 keys(T1) < key(x) < keys(T2) < key(y) < keys(T3)
So BST property is not violated anywhere.
```

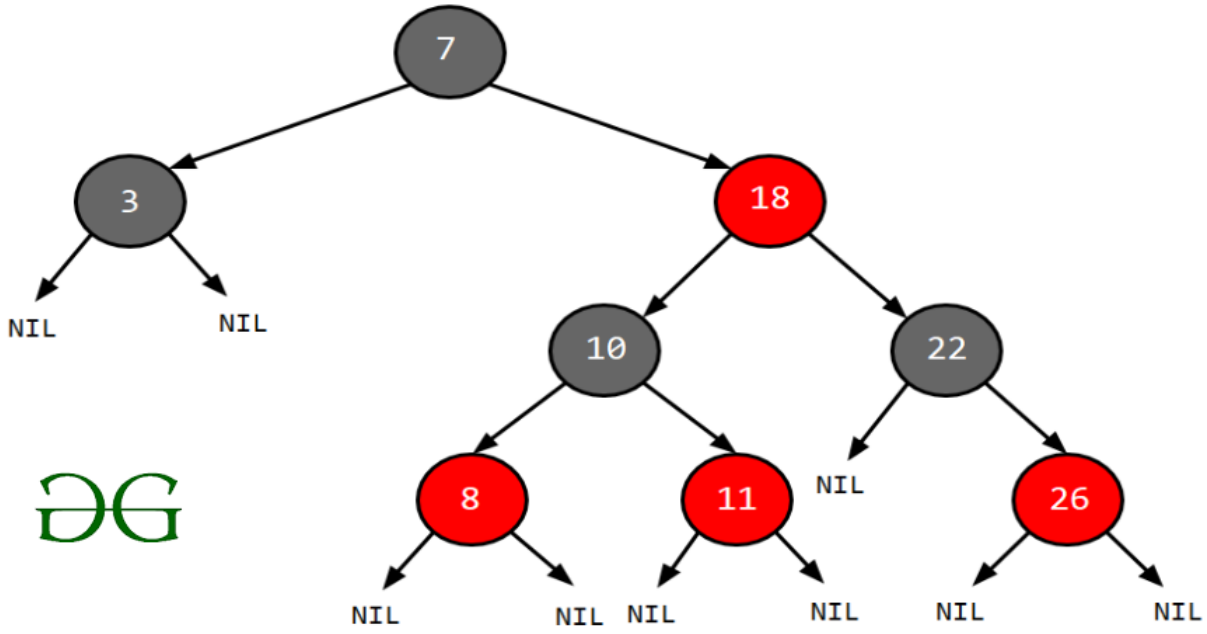
Examples: The most common examples of self-balancing binary search trees are

- AVL Tree
- Red Black Tree and
- Splay Tree

AVL Tree:
An AVL tree defined as a self-balancing Binary Search Tree (BST) where the difference between heights of left and right subtrees for any node cannot be more than one.



Red-Black Tree:
Red-Black tree is a self-balancing **binary search tree** in which every node is colored with either red or black. The root and leaf nodes (i.e., NULL nodes) are always marked as black.



Splay Tree:

Splay is a self-balancing binary search tree. The basic idea behind splay trees is to bring the most recently accessed or inserted element to the root of the tree by performing a sequence of tree rotations, called splaying.

Basic operations that are performed in a splay tree are:

- Insertion
- Searching
- Deletion
- Rotation (There are two types of rotation in a Splay tree named **zig rotation** and **zag rotation**)

Language Implementations of self-balancing BST:

- set and map in C++ STL.
- TreeSet and TreeMap in Java. Most of the library implementations use Red Black Tree.
- Python standard library does not support Self Balancing BST. In Python, we can use bisect module to keep a set of sorted data. We can also use **PyPi** modules like rbtree (implementation of red-black tree) and pyavl (implementation of AVL tree).

We will compare the efficiency of these trees:

Metric	RB Tree	AVL Tree	Splay Tree
Insertion in worst case	O(logn)	O(logn)	Amortized O(logn)
Maximum height of tree	**2*log(n)**	**1.44*log(n)**	**O(n)**
Search in worst case	O(logn), Moderate	O(logn), Faster	Amortized O(logn), Slower
Efficient Implementation requires	Three pointers with color bit per node	Two pointers with balance factor per node	Only two pointers with no extra information
Deletion in worst case	O(logn)	O(logn)	Amortized O(logn)
Mostly used	As universal data structure	When frequent lookups are required	When same element is retrieved again and again
Real world Application	Database Transactions	Multiset, Multimap, Map, Set, etc.	Cache implementation, Garbage collection Algorithms

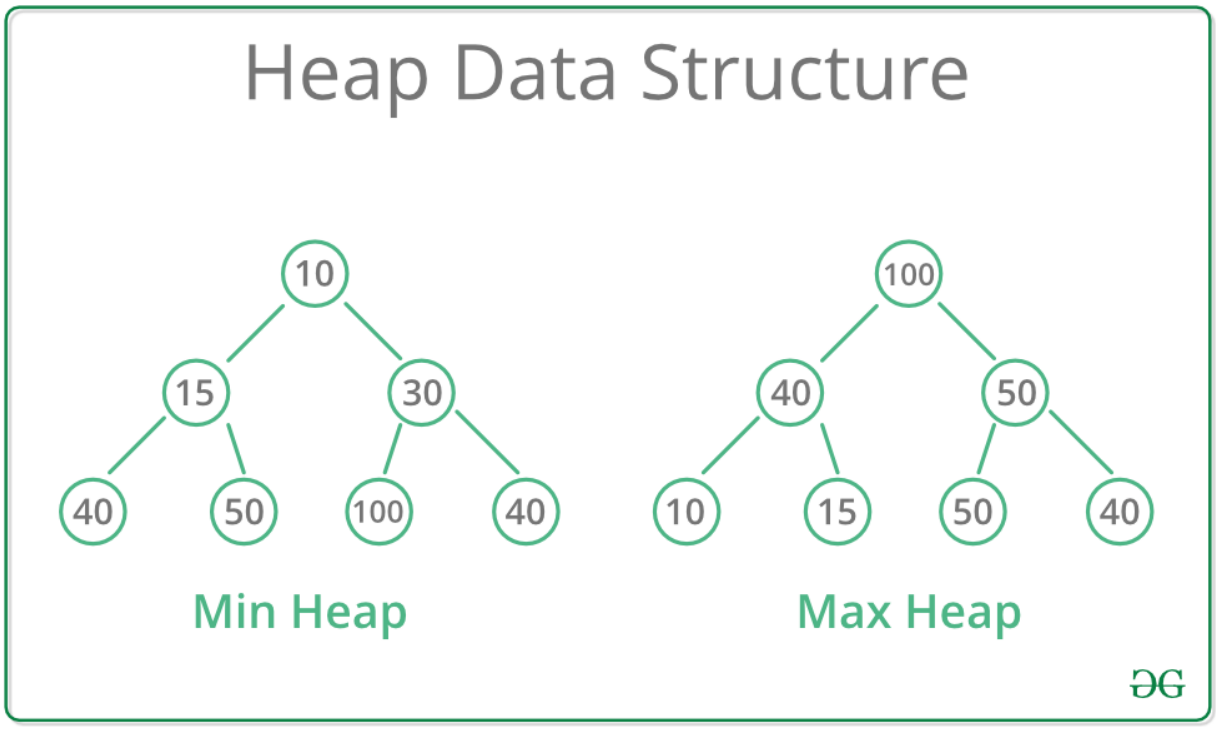
Heap

Binary Heap Introduction

A Heap is a Tree-based data structure, which satisfies the below properties:

1. A Heap is a complete tree (All levels are completely filled except possibly the last level and the last level has all keys as left as possible).
2. A Heap is either Min Heap or Max Heap. In a Min-Heap, the key at root must be minimum among all keys present in the Binary Heap. The same property must be recursively true for all nodes in the Tree. Max Heap is similar to MinHeap.

Binary Heap: A Binary Heap is a heap where each node can have at most two children. In other words, a Binary Heap is a complete Binary Tree satisfying the above-mentioned properties.



Representing Binary Heaps

Since a Binary Heap is a complete Binary Tree, it can be easily represented using Arrays.

- The root element will be at Arr[0].
- Below table shows indexes of other nodes for the ith node, i.e., Arr[i]:

Arr[(i-1)/2]	Returns the parent node
Arr[(2*i)+1]	Returns the left child node
Arr[(2*i)+2]	Returns the right child node

Getting Maximum Element: In a Max-Heap, the maximum element is always present at the root node which is the first element in the array used to represent the Heap. So, the maximum element from a max heap can be simply obtained by returning the root node as Arr[0] in O(1) time complexity.

Getting Minimum Element: In a Min-Heap, the minimum element is always present at the root node which is the first element in the array used to represent the Heap. So, the minimum element from a minheap can be simply obtained by returning the root node as Arr[0] in O(1) time complexity.

Priority Queue in C++

A **priority_queue** in C++ is always implemented using a heap data structure. In C++ this default priority queue we create is always MAX_HEAP. The maximum element by default is

always at the top. Priority queues are a type of container adapters, specifically designed such that the first element of the queue is the greatest of all elements in the queue and elements are in non-increasing order.

Syntax:

```
priority_queue<int> pq;

This creates an empty priority_queue pq
to accept integer values.
```

Few important functions of Priority Queue:

1. **priority_queue::push()** - The **push()** function is used to insert an element in the priority queue. The element is added to the priority queue container and the size of the queue is increased by 1. Firstly, the element is added at the back and at the same time the elements of the priority queue reorder themselves according to priority. By default, the maximum element is brought to the top. **Syntax:**
2. `priority_queue.push(value)`
   Parameters: The value of the element to be inserted is passed as the parameter. **Examples:**
   Input: `pqueue = {5, 2, 1}`
   Operation: `pqueue.push(3);`
   Output: `5, 3, 2, 1`
3. **priority_queue::pop()** - The **pop()** function is used to remove the top element from the priority queue. **Syntax:**
4. `priority_queue.pop()`
   Parameters: No parameters are passed. **Examples:**
   Input: `pqueue = {5, 3, 2, 1}`
   Operation: `pqueue.pop();`
   Output: `pqueue = {3, 2, 1}`

5. **priority_queue::top()** - The **top()** function is used to reference the top(or the largest) element of the priority queue. **Syntax:**
6. `priority_queue.top()`
   Parameters: No value is needed to pass as the parameter. **Return Value:** Direct reference to the top(or the largest) the element of the priority queue container. **Examples:**

```
Input: pqueue.push(5);
       pqueue.push(1);
       pqueue.top();
Output: 5

Input: pqueue.push(5);
       pqueue.push(1);
```

```
        pqueue.push(7);
        pqueue.top();
```
Output: 7

7. **priority_queue::empty()** - The **empty()** function is used to check if the priority queue container is empty or not. **Syntax:**
8. `pqueuename.empty()`

   Parameters: No parameters are passed **Returns:** The method returns True, if priority queue is empty, else False. **Examples:**

   Input : `pqueue = {3, 2, 1}`
   ```
   pqueue.empty();
   ```
   Output : False

   Input : `pqueue`
   ```
   pqueue.empty();
   ```
   Output : True

Program 1: This program shows the implementation of PriorityQueue along with various important above mentioned functions.

CPP

```cpp
// CPP code to illustrate
// priorit_queue and its
// various function
#include <iostream>
#include <queue>
using namespace std;

// Drivers Method
int main()
{

    // Creating a priority queue
    priority_queue<int> pq;

    // Pushing elements into
    // the priority_queue
    pq.push(10);
    pq.push(15);
    pq.push(5);

    // Displaying th e size of
    // the queue
    cout << pq.size() << " ";

    // Displaying the top elements
    // of the queue
    cout << pq.top() << " ";
```

```
    while (pq.empty() == false) {
        cout << pq.top() << " ";
        pq.pop();
    }
    return 0;
}
```

Output:

```
3 15 15 10 5
```

Program 2: Creating a priority_queue where the minimum element is always at the top (construction of a MIN_HEAP).

Syntax:

```
priority_queue<int, vector<int>,
            greater<int<>> pq;
```

Parameters: In the syntax, the *int* represents the data type of the elements of the priority queue. We have also passed a *vector*, which is *used by the priority_queue as an underlined container*. The priority_queue uses the *concept of Heap* but it is built on vector or a dynamic array as a base. Here the *greater()* function has been used to revere the order of the Heap. This converts the MAX_HEAP into a MIN_HEAP.

CPP

```
// CPP code to illustrate
// priority_queue with MIN_HEAP
#include <iostream>
#include <queue>
using namespace std;

// Drivers Method
int main()
{

    // Creating a priority queue
    priority_queue<int, vector<int>,
                   greater<int> >
        pq;
```

```
    // Pushing elements into
    // the priority_queue
    pq.push(10);
    pq.push(15);
    pq.push(5);

    // Displaying th e size of
    // the queue
    cout << pq.size() << " ";

    // Displaying the top elements
    // of the queue
    cout << pq.top() << " ";

    while (pq.empty() == false) {
        cout << pq.top() << " ";
        pq.pop();
    }
    return 0;
}
```

Output:

```
3 5 5 10 15
```

Program 3: Implementing priority queue with an existing vector or an array.
Syntax:

```
priority_queue
pq(begin_iterator, last_iterator);
```

Parameters: Here *begin_iterator* and *last_iterator* refers to the first and the last position of the array respectively. Here the last position refers to the index beyond the last element.

CPP

```
// CPP code to illustrate
// priority_queue
#include <iostream>
#include <queue>
using namespace std;
```

```
// Drivers Method
int main()
{
    // Sample array
    int arr[] = { 10, 5, 15 };

    // Creating priority_queue
    // out of the array
    priority_queue<int> pq(arr, arr + 3);

    // Displaying the priority_queue
    while (pq.empty() == false) {
        cout << pq.top() << " ";
        pq.pop();
    }
}
```

Output:

```
15 10 5
```

Note: Internally, the priority_queue calls make_heap() function and creates a MAX_HEAP by default out of the passed array. This method is more optimal than pushing each element into the queue as the construction of the heap takes O(n) time, where n is the number of elements in the array.

Time Complexities:

- top(): O(1) as the top element can be accessed directly in constant time.
- empty(): O(1) as the checking can be done in constant time.
- push(): O(logn)
- pop(): O(logn)

Program 4: Alternate way to create a MIN_HEAP using a simple method. The trick lies in converting all the array elements into its negative counterpart. This code uses the syntax of MAX_HEAP but the effect is seen that of MIN_HEAP.

CPP

```
// CPP code to illustrate
// priority_queue and MIN_HEAP
#include <iostream>
#include <queue>
```

```cpp
using namespace std;

// Drivers Method
int main()
{
    // Sample array
    int arr[] = { 10, 5, 15 };
    int n = 3;

    // Multiplying the array elements
    // with -1
    for (int i = 0; i < n; i++)
        arr[i] = -arr[i];

    // Creating priority_queue
    // out of the array
    priority_queue<int> pq(arr, arr + 3);

    // Displaying the priority_queue
    while (pq.empty() == false) {
        cout << (-pq.top()) << " ";
        pq.pop();
    }
}
```

Output:

```
5 10 15
```

Note: If you create a priority queue of pairs, elements would be considered according to the first element of pairs which is the default method, be it sorting, priority_queue, map or unordered map of pairs. To make any change to the order, you can use the customised comparator function.

Program 5: Taking the age and height of a person as a structure, write a program to create a priority queue of persons such that all the person can be traversed in decreasing order of heights.
Example:

Input:

```
Person p1: age - 21
           height - 5

Person p2: age - 22
```

```
            height - 6

Person p3: age - 23
            height - 7
```

Output: 7 6 5

CPP

```cpp
// CPP code to illustrate
// priorit_queue and its
// various function
#include <iostream>
#include <queue>
using namespace std;

struct Person {
    int age;
    float ht;

    Person(int a, float h)
    {
        age = a;
        ht = h;
    }
};

struct myCmp {
    bool operator()(Person const& p1, Person const& p2)
    {
```

```
        p1.ht < p2.ht;
    }
};

// Drivers Method
int main()
{

    // Creating a priority queue
    priority_queue<Person, vector<Person>, myCmp> pq;

    // Pushing elements into
    // the priority_queue
    // First Person
    Person p1(21, 5);
    pq.push(p1);

    // Second Person
    Person p2(22, 6);
    pq.push(p2);

    // Third Person
    Person p3(23, 7);
    pq.push(p3);

    // Displaying the top elements
    // of the queue
    while (pq.empty() == false) {
        cout << pq.top().ht << " ";
        pq.pop();
    }
    return 0;
}
```

Output:
7 6 5

Applications: The priority_queue can be used in various standard algorithms like,

- Dijkstra's Algorithm
- Prims Algorithm
- Huffman Algorithm
- Heap Sort
- Any other place where the heaps can be used.

Graph

Introduction to Graphs

A ***Graph*** is a data structure that consists of the following two components:

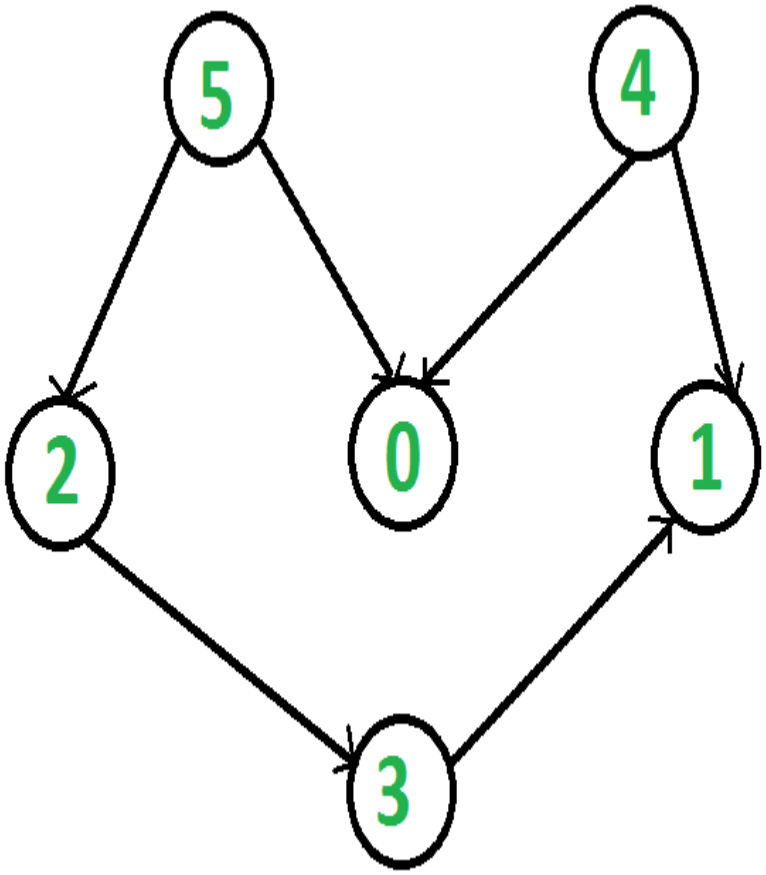
1. A finite set of vertices also called nodes.
2. A finite set of ordered pair of the form (u, v) called as edge. The pair is ordered because (u, v) is not the same as (v, u) in case of a directed graph(digraph). The pair of the form (u, v) indicates that there is an edge from vertex u to vertex v. The edges may contain weight/value/cost.

Graphs are used to represent many real-life applications:

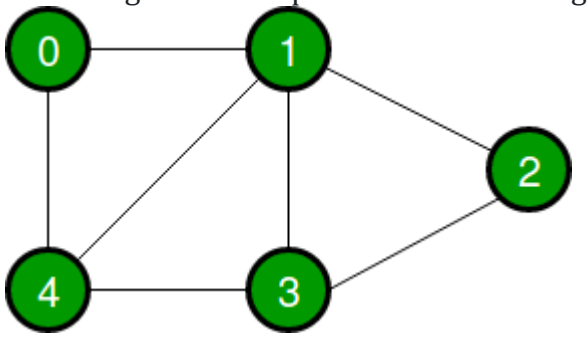
- Graphs are used to represent networks. The networks may include paths in a city or telephone network or circuit network. For example Google GPS
- Graphs are also used in social networks like linkedIn, Facebook. For example, in Facebook, each person is represented with a vertex(or node). Each node is a structure and contains information like person id, name, gender and locale.

Directed and Undirected Graphs

- **Directed Graphs**: The Directed graphs are such graphs in which edges are directed in a single direction. For Example, the below graph is a directed graph:



- **Undirected Graphs**: Undirected graphs are such graphs in which the edges are directionless or in other words bi-directional. That is, if there is an edge between vertices **u** and **v** then it means we can use the edge to go from both **u to v** and **v to u**. Following is an example of an undirected graph with 5 vertices:



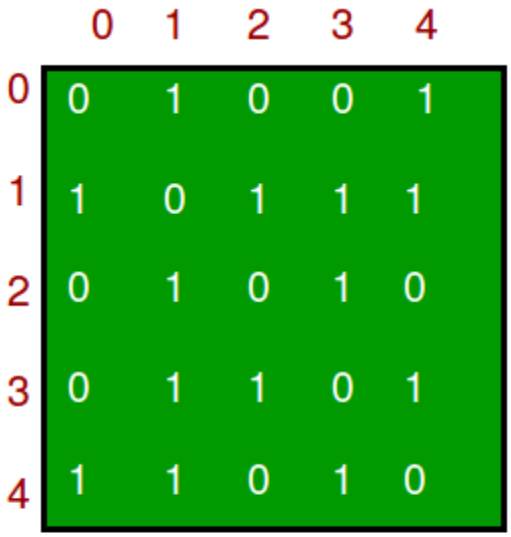
Representing Graphs

Following two are the most commonly used representations of a graph:

1. Adjacency Matrix.
2. Adjacency List.

Let us look at each one of the above two method in details:

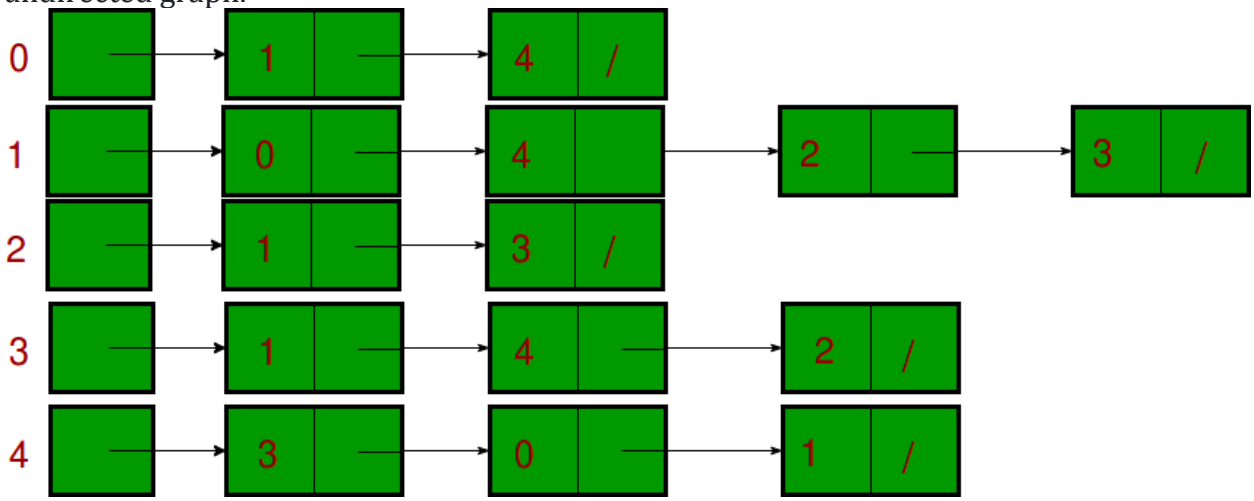
- **Adjacency Matrix:** The Adjacency Matrix is a 2D array of size V x V where V is the number of vertices in a graph. Let the 2D array be adj[][], a slot adj[i][j] = 1 indicates that there is an edge from vertex i to vertex j. Adjacency matrix for undirected graph is always symmetric. Adjacency Matrix is also used to represent weighted graphs. If adj[i][j] = w, then there is an edge from vertex i to vertex j with weight w. The adjacency matrix for the above example undirected graph is:



Pros: Representation is easier to implement and follow. Removing an edge takes O(1) time. Queries like whether there is an edge from vertex 'u' to vertex 'v' are efficient and can be done O(1). ***Cons:*** Consumes more space O(V^2). Even if the graph is sparse(contains less number of edges), it consumes the same space. Adding a vertex is O(V^2) time. Please see this for a sample Python implementation of adjacency matrix.

- **Adjacency List:** Graph can also be implemented using an array of lists. That is every index of the array will contain a complete list. Size of the array is equal to the number of vertices and every index **i** in the array will store the list of vertices

connected to the vertex numbered *i*. Let the array be array[]. An entry array[i] represents the list of vertices adjacent to the ***i***th vertex. This representation can also be used to represent a weighted graph. The weights of edges can be represented as lists of pairs. Following is the adjacency list representation of the above example undirected graph.



Below is the implementation of the adjacency list representation of Graphs: **Note**: In below implementation, we use dynamic arrays (vector in C++/ArrayList in Java) to represent adjacency lists instead of a linked list. The vector implementation has advantages of cache friendliness.

C++Java

```
// A simple representation of graph using STL
#include<bits/stdc++.h>
using namespace std;

// A utility function to add an edge in an
// undirected graph.
void addEdge(vector<int> adj[], int u, int v)
{
    adj[u].push_back(v);
    adj[v].push_back(u);
}

// A utility function to print the adjacency list
// representation of graph
void printGraph(vector<int> adj[], int V)
{
    for (int v = 0; v < V; ++v)
    {
        cout << "\n Adjacency list of vertex "
             << v << "\n head ";
        for (auto x : adj[v])
           cout << "-> " << x;
        printf("\n");
    }
```

```
}

// Driver code
int main()
{
    int V = 5;
    vector<int> adj[V];
    addEdge(adj, 0, 1);
    addEdge(adj, 0, 4);
    addEdge(adj, 1, 2);
    addEdge(adj, 1, 3);
    addEdge(adj, 1, 4);
    addEdge(adj, 2, 3);
    addEdge(adj, 3, 4);
    printGraph(adj, V);
    return 0;
}
```

Output:

```
 Adjacency list of vertex 0
 head -> 1-> 4

 Adjacency list of vertex 1
 head -> 0-> 2-> 3-> 4

 Adjacency list of vertex 2
 head -> 1-> 3

 Adjacency list of vertex 3
 head -> 1-> 2-> 4

 Adjacency list of vertex 4
 head -> 0-> 1-> 3
```

***Pros**:* Saves space O(|V|+|E|). In the worst case, there can be C(V, 2) number of edges in a graph thus consuming O(V^2) space. Adding a vertex is easier. ***Cons**:* Queries like whether there is an edge from vertex u to vertex v are not efficient and can be done O(V).

Breadth First Traversal of a Graph

The ***Breadth First Traversal*** or ***BFS*** traversal of a graph is similar to that of the Level Order Traversal of Trees.

The BFS traversal of Graphs also traverses the graph in levels. It starts the traversal with a

given vertex, visits all of the vertices adjacent to the initially given vertex and pushes them all to a queue in order of visiting. Then it pops an element from the front of the queue, visits all of its neighbours and pushes the neighbours which are not already visited into the queue and repeats the process until the queue is empty or all of the vertices are visited.

The BFS traversal uses an auxiliary boolean array say *visited[]* which keeps track of the visited vertices. That is if **visited[i] = true** then it means that the **i-th** vertex is already visited.

Complete Algorithm:

1. Create a boolean array say ***visited[]*** of size **V+1** where *V* is the number of vertices in the graph.
2. Create a Queue, mark the source vertex visited as **visited[s] = true** and push it into the queue.
3. Until the Queue is non-empty, repeat the below steps:
    - Pop an element from the queue and print the popped element.
    - Traverse all of the vertices adjacent to the vertex poped from the queue.
    - If any of the adjacent vertex is not already visited, mark it visited and push it to the queue.

Illustration:

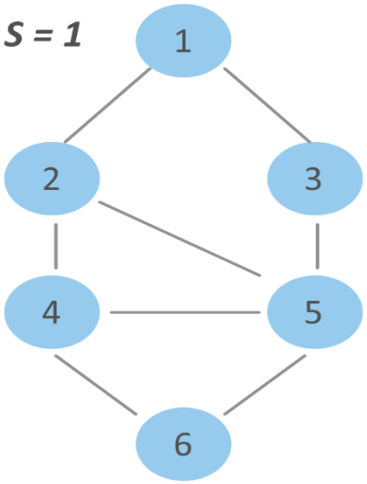
Consider the graph shown in the below Image. The vertices marked **blue** are *not-visited* vertices and the vertices marked **yellow** are *visited*. The vertex numbered **1** is the source vertex, i.e. the BFS traversal will start from the vertex 1.

Following the BFS algorithm:

- Mark the vertex 1 visited in the visited[] array and push it to the queue.

Step: 1

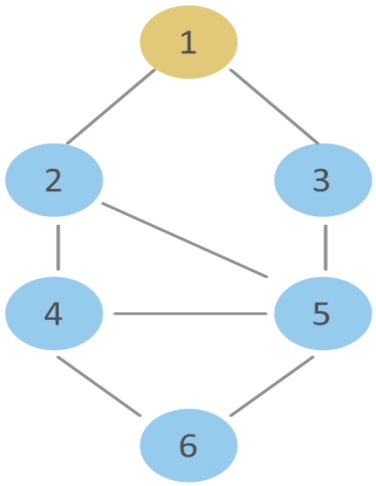
S = 1



	1	2	3	4	5	6
Visited	0	0	0	0	0	0

Queue

Step: 2

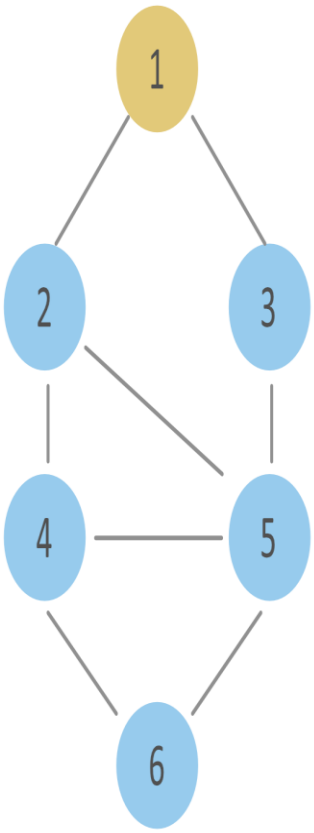


	1	2	3	4	5	6
Visited	1	0	0	0	0	0

Queue 1

Step 3: POP the vertex at the front of queue that is 1, and print it.

Step: 3



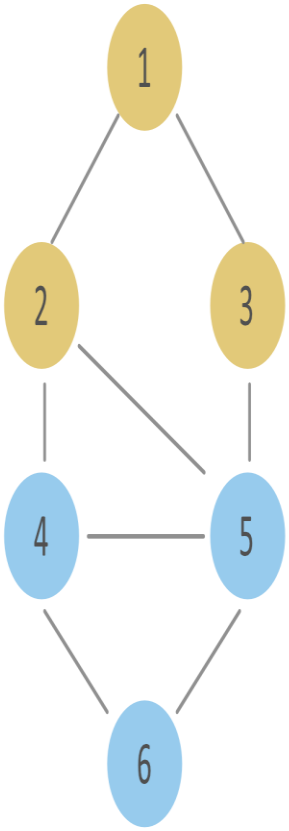
	1	2	3	4	5	6
Visited	1	0	0	0	0	0

Queue

Print 1

Step 4: Check if adjacent vertices of the vertex 1 are not already visited. If not, mark them visited and push them back to the queue.

Step: 4



	1	2	3	4	5	6
Visited	1	1	1	0	0	0

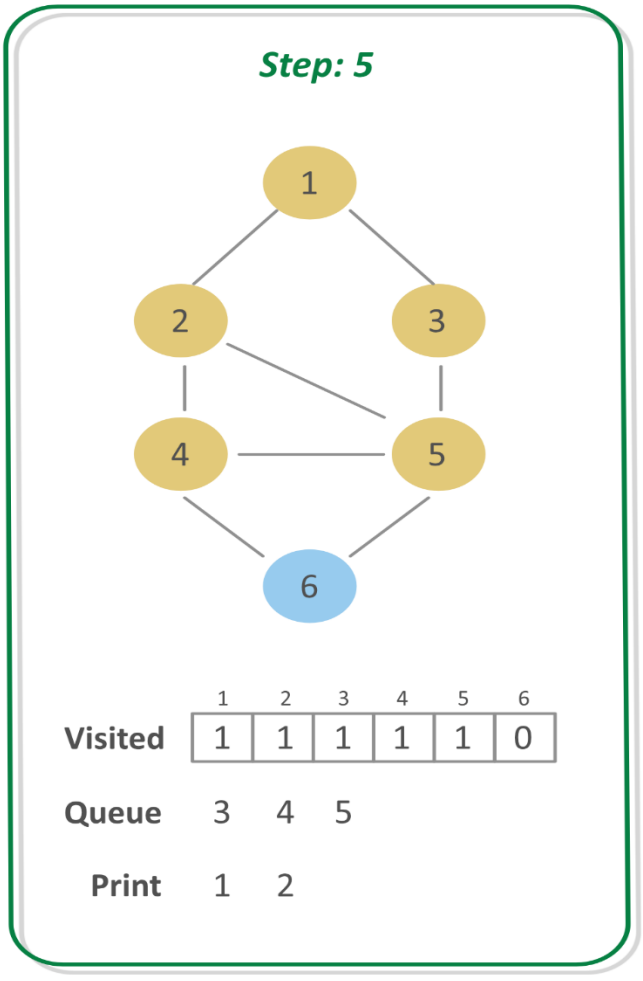
Queue 2 3

Print 1

After removing 1 from queue and printing it, we enqueue its non-vidited Nodes

Step 5:

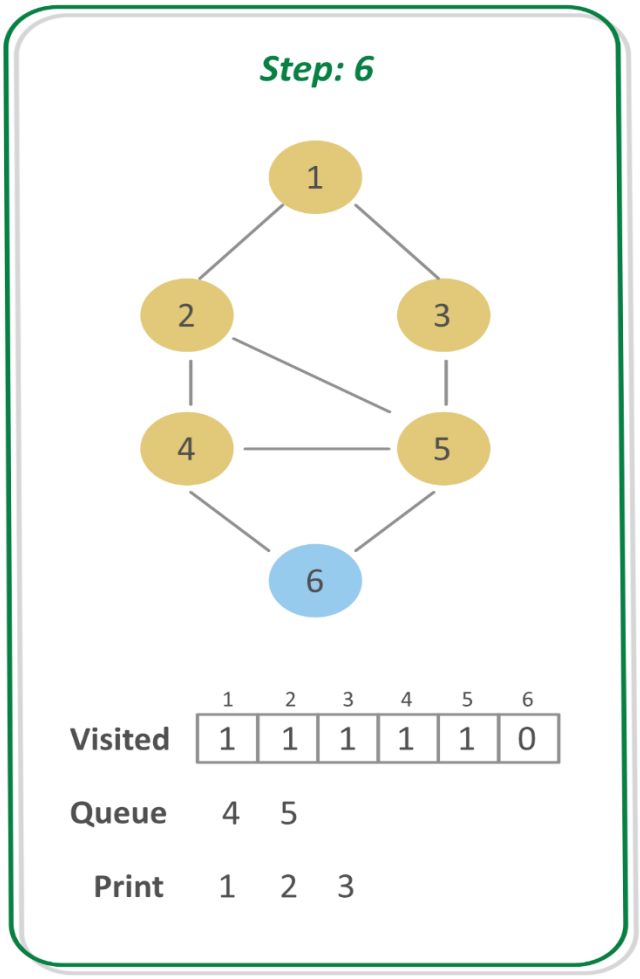
- POP the vertex at front that is 2 and print it.
- Check if the adjacent vertices of 2 are not already visited. If not, mark them visited and push them to queue. So, push 4 and 5.



Step 6:

- POP the vertex at front that is 3 and print it.

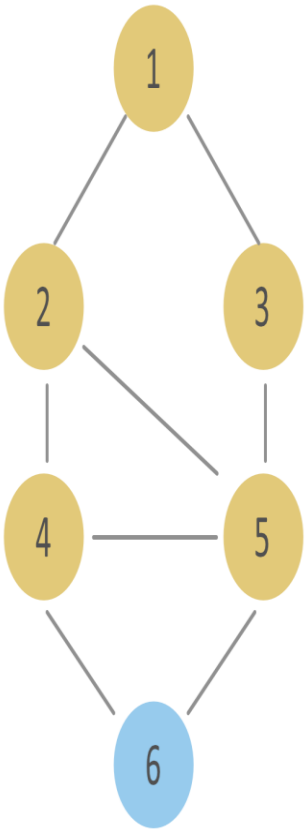
- Check if the adjacent vertices of 3 are not already visited. If not, mark them visited and push them to queue. So, donot push anything.



Step 7:

- POP the vertex at front that is 4 and print it.

Step: 7



	1	2	3	4	5	6
Visited	1	1	1	1	1	0

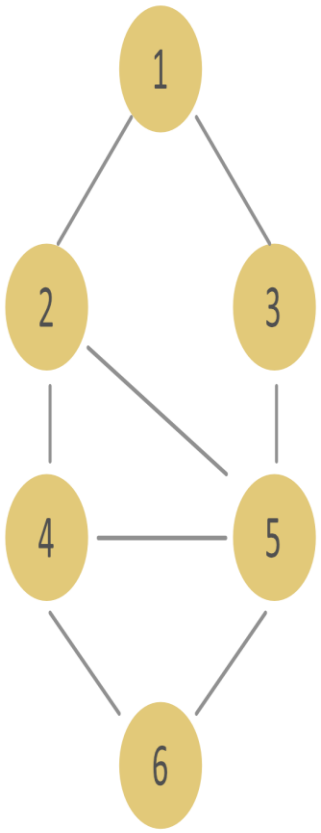
Queue 5

Print 1 2 3 4

Step 8:

- Check if the adjacent vertices of 4 are not already visited. If not, mark them visited and push them to queue. So, push 6 to the queue.

Step: 8



	1	2	3	4	5	6
Visited	1	1	1	1	1	1

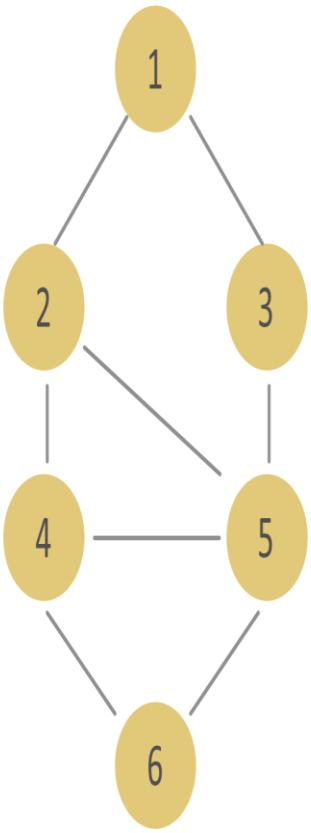
Queue 5 6

Print 1 2 3 4

Step 9:

- POP the vertex at front, that is 5 and print it.
- Since, all of its adjacent vertices are already visited, donot push anything.

Step: 9



	1	2	3	4	5	6
Visited	1	1	1	1	1	1

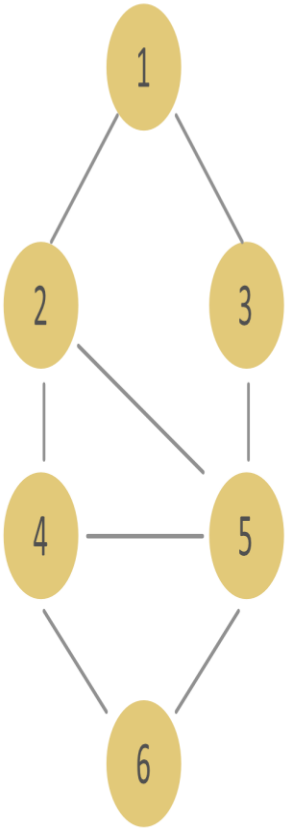
Queue 6

Print 1 2 3 4 5

Step 10:

- POP the vertex at front, that is 6 and print it.
- Since, all of its adjacent vertices are already visited, donot push anything.

Step: 10



	1	2	3	4	5	6
Visited	1	1	1	1	1	1

Queue

Print 1 2 3 4 5 6

Since the Queue is empty now, it means that the complete graph is traversed.

Implementation:

C++Java

```cpp
// C++ program to implement BFS traversal
// of a Graph

#include <bits/stdc++.h>
using namespace std;

// A utility function to add an edge in an
// undirected graph.
void addEdge(vector<int> adj[], int u, int v)
{
    adj[u].push_back(v);
    adj[v].push_back(u);
}

// Function to perform BFS traversal of the given Graph
void BFS(vector<int> adj[], int V)
{
    // Initialize a boolean array
    // to keep track of visited vertices
    bool visited[V + 1];

    // Mark all vertices not-visited initially
    for (int i = 1; i <= V; i++)
        visited[i] = false;

    // Create a Queue to perform BFS
    queue<int> q;

    // Our source vertex is vertex
    // numbered 1
    int s = 1;

    // Mark S visited and Push to queue
    visited[s] = true;
    q.push(s);
```

```
    while (!q.empty()) {
        // Pop element at front and print
        int node = q.front();
        q.pop();

        cout << node << " ";

        // Traverse the nodes adjacent to the currently
        // poped element and push those elements to the
        // queue which are not already visited
        for (int i = 0; i < adj[node].size(); i++) {
            if (visited[adj[node][i]] == false) {
                // Mark it visited
                visited[adj[node][i]] = true;

                // Push it to the Queue
                q.push(adj[node][i]);
            }
        }
    }
}

// Driver code
int main()
{
    int V = 6;
    vector<int> adj[V + 1];
    addEdge(adj, 1, 2);
    addEdge(adj, 1, 3);
    addEdge(adj, 2, 4);
    addEdge(adj, 2, 5);
    addEdge(adj, 3, 5);
    addEdge(adj, 4, 5);
    addEdge(adj, 4, 6);
    addEdge(adj, 5, 6);

    BFS(adj, V);

    return 0;
}
```

Output:

```
1 2 3 4 5 6
```

Applications of BFS

We have earlier discussed Breadth First Traversal Algorithm for Graphs. We have also discussed Applications of Depth First Traversal. In this article, applications of Breadth First Search are discussed.

1. **Shortest Path and Minimum Spanning Tree for unweighted graph** In an unweighted graph, the shortest path is the path with least number of edges. With Breadth First, we always reach a vertex from given source using the minimum number of edges. Also, in case of unweighted graphs, any spanning tree is Minimum Spanning Tree and we can use either Depth or Breadth first traversal for finding a spanning tree.
2. **Peer to Peer Networks.** In Peer to Peer Networks like BitTorrent, Breadth First Search is used to find all neighbor nodes.
3. **Crawlers in Search Engines:** Crawlers build index using Breadth First. The idea is to start from source page and follow all links from source and keep doing same. Depth First Traversal can also be used for crawlers, but the advantage with Breadth First Traversal is, depth or levels of the built tree can be limited.
4. **Social Networking Websites:** In social networks, we can find people within a given distance 'k' from a person using Breadth First Search till 'k' levels.
5. **GPS Navigation systems:** Breadth First Search is used to find all neighboring locations.
6. **Broadcasting in Network:** In networks, a broadcasted packet follows Breadth First Search to reach all nodes.
7. **In Garbage Collection:** Breadth First Search is used in copying garbage collection using Cheney's algorithm. Refer this and for details. Breadth First Search is preferred over Depth First Search because of better locality of reference:
8. **Cycle detection in undirected graph:** In undirected graphs, either Breadth First Search or Depth First Search can be used to detect cycle. We can use BFS to detect cycle in a directed graph also,
9. **Ford–Fulkerson algorithm** In Ford-Fulkerson algorithm, we can either use Breadth First or Depth First Traversal to find the maximum flow. Breadth First Traversal is preferred as it reduces worst case time complexity to $O(VE^2)$.
10. **To test if a graph is Bipartite** We can either use Breadth First or Depth First Traversal.
11. **Path Finding** We can either use Breadth First or Depth First Traversal to find if there is a path between two vertices.
12. **Finding all nodes within one connected component:** We can either use Breadth First or Depth First Traversal to find all nodes reachable from a given node.

Many algorithms like Prim's Minimum Spanning Tree and Dijkstra's Single Source Shortest Path use structure similar to Breadth First Search.

There can be many more applications as Breadth First Search is one of the core algorithms for Graphs.

Depth First Traversal of a Graph

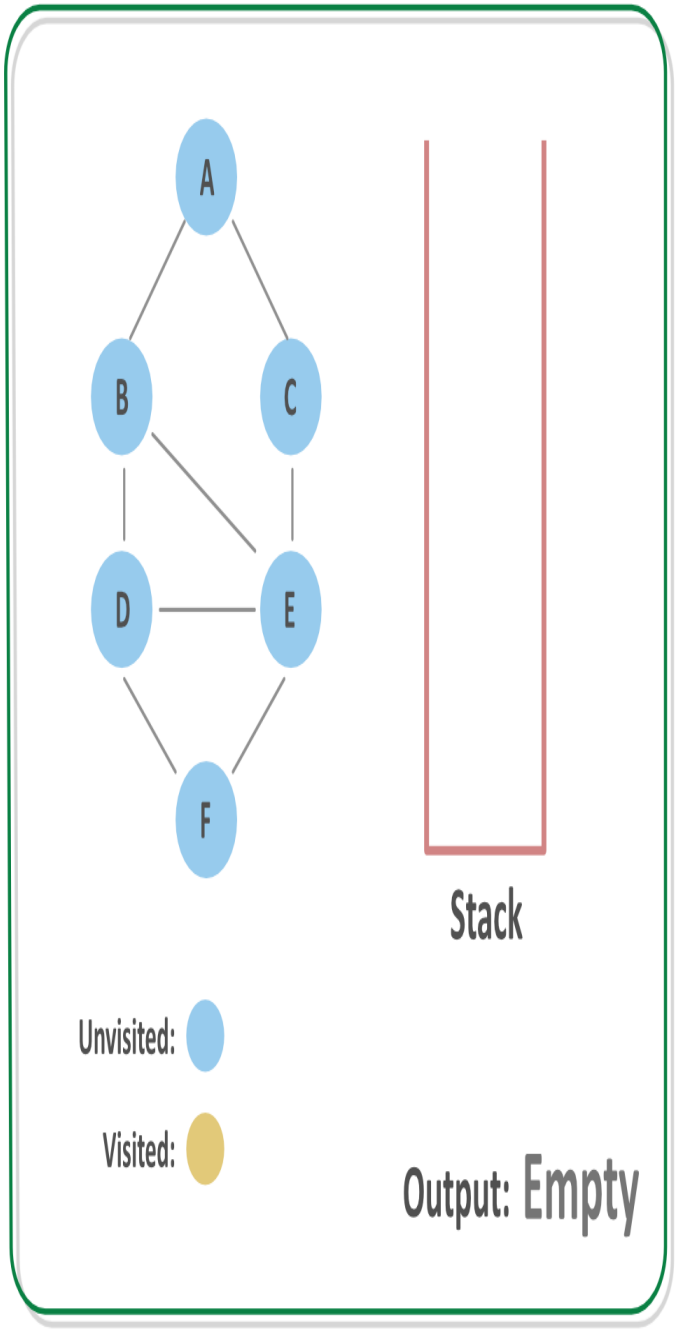
The Depth-First Traversal or the DFS traversal of a Graph is used to traverse a graph depth wise. That is, it in this traversal method, we start traversing the graph from a node and keep on going in the same direction as far as possible. When no nodes are left to be traversed along the current path, backtrack to find a new possible path and repeat this process until all of the nodes are visited.

We can implement the DFS traversal algorithm using a recursive approach. While performing the DFS traversal the graph may contain a cycle and the same node can be visited again, so in order to avoid this we can keep track of visited array using an auxiliary array. On each step of the recursion mark, the current vertex visited and call the recursive function again for all the adjacent vertices.

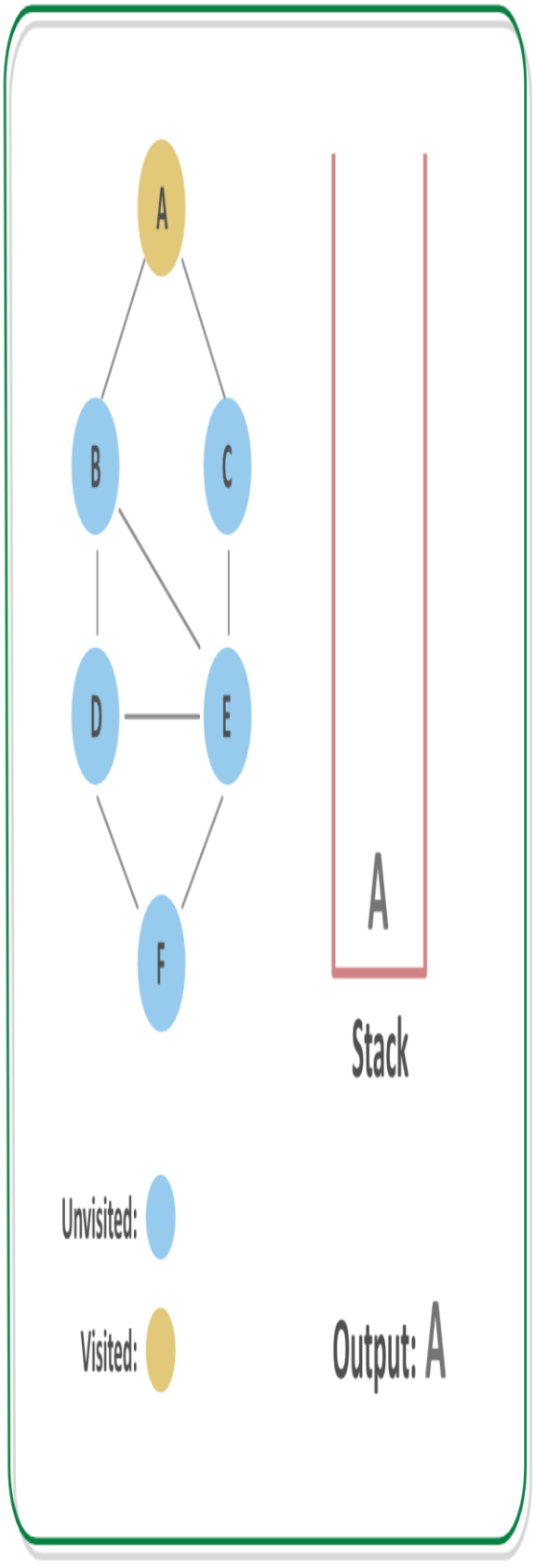
Illustration:

Step 1: Consider the below graph and apply the DFS algorithm recursively for every node

using an auxiliary stack for recursive calls and an array to keep track of visited vertices.

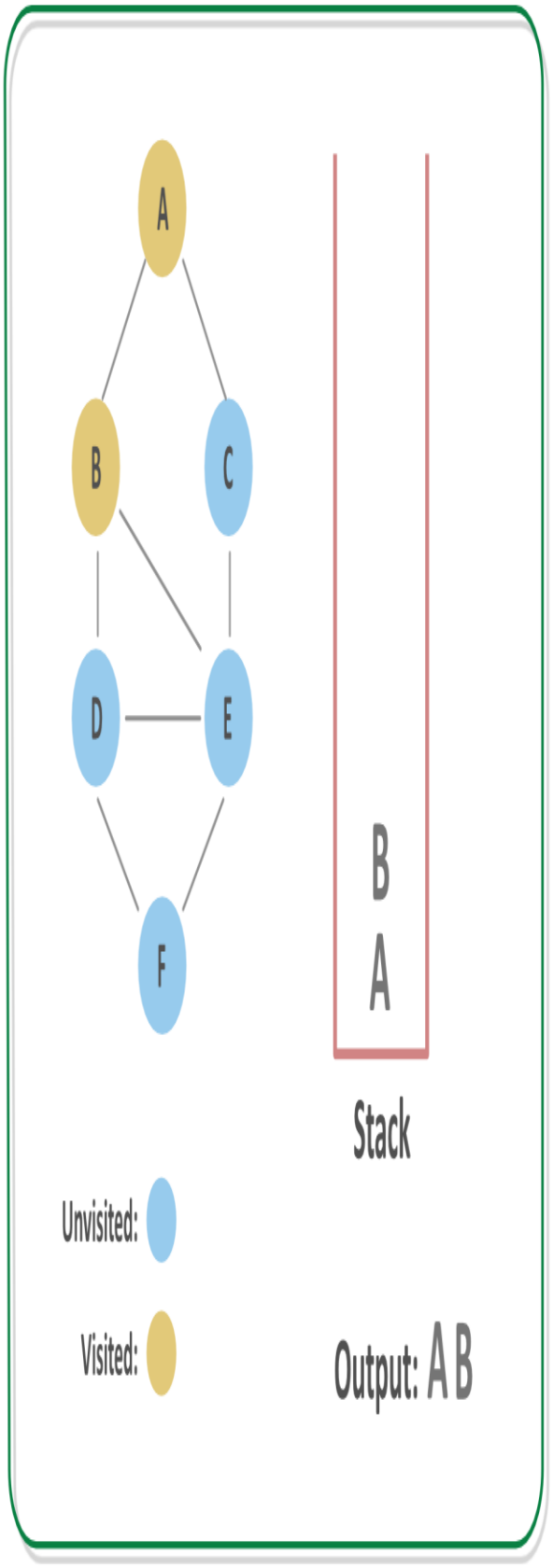


Step 2: Process the vertex A, mark it visited and call DFS for its adjacent vertices. Suppose the first adjacent vertex processed is B. The vertex A is put on the auxilary stack for now.


A
B
C
D
E
F
A
Stack
Unvisited:
Visited:
Output: A

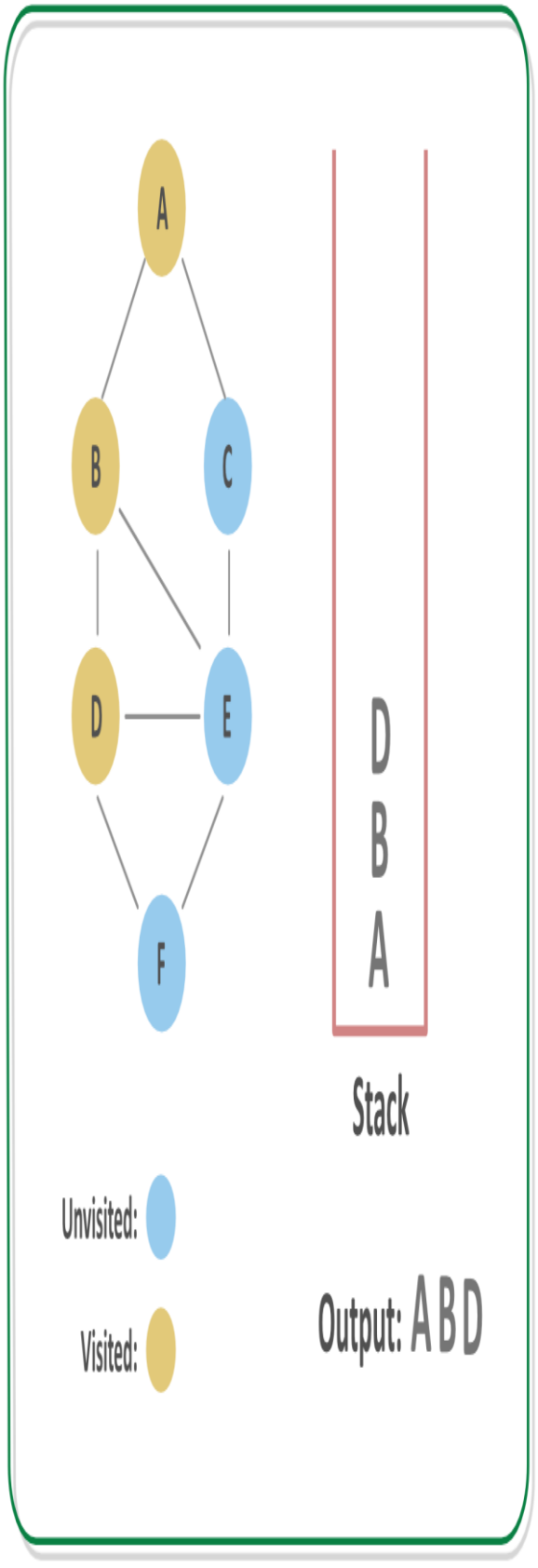
Step 3: Process the vertex B, mark it visited and call DFS for its adjacent vertices. Suppose

the first adjacent vertex processed is D. The vertex B is put on the auxilary stack for now.


A
B
C
D
E
F
B
A
Stack
Unvisited:
Visited:
Output: A B

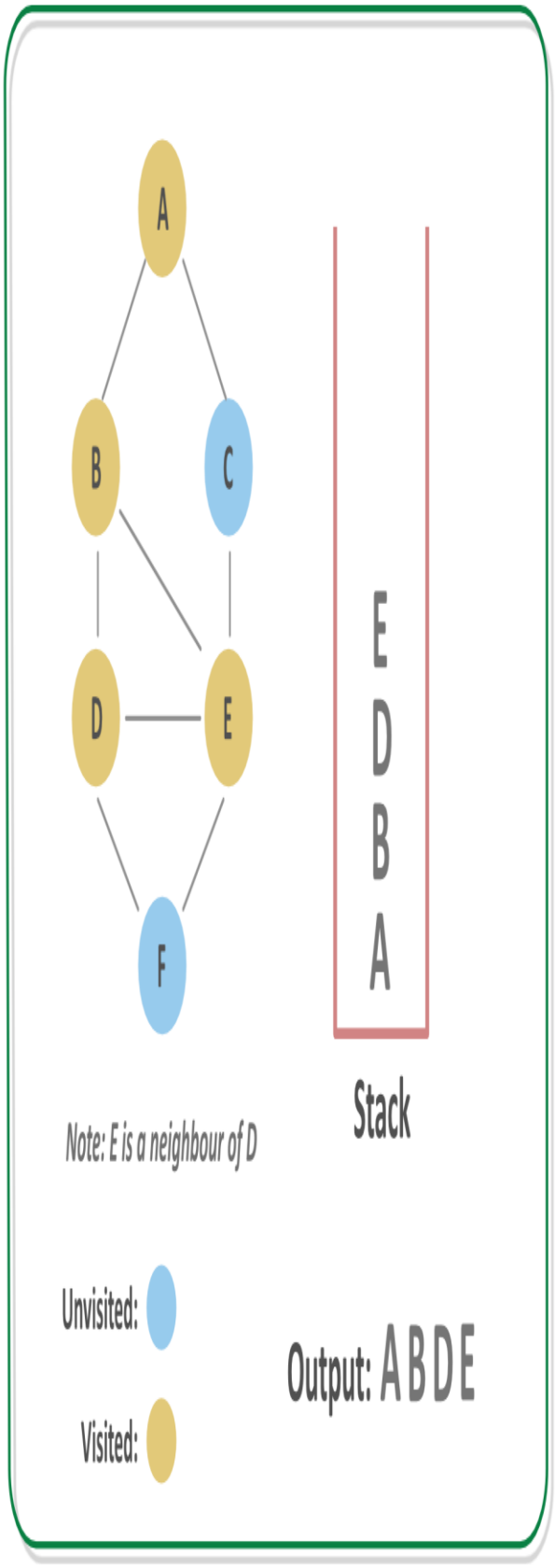
Step 4: Process the vertex D, mark it visited and call DFS for its adjacent vertices. Suppose

the first adjacent vertex processed is E. The vertex D is put on the auxilary stack for now.


A
B
C
D
E
F
D
B
A
Stack
Unvisited:
Visited:
Output: A B D

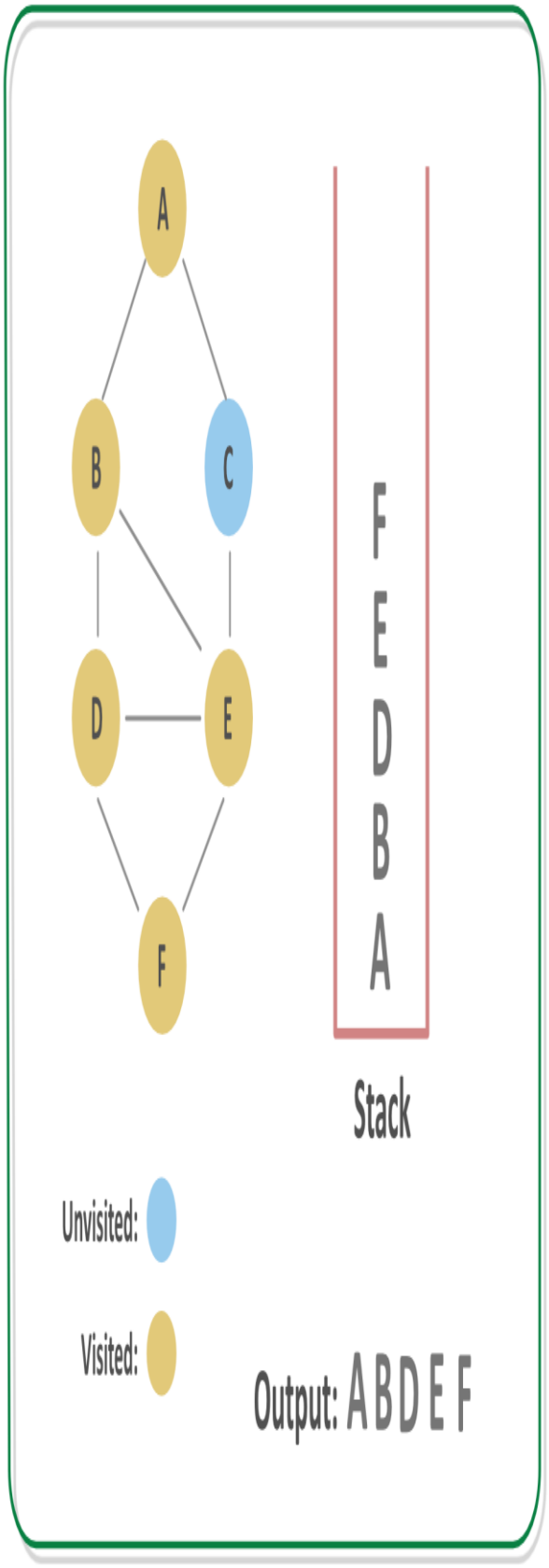
Step 5: Process the vertex E, mark it visited and call DFS for its adjacent vertices. Suppose the first adjacent vertex processed is F. The vertex E is put on the auxilary stack

for now.


A
B
C
D
E
F
E
D
B
A
Stack
Note: E is a neighbour of D
Unvisited:
Visited:
Output: A B D E

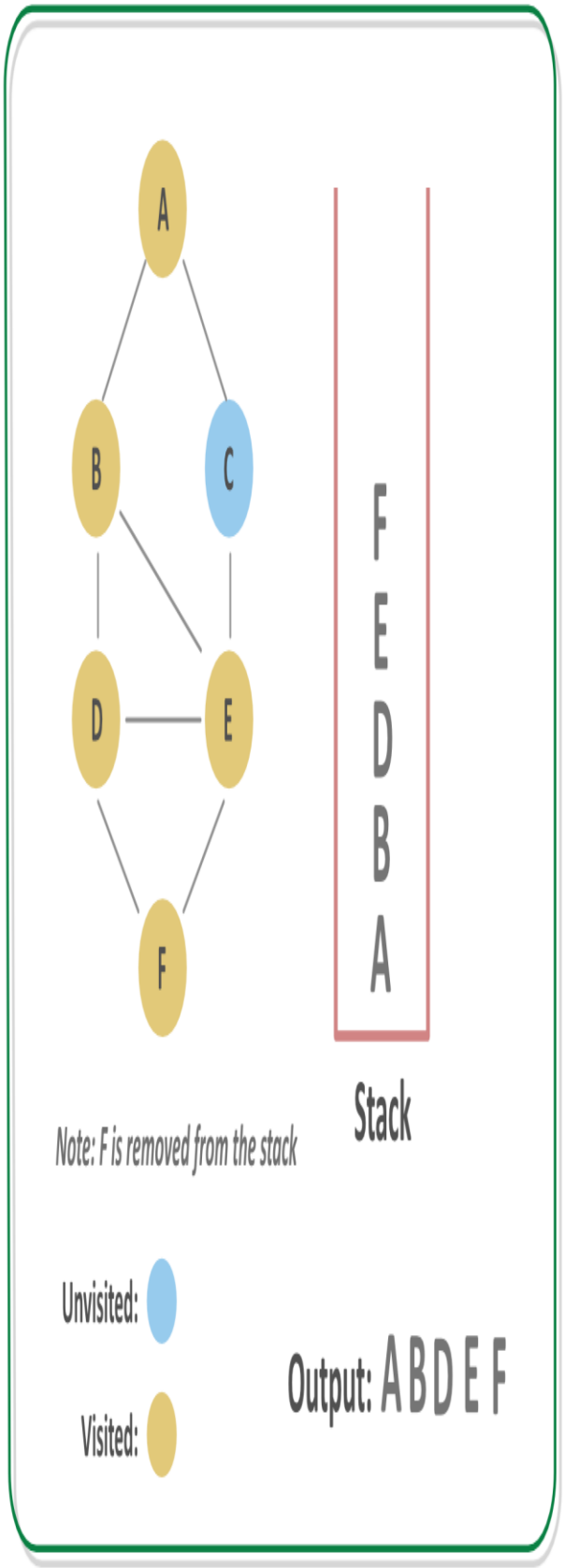
Step 6: Process the vertex F, mark it visited and call DFS for its adjacent vertices. There are no adjacent vertex of the vertex F, so backtrack to find a new path. The vertex F is put on

the auxilary stack for now.

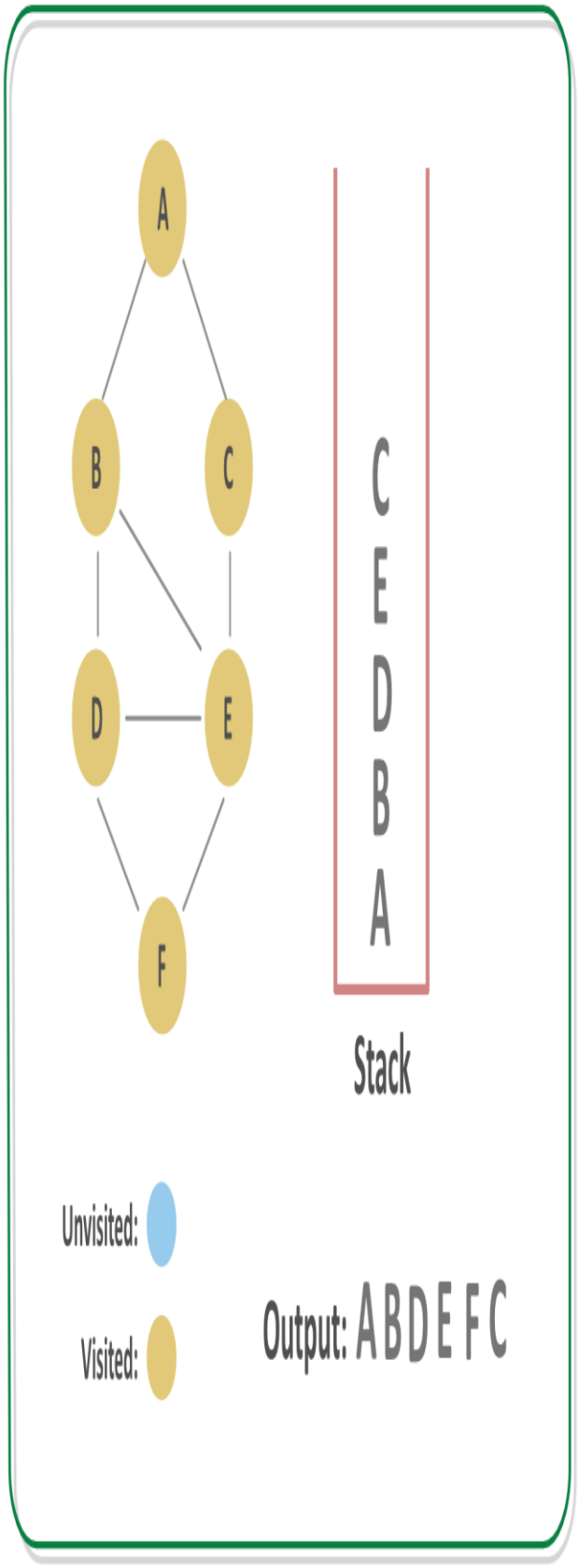

A
B
C
D
E
F
F
E
D
B
A
Stack
Unvisited:
Visited:
Output: A B D E F

Step 7: Since the vertex F has no adjacent vertices left unvisited, backtrack and go to

previous call, that is process any other adjacent vertex of node E, that is C.

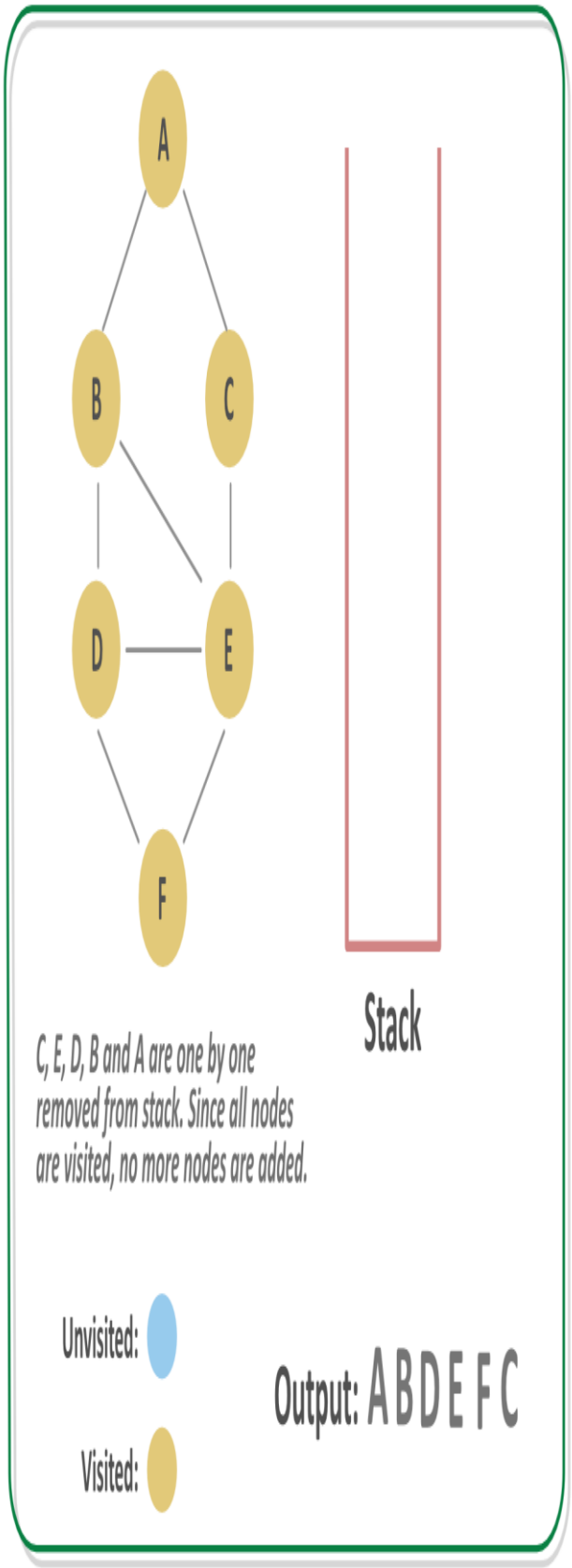

A
B
C
D
E
F
F
E
D
B
A
Stack
Note: F is removed from the stack
Unvisited:
Visited:
Output: A B D E F

Step 8: Process the vertex C, mark it visited and call DFS for its adjacent vertices. The vertex C is put on the auxilary stack for now.


A
B
C
D
E
F
C
E
D
B
A
Stack
Unvisited:
Visited:
Output: A B D E F C

Step 9: Since there are no adjacent vertex of C, backtrack to find a new path and keep removing nodes from stack until a new path is found. Since all of the nodes are processed

so the stack will get empty.


A
B
C
D
E
F
Stack
C, E, D, B and A are one by one removed from stack. Since all nodes are visited, no more nodes are added.
Unvisited:
Visited:
Output: A B D E F C

Implementation:

C++Java

```
// C++ program to print DFS traversal from
// a given vertex in a  given graph
#include<iostream>
#include<list>
using namespace std;

// Graph class represents a directed graph
// using adjacency list representation
class Graph
{
    int V;    // No. of vertices

    // Pointer to an array containing
    // adjacency lists
    list<int> *adj;

    // A recursive function used by DFS
    void DFSUtil(int v, bool visited[]);
public:
    Graph(int V);   // Constructor

    // function to add an edge to graph
    void addEdge(int v, int w);

    // DFS traversal of the vertices
    // reachable from v
    void DFS(int v);
};

Graph::Graph(int V)
{
    this->V = V;
    adj = new list<int>[V];
}

void Graph::addEdge(int v, int w)
{
    adj[v].push_back(w); // Add w to v's list.
}

void Graph::DFSUtil(int v, bool visited[])
{
    // Mark the current node as visited and
```

```
    // print it
    visited[v] = true;
    cout << v << " ";

    // Recur for all the vertices adjacent
    // to this vertex
    list<int>::iterator i;
    for (i = adj[v].begin(); i != adj[v].end(); ++i)
        if (!visited[*i])
            DFSUtil(*i, visited);
}

// DFS traversal of the vertices reachable from v.
// It uses recursive DFSUtil()
void Graph::DFS(int v)
{
    // Mark all the vertices as not visited
    bool *visited = new bool[V];
    for (int i = 0; i < V; i++)
        visited[i] = false;

    // Call the recursive helper function
    // to print DFS traversal
    DFSUtil(v, visited);
}

int main()
{
    // Create a graph given in the above diagram
    Graph g(4);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 0);
    g.addEdge(2, 3);
    g.addEdge(3, 3);

    cout << "Following is Depth First Traversal"
            " (starting from vertex 2) \n";
    g.DFS(2);

    return 0;
}
```

Output:

```
Following is Depth First Traversal (starting from vertex 2)
2 0 1 3
```

Applications of DFS

Depth-first search (DFS) is an algorithm (or technique) for traversing a graph. DFS uses a stack data structure for the traversal. A graph can have more than one DFS traversal.

Following are the problems that use DFS as a building block.

1) Detecting cycle in a graph
A graph has cycle if and only if we see a back edge during DFS. So we can run DFS for the graph and check for back edges. (See this for details)

2) Path Finding
We can specialize the DFS algorithm to find a path between two given vertices u and z.
i) Call DFS(G, u) with u as the start vertex.
ii) Use a stack S to keep track of the path between the start vertex and the current vertex.
iii) As soon as destination vertex z is encountered, return the path as the
contents of the stack

See this for details.

3) Topological Sorting
Topological Sorting is mainly used for scheduling jobs from the given dependencies among jobs. In computer science, applications of this type arise in instruction scheduling, ordering of formula cell evaluation when recomputing formula values in spreadsheets, logic synthesis, determining the order of compilation tasks to perform in makefiles, data serialization, and resolving symbol dependencies in linkers [2].

4) To test if a graph is bipartite
We can augment either BFS or DFS when we first discover a new vertex, color it opposite its parents, and for each other edge, check it doesn't link two vertices of the same color. The first vertex in any connected component can be red or black! See this for details.

5) Finding Strongly Connected Components of a graph A directed graph is called strongly connected if there is a path from each vertex in the graph to every other vertex. (See this for DFS-based algo for finding Strongly Connected Components)

6) Solving puzzles with only one solution, such as mazes. (DFS can be adapted to find all solutions to a maze by only including nodes on the current path in the visited set.)

Greedy

Introduction to Greedy Algorithms

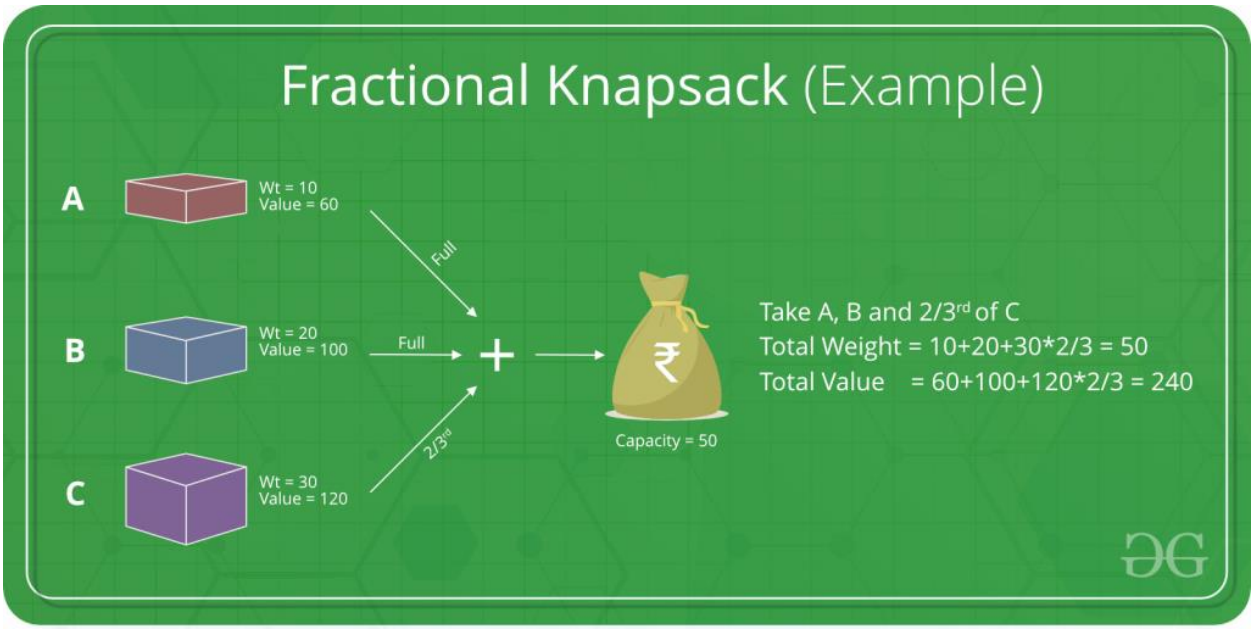
Greedy is an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit. So the problems where choosing locally optimal also leads to the global optimal solution are best fit for Greedy.

For example, consider the **Fractional Knapsack Problem**. The problem states that:

Given a list of elements with specific values and weights associated with them, the task is to fill a Knapsack of weight W using these elements such that the value of knapsack is maximum possible.

***Note**: You are allowed to take a fraction of an element also in order to maximize the value.*

The local optimal strategy is to choose the item that has maximum value vs weight ratio. This strategy also leads to global optimal solution because we are allowed to take fractions of an item.



In general, the **Greedy Algorithm** can be applied to solve a problem if it satisfies the below property:

At every step, we can make a choice that looks best at the moment, and we get the optimal solution of the complete problem.

Let us consider one more problem known as the **Activity Selection Problem** to understand the use of Greedy Algorithms.

Problem: You are given N activities with their start and finish times. Select the maximum number of activities that can be performed by a single person, assuming that a person can only work on a single activity at a time.

```
Example 1: Consider the following 3 activities sorted by
finish time.
     start[]  =  {10, 12, 20};
     finish[] =  {20, 25, 30};
A person can perform at most two activities. The
maximum set of activities that can be executed
is {0, 2} [ These are indexes in start[] and
finish[] ]
```

```
Example 2: Consider the following 6 activities
```

```
sorted by finish time.
     start[]  =  {1, 3, 0, 5, 8, 5};
     finish[] =  {2, 4, 6, 7, 9, 9};
A person can perform at most four activities. The
maximum set of activities that can be executed
is {0, 1, 3, 4} [ These are indexes in start[] and
finish[] ]
```

The greedy choice is to always pick the next activity whose finish time is least among the remaining activities and the start time is more than or equal to the finish time of previously selected activity. We can sort the activities according to their finishing time so that we always consider the next activity as minimum finishing time activity.

To do this:

1. Sort the activities according to their finishing time.
2. Select the first activity from the sorted array and print it.
3. Do following for remaining activities in the sorted array.
    - If the start time of this activity is greater than or equal to the finish time of previously selected activity then select this activity and print it.

Activity Selection Problem

You are given n activities with their start and finish times. Select the maximum number of activities that can be performed by a single person, assuming that a person can only work on a single activity at a time.

Example:

```
Example 1 : Consider the following 3 activities sorted by
by finish time.
     start[]  =  {10, 12, 20};
     finish[] =  {20, 25, 30};
A person can perform at most two activities. The
maximum set of activities that can be executed
is {0, 2} [ These are indexes in start[] and
finish[] ]
```

Example 2 : Consider the following 6 activities sorted by finish time.

```
     start[]  =  {1, 3, 0, 5, 8, 5};
     finish[] =  {2, 4, 6, 7, 9, 9};
```

A person can perform at most **four** activities. The maximum set of activities that can be executed is {0, 1, 3, 4} [These are indexes in start[] and finish[]]

The greedy choice is to always pick the next activity whose finish time is least among the remaining activities and the start time is more than or equal to the finish time of the previously selected activity. We can sort the activities according to their finishing time so that we always consider the next activity as minimum finishing time activity.

1) Sort the activities according to their finishing time
2) Select the first activity from the sorted array and print it.
3) Do the following for the remaining activities in the sorted array.
.....a) If the start time of this activity is greater than or equal to the finish time of the previously selected activity then select this activity and print it.

C++Java

```cpp
// C++ program for activity selection problem.
// The following implementation assumes that the activities
// are already sorted according to their finish time
#include <bits/stdc++.h>
using namespace std;

// Prints a maximum set of activities that can be done by a single
// person, one at a time.
//  n   -->  Total number of activities
//  s[] -->  An array that contains start time of all activities
//  f[] -->  An array that contains finish time of all activities
void printMaxActivities(int s[], int f[], int n)
{
    int i, j;

    cout <<"Following activities are selected "<< endl;

    // The first activity always gets selected
    i = 0;
    cout <<" "<< i;

    // Consider rest of the activities
    for (j = 1; j < n; j++)
    {
      // If this activity has start time greater than or
      // equal to the finish time of previously selected
```

```
      // activity, then select it
      if (s[j] >= f[i])
      {
          cout <<" " << j;
          i = j;
      }
    }
}

// driver program to test above function
int main()
{
    int s[] =  {1, 3, 0, 5, 8, 5};
    int f[] =  {2, 4, 6, 7, 9, 9};
    int n = sizeof(s)/sizeof(s[0]);
    printMaxActivities(s, f, n);
    return 0;
}
```

Output

```
Following activities are selected n0 1 3 4
```

How does Greedy Choice work for Activities sorted according to finish time?

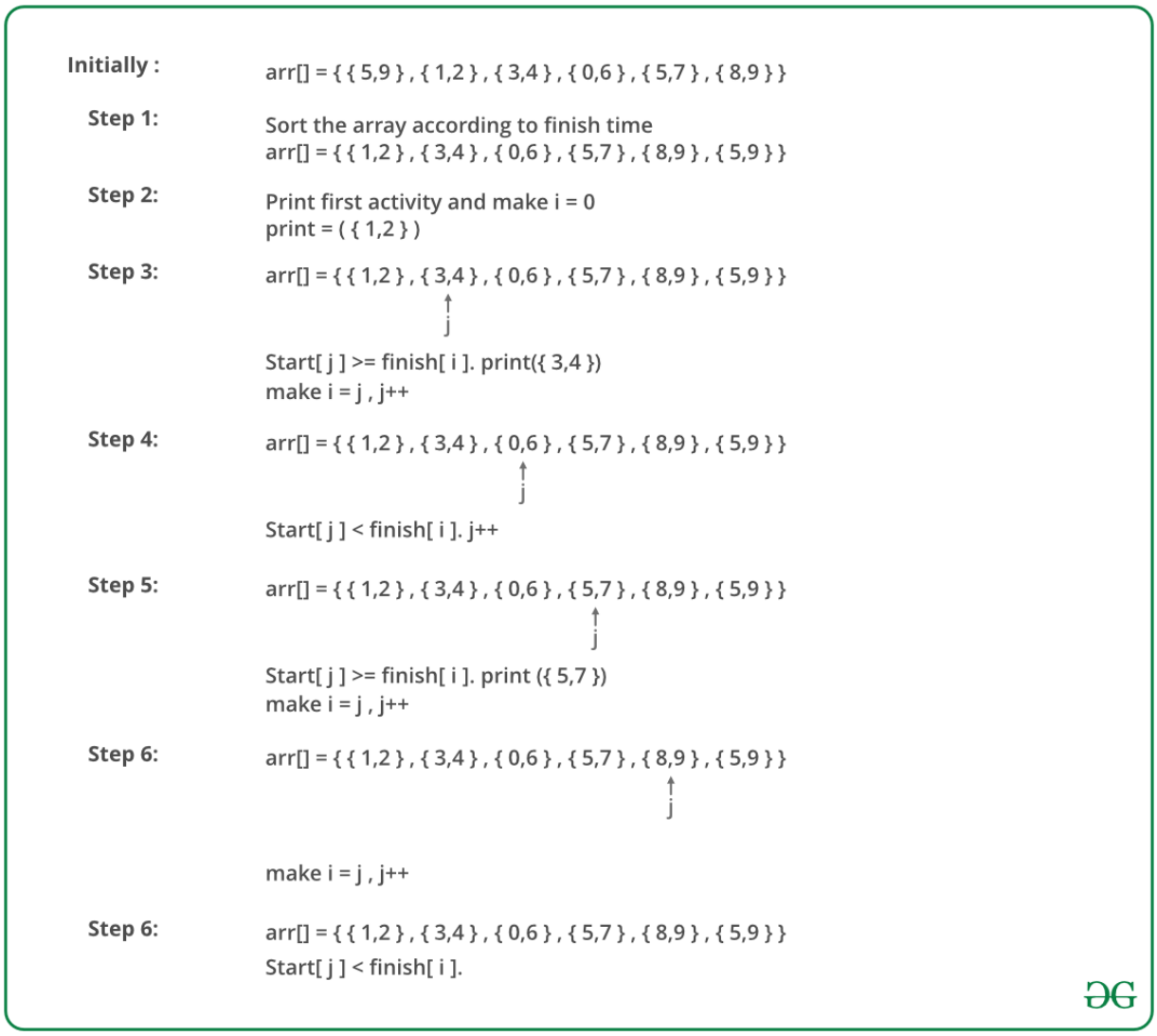
Let the given set of activities be S = {1, 2, 3, ...n} and activities are sorted by finish time. The greedy choice is to always pick activity 1. How come activity 1 always provides one of the optimal solutions. We can prove it by showing that if there is another solution B with the first activity other than 1, then there is also a solution A of the same size with activity 1 as the first activity. Let the first activity selected by B be k, then there always exist A = {B - {k}} U {1}.

(Note that the activities in B are independent and k has the smallest finishing time among all. Since k is not 1, finish(k) >= finish(1)).

How to implement when given activities are not sorted?

We create a structure/class for activities. We sort all activities by finish time (Refer sort in C++ STL). Once we have activities sorted, we apply the same algorithm.

Below image is an illustration of the above approach:



Below is the implementation of the above approach:

C++Java

```cpp
// C++ program for activity selection problem
// when input activities may not be sorted.
#include <bits/stdc++.h>
using namespace std;

// A job has a start time, finish time and profit.
struct Activitiy
{
    int start, finish;
};
```

```cpp
// A utility function that is used for sorting
// activities according to finish time
bool activityCompare(Activitiy s1, Activitiy s2)
{
    return (s1.finish < s2.finish);
}

// Returns count of the maximum set of activities that can
// be done by a single person, one at a time.
void printMaxActivities(Activitiy arr[], int n)
{
    // Sort jobs according to finish time
    sort(arr, arr+n, activityCompare);

    cout << "Following activities are selected n";

    // The first activity always gets selected
    int i = 0;
    cout << "(" << arr[i].start << ", " << arr[i].finish << "), ";

    // Consider rest of the activities
    for (int j = 1; j < n; j++)
    {
      // If this activity has start time greater than or
      // equal to the finish time of previously selected
      // activity, then select it
      if (arr[j].start >= arr[i].finish)
      {
          cout << "(" << arr[j].start << ", "
               << arr[j].finish << "), ";
          i = j;
      }
    }
}

// Driver program
int main()
{
    Activitiy arr[] = {{5, 9}, {1, 2}, {3, 4}, {0, 6},
                                        {5, 7}, {8, 9}};
    int n = sizeof(arr)/sizeof(arr[0]);
    printMaxActivities(arr, n);
    return 0;
}
```

Output:

```
Following activities are selected
(1, 2), (3, 4), (5, 7), (8, 9),
```

Time Complexity: It takes O(n log n) time if input activities may not be sorted. It takes O(n) time when it is given that input activities are always sorted.

Using STL we can solve it as follows:

CPPJava

```cpp
// C++ program for activity selection problem
// when input activities may not be sorted.
#include <bits/stdc++.h>
using namespace std;

void SelectActivities(vector<int>s,vector<int>f){
// Vector to store results.
    vector<pair<int,int>>ans;

// Minimum Priority Queue to sort activities in ascending order of
finishing time (f[i]).


priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>>
p;

    for(int i=0;i<s.size();i++){
        // Pushing elements in priority queue where the key is f[i]
        p.push(make_pair(f[i],s[i]));
    }

    auto it = p.top();
    int start = it.second;
    int end = it.first;
    p.pop();
    ans.push_back(make_pair(start,end));

    while(!p.empty()){
        auto itr = p.top();
        p.pop();
        if(itr.second >= end){
            start = itr.second;
            end = itr.first;
            ans.push_back(make_pair(start,end));
        }
```

```
    }
    cout << "Following Activities should be selected. " << endl << endl;

    for(auto itr=ans.begin();itr!=ans.end();itr++){
        cout << "Activity started at: " << (*itr).first << " and ends at
" << (*itr).second << endl;
    }
}

// Driver program
int main()
{
    vector<int>s = {1, 3, 0, 5, 8, 5};
    vector<int>f = {2, 4, 6, 7, 9, 9};
    SelectActivities(s,f);

    return 0;
}
```

Output

```
Following Activities should be selected.

Activity started at: 1 and ends at  2
Activity started at: 3 and ends at  4
Activity started at: 5 and ends at  7
Activity started at: 8 and ends at  9
```

Backtracking

Concept of Backtracking

Backtracking *can be defined as a general algorithmic technique that considers searching every possible combination in order to solve a computational problem.*

What is Backtracking Algorithm?

Backtracking is an algorithmic technique for solving problems recursively by trying to build a solution incrementally, one piece at a time, removing those solutions that fail to

satisfy the constraints of the problem at any point of time (by time, here, is referred to the time elapsed till reaching any level of the search tree).

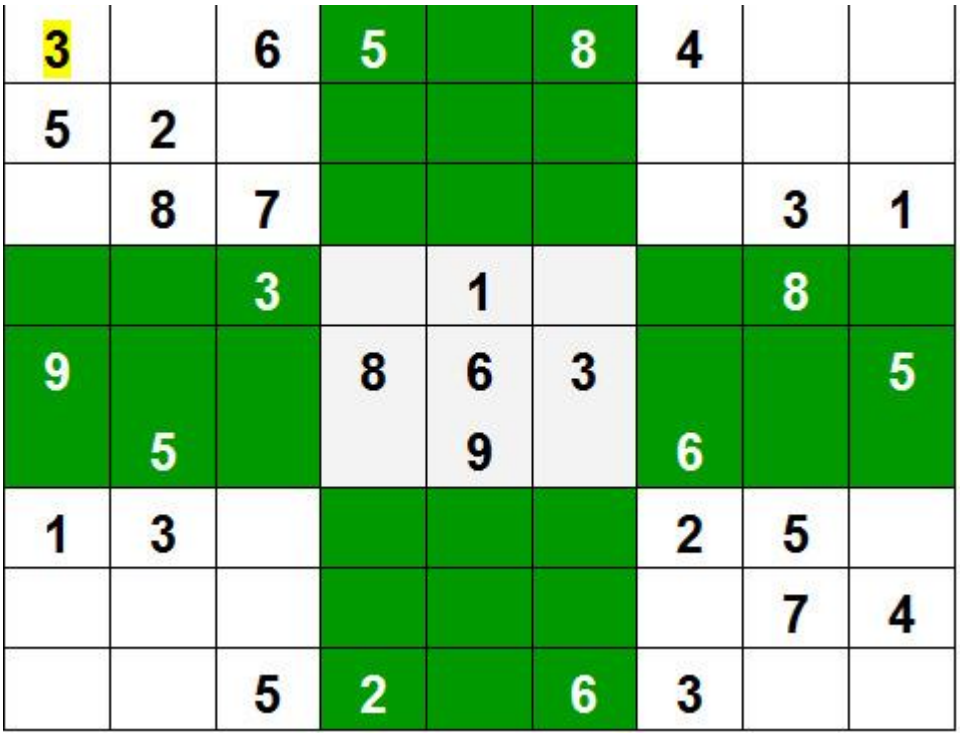
Types of Backtracking Algorithm

There are three types of problems in backtracking –

1. Decision Problem – In this, we search for a feasible solution.
2. Optimization Problem – In this, we search for the best solution.
3. Enumeration Problem – In this, we find all feasible solutions.

When can be Backtracking Algorithm used?

For example, consider the SudoKu solving Problem, we try filling digits one by one. Whenever we find that current digit cannot lead to a solution, we remove it (backtrack) and try next digit. This is better than naive approach (generating all possible combinations of digits and then trying every combination one by one) as it drops a set of permutations whenever it backtracks.



Rat in a Maze

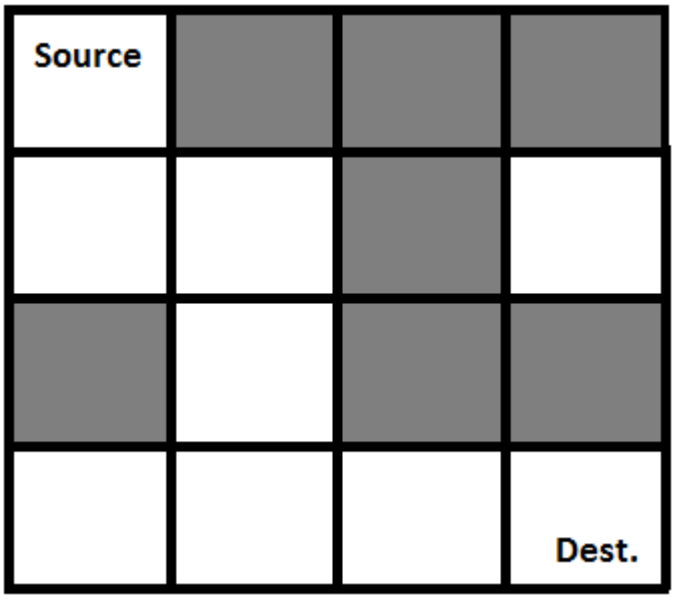
Let us discuss Rat in a Maze as another example problem that can be solved using Backtracking.

A Maze is given as N*N binary matrix of blocks where source block is the upper left most block i.e., maze[0][0] and destination block is lower rightmost block i.e., maze[N-1][N-1]. A rat starts from source and has to reach the destination. The rat can move only in two directions: forward and down.

In the maze matrix, 0 means the block is a dead end and 1 means the block can be used in the path from source to destination. Note that this is a simple version of the typical Maze problem. For example, a more complex version can be that the rat can move in 4 directions and a more complex version can be with a limited number of moves.

Following is an example maze.

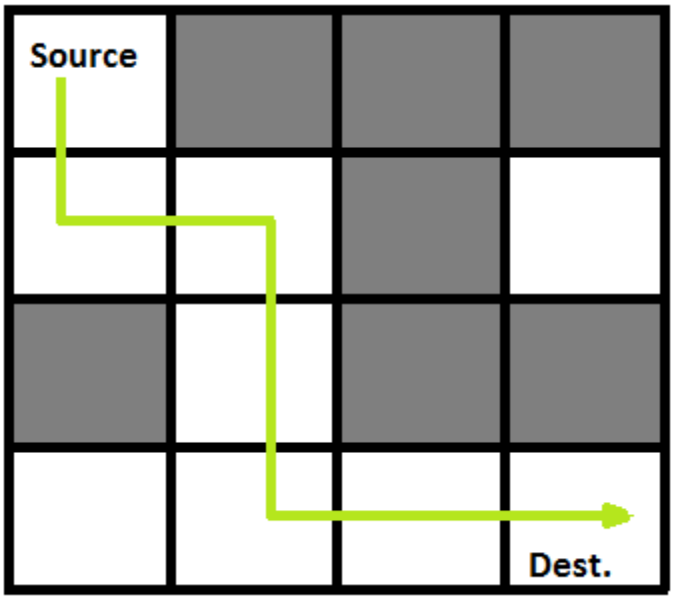
```
Gray blocks are dead ends (value = 0).
```



Following is a binary matrix representation of the above maze.

```
{1, 0, 0, 0}
{1, 1, 0, 1}
{0, 1, 0, 0}
{1, 1, 1, 1}
```

Following is a maze with highlighted solution path.



Following is the solution matrix (output of program) for the above input matrix.

```
{1, 0, 0, 0}
{1, 1, 0, 0}
{0, 1, 0, 0}
{0, 1, 1, 1}
All entries in solution path are marked as 1.
```

Backtracking Algorithm: Backtracking is an algorithmic-technique for solving problems recursively by trying to build a solution incrementally. Solving one piece at a time, and removing those solutions that fail to satisfy the constraints of the problem at any point of time (by time, here, is referred to the time elapsed till reaching any level of the search tree) is the process of backtracking.

Approach: Form a recursive function, which will follow a path and check if the path reaches the destination or not. If the path does not reach the destination then backtrack and try other paths.

Algorithm:

1. Create a solution matrix, initially filled with 0's.
2. Create a recursive function, which takes initial matrix, output matrix and position of rat (i, j).
3. if the position is out of the matrix or the position is not valid then return.
4. Mark the position output[i][j] as 1 and check if the current position is destination or not. If destination is reached print the output matrix and return.
5. Recursively call for position (i+1, j) and (i, j+1).

6. Unmark position (i, j), i.e output[i][j] = 0.

C++Java

```
#include <bits/stdc++.h>
using namespace std;

#define N 4

bool solveMazeRec(
    int maze[N][N], int x,
    int y, int sol[N][N]);

void printSolution(int sol[N][N])
{
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            printf(" %d ", sol[i][j]);
        printf("\n");
    }
}

bool isSafe(int maze[N][N], int i, int j)
{
    return ( i < N && j < N && maze[i][j] == 1);
}

bool solveMaze(int maze[N][N])
{
    int sol[N][N] = { { 0, 0, 0, 0 },
                      { 0, 0, 0, 0 },
                      { 0, 0, 0, 0 },
                      { 0, 0, 0, 0 } };

    if (solveMazeRec(maze, 0, 0, sol) == false) {
        printf("Solution doesn't exist");
        return false;
    }

    printSolution(sol);
    return true;
}

bool solveMazeRec( int maze[N][N], int i, int j, int sol[N][N])
{
    if ( i == N - 1 && j == N - 1 && maze[i][j] == 1) {
        sol[i][j] = 1;
        return true;
    }
```

```
    if (isSafe(maze, i, j) == true) {
        sol[i][j] = 1;

        if (solveMazeRec(maze, i + 1, j, sol) == true)
            return true;

        if (solveMazeRec(maze, i, j + 1, sol) == true)
            return true;

        sol[i][j] = 0;
    }

    return false;
}

int main() {

        int maze[N][N] = { { 1, 0, 0, 0 },
                           { 1, 1, 0, 1 },
                           { 0, 1, 0, 0 },
                           { 1, 1, 1, 1 } };

    solveMaze(maze);
    return 0;

}
```

Output:
The 1 values show the path for rat

```
1   0   0   0
1   1   0   0
0   1   0   0
0   1   1   1
```

Complexity Analysis:

- **Time Complexity:** O(2^(n^2)).
  The recursion can run upper-bound 2^(n^2) times.
- **Space Complexity:** O(n^2).
  Output matrix is required so an extra space of size n*n is needed.

Dynamic Programming

Introduction to Dynamic Programming

Dynamic Programming is an algorithmic approach to solve some complex problems easily and save time and number of comparisons by storing the results of past computations. The basic idea of dynamic programming is to store the results of previous calculation and reuse it in future instead of recalculating them.

We can also see Dynamic Programming as dividing a particular problem into subproblems and then storing the result of these subproblems to calculate the result of the actual problem.

Consider the problem to ***find the N-th Fibonacci number***.

We know that n-th fibonacci number fib(n) can be defined as:

```
fib(n) = fib(n-1) + fib(n-2), where n >= 2.

and,

fib(0) = 0
fib(1) = 1
```

We can see that the above function fib() to find the nth fibonacci number is divided into two subproblems fib(n-1) and fib(n-2) each one of which will be further divided into subproblems and so on.

The first few Fibonacci numbers are:

1, 1, 2, 3, 5, 8, 13, 21, 34,.....

The recursive program to find N-th Fibonacci number is shown below:

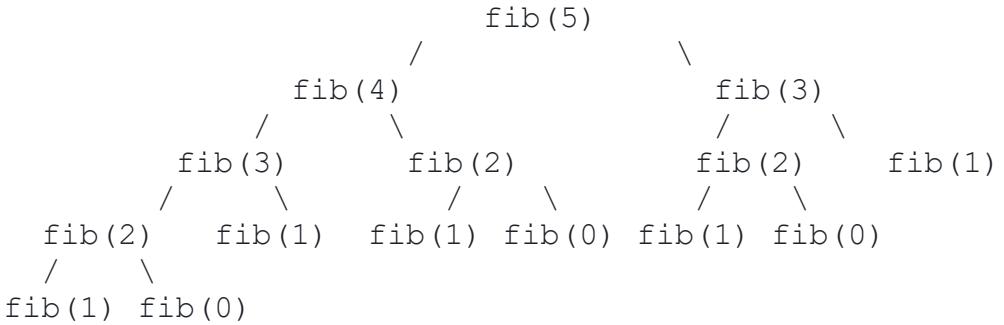
CPP

```cpp
int fib(int n)
{
    if (n <= 1)
        return n;

    return fib(n-1) + fib(n-2);
```

```
}
```

Below is the recursion tree for the recursive solution to find the N-th Fibonacci number:



We can see that the function fib(3) is being called 2 times. If we would have stored the value of fib(3), then instead of computing it again, we could have reused the old stored value.

The **time complexity** of the *recursive solution* is **exponential**. However, we can improve the time complexity by using Dynamic Programming approach and storing the results of the subproblems as shown below:

CPP

```
int fib(int n)
{
  // Declare an array to store Fibonacci numbers
  int f[n+2];   // 1 extra to handle case, n = 0
  int i;

  // 0th and 1st number of the series are 0 and 1
  f[0] = 0;
  f[1] = 1;

  for (i = 2; i <= n; i++)
  {
      // Add the previous 2 numbers in the series
      // and store it
      f[i] = f[i-1] + f[i-2];
  }

  return f[n];
}
```

The **time complexity** of the above solution is ***linear***.

Tabulation vs Memoization

There are two different ways to store the values so that the values of a sub-problem can be reused. Here, will discuss two patterns of solving dynamic programming (DP) problems:

1. **Tabulation:** Bottom Up
2. **Memoization:** Top Down

Before getting to the definitions of the above two terms consider the following statements:

- **Version 1**: I will study the theory of DP from GeeksforGeeks, then I will practice some problems on classic DP and hence I will master DP.
- **Version 2**: To Master DP, I would have to practice Dynamic problems and practice problems - Firstly, I would have to study some theories of DP from GeeksforGeeks

Both versions say the same thing, the difference simply lies in the way of conveying the message and that's exactly what Bottom-Up and Top-Down DP do. Version 1 can be related to Bottom-Up DP and Version-2 can be related as Top-Down DP.

Tabulation Method - Bottom Up Dynamic Programming

As the name itself suggests starting from the bottom and accumulating answers to the top. Let's discuss in terms of state transition.

Let's describe a state for our DP problem to be dp[x] with dp[0] as base state and dp[n] as our destination state. So, we need to find the value of destination state i.e dp[n].
If we start our transition from our base state i.e dp[0] and follow our state transition relation to reach our destination state dp[n], we call it the Bottom-Up approach as it is quite clear that we started our transition from the bottom base state and reached the topmost desired state.

Now, Why do we call it the tabulation method?

To know this let's first write some code to calculate the factorial of a number using a bottom-up approach. Once, again as our general procedure to solve a DP we first define a state. In this case, we define a state as dp[x], where dp[x] is to find the factorial of x.

Now, it is quite obvious that dp[x+1] = dp[x] * (x+1)

```
// Tabulated version to find factorial x.
int dp[MAXN];

// base case
int dp[0] = 1;
for (int i = 1; i< =n; i++)
{
    dp[i] = dp[i-1] * i;
}
```

The above code clearly follows the bottom-up approach as it starts its transition from the bottom-most base case dp[0] and reaches its destination state dp[n]. Here, we may notice that the DP table is being populated sequentially and we are directly accessing the calculated states from the table itself and hence, we call it the tabulation method.

Memoization Method - Top-Down Dynamic Programming

Once, again let's describe it in terms of state transition. If we need to find the value for some state say dp[n] and instead of starting from the base state that i.e dp[0] we ask our answer from the states that can reach the destination state dp[n] following the state transition relation, then it is the top-down fashion of DP.

Here, we start our journey from the top most destination state and compute its answer by taking in count the values of states that can reach the destination state, till we reach the bottom-most base state.

Once again, let's write the code for the factorial problem in the top-down fashion

```
// Memoized version to find factorial x.
// To speed up we store the values
// of calculated states

// initialized to -1
int dp[MAXN]
```

```
// return fact x!
int solve(int x)
{
    if (x==0)
        return 1;
    if (dp[x]!=-1)
        return dp[x];
    return (dp[x] = x * solve(x-1));
}
```

As we can see we are storing the most recent cache up to a limit so that if next time we got a call from the same state we simply return it from the memory. So, this is why we call it memoization as we are storing the most recent state values.

In this case, the memory layout is linear that's why it may seem that the memory is being filled in a sequential manner like the tabulation method, but you may consider any other top-down DP having 2D memory layout like Min Cost Path, here the memory is not filled in a sequential manner.

	Tabulation	Memoization
State	State Transition relation is difficult to think	State transition relation is easy to think
Code	Code gets complicated when lot of conditions are required	Code is easy and less complicated
Speed	Fast, as we directly access previous states from the table	Slow due to lot of recursive calls and return statements
Subproblem solving	If all subproblems must be solved at least once, a bottom-up dynamic-programming algorithm usually outperforms a top-down memoized algorithm by a constant factor	If some subproblems in the subproblem space need not be solved at all, the memoized solution has the advantage of solving only those subproblems that are definitely required
Table Entries	In Tabulated version, starting from the first entry, all entries are filled one by one	Unlike the Tabulated version, all entries of the lookup table are not necessarily filled in Memoized version. The table is filled on demand.

Longest Common Subsequence (Part 1)

LCS Problem Statement: Given two sequences, find the length of longest subsequence present in both of them. A subsequence is a sequence that appears in the same relative order, but not necessarily contiguous. For example, "abc", "abg", "bdf", "aeg", '"acefg", .. etc are subsequences of "abcdefg".

In order to find out the complexity of brute force approach, we need to first know the number of possible different subsequences of a string with length n, i.e., find the number of subsequences with lengths ranging from 1,2,..n-1. Recall from theory of permutation and combination that number of combinations with 1 element are ${}^{n}C_{1}$. Number of combinations with 2 elements are ${}^{n}C_{2}$ and so forth and so on. We know that ${}^{n}C_{0} + {}^{n}C_{1} + {}^{n}C_{2} + \ldots {}^{n}C_{n} = 2^{n}$. So a string of length n has 2^{n}-1 different possible subsequences since we do not consider the subsequence with length 0. This implies that the time complexity of the brute force approach will be $O(n * 2^{n})$. Note that it takes O(n) time to check if a subsequence is common to both the strings. This time complexity can be improved using dynamic programming.

It is a classic computer science problem, the basis of diff (a file comparison program that outputs the differences between two files), and has applications in bioinformatics.

Examples:
LCS for input Sequences "ABCDGH" and "AEDFHR" is "ADH" of length 3.
LCS for input Sequences "AGGTAB" and "GXTXAYB" is "GTAB" of length 4.

The naive solution for this problem is to generate all subsequences of both given sequences and find the longest matching subsequence. This solution is exponential in term of time complexity. Let us see how this problem possesses both important properties of a Dynamic Programming (DP) Problem.

1) Optimal Substructure:
Let the input sequences be X[0..m-1] and Y[0..n-1] of lengths m and n respectively. And let L(X[0..m-1], Y[0..n-1]) be the length of LCS of the two sequences X and Y. Following is the recursive definition of L(X[0..m-1], Y[0..n-1]).

- If last characters of both sequences match (or X[m-1] == Y[n-1]) then L(X[0..m-1], Y[0..n-1]) = 1 + L(X[0..m-2], Y[0..n-2])
- If last characters of both sequences do not match (or X[m-1] != Y[n-1]) then L(X[0..m-1], Y[0..n-1]) = MAX (L(X[0..m-2], Y[0..n-1]), L(X[0..m-1], Y[0..n-2]))

Examples:
1) Consider the input strings "AGGTAB" and "GXTXAYB". Last characters match for the

strings. So length of LCS can be written as:
L("AGGTAB", "GXTXAYB") = 1 + L("AGGTA", "GXTXAY")

	A	G	G	T	A	B
G	-	-	4	-	-	-
X	-	-	-	-	-	-
T	-	-	-	3	-	-
X	-	-	-	-	-	-
A	-	-	-	-	2	-
Y	-	-	-	-	-	-
B	-	-	-	-	-	1

2) Consider the input strings "ABCDGH" and "AEDFHR. Last characters do not match for the strings. So length of LCS can be written as:
L("ABCDGH", "AEDFHR") = MAX (L("ABCDG", "AEDFH**R**"), L("ABCDG**H**", "AEDFH"))
So the LCS problem has optimal substructure property as the main problem can be solved using solutions to subproblems.

2) Overlapping Subproblems:
Following is simple recursive implementation of the LCS problem. The implementation simply follows the recursive structure mentioned above.

C++Java

```
/* A Naive recursive implementation of LCS problem */
#include <bits/stdc++.h>
using namespace std;



/* Returns length of LCS for X[0..m-1], Y[0..n-1] */
int lcs( char *X, char *Y, int m, int n )
{
    if (m == 0 || n == 0)
        return 0;
    if (X[m-1] == Y[n-1])
        return 1 + lcs(X, Y, m-1, n-1);
    else
        return max(lcs(X, Y, m, n-1), lcs(X, Y, m-1, n));
}
```

```
/* Driver code */
int main()
{
    char X[] = "AGGTAB";
    char Y[] = "GXTXAYB";

    int m = strlen(X);
    int n = strlen(Y);

    cout<<"Length of LCS is "<< lcs( X, Y, m, n ) ;

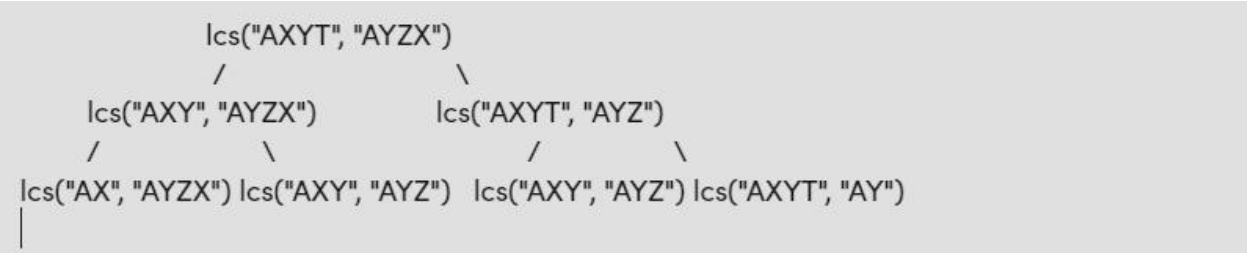
    return 0;
}
```

Output

```
Length of LCS is 4
```

Time complexity of the above naive recursive approach is O(2^n) in worst case and worst case happens when all characters of X and Y mismatch i.e., length of LCS is 0.

Considering the above implementation, following is a partial recursion tree for input strings "AXYT" and "AYZX"



In the above partial recursion tree, lcs("AXY", "AYZ") is being solved twice. If we draw the complete recursion tree, then we can see that there are many subproblems which are solved again and again. So this problem has Overlapping Substructure property and recomputation of same subproblems can be avoided by either using Memoization or Tabulation.

Following is a Memoization implementation for the LCS problem.

C++Java

```cpp
/* A Top-Down DP implementation of LCS problem */
#include <bits/stdc++.h>
using namespace std;

/* Returns length of LCS for X[0..m-1], Y[0..n-1] */
int lcs(char* X, char* Y, int m, int n,
        vector<vector<int> >& dp)
{
    if (m == 0 || n == 0)
        return 0;
    if (X[m - 1] == Y[n - 1])
        return dp[m][n] = 1 + lcs(X, Y, m - 1, n - 1, dp);

    if (dp[m][n] != -1) {
        return dp[m][n];
    }
    return dp[m][n] = max(lcs(X, Y, m, n - 1, dp),
                          lcs(X, Y, m - 1, n, dp));
}

/* Driver code */
int main()
{
    char X[] = "AGGTAB";
    char Y[] = "GXTXAYB";

    int m = strlen(X);
    int n = strlen(Y);
    vector<vector<int> > dp(m + 1, vector<int>(n + 1, -1));
    cout << "Length of LCS is " << lcs(X, Y, m, n, dp);

    return 0;
}
```

Output

```
Length of LCS is 4
```

Time Complexity : O(mn) ignoring recursion stack space

Longest Common Subsequence (Part 2)

LCS Problem Statement: Given two sequences, find the length of longest subsequence present in both of them. A subsequence is a sequence that appears in the same relative

order, but not necessarily contiguous. For example, "abc", "abg", "bdf", "aeg", '"acefg", .. etc are subsequences of "abcdefg".

In order to find out the complexity of brute force approach, we need to first know the number of possible different subsequences of a string with length n, i.e., find the number of subsequences with lengths ranging from 1,2,..n-1. Recall from theory of permutation and combination that number of combinations with 1 element are ${}^{n}C_{1}$. Number of combinations with 2 elements are ${}^{n}C_{2}$ and so forth and so on. We know that ${}^{n}C_{0} + {}^{n}C_{1} + {}^{n}C_{2} + ... {}^{n}C_{n} = 2^{n}$. So a string of length n has 2^{n}-1 different possible subsequences since we do not consider the subsequence with length 0. This implies that the time complexity of the brute force approach will be $O(n * 2^{n})$. Note that it takes O(n) time to check if a subsequence is common to both the strings. This time complexity can be improved using dynamic programming.

It is a classic computer science problem, the basis of diff (a file comparison program that outputs the differences between two files), and has applications in bioinformatics.

Examples:
LCS for input Sequences "ABCDGH" and "AEDFHR" is "ADH" of length 3.
LCS for input Sequences "AGGTAB" and "GXTXAYB" is "GTAB" of length 4.

The naive solution for this problem is to generate all subsequences of both given sequences and find the longest matching subsequence. This solution is exponential in term of time complexity. Let us see how this problem possesses both important properties of a Dynamic Programming (DP) Problem.

1) Optimal Substructure:
Let the input sequences be X[0..m-1] and Y[0..n-1] of lengths m and n respectively. And let L(X[0..m-1], Y[0..n-1]) be the length of LCS of the two sequences X and Y. Following is the recursive definition of L(X[0..m-1], Y[0..n-1]).

- If last characters of both sequences match (or X[m-1] == Y[n-1]) then L(X[0..m-1], Y[0..n-1]) = 1 + L(X[0..m-2], Y[0..n-2])
- If last characters of both sequences do not match (or X[m-1] != Y[n-1]) then L(X[0..m-1], Y[0..n-1]) = MAX (L(X[0..m-2], Y[0..n-1]), L(X[0..m-1], Y[0..n-2]))

Examples:
1) Consider the input strings "AGGTAB" and "GXTXAYB". Last characters match for the strings. So length of LCS can be written as:
L("AGGTAB", "GXTXAYB") = 1 + L("AGGTA", "GXTXAY")

	A	G	G	T	A	B
G	-	-	4	-	-	-
X	-	-	-	-	-	-
T	-	-	-	3	-	-
X	-	-	-	-	-	-
A	-	-	-	-	2	-
Y	-	-	-	-	-	-
B	-	-	-	-	-	1

2) Consider the input strings "ABCDGH" and "AEDFHR. Last characters do not match for the strings. So length of LCS can be written as:
L("ABCDGH", "AEDFHR") = MAX (L("ABCDG", "AEDFH**R**"), L("ABCDG**H**", "AEDFH"))
So the LCS problem has optimal substructure property as the main problem can be solved using solutions to subproblems.

2) Overlapping Subproblems:
Following is simple recursive implementation of the LCS problem. The implementation simply follows the recursive structure mentioned above.

C++Java

```
/* A Naive recursive implementation of LCS problem */
#include <bits/stdc++.h>
using namespace std;



/* Returns length of LCS for X[0..m-1], Y[0..n-1] */
int lcs( char *X, char *Y, int m, int n )
{
    if (m == 0 || n == 0)
        return 0;
    if (X[m-1] == Y[n-1])
        return 1 + lcs(X, Y, m-1, n-1);
    else
        return max(lcs(X, Y, m, n-1), lcs(X, Y, m-1, n));
}



/* Driver code */
```

```
int main()
{
    char X[] = "AGGTAB";
    char Y[] = "GXTXAYB";

    int m = strlen(X);
    int n = strlen(Y);

    cout<<"Length of LCS is "<< lcs( X, Y, m, n ) ;

    return 0;
}
```

Output

```
Length of LCS is 4
```

Time complexity of the above naive recursive approach is O(2^n) in worst case and worst case happens when all characters of X and Y mismatch i.e., length of LCS is 0.

Considering the above implementation, following is a partial recursion tree for input strings "AXYT" and "AYZX"

```
                         lcs("AXYT", "AYZX")
                        /
          lcs("AXY", "AYZX")              lcs("AXYT", "AYZ")
          /                                /
lcs("AX", "AYZX") lcs("AXY", "AYZ")    lcs("AXY", "AYZ")
lcs("AXYT", "AY")
```

In the above partial recursion tree, lcs("AXY", "AYZ") is being solved twice. If we draw the complete recursion tree, then we can see that there are many subproblems which are solved again and again. So this problem has Overlapping Substructure property and recomputation of same subproblems can be avoided by either using Memoization or Tabulation.

Following is a Memoization implementation for the LCS problem.

C++Java

```
/* A Top-Down DP implementation of LCS problem */
#include <bits/stdc++.h>
using namespace std;

/* Returns length of LCS for X[0..m-1], Y[0..n-1] */
int lcs(char* X, char* Y, int m, int n,
```

```
        vector<vector<int> >& dp)
{
    if (m == 0 || n == 0)
        return 0;
    if (X[m - 1] == Y[n - 1])
        return dp[m][n] = 1 + lcs(X, Y, m - 1, n - 1, dp);

    if (dp[m][n] != -1) {
        return dp[m][n];
    }
    return dp[m][n] = max(lcs(X, Y, m, n - 1, dp),
                          lcs(X, Y, m - 1, n, dp));
}

/* Driver code */
int main()
{
    char X[] = "AGGTAB";
    char Y[] = "GXTXAYB";

    int m = strlen(X);
    int n = strlen(Y);
    vector<vector<int> > dp(m + 1, vector<int>(n + 1, -1));
    cout << "Length of LCS is " << lcs(X, Y, m, n, dp);

    return 0;
}
```

Output

```
Length of LCS is 4
```

Time Complexity : O(mn) ignoring recursion stack space

Following is a tabulated implementation for the LCS problem.

C++Java

```
/* Dynamic Programming C implementation of LCS problem */
#include <bits/stdc++.h>
using namespace std;

/* Returns length of LCS for X[0..m-1], Y[0..n-1] */
int lcs(char *X, char *Y, int m, int n)
{
    // intitalizing a matrix of size (m+1)*(n+1)
    int L[m + 1][n + 1];
```

```
    /* Following steps build L[m+1][n+1] in bottom up fashion. Note
        that L[i][j] contains length of LCS of X[0..i-1] and Y[0..j-1] */
    for (int i = 0; i <= m; i++)
    {
        for (int j = 0; j <= n; j++)
        {
            if (i == 0 || j == 0)
                L[i][j] = 0;

            else if (X[i - 1] == Y[j - 1])
                L[i][j] = L[i - 1][j - 1] + 1;

            else
                L[i][j] = max(L[i - 1][j], L[i][j - 1]);
        }
    }

    // L[m][n] contains length of LCS for X[0..n-1] and Y[0..m-1]
    return L[m][n];
}

// Driver program to test above function
int main()
{

    char X[] = "AGGTAB";
    char Y[] = "GXTXAYB";

    int m = strlen(X);
    int n = strlen(Y);

    cout << "Length of LCS is: " << lcs(X, Y, m, n);

    return 0;
}
```

Output

```
Length of LCS is  4
```

Time Complexity of the above implementation is O(mn) which is much better than the worst-case time complexity of Naive Recursive implementation.

Auxiliary Space: O(m * n)

Longest Increasing Subsequence

The Longest Increasing Subsequence (LIS) problem is to find the length of the longest subsequence of a given sequence such that all elements of the subsequence are sorted in increasing order. For example, the length of LIS for {10, 22, 9, 33, 21, 50, 41, 60, 80} is 6 and LIS is {10, 22, 33, 50, 60, 80}.

arr[ ]	10	22	9	33	21	50	41	60	80
LIS	1	2		3		4		5	6

Examples:

```
Input: arr[] = {3, 10, 2, 1, 20}
Output: Length of LIS = 3
The longest increasing subsequence is 3, 10, 20

Input: arr[] = {3, 2}
Output: Length of LIS = 1
The longest increasing subsequences are {3} and {2}

Input: arr[] = {50, 3, 10, 7, 40, 80}
Output: Length of LIS = 4
The longest increasing subsequence is {3, 7, 40, 80}
```

Method 1: Recursion.

Optimal Substructure: Let arr[0..n-1] be the input array and L(i) be the length of the LIS ending at index i such that arr[i] is the last element of the LIS.

Then, L(i) can be recursively written as:

```
L(i) = 1 + max( L(j) ) where 0 < j < i and arr[j] < arr[i]; or
L(i) = 1, if no such j exists.
```

To find the LIS for a given array, we need to return max(L(i)) where 0 < i < n.
Formally, the length of the longest increasing subsequence ending at index i, will be 1 greater than the maximum of lengths of all longest increasing subsequences ending at indices before i, where arr[j] < arr[i] (j < i).
Thus, we see the LIS problem satisfies the optimal substructure property as the main problem can be solved using solutions to subproblems.

The recursive tree given below will make the approach clearer:

```
Input  : arr[] = {3, 10, 2, 11}
f(i): Denotes LIS of subarray ending at index 'i'

(LIS(1)=1)
```

```
       f(4)   {f(4) = 1 + max(f(1), f(2), f(3))}
  /     |     \
f(1)   f(2)   f(3) {f(3) = 1, f(2) and f(1) are > f(3)}
        |       |  \
       f(1)   f(2)  f(1) {f(2) = 1 + max(f(1)}
                |
               f(1) {f(1) = 1}
```

Below is the implementation of the recursive approach:

C++Java

```
/* A Naive C++ recursive implementation
of LIS problem */
#include <iostream>
using namespace std;

/* To make use of recursive calls, this
function must return two things:
1) Length of LIS ending with element arr[n-1].
       We use max_ending_here for this purpose
2) Overall maximum as the LIS may end with
       an element before arr[n-1] max_ref is
       used this purpose.
The value of LIS of full array of size n
is stored in *max_ref which is our final result
*/
int _lis(int arr[], int n, int* max_ref)
{
       /* Base case */
       if (n == 1)
              return 1;

       // 'max_ending_here' is length of LIS
       // ending with arr[n-1]
       int res, max_ending_here = 1;

       /* Recursively get all LIS ending with arr[0],
       arr[1] ... arr[n-2]. If arr[i-1] is smaller
       than arr[n-1], and max ending with arr[n-1]
       needs to be updated, then update it */
       for (int i = 1; i < n; i++) {
              res = _lis(arr, i, max_ref);
              if (arr[i - 1] < arr[n - 1]
                     && res + 1 > max_ending_here)
                     max_ending_here = res + 1;
       }

       // Compare max_ending_here with the overall
```

```
        // max. And update the overall max if needed
        if (*max_ref < max_ending_here)
                *max_ref = max_ending_here;

        // Return length of LIS ending with arr[n-1]
        return max_ending_here;
}

// The wrapper function for _lis()
int lis(int arr[], int n)
{
        // The max variable holds the result
        int max = 1;

        // The function _lis() stores its result in max
        _lis(arr, n, &max);

        // returns max
        return max;
}

/* Driver program to test above function */
int main()
{
        int arr[] = { 10, 22, 9, 33, 21, 50, 41, 60 };
        int n = sizeof(arr) / sizeof(arr[0]);
        cout <<"Length of lis is "<< lis(arr, n);
        return 0;
}
```

Output

```
Length of lis is 5
```

Complexity Analysis:

- **Time Complexity:** The time complexity of this recursive approach is exponential as there is a case of overlapping subproblems as explained in the recursive tree diagram above.
- **Auxiliary Space:** O(1). No external space used for storing values apart from the internal stack space.

Method 2: Dynamic Programming.

We can see that there are many subproblems in the above recursive solution which are solved again and again. So this problem has Overlapping Substructure property and recomputation of same subproblems can be avoided by either using Memoization or Tabulation.

The simulation of approach will make things clear:

```
Input   : arr[] = {3, 10, 2, 11}
LIS[] = {1, 1, 1, 1} (initially)
```

Iteration-wise simulation :

1. arr[2] > arr[1] {LIS[2] = max(LIS [2], LIS[1]+1)=2}
2. arr[3] < arr[1] {No change}
3. arr[3] < arr[2] {No change}
4. arr[4] > arr[1] {LIS[4] = max(LIS [4], LIS[1]+1)=2}
5. arr[4] > arr[2] {LIS[4] = max(LIS [4], LIS[2]+1)=3}
6. arr[4] > arr[3] {LIS[4] = max(LIS [4], LIS[3]+1)=3}

We can avoid recomputation of subproblems by using tabulation as shown in the below code:

Below is the implementation of the above approach:

C++Java

```cpp
/* Dynamic Programming C++ implementation
of LIS problem */
#include <bits/stdc++.h>
using namespace std;

/* lis() returns the length of the longest
increasing subsequence in arr[] of size n */
int lis(int arr[], int n)
{
	int lis[n];

	lis[0] = 1;

	/* Compute optimized LIS values in
	bottom up manner */
	for (int i = 1; i < n; i++) {
		lis[i] = 1;
		for (int j = 0; j < i; j++)
			if (arr[i] > arr[j] && lis[i] < lis[j] + 1)
				lis[i] = lis[j] + 1;
	}

	// Return maximum value in lis[]
	return *max_element(lis, lis + n);
}

/* Driver program to test above function */
int main()
{
```

```
        int arr[] = { 10, 22, 9, 33, 21, 50, 41, 60 };
        int n = sizeof(arr) / sizeof(arr[0]);
        printf("Length of lis is %d\n", lis(arr, n));

        return 0;
}
```

Output

```
Length of lis is 5
```

Complexity Analysis:

- **Time Complexity:** O(n^2).
  As nested loop is used.
- **Auxiliary Space:** O(n).
  Use of any array to store LIS values at each index.

Method 3 : Memoization DP

This is extension of recursive method

We can see that there are many subproblems in the above recursive solution which are solved again and again. So this problem has Overlapping Substructure property and recomputation of same subproblems can be avoided by either using Memoization

C++Java

```
/* A Naive C++ recursive implementation
of LIS problem */
#include <bits/stdc++.h>
#include <iostream>
using namespace std;

/* To make use of recursive calls, this
function must return two things:
1) Length of LIS ending with element arr[n-1].
       We use max_ending_here for this purpose
2) Overall maximum as the LIS may end with
       an element before arr[n-1] max_ref is
       used this purpose.
The value of LIS of full array of size n
is stored in *max_ref which is our final result
*/

int f(int idx, int prev_idx, int n, int a[],
       vector<vector<int> >& dp)
{
```

```
        if (idx == n) {
                return 0;
        }

        if (dp[idx][prev_idx + 1] != -1) {
                return dp[idx][prev_idx + 1];
        }

        int notTake = 0 + f(idx + 1, prev_idx, n, a, dp);
        int take = INT_MIN;
        if (prev_idx == -1 || a[idx] > a[prev_idx]) {
                take = 1 + f(idx + 1, idx, n, a, dp);
        }

        return dp[idx][prev_idx + 1] = max(take, notTake);
}
// Function to find length of longest increasing
// subsequence.
int longestSubsequence(int n, int a[])
{
        vector<vector<int> > dp(n + 1, vector<int>(n + 1, -1));
        return f(0, -1, n, a, dp);
}

/* Driver program to test above function */
int main()
{
        int a[] = { 3, 10, 2, 1, 20 };
        int n = sizeof(a) / sizeof(a[0]);
        cout << "Length of lis is " << longestSubsequence(n, a);
        return 0;
}
```

Output

```
Length of lis is 3
```

Complexity Analysis:

Time Complexity: $O(n^2)$.
Auxiliary Space: $O(n^2)$.

Longest Increasing Subsequence in O(nlogn)

Given an array of random numbers. Find *longest increasing subsequence* (LIS) in the array. I know many of you might have read recursive and dynamic programming (DP) solutions. There are few requests for O(N log N) algo in the forum posts.

For the time being, forget about recursive and DP solutions. Let us take small samples and extend the solution to large instances. Even though it may look complex at first time, once if we understood the logic, coding is simple.
Consider an input array A = {2, 5, 3}. I will extend the array during explanation.
By observation we know that the LIS is either {2, 3} or {2, 5}. ***Note that I am considering only strictly increasing sequences***.
Let us add two more elements, say 7, 11 to the array. These elements will extend the existing sequences. Now the increasing sequences are {2, 3, 7, 11} and {2, 5, 7, 11} for the input array {2, 5, 3, 7, 11}.
Further, we add one more element, say 8 to the array i.e. input array becomes {2, 5, 3, 7, 11, 8}. Note that the latest element 8 is greater than smallest element of any active sequence (*will discuss shortly about active sequences*). How can we extend the existing sequences with 8? First of all, can 8 be part of LIS? If yes, how? If we want to add 8, it should come after 7 (by replacing 11).
Since the approach is *offline (what we mean by offline?)*, we are not sure whether adding 8 will extend the series or not. Assume there is 9 in the input array, say {2, 5, 3, 7, 11, 8, 7, 9 ...}. We can replace 11 with 8, as there is potentially *best* candidate (9) that can extend the new series {2, 3, 7, 8} or {2, 5, 7, 8}.
Our observation is, assume that the end element of largest sequence is E. We can add (replace) current element A[i] to the existing sequence if there is an element A[j] (j > i) such that E < A[i] < A[j] or (E > A[i] < A[j] - for replace). In the above example, E = 11, A[i] = 8 and A[j] = 9.
In case of our original array {2, 5, 3}, note that we face same situation when we are adding 3 to increasing sequence {2, 5}. I just created two increasing sequences to make explanation simple. Instead of two sequences, 3 can replace 5 in the sequence {2, 5}.
I know it will be confusing, I will clear it shortly!
The question is, when will it be safe to add or replace an element in the existing sequence?
Let us consider another sample A = {2, 5, 3}. Say, the next element is 1. How can it extend the current sequences {2, 3} or {2, 5}. Obviously, it can't extend either. Yet, there is a potential that the new smallest element can be start of an LIS. To make it clear, consider the array is {2, 5, 3, 1, 2, 3, 4, 5, 6}. Making 1 as new sequence will create new sequence which is largest.
The observation is, when we encounter new smallest element in the array, it can be a

potential candidate to start new sequence.
From the observations, we need to maintain lists of increasing sequences.
In general, we have set of **active lists** of varying length. We are adding an element A[i] to these lists. We scan the lists (for end elements) in decreasing order of their length. We will verify the end elements of all the lists to find a list whose end element is smaller than A[i] (*floor* value).
Our strategy determined by the following conditions,

1. **If A[i] is smallest among all *end candidates of active lists, we will start new active list of length 1.***
2. **If A[i] is largest among all *end candidates of active lists, we will clone the largest active list, and extend it by A[i].***
3. **If A[i] is in between, we will find a list with *largest end element that is smaller than A[i]. Clone and extend this list by A[i]. We will discard all other lists of same length as that of this modified list.***

Note that at any instance during our construction of active lists, the following condition is maintained.
"end element of smaller list is smaller than end elements of larger lists".
It will be clear with an example, let us take example from wiki {0, 8, 4, 12, 2, 10, 6, 14, 1, 9, 5, 13, 3, 11, 7, 15}.

```
A[0] = 0. Case 1. There are no active lists, create one.
0.
-------------------------------------------------------------
-------------
A[1] = 8. Case 2. Clone and extend.
0.
0, 8.
-------------------------------------------------------------
-------------
A[2] = 4. Case 3. Clone, extend and discard.
0.
0, 4.
0, 8. Discarded
-------------------------------------------------------------
-------------
A[3] = 12. Case 2. Clone and extend.
0.
0, 4.
```

```
0, 4, 12.
------------------------------------------------------------------
------------
A[4] = 2. Case 3. Clone, extend and discard.
0.
0, 2.
0, 4. Discarded.
0, 4, 12.
------------------------------------------------------------------
------------
A[5] = 10. Case 3. Clone, extend and discard.
0.
0, 2.
0, 2, 10.
0, 4, 12. Discarded.
------------------------------------------------------------------
------------
A[6] = 6. Case 3. Clone, extend and discard.
0.
0, 2.
0, 2, 6.
0, 2, 10. Discarded.
------------------------------------------------------------------
------------
A[7] = 14. Case 2. Clone and extend.
0.
0, 2.
0, 2, 6.
0, 2, 6, 14.
------------------------------------------------------------------
------------
A[8] = 1. Case 3. Clone, extend and discard.
0.
0, 1.
0, 2. Discarded.
0, 2, 6.
0, 2, 6, 14.
------------------------------------------------------------------
------------
A[9] = 9. Case 3. Clone, extend and discard.
0.
0, 1.
0, 2, 6.
0, 2, 6, 9.
0, 2, 6, 14. Discarded.
------------------------------------------------------------------
------------
```

```
A[10] = 5. Case 3. Clone, extend and discard.
0.
0, 1.
0, 1, 5.
0, 2, 6. Discarded.
0, 2, 6, 9.
------------------------------------------------------------
------------
A[11] = 13. Case 2. Clone and extend.
0.
0, 1.
0, 1, 5.
0, 2, 6, 9.
0, 2, 6, 9, 13.
------------------------------------------------------------
------------
A[12] = 3. Case 3. Clone, extend and discard.
0.
0, 1.
0, 1, 3.
0, 1, 5. Discarded.
0, 2, 6, 9.
0, 2, 6, 9, 13.
------------------------------------------------------------
------------
A[13] = 11. Case 3. Clone, extend and discard.
0.
0, 1.
0, 1, 3.
0, 2, 6, 9.
0, 2, 6, 9, 11.
0, 2, 6, 9, 13. Discarded.
------------------------------------------------------------
------------
A[14] = 7. Case 3. Clone, extend and discard.
0.
0, 1.
0, 1, 3.
0, 1, 3, 7.
0, 2, 6, 9. Discarded.
0, 2, 6, 9, 11.
------------------------------------------------------------
------------
A[15] = 15. Case 2. Clone and extend.
0.
0, 1.
0, 1, 3.
```

```
0, 1, 3, 7.
0, 2, 6, 9, 11.
0, 2, 6, 9, 11, 15. <-- LIS List
------------------------------------------------------------
------------
```

It is required to understand above strategy to devise an algorithm. Also, ensure we have maintained the condition, "*end element of smaller list is smaller than end elements of larger lists*". Try with few other examples, before reading further. It is important to understand what happening to end elements.

Algorithm:

Querying length of longest is fairly easy. Note that we are dealing with end elements only. We need not to maintain all the lists. We can store the end elements in an array. Discarding operation can be simulated with replacement, and extending a list is analogous to adding more elements to array.

We will use an auxiliary array to keep end elements. The maximum length of this array is that of input. In the worst case the array divided into N lists of size one (*note that it doesn't lead to worst case complexity*). To discard an element, we will trace ceil value of A[i] in auxiliary array (again observe the end elements in your rough work), and replace ceil value with A[i]. We extend a list by adding element to auxiliary array. We also maintain a counter to keep track of auxiliary array length.

Bonus: You have learnt Patience Sorting technique partially :)

Here is a proverb, "*Tell me and I will forget. Show me and I will remember. Involve me and I will understand*." So, pick a suit from deck of cards. Find the longest increasing sub-sequence of cards from the shuffled suit. You will never forget the approach. :)

Update - 17 July, 2016: Quite impressive responses from the readers and few sites referring the post, feeling happy as my hardwork helping others. It looks like readers are not doing any homework prior to posting comments. Requesting to run through some examples after reading the article, and please do your work on paper (don't use editor/compiler). The request is to help yourself. Profess to 'know' is different from real understanding (no disrespect). Given below was my personal experience.

Initial content preparation took roughly 6 hours to me. But, it was a good lesson. I finished initial code in an hour. When I start writing content to explain the reader, I realized I didn't understand the cases. Took my note book (I have habit of maintaining binded note book to keep track of my rough work), and after few hours I filled nearly 15 pages of rough work. Whatever the content you are seeing in the gray colored example is from these pages. All the thought process for the solution triggered by a note in the book 'Introduction to Algorithms by Udi Manber', I strongly recommend to practice the book.

I suspect, many readers might not get the logic behind CeilIndex (binary search). I leave it as an exercise to the reader to understand how it works. Run through few examples on paper. I realized I have already covered the algorithm in another post.

Update - 5th August, 2016:

The following link worth referring after you do your work. I got to know the link via my recently created **Disqus** profile. The link has explanation of approach mentioned in the Wiki.

http://stackoverflow.com/questions/2631726/how-to-determine-the-longest-increasing-subsequence-using-dynamic-programming

Given below is code to find length of LIS (*updated to C++11 code, no C-style arrays*),

C++Java

```
#include <iostream>
#include <vector>

// Binary search (note boundaries in the caller)
int CeilIndex(std::vector<int>& v, int l, int r, int key)
{
    while (r - l > 1) {
        int m = l + (r - l) / 2;
        if (v[m] >= key)
            r = m;
        else
            l = m;
    }

    return r;
}

int LongestIncreasingSubsequenceLength(std::vector<int>& v)
{
    if (v.size() == 0)
        return 0;

    std::vector<int> tail(v.size(), 0);
    int length = 1; // always points empty slot in tail

    tail[0] = v[0];
    for (size_t i = 1; i < v.size(); i++) {

        // new smallest value
        if (v[i] < tail[0])
            tail[0] = v[i];
```

```
        // v[i] extends largest subsequence
        else if (v[i] > tail[length - 1])
            tail[length++] = v[i];

        // v[i] will become end candidate of an existing
        // subsequence or Throw away larger elements in all
        // LIS, to make room for upcoming greater elements
        // than v[i] (and also, v[i] would have already
        // appeared in one of LIS, identify the location
        // and replace it)
        else
            tail[CeilIndex(tail, -1, length - 1, v[i])] = v[i];
    }

    return length;
}

int main()
{
    std::vector<int> v{ 2, 5, 3, 7, 11, 8, 10, 13, 6 };
    std::cout << "Length of Longest Increasing Subsequence is "
              << LongestIncreasingSubsequenceLength(v) << '\n';
    return 0;
}
```

Output:

```
Length of Longest Increasing Subsequence is 6
```

Complexity:

The loop runs for N elements. In the worst case (what is worst case input?), we may end up querying ceil value using binary search (log *i*) for many A[i].

Therefore, T(n) < O(log N!) = O(N log N). Analyse to ensure that the upper and lower bounds are also O(N log N). The complexity is THETA (N log N).

Exercises:

1. Design an algorithm to construct the longest increasing list. Also, model your solution using DAGs.
2. Design an algorithm to construct **all** increasing list**s of equal longest size**.
3. Is the above algorithm an *online* algorithm?
4. Design an algorithm to construct the longest *decreasing* list..

Alternate implementation in various languages using their built in binary search functions are given below:

CPPJava

```
#include <bits/stdc++.h>
using namespace std;
```

```
int LongestIncreasingSubsequenceLength(std::vector<int>& v)
{
    if (v.size() == 0) // boundary case
        return 0;

    std::vector<int> tail(v.size(), 0);
    int length = 1; // always points empty slot in tail

    tail[0] = v[0];

    for (int i = 1; i < v.size(); i++) {

        // Do binary search for the element in
        // the range from begin to begin + length
        auto b = tail.begin(), e = tail.begin() + length;
        auto it = lower_bound(b, e, v[i]);

        // If not present change the tail element to v[i]
        if (it == tail.begin() + length)
            tail[length++] = v[i];
        else
            *it = v[i];
    }

    return length;
}

int main()
{
    std::vector<int> v{ 2, 5, 3, 7, 11, 8, 10, 13, 6 };
    std::cout
        << "Length of Longest Increasing Subsequence is "
        << LongestIncreasingSubsequenceLength(v);
    return 0;
}
```

Output:

```
Length of Longest Increasing Subsequence is 6
```

Time Complexity: O(nlogn)

Space Complexity: O(n)

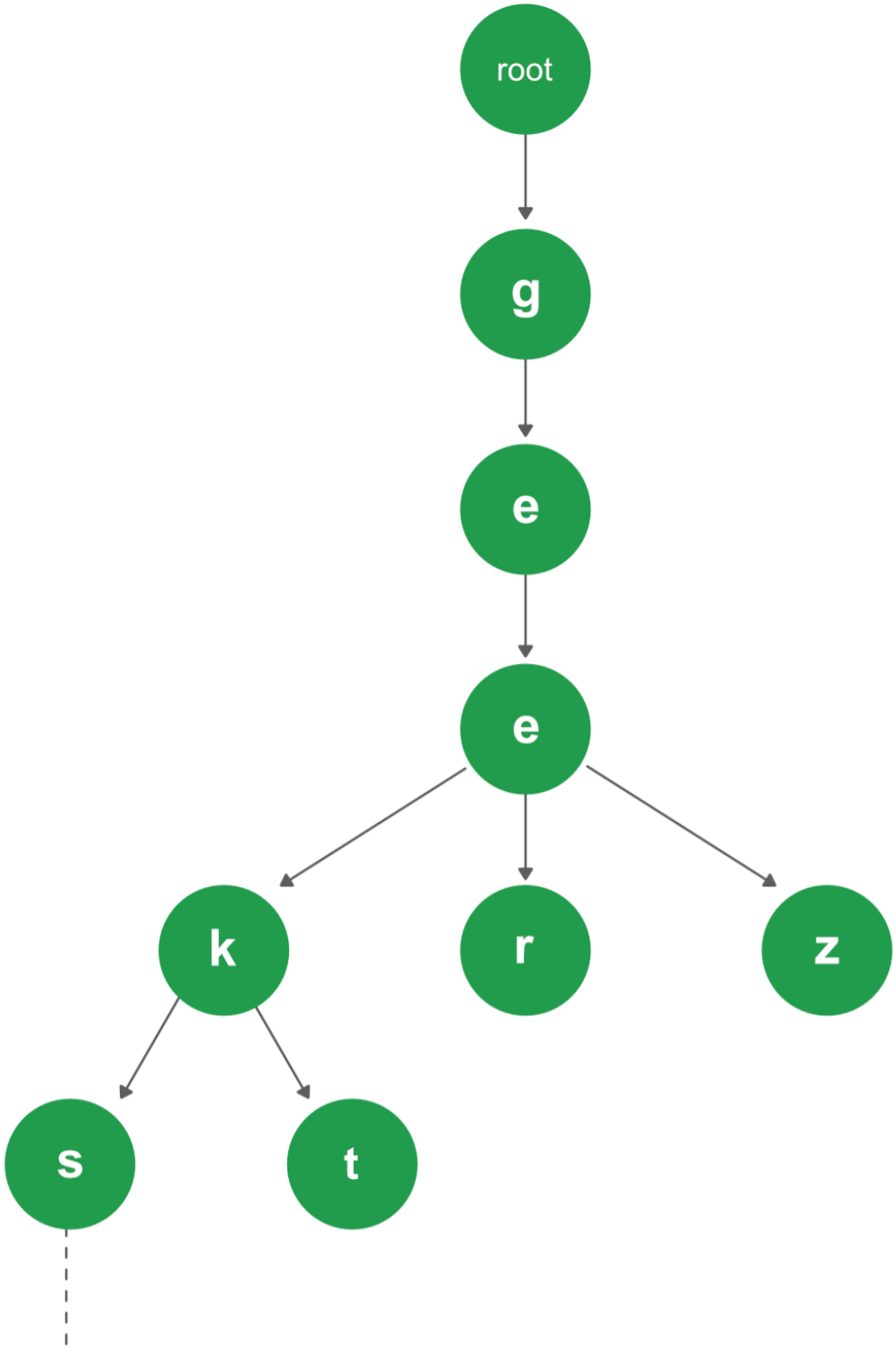
Trie

Insertion and Search in Trie Data Structure

The Trie data structure is an efficient information **re-*trie*-val** data structure. The Trie data struture is used to efficiently search for a particular string key among a list of such keys. Using the trie, the lookup operation can be performed in time complexity of **O(key_length)**.

Representation:
A trie is represented as a tree where each node contains 26 pointers which is equal to the number of characters in the English alphabets. In Trie basically, the common prefix of all strings are represented as a common path in the tree.

For Example, look at the below sample Trie:


root
g
e
e
k
r
z
s
t

There are 4 different strings in the Trie above: ["geeks", "geekt", "geer", "geez"]. Since the prefix **"gee"** is common among all of the strings, so it is a common path in the Trie as well. Again for the strings "geeks" and "geekt", the common prefix is "geek", so after dividing paths from "gee", the node with character "k" is common to these two strings.

Every node of Trie consists of multiple branches. Each branch represents a possible character of keys. We need to mark the last node of every key as the end of the word node. A Trie node field *isEndOfWord* is used to distinguish the node as the end of word node. A simple structure to represent nodes of the English alphabet can be as following,

```
// Trie node
struct TrieNode
{
     struct TrieNode *children[ALPHABET_SIZE];

     // isEndOfWord is true if the node
     // represents end of a word
     bool isEndOfWord;
};
```

Insertion in a Trie

Inserting a key to **Trie** is a simple approach. Every character of the input key is inserted as an individual Trie node. Note that the children are an array of pointers (or references) to next level trie nodes. The key character acts as an index into the array of children. If the input key is new or an extension of the existing key, we need to construct non-existing nodes of the key, and mark end of the word for the last node. If the input key is a prefix of the existing key in Trie, we simply mark the last node of the key as the end of a word. The key length determines Trie depth.

Searching a Key in Trie

Searching for a key is similar to the insert operation, however, we only compare the

characters and move down. The search can terminate due to the end of a string or lack of key in the trie. In the former case, if the *isEndofWord* field of the last node is true, then the key exists in the trie. In the second case, the search terminates without examining all the characters of the key, since the key is not present in the trie.

The following picture explains construction of trie using keys given in the example below,

In the picture, every character is of type *trie_node_t*. For example, the *root* is of type trie_node_t, and it's children *a*, *b* and *t* are filled, all other nodes of root will be NULL. Similarly, "a" at the next level is having only one child ("n"), all other children are NULL. The leaf nodes are in blue.

Implementation:

C++Java

```
// C++ implementation of search and insert
// operations on Trie
#include <bits/stdc++.h>
using namespace std;

const int ALPHABET_SIZE = 26;

// trie node
struct TrieNode
{
    struct TrieNode *children[ALPHABET_SIZE];

    // isEndOfWord is true if the node represents
    // end of a word
    bool isEndOfWord;
};

// Returns new trie node (initialized to NULLs)
struct TrieNode *getNode(void)
{
    struct TrieNode *pNode =  new TrieNode;

    pNode->isEndOfWord = false;
```

```
    for (int i = 0; i < ALPHABET_SIZE; i++)
        pNode->children[i] = NULL;

    return pNode;
}

// If not present, inserts key into trie
// If the key is prefix of trie node, just
// marks leaf node
void insert(struct TrieNode *root, string key)
{
    struct TrieNode *pCrawl = root;

    for (int i = 0; i < key.length(); i++)
    {
        int index = key[i] - 'a';
        if (!pCrawl->children[index])
            pCrawl->children[index] = getNode();

        pCrawl = pCrawl->children[index];
    }

    // mark last node as leaf
    pCrawl->isEndOfWord = true;
}

// Returns true if key presents in trie, else
// false
bool search(struct TrieNode *root, string key)
{
    struct TrieNode *pCrawl = root;

    for (int i = 0; i < key.length(); i++)
    {
        int index = key[i] - 'a';
        if (!pCrawl->children[index])
            return false;

        pCrawl = pCrawl->children[index];
    }

    return (pCrawl != NULL && pCrawl->isEndOfWord);
}

// Driver Code
int main()
{
    // Input keys (use only 'a' through 'z'
    // and lower case)
```

```
    string keys[] = {"the", "a", "there",
                     "answer", "any", "by",
                      "bye", "their" };
    int n = sizeof(keys)/sizeof(keys[0]);

    struct TrieNode *root = getNode();

    // Construct trie
    for (int i = 0; i < n; i++)
        insert(root, keys[i]);

    // Search for different keys
    search(root, "the")? cout << "Yes\n" :
                         cout << "No\n";
    search(root, "these")? cout << "Yes\n" :
                           cout << "No\n";
    return 0;
}
```

Output

```
Yes
No
```

Time Complexity: Both the insert and search operations in a Trie takes **O(M)** time. Where **M** is the length of the key being inserted or searched for in the Trie.

Deletion in a Trie

We had already discussed the representation of a Trie data structure, inserting a key and searching a key in a Trie. Let us look at the process of deleting a given key from a Trie.

During delete operation, we delete the key in a bottom-up manner using recursion. The following are possible cases which may occur while deleting the key from trie,

- **Case 1**: Key may not be present in the trie. In this case, the delete operation should not modify trie.
- **Case 2**: Key present as a unique key (no part of key contains another key (prefix), nor the key itself is a prefix of another key in trie). In this case, delete all the nodes of that key.
- **Case 3**: Key is a prefix key of another long key in the trie. In this case, simply unmark the leaf node.

- **Case 4**: Key present in the trie, having at least one other key as a prefix key. In this case, delete nodes from the end of key until first leaf node of longest prefix key.

Implementation:

C++Java

```
// C++ implementation of delete
// operations on Trie

#include <bits/stdc++.h>
using namespace std;

const int ALPHABET_SIZE = 26;

// trie node
struct TrieNode {
    struct TrieNode* children[ALPHABET_SIZE];

    // isEndOfWord is true if the node represents
    // end of a word
    bool isEndOfWord;
};

// Returns new trie node (initialized to NULLs)
struct TrieNode* getNode(void)
{
    struct TrieNode* pNode = new TrieNode;

    pNode->isEndOfWord = false;

    for (int i = 0; i < ALPHABET_SIZE; i++)
        pNode->children[i] = NULL;

    return pNode;
}

// If not present, inserts key into trie
// If the key is prefix of trie node, just
// marks leaf node
void insert(struct TrieNode* root, string key)
{
    struct TrieNode* pCrawl = root;

    for (int i = 0; i < key.length(); i++) {
        int index = key[i] - 'a';
        if (!pCrawl->children[index])
            pCrawl->children[index] = getNode();

        pCrawl = pCrawl->children[index];
```

```
    }

    // mark last node as leaf
    pCrawl->isEndOfWord = true;
}

// Returns true if key presents in trie, else
// false
bool search(struct TrieNode* root, string key)
{
    struct TrieNode* pCrawl = root;

    for (int i = 0; i < key.length(); i++) {
        int index = key[i] - 'a';
        if (!pCrawl->children[index])
            return false;

        pCrawl = pCrawl->children[index];
    }

    return (pCrawl != NULL && pCrawl->isEndOfWord);
}

// Returns true if root has no children, else false
bool isEmpty(TrieNode* root)
{
    for (int i = 0; i < ALPHABET_SIZE; i++)
        if (root->children[i])
            return false;
    return true;
}

// Recursive function to delete a key from given Trie
TrieNode* remove(TrieNode* root, string key, int depth = 0)
{
    // If tree is empty
    if (!root)
        return NULL;

    // If last character of key is being processed
    if (depth == key.size()) {

        // This node is no more end of word after
        // removal of given key
        if (root->isEndOfWord)
            root->isEndOfWord = false;

        // If given is not prefix of any other word
        if (isEmpty(root)) {
```

```
            delete (root);
            root = NULL;
        }

        return root;
    }

    // If not last character, recur for the child
    // obtained using ASCII value
    int index = key[depth] - 'a';
    root->children[index] =
          remove(root->children[index], key, depth + 1);

    // If root does not have any child (its only child got
    // deleted), and it is not end of another word.
    if (isEmpty(root) && root->isEndOfWord == false) {
        delete (root);
        root = NULL;
    }

    return root;
}

// Driver Code
int main()
{
    // Input keys (use only 'a' through 'z'
    // and lower case)
    string keys[] = { "the", "a", "there",
                      "answer", "any", "by",
                      "bye", "their", "hero", "heroplane" };
    int n = sizeof(keys) / sizeof(keys[0]);

    struct TrieNode* root = getNode();

    // Construct trie
    for (int i = 0; i < n; i++)
        insert(root, keys[i]);

    // Search for different keys
    search(root, "the") ? cout << "Yes\n" : cout << "No\n";
    search(root, "these") ? cout << "Yes\n" : cout << "No\n";

    remove(root, "heroplane");
    search(root, "hero") ? cout << "Yes\n" : cout << "No\n";
    return 0;
}
```

Output

```
Yes
No
Yes
```

Time Complexity: The time complexity of the deletion operation is O(n) where n is the key length.
Auxiliary Space: O(n*m), where n is the key length of the longest word and m is the total no of words.

Segment and Binary Indexed Tree

Introduction to Segment Trees

Segment Trees are **Binary Tree** which is used to store intervals or segments. That is each node in a segment tree basically stores the segment of an array. Segment Trees are generally used in problems where we need to solve queries on a range of elements in arrays.
Let us consider the following problem to understand Segment Trees.

Problem: We have an array arr[0 . . . n-1]. We should be able to perform the below operations on the array:

1. Find the sum of elements from index l to r where 0 <= l <= r <= n-1.
2. Change value of a specified element of the array to a new value x. We need to do arr[i] = x where 0 <= i <= n-1.

General Solution: A simple solution is to run a loop from index **l** to **r** and calculate the sum of elements in the given range. To update a value, simply do arr[i] = x. The first operation takes O(N) time for every query, where N is the number of elements in the range [l,r] and the second operation takes O(1) time.

Can we optimize the time complexity of the first operation in the above solution?
Yes, we can optimize the first operation to be solved in O(1) time complexity by storing **presum**. We can keep an auxiliary array say *sum[]* in which the **i-th** element will store the sum of first **i** elements of the original array. So, whenever we need to find the sum of a range of elements, we can simply calculate it by (**sum[r]-sum[l-1]**). But in this solution

the complexity to perform the second operation of updating an element increases from **O(1) to O(N)**.

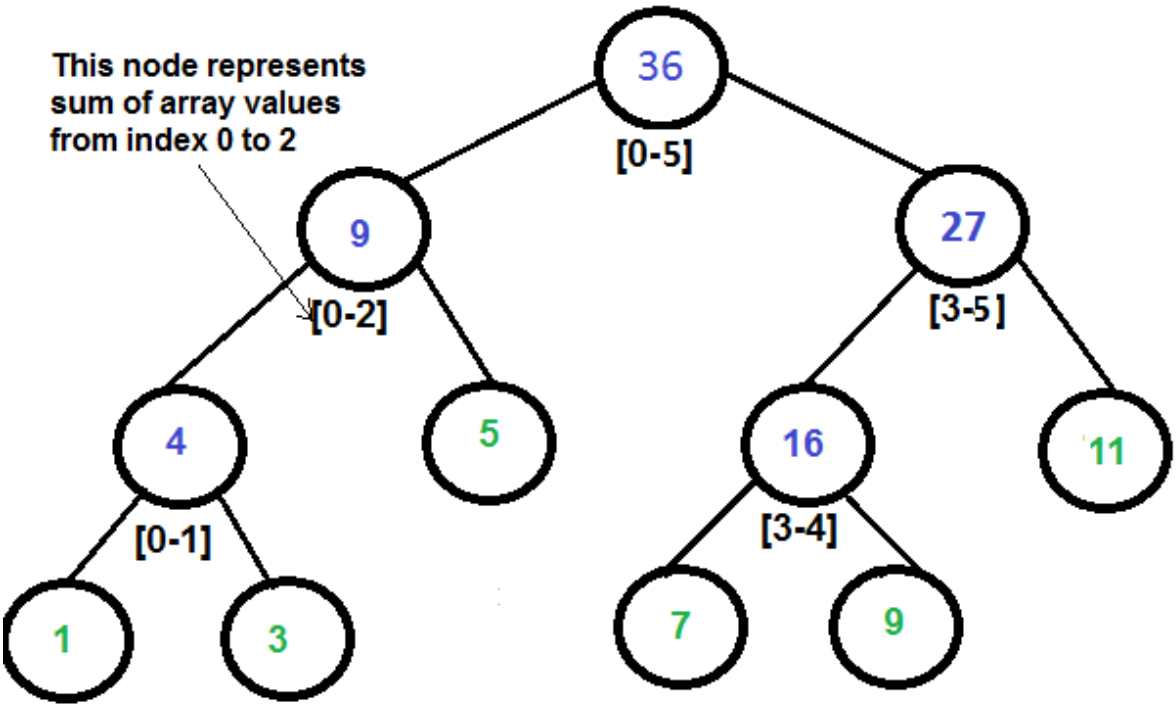
What if the number of query and updates are equal? Can we perform both the operations in O(log n) time once given the array?

We can use a segment tree to perform both of the operations in O(log N) time complexity.

Representation of Segment trees:

1. Leaf Nodes are the elements of the input array.
2. Each internal node represents some merging of the leaf nodes. The merging may be different for different problems. For this problem, merging is the sum of leaves under a node.

An array representation of the tree is used to represent Segment Trees. For each node at index **i**, the left child is at index **2*i + 1**, right child at **2*i + 2** and the parent is at **(i - 1)/2**.



Segment Tree for input array {1, 3, 5, 7, 9, 11}

Array representation of Segment Trees: Like Heap, segment tree is also represented as an array. The difference here is, it is not a complete binary tree. It is rather a full binary tree (every node has 0 or 2 children) and all levels are filled except possibly the last level. Unlike

Heap, the last level may have gaps between nodes. Below are the values in the segment tree array for the above diagram.

Array representation of segment tree for input array {1, 3, 5, 7, 9, 11} is,

st[] = {36, 9, 27, 4, 5, 16, 11, 1, 3, DUMMY, DUMMY, 7, 9, DUMMY, DUMMY}

The dummy values are never accessed and have no use. This is some wastage of space due to simple array representation. We may optimize this wastage using some clever implementations, but the code for sum and update becomes more complex.

Construction of Segment Tree from the given array: We start with a segment arr[0 . . . n-1] and every time we divide the current segment into two halves(if it has not yet become a segment of length 1), and then call the same procedure on both halves, and for each such segment, we store the sum in the corresponding node.

All levels of the constructed segment tree will be completely filled except the last level. Also, the tree will be a Full Binary Tree because we always divide segments into two halves at every level. Since the constructed tree is always a full binary tree with n leaves, there will be n-1 internal nodes. So the total number of nodes will be 2*n – 1. Note that this does not include dummy nodes.

What is the total size of the array representing segment tree?
If n is a power of 2, then there are no dummy nodes. So the size of the segment tree is 2n-1 (n leaf nodes and n-1) internal nodes. If n is not a power of 2, the size of the tree will be 2*x – 1 where x is the smallest of 2 greater than n. For example, when n = 10, then size of array representing segment tree is 2*16-1 = 31.

Query for Sum of given range
Following is the algorithm to get the sum of elements.

```
int getSum(node, l, r)
{
   if the range of the node is within l and r
        return value in the node
   else if the range of the node is completely outside l and r
        return 0
   else
        return getSum(node's left child, l, r) +
```

```
                    getSum(node's right child, l, r)
}
```

Updating a value

Like tree construction and query operations, the update can also be done recursively. We are given an index which needs to be updated. Let diff be the value to be added. We start from the root of the segment tree and add diff to all nodes which have given index in their range. If a node doesn't have given index in its range, we don't make any changes to that node.

Implementation:

C++Java

```
// C++ program to show segment tree operations like construction, query
// and update
#include <bits/stdc++.h>
using namespace std;

// A utility function to get the middle index from corner indexes.
int getMid(int s, int e) { return s + (e -s)/2; }

/* A recursive function to get the sum of values in given range
    of the array. The following are parameters for this function.

    st --> Pointer to segment tree
    si --> Index of current node in the segment tree. Initially
            0 is passed as root is always at index 0
    ss & se --> Starting and ending indexes of the segment represented
                by current node, i.e., st[si]
    qs & qe --> Starting and ending indexes of query range */
int getSumUtil(int *st, int ss, int se, int qs, int qe, int si)
{
    // If segment of this node is a part of given range, then return
    // the sum of the segment
    if (qs <= ss && qe >= se)
        return st[si];

    // If segment of this node is outside the given range
    if (se < qs || ss > qe)
        return 0;

    // If a part of this segment overlaps with the given range
    int mid = getMid(ss, se);
    return getSumUtil(st, ss, mid, qs, qe, 2*si+1) +
        getSumUtil(st, mid+1, se, qs, qe, 2*si+2);
}

/* A recursive function to update the nodes which have the given
```

```
index in their range. The following are parameters
    st, si, ss and se are same as getSumUtil()
    i --> index of the element to be updated. This index is
            in the input array.
diff --> Value to be added to all nodes which have i in range */
void updateValueUtil(int *st, int ss, int se, int i, int diff, int si)
{
    // Base Case: If the input index lies outside the range of
    // this segment
    if (i < ss || i > se)
        return;

    // If the input index is in range of this node, then update
    // the value of the node and its children
    st[si] = st[si] + diff;
    if (se != ss)
    {
        int mid = getMid(ss, se);
        updateValueUtil(st, ss, mid, i, diff, 2*si + 1);
        updateValueUtil(st, mid+1, se, i, diff, 2*si + 2);
    }
}

// The function to update a value in input array and segment tree.
// It uses updateValueUtil() to update the value in segment tree
void updateValue(int arr[], int *st, int n, int i, int new_val)
{
    // Check for erroneous input index
    if (i < 0 || i > n-1)
    {
        cout<<"Invalid Input";
        return;
    }

    // Get the difference between new value and old value
    int diff = new_val - arr[i];

    // Update the value in array
    arr[i] = new_val;

    // Update the values of nodes in segment tree
    updateValueUtil(st, 0, n-1, i, diff, 0);
}

// Return sum of elements in range from index qs (quey start)
// to qe (query end). It mainly uses getSumUtil()
int getSum(int *st, int n, int qs, int qe)
{
    // Check for erroneous input values
```

```
    if (qs < 0 || qe > n-1 || qs > qe)
    {
        cout<<"Invalid Input";
        return -1;
    }

    return getSumUtil(st, 0, n-1, qs, qe, 0);
}

// A recursive function that constructs Segment Tree for array[ss..se].
// si is index of current node in segment tree st
int constructSTUtil(int arr[], int ss, int se, int *st, int si)
{
    // If there is one element in array, store it in current node of
    // segment tree and return
    if (ss == se)
    {
        st[si] = arr[ss];
        return arr[ss];
    }

    // If there are more than one elements, then recur for left and
    // right subtrees and store the sum of values in this node
    int mid = getMid(ss, se);
    st[si] = constructSTUtil(arr, ss, mid, st, si*2+1) +
             constructSTUtil(arr, mid+1, se, st, si*2+2);
    return st[si];
}

/* Function to construct segment tree from given array. This function
allocates memory for segment tree and calls constructSTUtil() to
fill the allocated memory */
int *constructST(int arr[], int n)
{
    // Allocate memory for the segment tree

    //Height of segment tree
    int x = (int)(ceil(log2(n)));

    //Maximum size of segment tree
    int max_size = 2*(int)pow(2, x) - 1;

    // Allocate memory
    int *st = new int[max_size];

    // Fill the allocated memory st
    constructSTUtil(arr, 0, n-1, st, 0);

    // Return the constructed segment tree
```

```
    return st;
}

// Driver program to test above functions
int main()
{
    int arr[] = {1, 3, 5, 7, 9, 11};
    int n = sizeof(arr)/sizeof(arr[0]);

    // Build segment tree from given array
    int *st = constructST(arr, n);

    // Print sum of values in array from index 1 to 3
    cout<<"Sum of values in given range = "<<getSum(st, n, 1, 3)<<endl;

    // Update: set arr[1] = 10 and update corresponding
    // segment tree nodes
    updateValue(arr, st, n, 1, 10);

    // Find sum after the value is updated
    cout<<"Updated sum of values in given range = "
            <<getSum(st, n, 1, 3)<<endl;
    return 0;
}
```

Output

```
Sum of values in given range = 15
Updated sum of values in given range = 22
```

Time Complexity: The time Complexity for tree construction is O(n). There are total 2n-1 nodes, and the value of every node is calculated only once in tree construction.
Time complexity to query is O(Logn). To query a sum, we process at most four nodes at every level and number of levels is O(Logn).
The time complexity of the update is also O(Logn). To update a leaf value, we process one node at every level and number of levels is O(Logn).
Auxiliary Space: O(MAX)

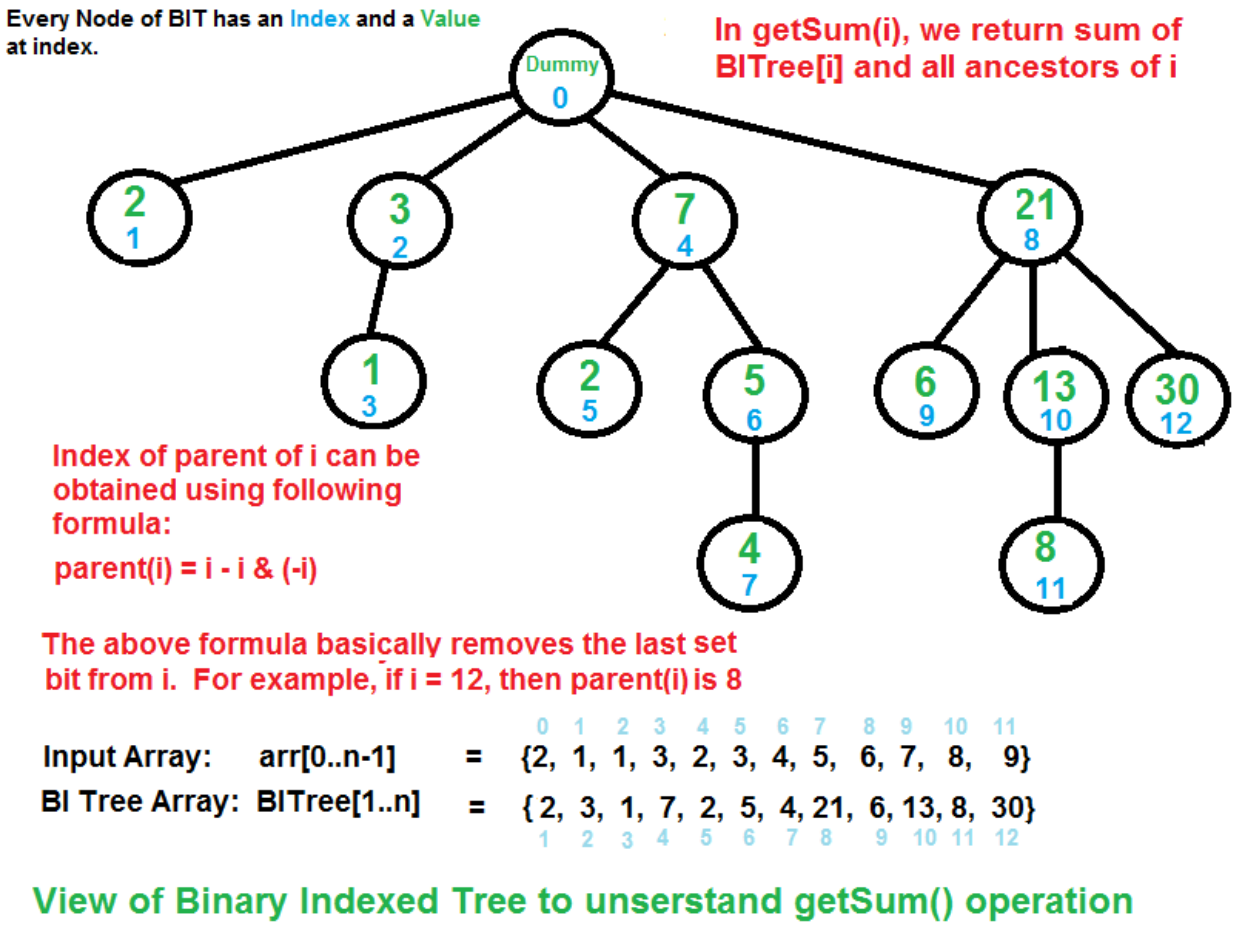
Binary Indexed Tree

Let us consider the following problem to understand Binary Indexed Tree.
We have an array arr[0 . . . n-1]. We would like to
1 Compute the sum of the first i elements.
2 Modify the value of a specified element of the array arr[i] = x where 0 <= i <= n-1.
A **simple solution** is to run a loop from 0 to i-1 and calculate the sum of the elements. To update a value, simply do arr[i] = x. The first operation takes O(n) time and the second operation takes O(1) time. Another simple solution is to create an extra array and store the sum of the first i-th elements at the i-th index in this new array. The sum of a given range can now be calculated in O(1) time, but the update operation takes O(n) time now. This works well if there are a large number of query operations but a very few number of update operations.

Could we perform both the query and update operations in O(log n) time?
One efficient solution is to use Segment Tree that performs both operations in O(Logn) time.
An alternative solution is Binary Indexed Tree, which also achieves O(Logn) time complexity for both operations. Compared with Segment Tree, Binary Indexed Tree requires less space and is easier to implement..
Representation
Binary Indexed Tree is represented as an array. Let the array be BITree[]. Each node of the Binary Indexed Tree stores the sum of some elements of the input array. The size of the Binary Indexed Tree is equal to the size of the input array, denoted as n. In the code below, we use a size of n+1 for ease of implementation.
Construction
We initialize all the values in BITree[] as 0. Then we call update() for all the indexes, the update() operation is discussed below.
Operations

getSum(x): Returns the sum of the sub-array arr[0,...,x]
// Returns the sum of the sub-array arr[0,...,x] using BITree[0..n], which is constructed from arr[0..n-1]
1) Initialize the output sum as 0, the current index as x+1.
2) Do following while the current index is greater than 0.
...a) Add BITree[index] to sum
...b) Go to the parent of BITree[index]. The parent can be obtained by removing the last set bit from the current index, i.e., index = index - (index & (-index))

3) Return sum.

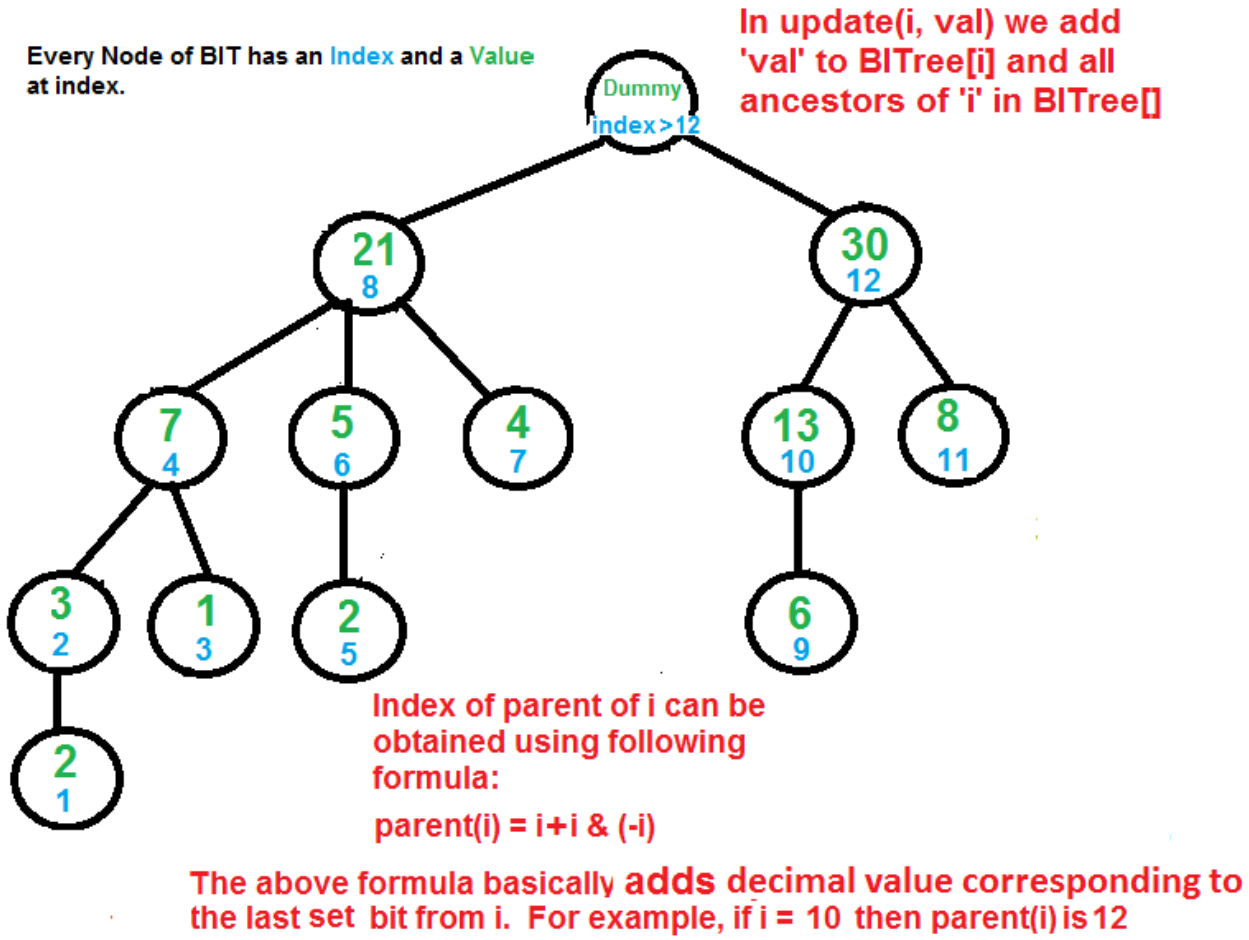


View of Binary Indexed Tree to unserstand getSum() operation

The diagram above provides an example of how getSum() is working. Here are some important observations.
BITree[0] is a dummy node.
BITree[y] is the parent of BITree[x], if and only if y can be obtained by removing the last set bit from the binary representation of x, that is y = x - (x & (-x)).
The child node BITree[x] of the node BITree[y] stores the sum of the elements between y(inclusive) and x(exclusive): arr[y,...,x].

update(x, val): Updates the Binary Indexed Tree (BIT) by performing arr[index] += val

// Note that the update(x, val) operation will not change arr[]. It only makes changes to BITree[]
1) Initialize the current index as x+1.
2) Do the following while the current index is smaller than or equal to n.
...a) Add the val to BITree[index]
...b) Go to next element of BITree[index]. The next element can be obtained by incrementing the last set bit of the current index, i.e., index = index + (index & (-index))



Contents of arr[] and BITree[] are same as above diagram for getSum()

View of Binary Indexed Tree to understand update() operation

The update function needs to make sure that all the BITree nodes which contain arr[i] within their ranges being updated. We loop over such nodes in the BITree by repeatedly adding the decimal number corresponding to the last set bit of the current index.

How does Binary Indexed Tree work?
The idea is based on the fact that all positive integers can be represented as the sum of powers of 2. For example 19 can be represented as 16 + 2 + 1. Every node of the BITree stores the sum of n elements where n is a power of 2. For example, in the first diagram above (the diagram for getSum()), the sum of the first 12 elements can be obtained by the sum of the last 4 elements (from 9 to 12) plus the sum of 8 elements (from 1 to 8). The number of set bits in the binary representation of a number n is O(Logn). Therefore, we traverse at-most O(Logn) nodes in both getSum() and update() operations. The time complexity of the construction is O(nLogn) as it calls update() for all n elements.

Implementation:
Following are the implementations of Binary Indexed Tree.

C++Java

```
// C++ code to demonstrate operations of Binary Index Tree
#include <iostream>

using namespace std;

/*          n --> No. of elements present in input array.
    BITree[0..n] --> Array that represents Binary Indexed Tree.
    arr[0..n-1] --> Input array for which prefix sum is evaluated. */

// Returns sum of arr[0..index]. This function assumes
// that the array is preprocessed and partial sums of
// array elements are stored in BITree[].
int getSum(int BITree[], int index)
{
    int sum = 0; // Initialize result

    // index in BITree[] is 1 more than the index in arr[]
    index = index + 1;

    // Traverse ancestors of BITree[index]
    while (index>0)
    {
        // Add current element of BITree to sum
```

```
        sum += BITree[index];

        // Move index to parent node in getSum View
        index -= index & (-index);
    }
    return sum;
}

// Updates a node in Binary Index Tree (BITree) at given index
// in BITree. The given value 'val' is added to BITree[i] and
// all of its ancestors in tree.
void updateBIT(int BITree[], int n, int index, int val)
{
    // index in BITree[] is 1 more than the index in arr[]
    index = index + 1;

    // Traverse all ancestors and add 'val'
    while (index <= n)
    {
    // Add 'val' to current node of BI Tree
    BITree[index] += val;

    // Update index to that of parent in update View
    index += index & (-index);
    }
}

// Constructs and returns a Binary Indexed Tree for given
// array of size n.
int *constructBITree(int arr[], int n)
{
    // Create and initialize BITree[] as 0
    int *BITree = new int[n+1];
    for (int i=1; i<=n; i++)
        BITree[i] = 0;

    // Store the actual values in BITree[] using update()
    for (int i=0; i<n; i++)
        updateBIT(BITree, n, i, arr[i]);

    // Uncomment below lines to see contents of BITree[]
    //for (int i=1; i<=n; i++)
    //     cout << BITree[i] << " ";

    return BITree;
}


// Driver program to test above functions
```

```
int main()
{
    int freq[] = {2, 1, 1, 3, 2, 3, 4, 5, 6, 7, 8, 9};
    int n = sizeof(freq)/sizeof(freq[0]);
    int *BITree = constructBITree(freq, n);
    cout << "Sum of elements in arr[0..5] is "
         << getSum(BITree, 5);

    // Let use test the update operation
    freq[3] += 6;
    updateBIT(BITree, n, 3, 6); //Update BIT for above change in arr[]

    cout << "\nSum of elements in arr[0..5] after update is "
         << getSum(BITree, 5);

    return 0;
}
```

Output

```
Sum of elements in arr[0..5] is 12
Sum of elements in arr[0..5] after update is 18
```

Time Complexity: O(NLogN)

Auxiliary Space: O(N)

Can we extend the Binary Indexed Tree to computing the sum of a range in O(Logn) time?

Yes. rangeSum(l, r) = getSum(r) - getSum(l-1).

Applications:

The implementation of the arithmetic coding algorithm. The development of the Binary Indexed Tree was primarily motivated by its application in this case.

Disjoint Set

Disjoint Set Data Structure | Introduction

A **disjoint-set** data structure is a data structure that keeps track of a set of elements partitioned into a number of disjoint (non-overlapping) subsets.

For Example:

*Consider that there are 5 students in a classroom namely, **A, B, C, D, and E**.*

*They will be denoted as 5 different subsets: **{A}, {B}, {C}, {D}, {E}**.*

After some point of time, **A** became friends with **B**, and **C** became friend with **D**. So, A and B will now belong to a same set and C and D will now belong to another same set.

The disjoint data structure will now be: **{A, B}, {C, D}, {E}**.

If at any point in time, we want to check that if any two students are friends or not, then we can simply check whether they belong to the same set.

As explained above, there are generally two types of operations performed on a Disjoint-Set data structure:

- **Union(A, B)**: This operation tells to merge the sets containing elements A and B respectively by performing a Union operation on the sets.
- **Find(A)**: This operation tells to find the subset to which the element A belongs.

Implementation: The disjoint set data structure can be implemented using a **Parent array** representation. If we are dealing with n items, i'th element of the array represents the i'th item. More precisely, the i'th element of the array is the parent of the i'th item.

- **Implementing Find Operation**: It can be implemented by recursively traversing the parent array until we hit a node who is the parent of itself.

```
// Finds the representative of the set
// that i is an element of
int find(int i)
{
    // If i is the parent of itself
    if (parent[i] == i)
    {
        // Then i is the representative of
        // this set
        return i;
    }
    else
    {
        // Else if i is not the parent of
        // itself, then i is not the
        // representative of his set. So we
```

```
        // recursively call Find on its parent
        return find(parent[i]);
    }
}
```

- **Implementing Union Operation**: It takes as input two elements and finds the representatives of their sets using the find operation, and then finally puts either one of the trees (representing the set) under the root node of the other tree, effectively merging the trees and the sets.

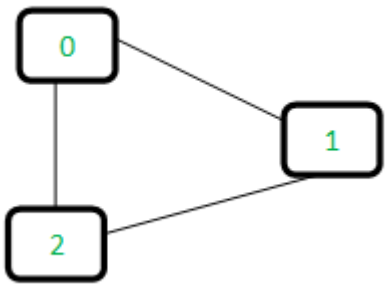
```
// Unites the set that includes i
// and the set that includes j
void union(int i, int j)
{
    // Find the representatives
    // (or the root nodes) for the set
    // that includes i
    int irep = find(i),

    // And do the same for the set
    // that includes j
    int jrep = find(j);

    // Make the parent of i's representative
    // be j's  representative effectively
    // moving all of i's set into j's set)
    Parent[irep] = jrep;
}
```

Application: There are a lot of applications of Disjoint-Set data structure. Consider the problem of **detecting a cycle in a Graph**. It can be easily solved using the Disjoint Set and Union-Find algorithm. This method assumes that the graph does not contain any self-loop.

Let us consider the following graph:



For each edge, make subsets using both the vertices of the edge. If both the vertices are in the same subset, a cycle is found.

Initially, all slots of parent array are initialized to -1 (means there is only one item in every subset).

```
0   1   2
-1 -1  -1
```

Now process all edges one by one.

- **Edge 0-1**: Find the subsets to which vertices 0 and 1 belong to. Since they are in different subsets, we take the union of them. For taking the union, either make node 0 as the parent of node 1 or vice-versa.
- `0   1   2    <----- 1 is made parent of 0`
- `                  (1 is now representative of subset {0, 1})`
- `1  -1  -1`
-
- **Edge 1-2**: 1 is in subset 1 and 2 is in subset 2. So, take union.
- `0   1   2    <----- 2 is made parent of 1`
- `                  (2 is now representative of subset {0, 1, 2})`
- `1   2  -1`
-
- **Edge 0-2**: 0 is in subset 2 and 2 is also in subset 2. Hence, including this edge forms a cycle.

Below is the implementation of the solution to detect cycle using Disjoint-Set:

C++Java

```cpp
// A union-find algorithm to detect cycle in a graph
#include <bits/stdc++.h>
using namespace std;

// a structure to represent an edge in graph
class Edge
{
    public:
    int src, dest;
};

// a structure to represent a graph
class Graph
```

```
{
    public:
    // V-> Number of vertices, E-> Number of edges
    int V, E;

    // graph is represented as an array of edges
    Edge* edge;
};

// Creates a graph with V vertices and E edges
Graph* createGraph(int V, int E)
{
    Graph* graph = new Graph();
    graph->V = V;
    graph->E = E;

    graph->edge = new Edge[graph->E * sizeof(Edge)];

    return graph;
}

// A utility function to find the subset of an element i
int find(int parent[], int i)
{
    if (parent[i] == -1)
        return i;
    return find(parent, parent[i]);
}

// A utility function to do union of two subsets
void Union(int parent[], int x, int y)
{
    int xset = find(parent, x);
    int yset = find(parent, y);
    if(xset != yset)
    {
        parent[xset] = yset;
    }
}

// The main function to check whether a given graph contains
// cycle or not
int isCycle( Graph* graph )
{
    // Allocate memory for creating V subsets
    int *parent = new int[graph->V * sizeof(int)];

    // Initialize all subsets as single element sets
    memset(parent, -1, sizeof(int) * graph->V);
```

```
    // Iterate through all edges of graph, find subset of both
    // vertices of every edge, if both subsets are same, then
    // there is cycle in graph.
    for(int i = 0; i < graph->E; ++i)
    {
        int x = find(parent, graph->edge[i].src);
        int y = find(parent, graph->edge[i].dest);

        if (x == y)
            return 1;

        Union(parent, x, y);
    }
    return 0;
}

// Driver code
int main()
{
    /* Let us create the following graph
        0
        | \
        | \
        1-----2 */
    int V = 3, E = 3;
    Graph* graph = createGraph(V, E);

    // add edge 0-1
    graph->edge[0].src = 0;
    graph->edge[0].dest = 1;

    // add edge 1-2
    graph->edge[1].src = 1;
    graph->edge[1].dest = 2;

    // add edge 0-2
    graph->edge[2].src = 0;
    graph->edge[2].dest = 2;

    if (isCycle(graph))
        cout<<"graph contains cycle";
    else
        cout<<"graph doesn't contain cycle";

    return 0;
}
```

Output:

```
graph contains cycle
```

Note: The implementation of *union()* and *find()* is naive and takes O(n) time in worst case. These methods can be improved to O(Logn) using *Union by Rank or Height*.

Union by Rank and Path Compression

In the previous post, we introduced *union find algorithm* and used it to detect cycle in a graph. We used the following *union()* and *find()* operations for subsets.

C++Java

```
// Naive implementation of find
int find(int parent[], int i)
{
if (parent[i] == -1)
return i;
return find(parent, parent[i]);
}

// Naive implementation of union()
void Union(int parent[], int x, int y)
{
int xset = find(parent, x);
int yset = find(parent, y);
parent[xset] = yset;
}
```

The above *union()* and *find()* are naïve and the worst case time complexity is linear. The trees created to represent subsets can be skewed and can become like a linked list. Following is an example of the worst case scenario.

```
Let there be 4 elements 0, 1, 2, 3

Initially, all elements are single element subsets.
0 1 2 3
```

```
Do Union(0, 1)
1 2 3
/
0

Do Union(1, 2)
2 3
/
1
/
0

Do Union(2, 3)
3
/
2
/
1
/
0
```

The above operations can be optimized to *O(Log n)* time complexity in worst case. The idea is to always attach smaller depth tree under the root of the deeper tree. This technique is called ***union by rank***. The term *rank* is preferred instead of height because if path compression technique (we have discussed it below) is used, then *rank* is not always equal to height. Also, size (in place of height) of trees can also be used as *rank*. Using size as *rank* also yields worst case time complexity as O(Logn).

```
Let us see the above example with the union by rank
Initially, all elements are single element subsets.
0 1 2 3

Do Union(0, 1)
1 2 3
/
0

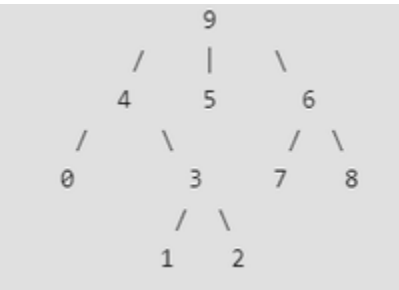
Do Union(1, 2)
1 3
/ \
0 2

Do Union(2, 3)
   1
```

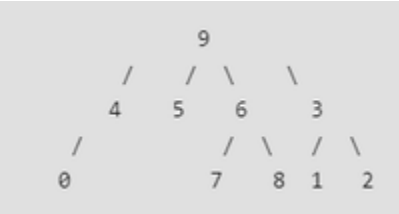
```
/   |   \
0   2    3
```

The second optimization to naive method is **Path Compression**. The idea is to flatten the tree when find() is called. When find() is called for an element x, root of the tree is returned. The *find()* operation traverses up from x to find root. The idea of path compression is to make the found root as parent of x so that we don't have to traverse all intermediate nodes again. If x is root of a subtree, then path (to root) from all nodes under x also compresses.

Let the subset {0, 1, .. 9} be represented as below and find() is called
for element 3.



When *find()* is called for 3, we traverse up and find 9 as representative
of this subset. With path compression, we also make 3 as the child of 9 so
that when find() is called next time for 1, 2 or 3, the path to root is reduced.



The two techniques complement each other. The time complexity of each operation becomes even smaller than O(Logn). In fact, amortized time complexity effectively becomes small constant.

Following is union by rank and path compression based implementation to find a cycle in a graph:

C++Java

```c
// A union by rank and path compression based program
// to detect cycle in a graph

#include <stdio.h>
#include <stdlib.h>

// a structure to represent an edge in the graph
struct Edge {
    int src, dest;
};

// a structure to represent a graph
struct Graph {
    // V-> Number of vertices, E-> Number of edges
    int V, E;

    // graph is represented as an array of edges
    struct Edge* edge;
};

struct subset {
    int parent;
    int rank;
};

// Creates a graph with V vertices and E edges
struct Graph* createGraph(int V, int E)
{
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = V;
    graph->E = E;

    graph->edge = (struct Edge*)malloc(graph->E * sizeof(struct Edge));

    return graph;
}

// A utility function to find set of an element i
// (uses path compression technique)
int find(struct subset subsets[], int i)
{
    // find root and make root as parent of i (path compression)
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);

    return subsets[i].parent;
}

// A function that does union of two sets of x and y
```

```c
// (uses union by rank)
void Union(struct subset subsets[], int x, int y)
{
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);

    // Attach smaller rank tree under root of high rank tree
    // (Union by Rank)
    if (subsets[xroot].rank < subsets[yroot].rank)
        subsets[xroot].parent = yroot;
    else if (subsets[xroot].rank > subsets[yroot].rank)
        subsets[yroot].parent = xroot;

    // If ranks are same, then make one as root and increment
    // its rank by one
    else {
        subsets[yroot].parent = xroot;
        subsets[xroot].rank++;
    }
}

// The main function to check whether a given graph contains cycle or not
int isCycle(struct Graph* graph)
{
    int V = graph->V;
    int E = graph->E;

    // Allocate memory for creating V sets
    struct subset* subsets = (struct subset*)malloc(V * sizeof(struct
subset));

    for (int v = 0; v < V; ++v) {
        subsets[v].parent = v;
        subsets[v].rank = 0;
    }

    // Iterate through all edges of graph, find sets of both
    // vertices of every edge, if sets are same, then there is
    // cycle in graph.
    for (int e = 0; e < E; ++e) {
        int x = find(subsets, graph->edge[e].src);
        int y = find(subsets, graph->edge[e].dest);

        if (x == y)
            return 1;

        Union(subsets, x, y);
    }
    return 0;
```

```
}

// Driver program to test above functions
int main()
{
    /* Let us create the following graph
         0
        |  \
        |    \
        1-----2 */

    int V = 3, E = 3;
    struct Graph* graph = createGraph(V, E);

    // add edge 0-1
    graph->edge[0].src = 0;
    graph->edge[0].dest = 1;

    // add edge 1-2
    graph->edge[1].src = 1;
    graph->edge[1].dest = 2;

    // add edge 0-2
    graph->edge[2].src = 0;
    graph->edge[2].dest = 2;

    if (isCycle(graph))
        printf("Graph contains cycle");
    else
        printf("Graph doesn't contain cycle");

    return 0;
}
```

Output

```
Graph contains cycle
```